GraphString

目录

4 树上算法与动态树 1

4.1 树的基础性质与预处理 1
4.2 LCA、倍增与路径关系 6
4.3 树链分解与树上序列化 11
4.4 树上离线、启发式与压缩 16
4.5 树分治 23
4.6 动态树 31
4.7 动态树应用 40

5 图论 47

5.1 基础、遍历与建图 47
5.2 最短路、环与约束 63
5.3 生成树与连接子图 92
5.4 连通性、缩点与支配 111
5.5 Euler 与基环结构 134
5.6 特殊图与图计数 158

6 匹配、网络流与图优化模型 172

6.1 二分图最大匹配（Hopcroft--Karp） 172
6.2 Dinic 最大流 186
6.3 最小费用流 201
6.4 上下界网络流 206
6.5 Stoer--Wagner 全局最小割 213
6.6 网络流建模与方案恢复索引 216

7 字符串 225

7.1 KMP 225
7.2 字符串哈希 228
7.3 Trie 233
7.4 AC 自动机 241
7.5 子序列自动机 250
7.6 Manacher 254
7.7 PAM（回文树） 257
7.8 Series PAM（回文系列优化） 261
7.9 SA（后缀数组） 265
7.10 SAM 269
7.11 循环最小表示与 Lyndon 分解 275
7.12 Bitset LCS 279
7.13 周期结构、Runs 与 Main – Lorentz 282

4. 树上算法与动态树

4.1 树的基础性质与预处理

树信息、DFS 序与子树

```
struct TreeInfo {
    int n, root{ -1 };
    vector<vector<pair<int, i64>>> g;
    vector<int> fa, dep, siz, in, out, seq;
    vector<i64> dis;
    explicit TreeInfo(int n_) : n(n_), g(n), fa(n, -1), dep(n), siz(n),
        in(n), out(n), seq(n), dis(n) {}
    void add(int u, int v, i64 w = 1) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    void build(int root_ = 0) {
        root = root_;
        fill(fa.begin(), fa.end(), -1);
        int cur{ 0 };
        vector<int> stk{ root };
        fa[root] = root;
        dep[root] = 0;
        dis[root] = 0;
        while (!stk.empty()) {
            int u{ stk.back() };
            stk.pop_back();
            in[u] = cur;
            seq[cur ++] = u;
            for (auto it = g[u].rbegin(); it != g[u].rend(); ++ it) {
                auto [v, w]{ *it };
                if (v == fa[u]) continue;
                fa[v] = u;
                dep[v] = dep[u] + 1;
                dis[v] = dis[u] + w;
                stk.push_back(v);
            }
        }
        fill(siz.begin(), siz.end(), 1);
        for (int i = n - 1; i > 0; -- i) {
            int u{ seq[i] };
            siz[fa[u]] += siz[u];
        }
        for (int u = 0; u < n; ++ u) out[u] = in[u] + siz[u];
    }
    bool is_anc(int u, int v) const { return in[u] <= in[v] && in[v] < out[u]; }
    pair<int, int> sub(int u) const { return {in[u], out[u]}; }
};
```

成员变量

- `n`：节点数，编号为 `0..n-1`
- `root`：最近一次 `build()` 使用的根
- `g[u]`：无向带权邻接表，无权树直接使用默认边权 `1`
- `fa[u]`：预处理根下的父节点，内部满足 `fa[root] = root`
- `dep[u]`：深度，`dep[root] = 0`
- `siz[u]`：子树大小
- `dis[u]`：从 `root` 到 `u` 的路径边权和
- `in[u]`、`out[u]`：子树对应 DFS 序区间 `[in[u],out[u])`
- `seq[i]`：DFS 序位置 `i` 对应的原节点，满足 `seq[in[u]] = u`

接口

- `TreeInfo(n)`：初始化 `n` 个节点，$O(n)$
- `add(u,v,w)`：加入无向边，默认 `w=1`，均摊 $O(1)$
- `build(root)`：预处理父节点、深度、根距离、子树大小和 DFS 序，$O(n)$
- `is_anc(u,v)`：判断 `u` 是否为 `v` 的祖先，$O(1)$
- `sub(u)`：返回 `u` 的子树区间 `[in[u],out[u])`，$O(1)$

注意事项

- 要求 `n≥1`，输入为含 `n-1` 条边的连通无向树
- 所有节点和区间均为 `0base`，区间使用 `[l,r)`
- `fa[root]=root` 是内部约定；判断根节点时直接与 `root` 比较
- `seq` 是 DFS 先序，不是 LCA 中长度约为 `2n-1` 的 Euler Tour
- `dis` 只是唯一树路径的权值和，即使有负边也能正常计算；但数值必须能够放入 `i64`
- `build()` 不修改邻接表，可以换根后重新调用

常见用法

子树转连续区间

```
TreeInfo tr(n);
// tr.add(...)
tr.build(0);
auto [l, r]{ tr.sub(u) };
```

点权按 DFS 序展开

```
vector<i64> b(n);
for (int u = 0; u < n; ++ u) b[tr.in[u]] = a[u];
```

自底向上遍历

```
for (int i = n - 1; i > 0; -- i) {
    int u{ tr.seq[i] }, p{ tr.fa[u] };
    // 用 u 的信息更新 p
}
```

复杂度

操作	时间复杂度
`add()`	均摊 $O(1)$
`build()`	$O(n)$
`is_anc()` / `sub()`	$O(1)$

空间复杂度：$O(n)$

直径、中心与重心

```
struct TreeProp {
    const TreeInfo &t;
    explicit TreeProp(const TreeInfo &t_) : t(t_) {}
    // ===== 距离与直径：diam() / ecc() / center() 依赖 =====
    vector<i64> dists(int s) const {
        vector<i64> d(t.n);
        vector<pair<int, int>> stk{{s, -1}};
        while (!stk.empty()) {
            auto [u, p]{ stk.back() };
            stk.pop_back();
            for (auto [v, w] : t.g[u]) {
                if (v == p) continue;
                d[v] = d[u] + w;
                stk.push_back({v, u});
            }
        }
        return d;
```

```
    }
    pair<int, i64> far(int s) const {
        auto d{ dists(s) };
        int u{ int(max_element(d.begin(), d.end()) - d.begin()) };
        return {u, d[u]};
    }
    tuple<int, int, i64> diam() const {
        int s{ far(0).first };
        auto [t, d]{ far(s) };
        return {s, t, d};
    }
    // ===== 离心率与中心 =====
    vector<i64> ecc() const {
        int s{ far(0).first };
        auto ds{ dists(s) };
        int t{ int(max_element(ds.begin(), ds.end()) - ds.begin()) };
        auto dt{ dists(t) };
        for (int u = 0; u < this->t.n; ++ u) ds[u] = max(ds[u], dt[u]);
        return ds;
    }
    pair<i64, vector<int>> center() const {
        auto e{ ecc() };
        i64 r{ *min_element(e.begin(), e.end()) };
        vector<int> res;
        for (int u = 0; u < t.n; ++ u) if (e[u] == r) res.push_back(u);
        return {r, res};
    }
    // ===== 重心 =====
    vector<int> centroid() const {
        vector<int> res;
        for (int u = 0; u < t.n; ++ u) {
            int mx{ t.n - t.siz[u] };
            for (auto [v, w] : t.g[u]) {
                if (t.fa[v] == u) mx = max(mx, t.siz[v]);
            }
            if (mx * 2 <= t.n) res.push_back(u);
        }
        return res;
    }
};
```

接口说明

- `TreeProp(t)`：绑定一棵 `TreeInfo`，$O(1)$
- `dists(s)`：返回 `s` 到所有节点的路径权值和，$O(n)$
- `far(s)`：返回距离 `s` 最远的节点及距离，要求边权非负，$O(n)$
- `diam()`：返回 `{s,t,d}`，其中 `s-t` 为一条直径、`d` 为直径长度，$O(n)$
- `ecc()`：返回每个节点的离心率，即到最远节点的距离，$O(n)$
- `center()`：返回 `{半径,所有节点中心}`，$O(n)$
- `centroid()`：返回一个或两个重心，需要先调用 `TreeInfo::build()`，$O(n)$

注意事项

- `far()`、`diam()`、`ecc()`、`center()` 的端点性质要求边权非负
- 若存在负边权，最大权路径应改用树形 DP，不能使用两次 `far()`
- `center()` 求的是**节点中心**；无权树中即经典树中心
- 带权树若允许中心落在边内部，则连续中心位于某条直径中点，本函数不表示边内部位置
- `centroid()` 只由树拓扑决定，与边权无关
- 树重心最多有两个；若有两个则相邻
- `centroid()` 依赖 `fa` 和 `siz`，因此调用前必须完成 `build()`

常见用法

```
TreeInfo tr(n);
// tr.add(...)
tr.build(0);
TreeProp tp(tr);

auto [s, t, d]{ tp.diam() };
```

```
auto [r, cen]{ tp.center() };
vector<int> cog{ tp.centroid() };
```

复杂度

操作	时间复杂度
dists() / far()	$O(n)$
diam()	$O(n)$
ecc() / center()	$O(n)$
centroid()	$O(n)$

额外空间复杂度：$O(n)$

树哈希与树同构

```
struct TreeIso {
    map<vector<int>, int> id;
    // ===== 有根树规范编号：核心功能 =====
    vector<int> rids(const vector<vector<int>> &g, int root) {
        int n{ int(g.size()) };
        vector<int> fa(n, -1), seq(n), val(n);
        int ord{ 1 };
        seq[0] = root;
        fa[root] = root;
        for (int i = 0; i < ord; ++ i) {
            int u{ seq[i] };
            for (int v : g[u]) {
                if (fa[v] != -1) continue;
                fa[v] = u;
                seq[ord ++] = v;
            }
        }
        for (int i = n - 1; i >= 0; -- i) {
            int u{ seq[i] };
            vector<int> ch;
            for (int v : g[u]) if (fa[v] == u) ch.push_back(val[v]);
            sort(ch.begin(), ch.end());
            auto [it, ok]{ id.try_emplace(move(ch), int(id.size())) };
            val[u] = it->second;
        }
        return val;
    }
    int rid(const vector<vector<int>> &g, int root) { return rids(g, root)[root]; }
    bool same_root(const vector<vector<int>> &a, int ra,
                   const vector<vector<int>> &b, int rb) {
        return a.size() == b.size() && rid(a, ra) == rid(b, rb);
    }
    // ===== 无根树同构 =====
    vector<int> cent(const vector<vector<int>> &g) const {
        int n{ int(g.size()) };
        if (n <= 2) {
            vector<int> res(n);
            iota(res.begin(), res.end(), 0);
            return res;
        }
        vector<int> deg(n);
        queue<int> q;
        for (int u = 0; u < n; ++ u) {
            deg[u] = g[u].size();
            if (deg[u] == 1) q.push(u);
        }
        int rem{ n };
        while (rem > 2) {
            int cnt{ int(q.size()) };
            rem -= cnt;
            while (cnt --) {
```

```
                int u{ q.front() };
                q.pop();
                for (int v : g[u]) if (-- deg[v] == 1) q.push(v);
            }
        }
        vector<int> res;
        while (!q.empty()) {
            res.push_back(q.front());
            q.pop();
        }
        return res;
    }
    vector<int> uid(const vector<vector<int>> &g) {
        vector<int> res;
        for (int u : cent(g)) res.push_back(rid(g, u));
        sort(res.begin(), res.end());
        return res;
    }
    bool same(const vector<vector<int>> &a, const vector<vector<int>> &b) {
        return a.size() == b.size() && uid(a) == uid(b);
    }
};
```

成员变量

- `id`：规范子树结构到唯一整数编号的映射；同一组比较中必须共享

接口

- `rids(g,root)`：返回以 `root` 为根时每个节点子树的规范编号，$O(n \log n)$
- `rid(g,root)`：返回整棵有根树的规范编号，$O(n \log n)$
- `same_root(a,ra,b,rb)`：判断两棵有根树是否同构，$O(n \log n)$
- `cent(g)`：返回无根树的一个或两个拓扑中心，$O(n)$
- `uid(g)`：返回无根树规范编号，$O(n \log n)$
- `same(a,b)`：判断两棵无根树是否同构，$O(n \log n)$

注意事项

- `g` 必须表示一棵 `0base` 连通无向树，且节点数至少为 `1`
- AHU 规范编号为确定性算法，不存在随机哈希碰撞
- **需要比较的多棵树必须使用同一个 `TreeIso` 对象**，否则整数编号没有直接可比性
- `rids()` 可以直接用于判断同一棵有根树中的两个子树是否同构
- `id` 会跨调用保留已经出现过的规范类型；一组独立比较结束后可重新构造 `TreeIso`。表中已有 S 种类型时，单次时间更准确地为 O(n log n + n log(S+n))；表的空间由所有规范键的总长度决定。使用 try_emplace，已存在的键不再构造节点或复制键
- 当前版本只考虑树的拓扑。如果节点有颜色，应把规范键改成 `{color[u],ch}`；边有类型时也应把边类型纳入规范键

常见用法

有根树同构

```
TreeIso iso;
bool ok{ iso.same_root(a, ra, b, rb) };
```

无根树同构

```
TreeIso iso;
bool ok{ iso.same(a, b) };
```

判断两棵子树是否同构

```
TreeIso iso;
auto h{ iso.rids(g, root) };
bool ok{ h[u] == h[v] };
```

复杂度

操作	时间复杂度
`rids()` / `rid()`	$O(n \log n)$
`cent()`	$O(n)$

操作	时间复杂度
`uid()` / `same()`	$O(n \log n)$

单次处理额外空间复杂度：$O(n)$；`id` 的空间取决于累计出现的规范子树类型数

4.2 LCA、倍增与路径关系

解决问题： 预处理最近公共祖先，支持祖先跳跃、树上距离、路径第 k 点、动态换根、路径交并关系以及基于 LCA 的树上差分

版本选择

需求	选择
普通 LCA、祖先跳跃、路径第 k 点	倍增 `LCA`
大量纯 LCA / 距离查询	`ELCA` + 状压 RMQ
已经需要完整 HLD	直接使用后面 4.3 HLD 的 LCA，不再复制本节
所有询问离线	一般仍可用前两版；Tarjan 离线 LCA 不单独保留板子
路径交 / 包含 / 合并	`PathRel<LCA>` 或 `PathRel<ELCA>`
批量路径点 / 边计数	`TreeDiff`

倍增 LCA

依赖 4.1.1 `TreeInfo`，必须先调用 `TreeInfo::build()`

```
struct LCA {
    const TreeInfo &t;
    int lg;
    vector<vector<int>> up;
    explicit LCA(const TreeInfo &t_) : t(t_), lg(__lg(max(1, t.n)) + 1),
        up(lg, vector<int>(t.n)) {
        up[0] = t.fa;
        for (int k = 1; k < lg; ++ k) {
            for (int u = 0; u < t.n; ++ u) {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
        }
    }
    // ===== 基础 LCA 与距离 =====
    int lca(int u, int v) const {
        if (t.is_anc(u, v)) return u;
        if (t.is_anc(v, u)) return v;
        for (int k = lg - 1; k >= 0; -- k) {
            if (!t.is_anc(up[k][u], v)) u = up[k][u];
        }
        return t.fa[u];
    }
    int len(int u, int v) const {
        int p{ lca(u, v) };
        return t.dep[u] + t.dep[v] - 2 * t.dep[p];
    }
    i64 dist(int u, int v) const {
        int p{ lca(u, v) };
        return i128(t.dis[u]) + t.dis[v] - 2 * i128(t.dis[p]);
    }
    // ===== 祖先与路径跳跃 =====
    int jump(int u, int k) const {
        if (k < 0 || k > t.dep[u]) return -1;
        for (int i = 0; i < lg; ++ i) if (k >> i & 1) u = up[i][u];
        return u;
    }
    int kth(int u, int v, int k) const {
        int p{ lca(u, v) };
        int a{ t.dep[u] - t.dep[p] }, b{ t.dep[v] - t.dep[p] };
```

```
        if (k < 0 || k > a + b) return -1;
        return k <= a ? jump(u, k) : jump(v, a + b - k);
    }
    // ===== 动态换根 =====
    int lca_r(int u, int v, int r) const {
        int a{ lca(u, v) }, b{ lca(u, r) }, c{ lca(v, r) };
        if (t.dep[b] > t.dep[a]) a = b;
        if (t.dep[c] > t.dep[a]) a = c;
        return a;
    }
    int par_r(int u, int r) const {
        if (u == r) return -1;
        if (!t.is_anc(u, r)) return t.fa[u];
        return jump(r, t.dep[r] - t.dep[u] - 1);
    }
    int siz_r(int u, int r) const {
        if (u == r) return t.n;
        if (!t.is_anc(u, r)) return t.siz[u];
        int v{ jump(r, t.dep[r] - t.dep[u] - 1) };
        return t.n - t.siz[v];
    }
};
```

成员变量

- `t`：已经完成 `build()` 的 `TreeInfo`
- `lg`：倍增层数
- `up[k][u]`：`u` 向上的第 2^k 个祖先，根的所有倍增祖先均为根

接口

- `LCA(t)`：建立倍增表，$O(n \log n)$
- `lca(u,v)`：最近公共祖先，$O(\log n)$
- `len(u,v)`：路径边数，$O(\log n)$
- `dist(u,v)`：路径边权和，$O(\log n)$
- `jump(u,k)`：`u` 的第 `k` 级祖先，不存在返回 `-1`，$O(\log n)$
- `kth(u,v,k)`：路径 `u → v` 上从 `u` 开始第 `k` 个点，`k=0` 时为 `u`，不存在返回 `-1`，$O(\log n)$
- `lca_r(u,v,r)`：临时以 `r` 为根时的 LCA，$O(\log n)$
- `par_r(u,r)`：临时以 `r` 为根时 `u` 的父节点，`u=r` 返回 `-1`，$O(\log n)$
- `siz_r(u,r)`：临时以 `r` 为根时 `u` 的子树大小，$O(\log n)$

注意事项

- 构造 `LCA` 前必须执行 `TreeInfo::build()`
- 若之后重新调用 `TreeInfo::build()` 换根，必须重新构造 `LCA`，原 `up` 表已经失效
- `dist()` 是唯一树路径的边权和，允许负边权；根距离与最终答案须能放入 i64，中间表达式使用 i128，避免相消前溢出
- `len()` 只计算路径边数，不受边权影响
- `jump(u,k)` 中 `k<0` 或 `k>dep[u]` 时返回 `-1`
- `lca_r()`、`par_r()`、`siz_r()` 不会真正重新建树，只按照临时根解释原树

常见用法

路径第 k 个点

```
TreeInfo tr(n);
// tr.add(...)
tr.build(0);
LCA lca(tr);

int x{ lca.kth(u, v, k) };
```

动态换根

```
int p{ lca.lca_r(u, v, r) };
int fa{ lca.par_r(u, r) };
int siz{ lca.siz_r(u, r) };
```

复杂度

操作	时间复杂度
构造	$O(n \log n)$
`lca()` / `jump()` / `kth()`	$O(\log n)$
`len()` / `dist()`	$O(\log n)$
动态换根接口	$O(\log n)$

空间复杂度：$O(n \log n)$

欧拉序 + 状压 RMQ LCA

当题目有大量 LCA / 距离查询，而不需要 `jump()`、`kth()` 时可以使用本版
依赖前面的 `TreeInfo` 以及通用 `RMQ<T,Cmp>`

```
struct ELCA {
    const TreeInfo &t;
    vector<int> fir, eul;
    optional<RMQ<int>> rmq;
    explicit ELCA(const TreeInfo &t_) : t(t_), fir(t.n, -1) {
        struct F { int u, it; };
        vector<F> stk{{t.root, 0}};
        eul.reserve(2 * t.n - 1);
        fir[t.root] = 0;
        eul.push_back(t.root);
        while (!stk.empty()) {
            F &f{ stk.back() };
            if (f.it == int(t.g[f.u].size())) {
                stk.pop_back();
                if (!stk.empty()) eul.push_back(stk.back().u);
                continue;
            }
            int v{ t.g[f.u][f.it ++].first };
            if (t.fa[v] != f.u) continue;
            fir[v] = eul.size();
            eul.push_back(v);
            stk.push_back({v, 0});
        }
        vector<int> d(eul.size());
        for (int i = 0; i < int(eul.size()); ++ i) d[i] = t.dep[eul[i]];
        rmq.emplace(move(d));
    }
    int lca(int u, int v) const {
        int l{ fir[u] }, r{ fir[v] };
        if (l > r) swap(l, r);
        return eul[rmq->query_index(l, r + 1)];
    }
    int len(int u, int v) const {
        int p{ lca(u, v) };
        return t.dep[u] + t.dep[v] - 2 * t.dep[p];
    }
    i64 dist(int u, int v) const {
        int p{ lca(u, v) };
        return i128(t.dis[u]) + t.dis[v] - 2 * i128(t.dis[p]);
    }
    // ===== 动态换根 =====
    int lca_r(int u, int v, int r) const {
        int a{ lca(u, v) }, b{ lca(u, r) }, c{ lca(v, r) };
        if (t.dep[b] > t.dep[a]) a = b;
        if (t.dep[c] > t.dep[a]) a = c;
        return a;
    }
};
```

接口说明

- `fir[u]`：`u` 第一次出现在 Euler Tour 中的位置
- `eul`：Euler Tour，长度为 $2n-1$
- `ELCA(t)`：构造欧拉序并建立 RMQ
- `lca(u,v)`：最近公共祖先，$O(1)$

- `len(u,v)`：路径边数，$O(1)$
- `dist(u,v)`：路径边权和，$O(1)$
- `lca_r(u,v,r)`：临时换根 LCA，$O(1)$

注意事项

- 同样要求 `TreeInfo::build()` 已完成，重新 `build()` 后必须重新构造 `ELCA`
- 本版**没有祖先跳跃**，需要 `jump()`、`kth()`、`par_r()`、`siz_r()` 时优先使用倍增版
- `eul` 是真正的 Euler Tour，与 4.1 中长度为 `n` 的 DFS 先序 `seq` 不同
- RMQ 查询的是 Euler Tour 上的**最小深度位置**，所以直接使用 `RMQ<int>` 的默认 `less<int>`
- 不需要把整棵树节点编号放进 RMQ；RMQ 内保存的是 Euler Tour 对应的深度数组

复杂度

令 $N = 2n - 1$，RMQ 块大小为 $B = 64$：

操作	时间复杂度
建 Euler Tour	$O(n)$
建 RMQ	$O(N + \frac{N}{B}\log\frac{N}{B})$
`lca()` / `len()` / `dist()`	$O(1)$

空间复杂度为 $O(N + \frac{N}{B}\log\frac{N}{B})$

> Tip
>
> 实际竞赛规模下 $B = 64$，这部分非常接近线性；但严格写复杂度时没必要把块上 ST 强行写成 $O(n)$

路径关系

使用 `LCA` 或 `ELCA` 均可，所有判定都使用**路径边数**，因此与边权正负无关

```
template<class L>
struct PathRel {
    const L &l;
    explicit PathRel(const L &l_) : l(l_) {}
    bool on(int x, int u, int v) const {
        return l.len(u, x) + l.len(x, v) == l.len(u, v);
    }
    bool subset(int a, int b, int c, int d) const {
        return on(a, c, d) && on(b, c, d);
    }
    optional<pair<int, int>> inter(int a, int b, int c, int d) const {
        array<int, 4> p{
            l.lca(a, c), l.lca(a, d),
            l.lca(b, c), l.lca(b, d)
        };
        sort(p.begin(), p.end(), [&](int x, int y) {
            if (l.t.dep[x] != l.t.dep[y]) return l.t.dep[x] < l.t.dep[y];
            return l.t.in[x] < l.t.in[y];
        });
        int x{ l.lca(a, b) }, y{ l.lca(c, d) };
        if (l.t.dep[p[0]] < min(l.t.dep[x], l.t.dep[y])) return nullopt;
        if (l.t.dep[p[2]] < max(l.t.dep[x], l.t.dep[y])) return nullopt;
        return pair{p[2], p[3]};
    }
    int inter_size(int a, int b, int c, int d) const {
        auto p{ inter(a, b, c, d) };
        return p ? l.len(p->first, p->second) + 1 : 0;
    }
    optional<pair<int, int>> join(int a, int b, int c, int d) const {
        if (!inter(a, b, c, d)) return nullopt;
        array<int, 4> p{a, b, c, d};
        pair<int, int> ans{a, a};
        for (int i = 0; i < 4; ++ i) {
            for (int j = i + 1; j < 4; ++ j) {
                if (l.len(p[i], p[j]) > l.len(ans.first, ans.second)) {
```

```
                    ans = {p[i], p[j]};
                }
            }
        }
        for (int x : p) if (!on(x, ans.first, ans.second)) return nullopt;
        return ans;
    }
};
```

接口说明

- `PathRel(l)`：绑定 `LCA` 或 `ELCA`
- `on(x,u,v)`：判断 `x` 是否位于路径 `u-v` 上
- `subset(a,b,c,d)`：判断路径 `a-b` 是否完全包含于路径 `c-d`
- `inter(a,b,c,d)`：返回两条路径交集的端点；无交集返回 `nullopt`
- `inter_size(a,b,c,d)`：返回交集包含的节点数；无交集返回 `0`
- `join(a,b,c,d)`：若两条路径的并仍是一条路径，返回并路径端点，否则返回 `nullopt`
  使用倍增 `LCA` 时以上接口均为 $O(\log n)$；使用 `ELCA` 时均为 $O(1)$

注意事项

- 路径包含端点
- 两条路径仅在一个节点相交时，`inter()` 返回 `{x,x}`
- `join()` 要求两条路径本身相交；若两条路径之间存在一段未被覆盖的空隙，即使四个端点共线也返回 `nullopt`
- 两条相交路径的并不一定仍是一条路径，例如在某个节点形成三叉形时 `join()` 会返回空
- 这里完全使用 `len()` 判断路径关系，因此负边权不会影响结果

常见用法

```
LCA lca(tr);
PathRel rel(lca);

bool ok{ rel.on(x, u, v) };
auto p{ rel.inter(a, b, c, d) };
if (p) {
    auto [x, y]{ *p };
}
```

复杂度

实现	路径关系单次复杂度
`PathRel<LCA>`	$O(\log n)$
`PathRel<ELCA>`	$O(1)$

额外空间复杂度：$O(1)$

树上差分

用于批量给若干条路径增加贡献，最后一次自底向上累加得到每个点或每条边的总贡献

```
template<class L, class T = i64>
struct TreeDiff {
    const L &l;
    vector<T> node, edge;
    explicit TreeDiff(const L &l_) : l(l_), node(l.t.n), edge(l.t.n) {}
    void clear() {
        fill(node.begin(), node.end(), T{});
        fill(edge.begin(), edge.end(), T{});
    }
    // ===== 点差分 =====
    void add_node(int u, int v, const T &x = T{ 1 }) {
        int p{ l.lca(u, v) };
        node[u] += x;
        node[v] += x;
        node[p] -= x;
        if (p != l.t.root) node[l.t.fa[p]] -= x;
    }
```

```
    // ===== 边差分 =====
    void add_edge(int u, int v, const T &x = T{ 1 }) {
        int p{ l.lca(u, v) };
        edge[u] += x;
        edge[v] += x;
        edge[p] -= x + x;
    }
    // ===== 所有修改完成后调用一次 =====
    void calc() {
        for (int i = l.t.n - 1; i > 0; -- i) {
            int u{ l.t.seq[i] }, p{ l.t.fa[u] };
            node[p] += node[u];
            edge[p] += edge[u];
        }
    }
};
```

接口说明

- `TreeDiff(l)`：绑定 `LCA` 或 `ELCA`
- `clear()`：清空全部差分，$O(n)$
- `add_node(u,v,x)`：路径 `u-v` 上所有节点增加 `x`
- `add_edge(u,v,x)`：路径 `u-v` 上所有边增加 `x`
- `calc()`：自底向上累加全部差分，$O(n)$

  调用 `calc()` 后：
- `node[u]`：节点 `u` 的最终贡献
- 对 `u ≠ root`，`edge[u]`：边 `(fa[u],u)` 的最终贡献
- `edge[root]` 没有对应的父边，应忽略

注意事项

- 固定调用顺序为：加入所有路径差分 → `calc()` → 读取答案
- `calc()` 会原地把差分数组转为答案，调用后不能继续添加修改再重复累加；新一轮先 `clear()`
- `T` 需要支持 `+`、`-`、`+=`、`-=`
- 若只使用点差分，可以删除 `edge`、`add_edge()` 及 `calc()` 中对应语句；只使用边差分同理
- 子树批量修改不需要树上差分，直接利用 `[in[u],out[u])` 做普通一维差分即可

常见用法

统计节点经过次数

```
LCA lca(tr);
TreeDiff diff(lca);

for (auto [u, v] : qry) diff.add_node(u, v);
diff.calc();

cout << diff.node[u] << '\n';
```

统计每条边经过次数

```
for (auto [u, v] : qry) diff.add_edge(u, v);
diff.calc();

int x{ tr.dep[u] > tr.dep[v] ? u : v };
cout << diff.edge[x] << '\n';
```

复杂度

设路径修改数为 q：

LCA 版本	单次差分	总复杂度
倍增 `LCA`	$O(\log n)$	$O(n+q\log n)$
`ELCA`	$O(1)$	$O(n+q)$，不计 RMQ 预处理

差分本身空间复杂度：$O(n)$

4.3 树链分解与树上序列化

解决问题： 将子树和树上路径映射为少量连续区间，接入 BIT / 线段树等序列数据结构；或利用长儿子优先顺序压缩深度相关树形 DP 的存储与合并

选型

需求	使用
子树转连续区间	`HLD::sub()`
任意路径转少量区间	`HLD::for_path()`
非交换路径信息	`HLD::path()`
点权 / 边权路径数据结构	HLD
换根后的子树转区间	`HLD::sub_r()`
LCA / 第 k 祖先且已经复制 HLD	HLD 自带接口
只需要 LCA	4.2
按深度维护树形 DP	`LongChain`
希望重儿子 DP 与父亲错位共享内存	`LongChain`

HLD / 重链剖分

依赖 4.1.1 `TreeInfo`，构造前必须已经执行 `TreeInfo::build()`

```
struct HLD {
    struct Seg {
        int l, r;
        bool rev;
    };
    const TreeInfo &t;
    vector<int> son, top, in, out, seq;
    explicit HLD(const TreeInfo &t_) : t(t_), son(t.n, -1), top(t.n),
        in(t.n), out(t.n), seq(t.n) { build(); }
    // ===== 建树：按子树大小选择重儿子，并生成重儿子优先 DFS 序 =====
    void build() {
        fill(son.begin(), son.end(), -1);
        for (int u = 0; u < t.n; ++ u) {
            for (auto [v, w] : t.g[u]) {
                if (t.fa[v] == u && (son[u] == -1 || t.siz[v] > t.siz[son[u]])) son[u] = v;
            }
        }
        int cur{ 0 };
        vector<pair<int, int>> stk{{t.root, t.root}};
        while (!stk.empty()) {
            auto [u, tp]{ stk.back() };
            stk.pop_back();
            for (int x = u; x != -1; x = son[x]) {
                top[x] = tp;
                in[x] = cur;
                seq[cur ++] = x;
                for (auto it = t.g[x].rbegin(); it != t.g[x].rend(); ++ it) {
                    int v{ it->first };
                    if (t.fa[v] == x && v != son[x]) stk.push_back({v, v});
                }
            }
        }
        for (int u = 0; u < t.n; ++ u) out[u] = in[u] + t.siz[u];
    }
    // ===== 固定根子树与无序路径 =====
    pair<int, int> sub(int u, bool edge = false) const {
        return {in[u] + edge, out[u]};
    }
    template<class F>
    void for_path(int u, int v, F &&f, bool edge = false) const {
        while (top[u] != top[v]) {
            if (t.dep[top[u]] < t.dep[top[v]]) swap(u, v);
            f(in[top[u]], in[u] + 1);
```

```
            u = t.fa[top[u]];
        }
        if (t.dep[u] > t.dep[v]) swap(u, v);
        int l{ in[u] + edge }, r{ in[v] + 1 };
        if (l < r) f(l, r);
    }
    // ===== 有序路径：若所有维护信息满足交换律，可删除 Seg 和以下函数 =====
    vector<Seg> path(int u, int v, bool edge = false) const {
        vector<Seg> a, b;
        while (top[u] != top[v]) {
            if (t.dep[top[u]] >= t.dep[top[v]]) {
                a.push_back({in[top[u]], in[u] + 1, true});
                u = t.fa[top[u]];
            } else {
                b.push_back({in[top[v]], in[v] + 1, false});
                v = t.fa[top[v]];
            }
        }
        if (t.dep[u] >= t.dep[v]) {
            int l{ in[v] + edge }, r{ in[u] + 1 };
            if (l < r) a.push_back({l, r, true});
        } else {
            int l{ in[u] + edge }, r{ in[v] + 1 };
            if (l < r) b.push_back({l, r, false});
        }
        reverse(b.begin(), b.end());
        a.insert(a.end(), b.begin(), b.end());
        return a;
    }
    // ===== 定位接口 =====
    int lca(int u, int v) const {
        while (top[u] != top[v]) {
            if (t.dep[top[u]] < t.dep[top[v]]) swap(u, v);
            u = t.fa[top[u]];
        }
        return t.dep[u] < t.dep[v] ? u : v;
    }
    int jump(int u, int k) const {
        if (k < 0 || k > t.dep[u]) return -1;
        int d{ t.dep[u] - k };
        while (t.dep[top[u]] > d) u = t.fa[top[u]];
        return seq[in[u] - (t.dep[u] - d)];
    }
    int kth(int u, int v, int k) const {
        int p{ lca(u, v) };
        int a{ t.dep[u] - t.dep[p] }, b{ t.dep[v] - t.dep[p] };
        if (k < 0 || k > a + b) return -1;
        return k <= a ? jump(u, k) : jump(v, a + b - k);
    }
    // ===== 换根子树：依赖 jump() =====
    vector<pair<int, int>> sub_r(int u, int r) const {
        if (u == r) return {{0, t.n}};
        if (!t.is_anc(u, r)) return {{in[u], out[u]}};
        int v{ jump(r, t.dep[r] - t.dep[u] - 1) };
        vector<pair<int, int>> res;
        if (in[v]) res.push_back({0, in[v]});
        if (out[v] < t.n) res.push_back({out[v], t.n});
        return res;
    }
};
```

成员变量

- `son[u]`：`u` 的重儿子，即子树大小最大的儿子，不存在为 `-1`
- `top[u]`：`u` 所在重链的链顶
- `in[u]`：`u` 在重儿子优先 DFS 序中的位置
- `out[u]`：子树右端点，子树对应 `[in[u],out[u])`
- `seq[i]`：位置 `i` 对应的原树节点
- `Seg{l,r,rev}`：有向路径段 `[l,r)`；`rev=true` 表示实际经过顺序为 `r-1 → l`

接口

- `HLD(t)`：基于已完成预处理的 `TreeInfo` 建立 HLD，$O(n)$
- `build()`：重新构建 HLD，$O(n)$
- `sub(u,edge)`：返回固定根下 `u` 的子树区间，$O(1)$
- `for_path(u,v,f,edge)`：把路径拆成 $O(\log n)$ 个区间并调用 `f(l,r)`，$O(\log n)$
- `path(u,v,edge)`：按实际 `u → v` 顺序返回路径段，$O(\log n)$
- `lca(u,v)`：LCA，$O(\log n)$
- `jump(u,k)`：向上跳 `k` 条边，不存在返回 `-1`，$O(\log n)$
- `kth(u,v,k)`：路径 `u → v` 上从 `u` 起第 `k` 个点，不存在返回 `-1`，$O(\log n)$
- `sub_r(u,r)`：临时以 `r` 为根时，将 `u` 的子树表示成至多两个序列区间，$O(\log n)$

注意事项

- `TreeInfo::build()` 与 `HLD::build()` 必须使用同一棵有根树；若重新调用了前者，必须再次调用后者
- `HLD::in/out` 与 `TreeInfo::in/out` 是两套不同序列；接入数据结构时使用 HLD 自己的 `in/out`
- `for_path()` 不保证回调顺序等于 `u → v` 的路径顺序，只适合加法、最大值、最小值、异或等交换型维护
- 矩阵乘法、字符串拼接等非交换信息使用 `path()`，并按 `rev` 选择正向或反向区间信息
- `edge=false` 为点权模式；`edge=true` 会排除 LCA 对应位置，用于边权模式。
- 边权模式把边 `(fa[u],u)` 的权值存到位置 `in[u]`，根节点位置不对应任何父边
- 一条树路径只会跨过 $O(\log n)$ 条重链，因此配合单次 $O(\log n)$ 的线段树操作时，整条路径通常为 $O(\log^2 n)$

常见用法

点权序列化

```
vector<i64> a(n);
for (int u = 0; u < n; ++ u) a[hld.in[u]] = w[u];
```

子树查询

```
auto [l, r]{ hld.sub(u) };
auto ans{ seg.query(l, r) };
```

路径查询 / 修改

```
i64 ans{};
hld.for_path(u, v, [&](int l, int r) {
    ans += seg.query(l, r);
});
```

路径修改同理：

```
hld.for_path(u, v, [&](int l, int r) {
    seg.apply(l, r, x);
});
```

边权路径

每条边挂到较深端点：

```
int x{ tr.dep[u] > tr.dep[v] ? u : v };
a[hld.in[x]] = w;
```

查询时：

```
hld.for_path(u, v, [&](int l, int r) {
    ans += seg.query(l, r);
}, true);
```

非交换路径信息

```
for (auto [l, r, rev] : hld.path(u, v)) {
    // 查询 [l,r);
    // rev=true 时使用该区间的反向聚合信息
}
```

动态根子树

```
for (auto [l, r] : hld.sub_r(u, root)) {
    ans += seg.query(l, r);
}
```

复杂度

操作	时间复杂度
构建	$O(n)$
`sub()`	$O(1)$
`for_path()` / `path()`	$O(\log n)$ 个区间
`lca()` / `jump()` / `kth()`	$O(\log n)$
`sub_r()`	$O(\log n)$

空间复杂度：$O(n)$

> Tip
>
> 如果已经复制 HLD，就不必为了 `lca()`、`jump()`、`kth()` 再复制 4.2 的倍增 LCA；反过来，如果题目完全没有路径 / 子树数据结构操作，只需要 LCA，则 4.2 的版本更合适

长链剖分

长链剖分选择的是向下最深的儿子，而不是 HLD 中子树最大的儿子。旧版也明确将 `len[u]` 定义为从 `u` 向下的最大节点数，并据此选择长儿子

```
struct LongChain {
    const TreeInfo &t;
    vector<int> len, son, top, in, seq;
    explicit LongChain(const TreeInfo &t_) : t(t_), len(t.n, 1), son(t.n, -1), top(t.n), in(t.n), seq(t.n) { build(); }
    void build() {
        fill(len.begin(), len.end(), 1);
        fill(son.begin(), son.end(), -1);
        for (int i = t.n - 1; i > 0; -- i) {
            int u{ t.seq[i] }, p{ t.fa[u] };
            if (len[u] + 1 > len[p]) {
                len[p] = len[u] + 1;
                son[p] = u;
            }
        }
        int cur{ 0 };
        vector<pair<int, int>> stk{{t.root, t.root}};
        while (!stk.empty()) {
            auto [u, tp]{ stk.back() };
            stk.pop_back();
            for (int x = u; x != -1; x = son[x]) {
                top[x] = tp;
                in[x] = cur;
                seq[cur ++] = x;
                for (auto it = t.g[x].rbegin(); it != t.g[x].rend(); ++ it) {
                    int v{ it->first };
                    if (t.fa[v] == x && v != son[x]) stk.push_back({v, v});
                }
            }
        }
    }
    // ===== 长儿子路径区间：常用于深度 DP 的共享存储 =====
    pair<int, int> down(int u) const { return {in[u], in[u] + len[u]}; }
};
```

成员变量

- `len[u]`：从 `u` 向下的最长路径包含的节点数，叶子为 `1`
- `son[u]`：使 `len[v]` 最大的长儿子，不存在为 `-1`
- `top[u]`：`u` 所在长链的链顶
- `in[u]`：长儿子优先序中的位置

- `seq[i]`：位置 `i` 对应的原树节点

满足：

```
son[u] ≠ -1  ⇒  in[son[u]] = in[u] + 1
```

并且：

```
seq[in[u]], seq[in[u]+1], ..., seq[in[u]+len[u]-1]
```

恰好是从 `u` 不断沿 `son` 向下得到的最长链

接口

- `LongChain(t)`：构建长链剖分，$O(n)$
- `build()`：重新构建，$O(n)$
- `down(u)`：返回从 `u` 开始沿长儿子向下的连续区间 `[l,r)`，$O(1)$

注意事项

- HLD 的 `son[u]` 按 `siz` 最大选择；长链剖分的 `son[u]` 按 `len` 最大选择，二者不能混用
- 长链剖分的主要用途是与深度有关的树形 DP，不是把任意路径拆成 $O(\log n)$ 段
- `in[u]+len[u]` 不超过当前长链分配的连续区域，可以直接用于 DP 数组错位共享
- `TreeInfo` 换根后必须重新 `build()`
- 长链剖分本身是 $O(n)$；具体 DP 是否能做到 $O(n)$ 取决于轻儿子状态的合并方式，并不是用了长链剖分就自动线性。旧版也特别强调只有短链元素总共被合并 $O(1)$ 次时才能得到线性复杂度

常见用法

深度 DP 共享存储

若 `f[u][d]` 表示距 `u` 深度为 `d` 的某类状态，可以把所有 DP 统一放进长度为 `n` 的缓冲区：

```
LongChain lc(tr);
vector<i64> buf(n);
auto f = [&](int u) { return buf.data() + lc.in[u]; };
```

于是长儿子天然满足：

```
if (lc.son[u] ≠ -1) {
    // f(lc.son[u]) == f(u) + 1
}
```

即长儿子的：

```
f[son[u]][d]
```

与父节点的：

```
f[u][d+1]
```

直接共享同一位置

枚举长儿子路径

```
auto [l, r]{ lc.down(u) };
for (int i = l; i < r; ++ i) {
    int v{ lc.seq[i] };
}
```

复杂度

操作	时间复杂度
构建	$O(n)$
`down()`	$O(1)$

空间复杂度：$O(n)$

4.4 树上离线、启发式与压缩

选型

问题形态	优先选择
求每个节点的子树统计，单点可增删	DSU on Tree
每个子树天然维护一个可合并容器	Small-to-Large
每次只关心少量关键点	虚树
大量离线路径统计，单点可增删	树上莫队
路径统计满足结合律，可在线区间维护	优先 HLD + 数据结构

DSU on Tree

解决问题： 离线计算每个节点子树的信息，要求单个节点能够快速加入 / 删除当前状态，例如子树颜色种数、众数、频次统计等
核心顺序是：轻儿子先算完并清空，重儿子计算后保留，再把所有轻子树重新加入，得到当前整个子树的信息
依赖 4.1.1 `TreeInfo`

```
struct DSUTree {
    const TreeInfo &t;
    vector<int> son;
    explicit DSUTree(const TreeInfo &t_) : t(t_), son(t.n, -1) {
        for (int u = 0; u < t.n; ++ u) {
            for (auto [v, w] : t.g[u]) {
                if (t.fa[v] != u) continue;
                if (son[u] == -1 || t.siz[v] > t.siz[son[u]]) son[u] = v;
            }
        }
    }
    template<class Add, class Del, class Calc>
    void run(Add add, Del del, Calc calc) const {
        auto sub = [&](int u, auto &&f) {
            for (int i = t.in[u]; i < t.out[u]; ++ i) f(t.seq[i]);
        };
        struct F { int u; bool keep, end; };
        vector<F> stk{{t.root, false, false}};
        while (!stk.empty()) {
            auto [u, keep, end]{ stk.back() };
            stk.pop_back();
            if (!end) {
                stk.push_back({u, keep, true});
                if (son[u] != -1) stk.push_back({son[u], true, false});
                for (auto [v, w] : t.g[u]) {
                    if (t.fa[v] == u && v != son[u]) stk.push_back({v, false, false});
                }
                continue;
            }
            for (auto [v, w] : t.g[u]) {
                if (t.fa[v] == u && v != son[u]) sub(v, add);
            }
            add(u);
            calc(u);
            if (!keep) sub(u, del);
        }
    }
};
```

成员变量

- `son[u]`：`u` 的重儿子，即子树大小最大的儿子，不存在为 `-1`

接口

- `DSUTree(t)`：预处理重儿子，$O(n)$
- `run(add,del,calc)`：依次求所有节点的子树答案
  - `add(u)`：将节点 `u` 加入当前状态
  - `del(u)`：将节点 `u` 从当前状态删除
  - `calc(u)`：当前状态恰好为 `u` 的整棵子树时调用

注意事项

- 构造前必须完成 `TreeInfo::build()`
- `add()` 和 `del()` 必须互为逆操作
- `calc(u)` 只能读取当前状态，不应破坏全局统计
- `run()` 结束后会把根子树也清空，因此外部状态恢复为空
- `sub()` 利用 `TreeInfo` 的 `[in[u],out[u])` 枚举整棵子树
- 使用显式事件栈，轻儿子清空、重儿子保留的顺序不变；链状树不会消耗 O(n) 层 C++ 调用栈
- DSU on Tree 的"重儿子"按子树大小定义，与 4.3.2 长链剖分的长儿子不同

常见用法

每个子树有多少种颜色

```
vector<int> cnt(m), ans(n);
int cur{};

auto add = [&](int u) {
    if (cnt[col[u]] ++ == 0) ++ cur;
};
auto del = [&](int u) {
    if (-- cnt[col[u]] == 0) -- cur;
};
auto calc = [&](int u) {
    ans[u] = cur;
};

DSUTree dsu(tr);
dsu.run(add, del, calc);
```

复杂状态也只需要修改三个回调

复杂度

若一次 `add/del` 为 $O(T)$:

操作	复杂度
构造	$O(n)$
`add/del` 调用总次数	$O(n \log n)$
`run()`	$O(n \log n \cdot T)$

额外空间复杂度：$O(n)$，不计用户维护的状态

Small-to-Large / 树上小并大

解决问题： 每棵子树维护一个可以合并的容器，将较小状态合入较大状态，使每个元素只被搬运少量次数
核心原则就是始终把"小容器"合并到"大容器"
下面给最常用的 子树 `map` 频次合并骨架

```
template<class K>
struct SmallMerge {
    const TreeInfo &t;
    vector<map<K, int>> f;
    vector<int> num;
    explicit SmallMerge(const TreeInfo &t_) : t(t_), f(t.n), num(t.n) {}
    void merge(int u, int v) {
        if (num[u] < num[v]) {
            swap(f[u], f[v]);
            swap(num[u], num[v]);
        }
        for (auto &[x, c] : f[v]) f[u][x] += c;
        num[u] += num[v];
        num[v] = 0;
        f[v].clear();
    }
    template<class F>
    void run(const vector<K> &a, F calc) {
```

```
        for (auto &mp : f) mp.clear();
        fill(num.begin(), num.end(), 0);
        for (int i = t.n - 1; i >= 0; -- i) {
            int u{ t.seq[i] };
            ++ f[u][a[u]];
            ++ num[u];
            calc(u, f[u]);
            if (u != t.root) merge(t.fa[u], u);
        }
    }
};
```

接口说明

- `f[u]`：当前归属于 `u` 的频次表
- `num[u]`：当前容器实际代表的原树节点数
- `merge(u,v)`：将 `v` 的状态合并进 `u`，小并大
- `run(a,calc)`：
  - 节点 `u` 自身贡献键 `a[u]`
  - 当 `calc(u,f[u])` 被调用时，`f[u]` 正好表示整个 `u` 子树
  - 随后 `f[u]` 会被合并进父节点

注意事项

- `calc()` 必须在回调中立刻保存答案；之后 `f[u]` 可能被交换并清空
- 本板子默认一个节点贡献一个键 `a[u]`；如果一个节点贡献多个元素，修改加入自身信息的部分即可
- `num` 按实际代表的节点数比较大小，而不是直接使用 `map::size()`；有重复键时这样更稳妥
- `map` 是确定性版本；如果数据很大且已有安全哈希表，可以换成 `unordered_map` / `FlatHashMap`
- DSU on Tree 使用一个全局状态反复增删；Small-to-Large 使用每棵子树自己的容器并不断合并，两者不要混为一个板子

常见用法

每棵子树颜色出现次数

```
SmallMerge<int> sm(tr);
vector<int> ans(n);

sm.run(col, [&](int u, const auto &mp) {
    ans[u] = mp.size();
});
```

若要求某种更复杂统计，直接根据 `mp` 维护附加信息通常比每次重新遍历整个 `map` 更好

复杂度

当前使用 `map`：

- 每个节点代表的信息只会由较小集合向较大集合移动 $O(\log n)$ 次
- 单次 `map` 插入 / 修改为 $O(\log n)$
  因此总时间复杂度为 $O(n \log^2 n)$，空间复杂度为 $O(n)$
  若换成均摊 $O(1)$ 的哈希容器，典型总复杂度可降为均摊 $O(n \log n)$

虚树

解决问题：一次询问只关心原树中的 k 个关键点时，只保留关键点及必要 LCA，把原树压缩成 $O(k)$ 个节点的树，再在虚树上 DP
经典构造就是：按 DFS 序排序关键点，加入相邻点 LCA，再排序去重并建立父子关系
依赖任意具有 `lca(u,v)` 和成员 `t` 的 LCA 结构，例如 4.2 `LCA / ELCA` 或 4.3 `HLD`

```
template<class L>
struct VTree {
    const L &l;
    vector<vector<int>> g;
    vector<int> ver;
    explicit VTree(const L &l_) : l(l_), g(l.t.n) {}
    int build(vector<int> a) {
        for (int u : ver) g[u].clear();
        ver.clear();
        if (a.empty()) return -1;
        auto cmp = [&](int u, int v) {
```

```
            return l.t.in[u] < l.t.in[v];
        };
        sort(a.begin(), a.end(), cmp);
        a.erase(unique(a.begin(), a.end()), a.end());
        int m{ int(a.size()) };
        for (int i = 1; i < m; ++ i) a.push_back(l.lca(a[i - 1], a[i]));
        sort(a.begin(), a.end(), cmp);
        a.erase(unique(a.begin(), a.end()), a.end());
        ver = a;
        vector<int> stk{ a[0] };
        for (int i = 1; i < int(a.size()); ++ i) {
            while (!l.t.is_anc(stk.back(), a[i])) stk.pop_back();
            g[stk.back()].push_back(a[i]);
            stk.push_back(a[i]);
        }
        return a[0];
    }
};
```

成员变量

- `g[u]`：虚树中 `u` 的儿子
- `ver`：本次虚树包含的全部节点，按原树 DFS 序排列

接口

- `VTree(l)`：绑定 LCA 结构，$O(n)$ 初始化空邻接表
- `build(a)`：以 `a` 中节点为关键点建立虚树，返回虚树根
  - `a` 为空时返回 `-1`
  - 自动去除重复关键点
  - 自动补入必要 LCA

注意事项

- `g` 是从虚树父亲指向儿子的有向邻接表
- 虚树节点包括原关键点与补入的 LCA；若需要区分关键点，应在外部自己维护标记数组
- 若必须让原树根出现在虚树中，直接把 `t.root` 加入关键点
- 每次 `build()` 会自动清空上一棵虚树用过的节点，不需要 $O(n)$ 清空整张 `g`
- 对一条虚边 `u → v`：
  - 原树边数为 `l.t.dep[v] - l.t.dep[u]`
  - 原树路径权值和为 `l.t.dis[v] - l.t.dis[u]`
- 因为 `ver` 中祖先一定早于后代，所以反向遍历 `ver` 可以直接做自底向上的虚树 DP

常见用法

```
LCA lca(tr);
VTree vt(lca);

int root{ vt.build(key) };

for (int i = int(vt.ver.size()) - 1; i > 0; -- i) {
    int u{ vt.ver[i] };
    // 处理 u，再向虚树父节点合并
}
```

直接遍历虚树：

```
auto dfs = [&](auto &&self, int u) -> void {
    for (int v : vt.g[u]) {
        i64 w{ tr.dis[v] - tr.dis[u] };
        self(self, v);
    }
};
if (root != -1) dfs(dfs, root);
```

复杂度

设关键点数为 k，最终虚树节点数不超过 $2k-1$
使用倍增 `LCA`：

$$O(k\log k + k\log n)$$

使用 $O(1)$ 查询的 `ELCA`：

$$O(k\log k)$$

单次虚树空间为 $O(k)$；对象的邻接表本身为 $O(n)$

树上莫队

解决问题： 大量离线树上路径询问，且答案能够通过单个节点的加入 / 删除快速维护
树上莫队的核心是每个节点在 DFS 时记录**进入和离开各一次**，再结合 LCA 把树路径转换为欧拉序区间，之后运行普通莫队
依赖任意具有 `lca(u,v)` 和成员 `t` 的 LCA 结构

```
template<class L>
struct TreeMo {
    struct Q {
        int l, r, id, ex;
    };
    const L &l;
    const TreeInfo &t;
    vector<int> fir, las, eul;
    vector<Q> q;

    explicit TreeMo(const L &l_) : l(l_), t(l_.t), fir(t.n), las(t.n) {
        build();
    }

    // ===== 双欧拉序：每个节点进入、离开各出现一次 =====
    void build() {
        eul.clear();
        q.clear();
        struct F { int u, it; };
        vector<F> stk{{t.root, 0}};
        eul.reserve(2 * t.n);
        fir[t.root] = eul.size();
        eul.push_back(t.root);
        while (!stk.empty()) {
            F &f{ stk.back() };
            if (f.it == int(t.g[f.u].size())) {
                las[f.u] = eul.size();
                eul.push_back(f.u);
                stk.pop_back();
                continue;
            }
            int v{ t.g[f.u][f.it ++].first };
            if (t.fa[v] != f.u) continue;
            fir[v] = eul.size();
            eul.push_back(v);
            stk.push_back({v, 0});
        }
    }

    // ===== 加入路径询问；edge=true 时按边权模式 =====
    int add(int u, int v, bool edge = false) {
        if (fir[u] > fir[v]) swap(u, v);
        int p{ l.lca(u, v) }, id{ int(q.size()) };
        if (p == u) q.push_back({fir[u], fir[v] + 1, id, edge ? p : -1});
        else q.push_back({las[u], fir[v] + 1, id, edge ? -1 : p});
        return id;
    }

    // ===== 莫队：add/del 维护节点状态，calc(id) 保存当前询问答案 =====
    template<class Add, class Del, class Calc>
    void run(Add add, Del del, Calc calc) const {
        if (q.empty()) return;
        vector<Q> a{ q };
        int bs{ max(1, int(eul.size() / sqrt(q.size()))) };
```

```
        sort(a.begin(), a.end(), [&](const Q &x, const Q &y) {
            int bx{ x.l / bs }, by{ y.l / bs };
            if (bx != by) return bx < by;
            return bx & 1 ? x.r > y.r : x.r < y.r;
        });

        vector<char> on(t.n);
        auto flip = [&](int u) {
            if (on[u]) del(u);
            else add(u);
            on[u] ^= 1;
        };

        int lp{}, rp{};
        for (auto [ql, qr, id, ex] : a) {
            while (lp > ql) flip(eul[--lp]);
            while (rp < qr) flip(eul[rp++]);
            while (lp < ql) flip(eul[lp++]);
            while (rp > qr) flip(eul[--rp]);

            if (ex != -1) flip(ex);
            calc(id);
            if (ex != -1) flip(ex);
        }
        while (lp < rp) flip(eul[lp++]);
    }
};
```

接口说明

成员变量

- `fir[u]`：`u` 第一次进入 DFS 时的位置
- `las[u]`：离开 `u` 时第二次出现的位置
- `eul`：长度为 `2n` 的双欧拉序
- `Q::ex`：区间之外需要临时翻转一次的 LCA，`-1` 表示不需要
- `q`：全部离线路径询问

接口

- `TreeMo(l)`：建立双欧拉序，$O(n)$
- `build()`：清空旧序列和旧询问，再建立双欧拉序；若原树换根，先重建绑定的 LCA
- `add(u,v,edge)`：加入路径询问并返回编号，LCA 查询复杂度取决于 `L`
- `run(add,del,calc)`：
  - `add(u)`：节点进入当前路径集合
  - `del(u)`：节点离开当前路径集合
  - `calc(id)`：保存编号为 `id` 的答案

路径转换

设 `p=lca(u,v)`，且调整后 `fir[u]≤fir[v]`。

点权模式：

- `p=u`：使用 `[fir[u],fir[v]+1)`
- `p≠u`：使用 `[las[u],fir[v]+1)`，再额外加入 `p`

因为区间中出现两次的节点会被 `flip()` 抵消，只出现一次的节点恰好组成路径。

边权模式

把边 `(fa[u],u)` 的信息挂在节点 `u` 上，然后：

```
mo.add(u, v, true);
```

此时 LCA 对应节点会自动排除，所以当前集合恰好表示路径上的所有边。

注意事项

- 本版只处理**静态点 / 边信息 + 离线路径询问**。
- 若查询之间还存在点权修改，需要扩展为带时间维的树上莫队，本板子不包含。
- `add()` / `del()` 必须互为逆操作。

- `calc(id)` 只读取当前状态，不修改莫队维护的信息。
- `run()` 结束时会恢复为空状态。
- `fir/las` 与 `TreeInfo::in/out` 不是同一种欧拉序：这里每个节点恰好出现两次。
- 查询很多但能够直接用 HLD + 数据结构在线完成时，不要为了“高级”硬上莫队；树上莫队主要适合不好用结合律数据结构维护、但单点增删很简单的统计问题。

常见用法

路径不同颜色数

```
LCA lca(tr);
TreeMo mo(lca);

for (auto [u, v] : qry) mo.add(u, v);

vector<int> cnt(m), ans(qry.size());
int cur{};

auto add = [&](int u) {
    if (cnt[col[u]] ++ == 0) ++ cur;
};
auto del = [&](int u) {
    if (-- cnt[col[u]] == 0) -- cur;
};

mo.run(add, del, [&](int id) {
    ans[id] = cur;
});
```

复杂度

设询问数为 q，一次 `add/del` 为 $O(T)$。块长取 max(1, floor(2n / sqrt(q)))，适应查询数远小于或大于 n 的情况；q=0 直接返回。

操作	复杂度
双欧拉序	$O(n)$
查询排序	$O(q \log q)$
节点翻转总次数	$O(n\sqrt{q}+q)$
总计	$O(q \log q + (n\sqrt{q}+q)\cdot T)$

空间复杂度：$O(n+q)$

4.5 树分治

选型

问题	优先方案
静态统计所有树路径	`PointDiv`
距离 $\le k$ 的点对计数	`PointDiv::count_leq()`
节点状态动态变化、距离查询	`CDTree`
动态最近标记点	`CDNear`
动态距离范围计数 / 求和	`CDTree::up` + Fenwick 等
点分治难以得到合适的跨点结构，而跨边更自然	`EdgeDiv`
特殊动态路径问题	再考虑链分治等专用技巧

点分治与路径统计

解决问题： 对所有树上路径进行离线统计。每层选择当前连通块重心，只处理“路径经过当前重心”的部分，再递归处理删除重心后的各个连通块
依赖 4.1.1 `TreeInfo`，这里只使用 `n` 与 `g`，不要求提前 `build()`

```
struct PointDiv {
    const TreeInfo &t;
    vector<int> siz, pre;
    vector<char> ban;
    explicit PointDiv(const TreeInfo &t_) : t(t_), siz(t.n), pre(t.n), ban(t.n) {}
private:
    // ===== 找当前连通块重心 =====
    int cen(int s) {
        vector<int> ord, stk{s};
        pre[s] = -1;
        while (!stk.empty()) {
            int u{ stk.back() };
            stk.pop_back();
            ord.push_back(u);
            for (auto [v, w] : t.g[u]) {
                if (v == pre[u] || ban[v]) continue;
                pre[v] = u;
                stk.push_back(v);
            }
        }
        for (int i = int(ord.size()) - 1; i >= 0; -- i) {
            int u{ ord[i] };
            siz[u] = 1;
            for (auto [v, w] : t.g[u]) if (!ban[v] && pre[v] == u) siz[u] += siz[v];
        }
        int rt{ s }, best{ int(ord.size()) + 1 }, tot{ int(ord.size()) };
        for (int u : ord) {
            int mx{ tot - siz[u] };
            for (auto [v, w] : t.g[u]) if (!ban[v] && pre[v] == u) mx = max(mx, siz[v]);
            if (mx < best) best = mx, rt = u;
        }
        return rt;
    }
    // 收集某个重心分支内所有点到重心的距离
    vector<i64> dists(int s, int p, i64 d) const {
        vector<i64> res;
        vector<tuple<int, int, i64>> stk{{s, p, d}};
        while (!stk.empty()) {
            auto [u, fa, dis]{ stk.back() };
            stk.pop_back();
            res.push_back(dis);
            for (auto [v, w] : t.g[u]) {
                if (v == fa || ban[v]) continue;
                stk.push_back({v, u, dis + w});
            }
        }
        return res;
    }
    static i64 cnt(vector<i64> &a, i64 k) {
        sort(a.begin(), a.end());
        i64 res{};
        for (int l = 0, r = int(a.size()) - 1; l < r;) {
            if (i128(a[l]) + a[r] <= k) res += r - l, ++ l;
            else -- r;
        }
        return res;
    }
public:
    // ===== 通用点分治：f(c, all, sub) 处理当前重心 =====
    template<class F>
    void run(F f) {
        fill(ban.begin(), ban.end(), false);
        auto dfs = [&](auto &&self, int s) -> void {
            int c{ cen(s) };
            vector<i64> all{0};
            vector<vector<i64>> sub;
            for (auto [v, w] : t.g[c]) {
                if (ban[v]) continue;
                sub.push_back(dists(v, c, w));
                all.insert(all.end(), sub.back().begin(), sub.back().end());
```

```
            }
            f(c, all, sub);
            ban[c] = true;
            for (auto [v, w] : t.g[c]) if (!ban[v]) self(self, v);
        };
        dfs(dfs, 0);
    }
    // ===== 距离不超过 k 的点对计数 =====
    i64 count_leq(i64 k) {
        i64 ans{};
        run([&](int, vector<i64> &all, vector<vector<i64>> &sub) {
            ans += cnt(all, k);
            for (auto &a : sub) ans -= cnt(a, k);
        });
        return ans;
    }
};
```

成员变量

- `ban[u]`：`u` 是否已经作为某层分治重心被删除
- `siz[u]`、`pre[u]`：寻找当前连通块重心时使用的临时数组

接口

- `PointDiv(t)`：绑定原树，$O(n)$
- `run(f)`：执行一次完整点分治
- `f(c,all,sub)`：处理重心 `c`
  - `all`：当前连通块所有点到 `c` 的距离，包含 `c` 自己的 `0`
  - `sub[i]`：删除 `c` 后第 `i` 个分支内所有点到 `c` 的距离
- `count_leq(k)`：返回距离不超过 `k` 的无序不同点对数量

核心统计方式

若某类路径信息可以由距离集合函数 $F(S)$ 求出，则所有经过重心 `c` 的路径通常写成：

$$F(\text{all}) - \sum_i F(\text{sub}_i)$$

因为 `all` 会统计整个连通块的点对，而同一个分支内部的路径不经过当前重心，需要减掉
这正是点分治来源中“处理经过当前根的路径，再对子树继续分治”的结构

注意事项

- `run()` 每次都会清空 `ban`，因此同一个对象可以重复执行不同统计
- `f()` 必须在当前调用中完成该重心的信息统计；`all/sub` 调用结束后即销毁
- 点分治递归的是重心分治树，深度为 $O(\log n)$；节点遍历本身使用显式栈，不会因为原树是一条链而爆 DFS 栈
- 距离使用 `i64`，需要保证单条树路径权值和不溢出
- `count_leq()` 的双指针比较使用 `i128`，避免两个 `i64` 距离相加溢出
- `count_leq()` 对边权正负都可工作；这里只使用唯一树路径的权值和

常见用法

距离不超过 k 的点对数

```
PointDiv pd(tr);
i64 ans{ pd.count_leq(k) };
```

自定义路径统计

例如要求“距离恰好出现于某个集合”时，可以自己处理 `all/sub`：

```
PointDiv pd(tr);
pd.run([&](int c, vector<i64> &all, vector<vector<i64>> &sub) {
    // 统计 all
    // 减去每个 sub
});
```

复杂度

- 通用 `run()`：$O(n \log n)$ 次节点访问，不计回调内部复杂度
- 临时空间：$O(n)$
- `count_leq(k)`：由于每层还需要排序，最坏 $O(n \log^2 n)$

点分树与动态查询

解决问题： 原树不变，但节点状态不断修改，在线询问与某点距离有关的信息，例如：

- 到最近标记点的距离
- 某距离范围内标记点个数 / 权值和
- 点权修改后询问距离限制内的贡献
  已有动态点分治模板会为一个原树节点保存它到各级分治重心的距离，并在每个重心维护总结构、分支结构，通过“总贡献减当前分支贡献”避免重复

```
struct CDTree {
    struct A {
        int c, s;
        i64 d;
    };
    const TreeInfo &t;
    int root{ -1 }, sc{};
    vector<int> siz, pre, par, dep;
    vector<char> ban;
    vector<vector<A>> up;
    explicit CDTree(const TreeInfo &t_) : t(t_), siz(t.n), pre(t.n),
        par(t.n, -1), dep(t.n), ban(t.n), up(t.n) { build(); }
private:
    int cen(int s) {
        vector<int> ord, stk{s};
        pre[s] = -1;
        while (!stk.empty()) {
            int u{ stk.back() };
            stk.pop_back();
            ord.push_back(u);
            for (auto [v, w] : t.g[u]) {
                if (v == pre[u] || ban[v]) continue;
                pre[v] = u;
                stk.push_back(v);
            }
        }
        for (int i = int(ord.size()) - 1; i >= 0; -- i) {
            int u{ ord[i] };
            siz[u] = 1;
            for (auto [v, w] : t.g[u]) if (!ban[v] && pre[v] == u) siz[u] += siz[v];
        }
        int rt{ s }, best{ int(ord.size()) + 1 }, tot{ int(ord.size()) };
        for (int u : ord) {
            int mx{ tot - siz[u] };
            for (auto [v, w] : t.g[u]) if (!ban[v] && pre[v] == u) mx = max(mx, siz[v]);
            if (mx < best) best = mx, rt = u;
        }
        return rt;
    }
    void add_up(int s, int p, i64 d, int c, int id) {
        vector<tuple<int, int, i64>> stk{{s, p, d}};
        while (!stk.empty()) {
            auto [u, fa, dis]{ stk.back() };
            stk.pop_back();
            up[u].push_back({c, id, dis});
            for (auto [v, w] : t.g[u]) {
                if (v == fa || ban[v]) continue;
                stk.push_back({v, u, dis + w});
            }
        }
    }
public:
    void build() {
        fill(ban.begin(), ban.end(), false);
        fill(par.begin(), par.end(), -1);
        fill(dep.begin(), dep.end(), 0);
```

```
        for (auto &a : up) a.clear();
        sc = 0;
        auto dfs = [&](auto &&self, int s, int p) -> int {
            int c{ cen(s) };
            par[c] = p;
            dep[c] = p == -1 ? 0 : dep[p] + 1;
            up[c].push_back({c, -1, 0});
            ban[c] = true;
            for (auto [v, w] : t.g[c]) {
                if (ban[v]) continue;
                int id{ sc ++ };
                add_up(v, c, w, c, id);
                self(self, v, c);
            }
            return c;
        };
        root = dfs(dfs, 0, -1);
    }
};
```

成员变量

- `root`：点分树根
- `par[c]`：重心 `c` 在点分树中的父节点
- `dep[c]`：点分树深度
- `up[u]`：原树节点 `u` 到各级分治重心的信息
- `A::c`：当前分治重心
- `A::d`：原树中 `u` 到 `c` 的距离
- `A::s`：`u` 位于 `c` 的哪个分支；`u==c` 时为 `-1`
- `sc`：所有分支编号总数

接口

- `CDTree(t)`：构造点分树并预处理所有 `up[u]`
- `build()`：重新构造点分树
  每个 `up[u]` 的长度为 $O(\log n)$

注意事项

- `par` 是点分树父节点，与 `TreeInfo::fa` 完全不同
- `d` 是原树距离，不是点分树距离
- `s` 专门用于“总贡献减分支贡献”的动态统计；如果只需要最近点，可以直接忽略
- 原树结构发生变化后必须重建；单纯点权 / 标记变化不需要重建
- `CDTree` 不依赖 `TreeInfo::fa`，因此只要 `TreeInfo::g` 已建好即可

最近标记点

下面是点分树最常见的应用：支持标记 / 取消标记节点，并查询距离某点最近的标记点

```
struct CDNear {
    const CDTree &cd;
    vector<multiset<i64>> s;
    vector<char> on;
    explicit CDNear(const CDTree &cd_) : cd(cd_), s(cd.t.n), on(cd.t.n) {}
    void set(int u, bool v) {
        if (on[u] == v) return;
        on[u] = v;
        for (auto [c, id, d] : cd.up[u]) {
            if (v) s[c].insert(d);
            else s[c].erase(s[c].find(d));
        }
    }
    void flip(int u) { set(u, !on[u]); }
    i64 query(int u) const {
        constexpr i128 INF{ numeric_limits<i128>::max() };
        i128 ans{ INF };
        for (auto [c, id, d] : cd.up[u]) {
            if (!s[c].empty()) ans = min(ans, i128(d) + *s[c].begin());
        }
        return ans == INF ? -1 : i64(ans);
```

```
    }
};
```

常见用法

```
CDTree cd(tr);
CDNear ds(cd);

ds.set(u, true);   // 标记
ds.set(u, false);  // 取消
ds.flip(u);        // 翻转状态

i64 d{ ds.query(v) };
```

没有任何标记点时 `query()` 返回 `-1`。CDNear 要求边权非负，才能使用三角不等式取各级候选最小值；负边会使绕行重心的候选低估真实距离。根到点及任意原树路径距离须能放入 i64，候选相加使用 i128，支持包括 LLONG_MAX 在内的合法答案。CDTree 本身与总贡献减分支的统计不要求非负边。重建 CDTree 后应重新构造 CDNear。

距离限制统计的扩展

若要支持:

```
修改节点权值
查询 dist(u,v) ≤ k 的所有 v 的权值和
```

对 `up[u]` 中每个 `{c,s,d}`:

- 修改时向重心 `c` 的总结构加入距离 `d`
- 若 `s≠-1`，同时向分支 `s` 的结构加入距离 `d`
- 查询时统计总结构中距离不超过 `k-d` 的贡献
- 再减去分支 `s` 中距离不超过 `k-d` 的贡献
  即:

$$\sum_c \mathrm{all}_c(k-d) - \sum_c \mathrm{sub}_s(k-d)$$

  距离离散后可为各重心 / 分支挂 Fenwick、前缀数组或其他数据结构

复杂度

`CDTree`:

- 构造时间：$O(n \log n)$
- `up` 总空间：$O(n \log n)$
  `CDNear`:

操作	时间复杂度
`set()` / `flip()`	$O(\log^2 n)$
`query()`	$O(\log n)$
空间	$O(n \log n)$ 最坏

`set()` 多一个 $\log n$ 来自 `multiset` 插入 / 删除

边分治

点分治保证每次按点把规模折半；边分治则希望找到一条较平衡的边，删除后递归两侧，并统计所有跨过该边的路径
普通树中未必存在足够平衡的边，例如菊花图。因此常先把高度数节点拆成若干零权虚点，使新树最大度数不超过 `3`，且所有原节点之间的距离保持不变，再进行边分治
依赖已经执行 `TreeInfo::build()` 的 `TreeInfo`，因为需要借助 `fa` 将原树定向后拆高度数点

```
struct EdgeDiv {
    struct E {
        int to, id;
        i64 w;
    };
    const TreeInfo &t;
    int n0;
    vector<vector<E>> g;
```

```
    vector<int> eu, ev, siz, fa, pe;
    vector<i64> ew;
    vector<char> cut;
    explicit EdgeDiv(const TreeInfo &t_) : t(t_), n0(t.n), g(t.n) { build(); }
private:
    int node() {
        g.push_back({});
        return int(g.size()) - 1;
    }
    int link(int u, int v, i64 w) {
        int id{ int(eu.size()) };
        eu.push_back(u);
        ev.push_back(v);
        ew.push_back(w);
        cut.push_back(false);
        g[u].push_back({v, id, w});
        g[v].push_back({u, id, w});
        return id;
    }
    // ===== 将高儿子数节点拆成零权虚点链，使最大度数 ≤ 3 =====
    void build() {
        for (int u = 0; u < n0; ++ u) {
            vector<pair<int, i64>> ch;
            for (auto [v, w] : t.g[u]) if (t.fa[v] == u) ch.push_back({v, w});
            if (ch.size() <= 2) {
                for (auto [v, w] : ch) link(u, v, w);
                continue;
            }
            link(u, ch[0].first, ch[0].second);
            int cur{ u };
            for (int i = 1; i + 1 < int(ch.size()); ++ i) {
                int x{ node() };
                link(cur, x, 0);
                link(x, ch[i].first, ch[i].second);
                cur = x;
            }
            link(cur, ch.back().first, ch.back().second);
        }
        siz.resize(g.size());
        fa.resize(g.size());
        pe.resize(g.size());
    }
    // ===== 找当前连通块的平衡边 =====
    int cen(int s) {
        vector<int> ord, stk{s};
        fa[s] = pe[s] = -1;
        while (!stk.empty()) {
            int u{ stk.back() };
            stk.pop_back();
            ord.push_back(u);
            for (auto [v, id, w] : g[u]) {
                if (cut[id] || v == fa[u]) continue;
                fa[v] = u;
                pe[v] = id;
                stk.push_back(v);
            }
        }
        for (int i = int(ord.size()) - 1; i >= 0; -- i) {
            int u{ ord[i] };
            siz[u] = 1;
            for (auto [v, id, w] : g[u]) {
                if (!cut[id] && fa[v] == u) siz[u] += siz[v];
            }
        }
        int id{ -1 }, best{ int(ord.size()) + 1 }, tot{ int(ord.size()) };
        for (int u : ord) {
            if (pe[u] == -1) continue;
            int mx{ max(siz[u], tot - siz[u]) };
            if (mx < best) best = mx, id = pe[u];
        }
        return id;
```

```
    }
    // 返回某一侧所有原节点及其到该侧端点的距离
    vector<pair<int, i64>> side(int s, int ban_id) const {
        vector<pair<int, i64>> res;
        vector<tuple<int, int, i64>> stk{{s, -1, 0}};
        while (!stk.empty()) {
            auto [u, p, d]{ stk.back() };
            stk.pop_back();
            if (u < n0) res.push_back({u, d});
            for (auto [v, id, w] : g[u]) {
                if (id == ban_id || cut[id] || v == p) continue;
                stk.push_back({v, u, d + w});
            }
        }
        return res;
    }
public:
    // ===== 通用边分治 =====
    template<class F>
    void run(F f) {
        fill(cut.begin(), cut.end(), false);
        auto dfs = [&](auto &&self, int s) -> void {
            int id{ cen(s) };
            if (id == -1) return;
            auto a{ side(eu[id], id) };
            auto b{ side(ev[id], id) };
            f(eu[id], ev[id], ew[id], a, b);
            cut[id] = true;
            self(self, eu[id]);
            self(self, ev[id]);
        };
        dfs(dfs, 0);
    }
};
```

编号

- `0..n0-1`：原树节点
- `n0..g.size()-1`：自动加入的零权虚点
- 虚点只为保证分治平衡，不应被当作题目中的真实节点

接口

`run(f)` 每次找到分治边 `(u,v,w)`，调用：

```
f(u, v, w, a, b);
```

其中：

- `a`：`u` 一侧所有**原节点**，元素为 `{x,dist(x,u)}`
- `b`：`v` 一侧所有**原节点**，元素为 `{y,dist(y,v)}`
- `w`：分治边权
  任取：

```
{x, dx} ∈ a
{y, dy} ∈ b
```

则原树中：

$$\text{dist}(x,y) = dx + w + dy$$

因此所有跨越当前分治边的路径都可以在回调中统计

常见用法

```
EdgeDiv ed(tr);
ed.run([&](int u, int v, i64 w, auto &a, auto &b) {
    // 枚举 / 排序 / 数据结构统计 a 与 b 之间的跨边路径
});
```

注意事项

- 构造前必须先执行 `TreeInfo::build()`
- 拆点加入的边权全部为 `0`，因此原节点之间的树上距离保持不变
- 拆点后节点总数仍为 $O(n)$，且最大度数不超过 `3`
- 与点分治类似，回调只统计当前分治边两侧之间的路径；同侧路径留给后续递归
- 若题目天然更适合点分治，一般不要主动使用边分治：它代码更长、虚点处理也更容易出错

复杂度

拆点后设节点数为 $N = O(n)$：

- 建辅助树：$O(n)$
- 分治层数：$O(\log n)$
- 不计回调，所有节点枚举总复杂度：$O(n \log n)$
- 空间复杂度：$O(n)$

4.6 动态树

选型

需求	推荐
动态 `link / cut / connected`	`LCT`
动态路径和 / 最大值 / 异或	`LCT`
非交换路径信息	`LCT` 双向聚合
路径第 k 点	`LCT::kth()`
动态边权	LCT 边转点
动态 MST 中查询路径最大边	LCT 边转点 + `MaxOp`
动态整棵连通分量大小 / 和	`ETT`
动态子树 / 分量类问题	优先考虑 ETT
非常复杂的动态树 DP	Top Tree

Link-Cut Tree

解决问题： 动态维护森林，支持连边、删边、连通性以及树上路径信息查询；可通过边转点维护边权
`Op` 只要求满足结合律，`id` 为单位元。内部额外维护正反两个聚合值，因此也支持矩阵乘法、字符串拼接等非交换路径信息

```
template<class T, class Op = plus<T>>
struct LCT {
    struct Node {
        int ch[2]{}, fa{}, siz{ 1 };
        bool rev{};
        T val{}, sum[2]{};
    };
    int n;
    T id;
    Op op;
    vector<Node> tr;
    vector<int> stk;
    explicit LCT(int n_, T id_ = T{}, Op op_ = Op())
        : n(n_), id(id_), op(op_), tr(n + 1) {
        tr[0].siz = 0;
        tr[0].sum[0] = tr[0].sum[1] = id;
        stk.reserve(n + 1);
        for (int x = 1; x <= n; ++ x) tr[x].sum[0] = tr[x].sum[1] = tr[x].val;
    }
    LCT(const vector<T> &a, T id_ = T{}, Op op_ = Op())
        : LCT(int(a.size()), id_, op_) {
        for (int u = 0; u < n; ++ u) {
            int x{ u + 1 };
            tr[x].val = tr[x].sum[0] = tr[x].sum[1] = a[u];
        }
```

```
    }
private:
    // ===== Splay 基础维护 =====
    bool root(int x) const {
        int f{ tr[x].fa };
        return !f || (tr[f].ch[0] != x && tr[f].ch[1] != x);
    }
    void pull(int x) {
        auto &u{ tr[x] };
        int l{ u.ch[0] }, r{ u.ch[1] };
        u.siz = tr[l].siz + 1 + tr[r].siz;
        u.sum[0] = op(op(tr[l].sum[0], u.val), tr[r].sum[0]);
        u.sum[1] = op(op(tr[r].sum[1], u.val), tr[l].sum[1]);
    }
    void apply_rev(int x) {
        if (!x) return;
        swap(tr[x].ch[0], tr[x].ch[1]);
        swap(tr[x].sum[0], tr[x].sum[1]);
        tr[x].rev ^= 1;
    }
    void push(int x) {
        if (!tr[x].rev) return;
        apply_rev(tr[x].ch[0]);
        apply_rev(tr[x].ch[1]);
        tr[x].rev = false;
    }
    void rotate(int x) {
        int y{ tr[x].fa }, z{ tr[y].fa };
        int k{ tr[y].ch[1] == x }, w{ tr[x].ch[k ^ 1] };
        if (!root(y)) tr[z].ch[tr[z].ch[1] == y] = x;
        tr[x].fa = z;
        tr[y].ch[k] = w;
        if (w) tr[w].fa = y;
        tr[x].ch[k ^ 1] = y;
        tr[y].fa = x;
        pull(y);
        pull(x);
    }
    void splay(int x) {
        stk.clear();
        stk.push_back(x);
        for (int y{ x }; !root(y); y = tr[y].fa) stk.push_back(tr[y].fa);
        while (!stk.empty()) {
            push(stk.back());
            stk.pop_back();
        }
        while (!root(x)) {
            int y{ tr[x].fa }, z{ tr[y].fa };
            if (!root(y)) rotate((tr[y].ch[1] == x) == (tr[z].ch[1] == y) ? y : x);
            rotate(x);
        }
        pull(x);
    }
    // ===== 实链操作：link / cut / query 等均依赖，不建议删除 =====
    int access(int x) {
        int y{};
        for (int z{ x }; z; z = tr[z].fa) {
            splay(z);
            tr[z].ch[1] = y;
            pull(z);
            y = z;
        }
        splay(x);
        return y;
    }
    void mkrt(int x) {
        access(x);
        apply_rev(x);
    }
    int froot(int x) {
        access(x);
```

```
        while (true) {
            push(x);
            if (!tr[x].ch[0]) break;
            x = tr[x].ch[0];
        }
        splay(x);
        return x;
    }
    void split(int x, int y) {
        mkrt(x);
        access(y);
    }

public:
    // ===== 节点与森林基础操作 =====
    int add_node(const T &v = T{}) {
        ++ n;
        tr.push_back(Node());
        int x{ n };
        tr[x].val = tr[x].sum[0] = tr[x].sum[1] = v;
        return n - 1;
    }
    void set(int u, const T &v) {
        int x{ u + 1 };
        access(x);
        tr[x].val = v;
        pull(x);
    }
    bool conn(int u, int v) {
        int x{ u + 1 }, y{ v + 1 };
        if (x == y) return true;
        mkrt(x);
        return froot(y) == x;
    }
    bool link(int u, int v) {
        int x{ u + 1 }, y{ v + 1 };
        mkrt(x);
        if (froot(y) == x) return false;
        tr[x].fa = y;
        return true;
    }
    bool cut(int u, int v) {
        int x{ u + 1 }, y{ v + 1 };
        mkrt(x);
        access(y);
        if (tr[y].ch[0] != x || tr[x].ch[1]) return false;
        tr[y].ch[0] = tr[x].fa = 0;
        pull(y);
        return true;
    }

    // ===== 路径信息 =====
    T query(int u, int v) {
        int x{ u + 1 }, y{ v + 1 };
        split(x, y);
        return tr[y].sum[0];
    }
    int plen(int u, int v) {
        int x{ u + 1 }, y{ v + 1 };
        split(x, y);
        return tr[y].siz;
    }
    int kth(int u, int v, int k) {
        int x{ u + 1 }, y{ v + 1 };
        split(x, y);
        if (k < 0 || k >= tr[y].siz) return -1;
        int z{ y };
        while (true) {
            push(z);
            int l{ tr[z].ch[0] }, s{ tr[l].siz };
            if (k < s) z = l;
```

```
            else if (k == s) break;
            else k -= s + 1, z = tr[z].ch[1];
        }
        splay(z);
        return z - 1;
    }
};
```

模板参数

- `T`：单点值以及路径聚合值类型
- `Op`：结合运算
- `id`：`Op` 的单位元
  例如求和：

```
LCT<i64> lct(a, 0);
```

路径异或：

```
struct Xor {
    i64 operator()(i64 a, i64 b) const { return a ^ b; }
};
LCT<i64, Xor> lct(a, 0);
```

接口

- `LCT(a,id,op)`：建立 `n=a.size()` 个互不相连的节点
- `add_node(v)`：新建值为 `v` 的节点，返回其 `0base` 编号
- `set(u,v)`：修改节点值
- `conn(u,v)`：判断是否连通
- `link(u,v)`：若不连通则加入边，成功返回 `true`
- `cut(u,v)`：若 `(u,v)` 是现有边则删除，成功返回 `true`
- `query(u,v)`：返回路径 `u → v` 的聚合值
- `plen(u,v)`：路径节点数
- `kth(u,v,k)`：路径 `u → v` 上第 `k` 个节点，`k=0` 为 `u`
  标准 LCT 中 `access()` 暴露根到当前节点的实链，`makeroot()` 通过翻转路径改变表示根，随后 `split(x,y)` 即可让整条 `x-y` 路径落入一个 Splay

注意事项

- 对外节点编号是 `0..n-1`；内部使用 `1..n`，`0` 为哨兵
- `query()`、`plen()`、`kth()` 要求 `u,v` 已连通
- `link()` 会阻止成环；LCT 维护的是森林，不是一般动态图
- `cut(u,v)` 只删除真实存在的直接边，不能传路径上两个不相邻节点
- 所有操作都会改变 preferred path，因此不要把某个 Splay 的形态当成原树结构
- `sum[0]` 表示当前辅助树左到右的信息，`sum[1]` 表示右到左；翻转时交换两者，所以 `Op` 不要求交换律
- `Op` 必须满足结合律；若使用非默认单位元，构造时必须传入正确的 `id`
- 单次 Splay 操作最坏可能达到线性，但 LCT 操作的均摊复杂度为 $O(\log n)$
  Splay 中需要正确下传翻转标记后才能旋转

常见用法

动态森林路径和

```
LCT<i64> lct(a, 0);

lct.link(u, v);
i64 ans{ lct.query(u, v) };
lct.set(u, x);
lct.cut(u, v);
```

非交换路径信息

例如 `Op(a,b)` 表示矩阵乘法，则：

```
auto x{ lct.query(u, v) };
```

得到的是严格按照 `u → v` 顺序的乘积，而：

```
auto y{ lct.query(v, u) };
```

顺序相反

复杂度

操作	均摊复杂度
`set()`	$O(\log n)$
`conn()`	$O(\log n)$
`link()` / `cut()`	$O(\log n)$
`query()` / `plen()` / `kth()`	$O(\log n)$

空间复杂度：$O(n)$

LCT 常见扩展

边权转点权

LCT 天然维护节点，因此动态边权最稳妥的写法是：**每条边新建一个边节点**
原边：

```
u --w-- v
```

改成：

```
u -- e -- v
```

其中 `e` 的节点值为边权
例如维护路径最大边：

```
using P = pair<i64, int>;

struct MaxOp {
    P operator()(const P &a, const P &b) const { return max(a, b); }
};

constexpr i64 NEG{ numeric_limits<i64>::lowest() };
LCT<P, MaxOp> lct(vector<P>(n, {NEG, -1}), {NEG, -1});

int e{ lct.add_node({w, id}) };
lct.link(u, e);
lct.link(e, v);
```

查询：

```
auto [mx, id]{ lct.query(u, v) };
```

删除该边：

```
lct.cut(u, e);
lct.cut(e, v);
```

这也是后面**增量 MST / 动态 MST**最值得直接复用的写法

路径加 + 路径和

不建议默认把懒标记塞进核心 LCT，否则普通动态连通 / 动态 MST 时会白抄一坨代码
若确实需要数值路径加，在前面的求和版上做以下扩展即可
`Node` 增加：

```
T add{};
```

增加：

```
void apply_add(int x, const T &v) {
    if (!x) return;
    tr[x].val += v;
    tr[x].sum[0] += v * tr[x].siz;
    tr[x].sum[1] += v * tr[x].siz;
    tr[x].add += v;
}
```

把 `push()` 改为：

```
void push(int x) {
    if (tr[x].rev) {
        apply_rev(tr[x].ch[0]);
        apply_rev(tr[x].ch[1]);
        tr[x].rev = false;
    }
    if (tr[x].add != T{}) {
        apply_add(tr[x].ch[0], tr[x].add);
        apply_add(tr[x].ch[1], tr[x].add);
        tr[x].add = T{};
    }
}
```

再加入：

```
void add_path(int u, int v, const T &x) {
    int a{ u + 1 }, b{ v + 1 };
    split(a, b);
    apply_add(b, x);
}
```

此扩展只适用于：

```
Op = plus<T>
```

并要求 `T` 支持加法以及乘以路径长度

路径赋值 / 仿射修改

结构不变，本质仍然是：

```
split(u, v);
apply(root, tag);
```

区别仅在于 `Tag` 的复合以及它如何作用于 `val / sum`。如果既有赋值又有加法，应保证标记复合顺序与线段树中完全一致

路径第 k 点

核心版已经实现：

```
lct.kth(u, v, k);
```

因此不需要再准备单独的倍增结构

子树信息

普通 LCT 的辅助树只天然维护实链路径，不能直接把：

```
tr[x].sum
```

当成原树子树信息。
若一定要维护动态子树和，需要在 `access()` 中维护虚儿子贡献，例如：

```
all[x] = 左右实儿子贡献 + 自身 + virtual[x]
```

并在实边 / 虚边切换时增减 `virtual[x]`。这个版本与具体 `Info`、可逆性、根定义关系很强，不建议塞进通用核心；只需要动态连通分量统计时，下面的 ETT 更自然

Euler Tour Tree

解决问题： 动态森林中的连通性、连边、删边以及**整棵连通分量信息**。它把一棵树的 Euler Tour 存进隐式 Treap
相比 LCT：

- LCT 更擅长**路径**
- ETT 更擅长**整个连通分量 / 根树子树类信息**
  下面维护：

```
动态连通
连边 / 按边编号删边
连通分量节点数
连通分量点权和
单点修改
```

```
struct ETT {
    struct Node {
        int ch[2]{}, fa{}, siz{ 1 }, cnt{}, own{};
        u64 pri{};
        i64 val{}, sum{};
    };
    struct Edge {
        int a{}, b{};
        bool on{};
    };
    vector<Node> tr{{}};
    vector<int> self, free;
    vector<Edge> e;

    explicit ETT(int n) : self(n) {
        tr.reserve(n + 1);
        for (int u = 0; u < n; ++ u) self[u] = node(1);
    }
    explicit ETT(const vector<i64> &a) : self(a.size()) {
        tr.reserve(a.size() + 1);
        for (int u = 0; u < int(a.size()); ++ u) self[u] = node(1, a[u]);
    }
private:
    // ===== 隐式 Treap =====
    static u64 rnd(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    int node(int own = 0, i64 val = 0) {
        int x;
        if (free.empty()) x = tr.size(), tr.push_back(Node());
        else x = free.back(), free.pop_back(), tr[x] = Node();
        tr[x].own = tr[x].cnt = own;
        tr[x].val = tr[x].sum = val;
        tr[x].pri = rnd(x);
        return x;
    }
    int sz(int x) const { return x ? tr[x].siz : 0; }
    int cnt(int x) const { return x ? tr[x].cnt : 0; }
    i64 sum(int x) const { return x ? tr[x].sum : 0; }
    void pull(int x) {
        tr[x].siz = sz(tr[x].ch[0]) + 1 + sz(tr[x].ch[1]);
        tr[x].cnt = cnt(tr[x].ch[0]) + tr[x].own + cnt(tr[x].ch[1]);
        tr[x].sum = sum(tr[x].ch[0]) + tr[x].val + sum(tr[x].ch[1]);
        for (int k = 0; k < 2; ++ k) if (tr[x].ch[k]) tr[tr[x].ch[k]].fa = x;
    }
    int merge(int a, int b) {
        if (!a || !b) {
            int x{ a ? a : b };
            if (x) tr[x].fa = 0;
            return x;
        }
        if (tr[a].pri < tr[b].pri) {
            tr[a].ch[1] = merge(tr[a].ch[1], b);
```

```
            pull(a);
            tr[a].fa = 0;
            return a;
        }
        tr[b].ch[0] = merge(a, tr[b].ch[0]);
        pull(b);
        tr[b].fa = 0;
        return b;
    }
    pair<int, int> split(int x, int k) {
        if (!x) return {};
        if (sz(tr[x].ch[0]) >= k) {
            auto [a, b]{ split(tr[x].ch[0], k) };
            tr[x].ch[0] = b;
            pull(x);
            tr[x].fa = 0;
            if (a) tr[a].fa = 0;
            return {a, x};
        }
        auto [a, b]{ split(tr[x].ch[1], k - sz(tr[x].ch[0]) - 1) };
        tr[x].ch[1] = a;
        pull(x);
        tr[x].fa = 0;
        if (b) tr[b].fa = 0;
        return {x, b};
    }
    // ===== Euler Tour 定位与循环移位 =====
    int root(int x) const {
        while (tr[x].fa) x = tr[x].fa;
        return x;
    }
    int pos(int x) const {
        int res{ sz(tr[x].ch[0]) };
        while (tr[x].fa) {
            int p{ tr[x].fa };
            if (tr[p].ch[1] == x) res += sz(tr[p].ch[0]) + 1;
            x = p;
        }
        return res;
    }
    int reroot(int x) {
        int r{ root(x) }, k{ pos(x) };
        auto [a, b]{ split(r, k) };
        return merge(b, a);
    }
    void pull_up(int x) {
        while (x) {
            pull(x);
            x = tr[x].fa;
        }
    }

public:
    // ===== 动态森林接口 =====
    bool conn(int u, int v) const { return root(self[u]) == root(self[v]); }
    int size(int u) const { return cnt(root(self[u])); }
    i64 query(int u) const { return sum(root(self[u])); }
    void set(int u, i64 v) {
        int x{ self[u] };
        tr[x].val = v;
        pull_up(x);
    }
    int link(int u, int v) {
        if (conn(u, v)) return -1;
        int x{ reroot(self[u]) }, y{ reroot(self[v]) };
        int a{ node() }, b{ node() }, id{ int(e.size()) };
        e.push_back({a, b, true});
        merge(merge(merge(x, a), y), b);
        return id;
    }
    bool cut(int id) {
```

```
        if (id < 0 || id >= int(e.size()) || !e[id].on) return false;
        int a{ e[id].a }, b{ e[id].b };
        int r{ reroot(a) };
        auto [x, s]{ split(r, 1) };
        int k{ pos(b) };
        auto [l, y]{ split(s, k) };
        auto [z, rr]{ split(y, 1) };
        (void)x;
        (void)l;
        (void)z;
        (void)rr;
        tr[a].ch[0] = tr[a].ch[1] = tr[a].fa = 0;
        tr[b].ch[0] = tr[b].ch[1] = tr[b].fa = 0;
        e[id].on = false;
        free.push_back(a);
        free.push_back(b);
        return true;
    }
};
```

原理

每个真实节点有一个唯一的：

```
self[u]
```

节点；每条树边 `(u,v)` 在 Euler Tour 中加入两个有向出现：

```
u → v
v → u
```

因此一棵树对应一个循环 Euler Tour
连接两棵树时：

```
Tour(u) + (u,v) + Tour(v) + (v,u)
```

删除边时，删掉这两个有向出现，循环序列自然被切成两棵树
`own=1` 只出现在真实节点的 `self[u]` 上，因此：

```
size(u)
```

不会把边的两个 Euler Tour 节点计入

接口说明

- `ETT(n)`：建立 `n` 个孤立点，初始点权为 `0`
- `ETT(a)`：建立 `n=a.size()` 个孤立点
- `conn(u,v)`：是否位于同一棵树
- `link(u,v)`：若不连通则连接，返回**边编号**；失败返回 `-1`
- `cut(id)`：删除 `link()` 返回的边编号
- `set(u,x)`：修改点权
- `size(u)`：`u` 所在连通分量节点数
- `query(u)`：`u` 所在连通分量点权和

常见用法

```
ETT ett(a);

int id{ ett.link(u, v) };
cout << ett.size(u) << '\n';
cout << ett.query(u) << '\n';

ett.set(u, x);
ett.cut(id);
```

注意事项

- 必须保存 `link()` 返回的边编号，当前板子按编号删边
- ETT 同样只维护森林，不能 `link()` 两个已连通节点

- 一条边使用两个 Treap 节点
- 删除边后两个 Treap 节点进入空闲表，后续 link 复用；活跃森林最多 n-1 条边，节点池为 O(n)。边编号不复用，历史 Edge 记录仍为 O(q)，旧编号重复 cut 返回 false
- Treap 使用 SplitMix 生成固定伪随机优先级，表中期望复杂度采用随机优先级模型；固定公开序列对针对性构造不提供最坏对数保证
- 当前版本维护的是**整个连通分量的点数与点权和**；若需要其他满足结合律的分量信息，可以修改 `Node::sum / pull()`
- 如果题目核心是 `u-v` 路径最大值、路径和等，LCT 明显更直接，不要为了 ETT 而 ETT

复杂度

设累计创建的 Treap 节点数为 N：

操作	期望复杂度
`conn()`	$O(\log N)$
`link()` / `cut()`	$O(\log N)$
`set()`	$O(\log N)$
`size()` / `query()`	$O(\log N)$

空间复杂度：$O(n+q)$

Top Tree

Top Tree 将树拆成若干 Cluster。每个 Cluster 至多有两个边界点，通过不断合并 Cluster 维护整棵动态树
主要合并形态可以理解为：

```
Compress：两段路径首尾连接
Rake：把旁支子树挂到主路径
```

它能够同时覆盖很多 LCT 不自然的问题，例如动态维护：

```
树直径
全局 DP
与根无关的子树信息
路径 + 旁支信息
更复杂的动态树 DP
```

与 LCT / ETT 的区别

数据结构	最自然的维护对象	`link/cut`	路径信息	分量/子树信息	实现难度
LCT	preferred path	$O(\log n)$ 均摊	很强	较麻烦	中高
ETT	Euler Tour 序列	$O(\log n)$ 期望	较弱	很强	高
Top Tree	Cluster	$O(\log n)$	很强	很强	很高

赛场上建议按下面的优先级判断：

```
动态路径问题
    → LCT

动态连通 + 整个分量统计
    → ETT

动态树 DP / 动态直径等 LCT 很难自然维护的问题
    → 再考虑 Top Tree
```

对于绝大多数 ICPC / OI 题，LCT + 边转点已经覆盖真正需要现场模板支持的动态树问题。Top Tree 更适合作为“知道它能干什么”的能力索引，而不是打印模板的一部分

4.7 动态树应用

选型

操作模型	推荐
边只增加	`IncMST`
边增加 + 权值只降低	`IncMST::dec()`
只有删边，全部操作离线	倒序 `IncMST`
动态路径最大边	4.6 `LCT`
删除 MST 树边后寻找替代边	已超出普通 LCT
任意在线加边 / 删边 MST	全动态 MST，高级专项
只要求动态连通，不要求 MST	DSU / rollback DSU / LCT / ETT 按模型选择

增量最小生成树 / 最小生成森林

解决问题： 图中不断加入带权无向边，每次维护当前所有已加入边构成图的最小生成森林；图连通后即为 MST
核心结论：若新边 `(u,v,w)` 两端已经连通，则在当前 MST 中的 `u-v` 路径上找到最大权边 `e`
若 `w < weight(e)`，用新边替换 `e`；否则新边永久可以忽略
依赖 4.6.1 `LCT`，使用"边转点 + 路径最大值"

```
struct IncMST {
    using P = pair<i64, int>;
    struct MaxOp {
        P operator()(const P &a, const P &b) const { return max(a, b); }
    };
    struct Edge {
        int u, v, nd{ -1 };
        i64 w;
        bool in{};
    };
    static constexpr i64 NEG{ numeric_limits<i64>::lowest() };
    int n, cc;
    i128 sum{};
    LCT<P, MaxOp> lct;
    vector<Edge> e;
    explicit IncMST(int n_) : n(n_), cc(n_), lct(vector<P>(n_, {NEG, -1}), {NEG, -1}) {}
private:
    // ===== 将边加入当前 MSF =====
    void use(int id) {
        Edge &a{ e[id] };
        if (a.nd == -1) a.nd = lct.add_node({a.w, id});
        else lct.set(a.nd, {a.w, id});
        lct.link(a.u, a.nd);
        lct.link(a.nd, a.v);
        a.in = true;
        sum += a.w;
    }
    // ===== 从当前 MSF 删除边：仅替换边时调用 =====
    void drop(int id) {
        Edge &a{ e[id] };
        lct.cut(a.u, a.nd);
        lct.cut(a.nd, a.v);
        a.in = false;
        sum -= a.w;
    }
    // ===== 尝试让一条新边 / 降权边进入 MSF =====
    void relax(int id) {
        Edge &a{ e[id] };
        if (a.u == a.v) return;
        if (!lct.conn(a.u, a.v)) {
            use(id);
            -- cc;
            return;
        }
        auto [mx, old]{ lct.query(a.u, a.v) };
        if (a.w < mx) {
            drop(old);
            use(id);
        }
```

```
    }
public:
    // ===== 基础接口 =====
    int add(int u, int v, i64 w) {
        int id{ int(e.size()) };
        e.push_back({u, v, -1, w, false});
        relax(id);
        return id;
    }
    // ===== 边权降低 =====
    bool dec(int id, i64 w) {
        Edge &a{ e[id] };
        if (a.in) {
            sum += i128(w) - a.w;
            a.w = w;
            lct.set(a.nd, {w, id});
            return true;
        }
        a.w = w;
        relax(id);
        return a.in;
    }
    bool full() const { return cc == 1; }
    bool used(int id) const { return e[id].in; }
    i64 msf() const { return sum; }
    optional<i64> mst() const {
        if (!full()) return nullopt;
        return sum;
    }
};
```

接口说明

- `IncMST(n)`：初始化 `n` 个孤立节点
- `add(u,v,w)`：加入一条边，返回边编号
- `dec(id,w)`：把已有边 `id` 的权值降低到 `w`
- `used(id)`：该边当前是否位于最小生成森林中
- `full()`：当前图是否连通
- `msf()`：当前最小生成森林总权值
- `mst()`：若图已连通返回 MST 权值，否则返回 `nullopt`
- `cc`：当前图的连通块数

`dec(id,w)` 要求：

```
0 ≤ id < e.size()
w ≤ e[id].w
```

为什么只需要当前 MST

设当前最小生成树为 `T`，插入新边 `e=(u,v)`
若 `u,v` 已连通，则：

```
T + e
```

恰好产生一个环。设环中原树最大边为 `f`：

```
w(e) < w(f)  →  删除 f，加入 e
w(e) ≥ w(f) →  T 不需要变化
```

因此一次插边只需要查询当前树路径最大边，不需要保存所有被淘汰的非树边
这就是 LCT 特别适合增量 MST 的原因：`split(u,v)` 能把整条路径暴露成一个 Splay，从而查询路径最大边。标准 LCT 来源同样将 `split(x,y)` 定义为暴露整条 `x-y` 路径

边转点

原图节点全部赋：

```
{NEG, -1}
```

边：

```
u --w-- v
```

变成：

```
u -- edge_node -- v
```

边节点值为：

```
{w, id}
```

于是：

```
lct.query(u, v)
```

返回的就是路径最大边 `{权值,边编号}`

常见用法

```
IncMST mst(n);

for (auto [u, v, w] : edges) {
    int id{ mst.add(u, v, w) };

    if (mst.full()) {
        cout << mst.msf() << '\n';
    } else {
        cout << -1 << '\n';
    }
}
```

判断某条边当前是否在 MST：

```
if (mst.used(id)) {
    // 当前生成森林边
}
```

动态降低边权：

```
mst.dec(id, nw);
```

注意事项

- 支持负边权
- 支持重边和自环；自环永远不会进入 MSF
- 相同权值时不主动替换旧边，仍然得到合法 MST
- 内部 `sum` 和降权差值使用 i128，避免替换时的临时森林总和或 w-old 溢出。msf()/mst() 仍返回 i64，要求每次操作完成后的 MSF 总权值在 i64 范围内；若需要更大答案，同时将这两个接口与 DelMST 的答案类型改为 i128
- 一条边最多创建一个 LCT 边节点，即使曾经进入 MST、后来被替换、再因降权重新进入，也会复用原节点
- 被插入后淘汰的边，在纯插入模型中可以永久丢弃；这一点到了真正全动态 MST 中就不成立

复杂度

设累计加入 `m` 条边：

操作	均摊复杂度
`add()`	$O(\log(n+m))$
`dec()`	$O(\log(n+m))$
`used()` / `full()` / `msf()`	$O(1)$

空间复杂度为 $O(n+m)$

仅删边 MST 的离线倒序

如果操作只有：

```
删除某条初始边
询问当前 MST
```

那么没必要写真正的动态删除 MST
把时间倒过来：

```
删除边 → 加入边
```

就重新变成了 增量 MST

```cpp
struct DelMST {
    struct Edge {
        int u, v;
        i64 w;
    };
    struct Op {
        int tp, id;
    };
    // tp = 0: 删除边 id
    // tp = 1: 询问 MST
    static vector<optional<i64>> solve(int n, const vector<Edge> &e, const vector<Op> &op) {
        vector<char> del(e.size());
        for (auto [tp, id] : op) if (tp == 0) del[id] = true;

        IncMST mst(n);
        for (int i = 0; i < int(e.size()); ++ i) {
            if (!del[i]) mst.add(e[i].u, e[i].v, e[i].w);
        }

        vector<optional<i64>> ans;
        for (int i = int(op.size()) - 1; i >= 0; -- i) {
            auto [tp, id]{ op[i] };
            if (tp == 1) {
                ans.push_back(mst.mst());
            } else {
                mst.add(e[id].u, e[id].v, e[id].w);
            }
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

接口说明

`DelMST::solve(n,e,op)` 返回所有询问的答案
若某次图不连通：

```
ans[i] == nullopt
```

否则：

```
*ans[i]
```

即 MST 权值

注意事项

要求：

```
所有边最开始存在
每条边最多删除一次
不存在重新加入操作
```

如果操作同时有：

```
add
delete
```

倒序后仍然同时存在：

```
delete
add
```

就不能靠这一招解决

常见用法

```
auto ans{ DelMST::solve(n, edges, ops) };

for (auto x : ans) {
    if (x) cout << *x << '\n';
    else cout << -1 << '\n';
}
```

复杂度

每条原边恰好被加入 `IncMST` 一次，因此总复杂度：

$$O(m \log(n+m) + q)$$

空间复杂度：

$$O(n+m+q)$$

动态 MST 的能力边界

这一部分非常重要。LCT 能维护动态树，不代表“LCT = 全动态 MST”
当前 `IncMST` 真正容易支持的操作如下：

图操作	是否容易支持	原因
插入边	✅	环上删除最大边
非树边降权	✅	重新尝试环替换
树边降权	✅	当前 MST 仍然最优
删除非树边	当前 MST 不变	但全动态场景不能永久忘掉它
非树边升权	当前 MST 不变	不影响当前树
删除树边	❌	需要寻找割两侧最小替代边
树边升权	❌	可能需要寻找替代边
任意插入 + 删除	❌	必须维护非树边候选关系

真正困难的地方

假设 MST 中删除了树边：

```
u ----- v
```

森林被切成两个连通块：

```
A | B
```

此时必须在所有非树边中寻找：

```
连接 A 与 B 的最小权边
```

而普通 LCT 擅长的是：

```
给定 u,v
查询树上 u-v 路径
```

它并不能直接回答：

```
当前两个连通块之间有哪些非树边？
其中最小的是谁？
```

所以：

```
路径最大边
```

和：

```
割上的最小替代边
```

是两个完全不同的问题

为什么增量 MST 可以丢弃非树边

纯插入时，一条边在加入那一刻如果不能改善当前 MST，那么它以后也不需要重新考虑
但是允许删树边以后：

```
曾经没进入 MST 的边
```

可能突然成为最佳替代边
因此全动态 MST 必须保留并组织大量非树边，这也是复杂度陡增的根源

ETT 能不能直接解决

不能
4.6 的 ETT 可以很好地维护：

```
link / cut
连通块大小
连通块聚合信息
```

但它同样没有自动维护：

```
两个连通块之间最小的非树边
```

所以：

```
ETT + 最小生成树
```

仍然需要额外的 replacement-edge 结构

Top Tree 能不能直接解决

也不能“自动解决”
Top Tree 比 LCT 更擅长维护复杂树 DP 和 Cluster 信息，但全动态 MST 的非树边管理依然是额外问题

真正的全动态 MST

理论上的全动态 MST 通常需要维护多层动态森林以及不同层级的非树边，并在树边删除后高效寻找替代边。实现复杂度远高于普通 LCT，不适合把几百行实现作为常规赛场板子
竞赛中更应该先根据操作限制降级问题：

```
只有插边
→ IncMST

插边 + 降权
→ IncMST

只有删边且离线
→ 倒序 IncMST

静态图，只询问删某一条 MST 边后的结果
→ MST + 非树边替换预处理

任意 add / delete，但数据较小
→ 分块 / 周期重构
```

```
任意 add / delete，且必须在线
→ 真正 fully dynamic MST
```

一个常见误区

下面这种东西：

```
时间线段树 + rollback DSU
```

非常适合解决离线动态连通性，但它不能直接维护动态 MST 权值。
因为 DSU 只能回答：

```
两个点是否已经连通
```

而 Kruskal 还依赖边的全局权值顺序。一条边在某个时间区间有效，并不能简单地按时间线段树 DFS 顺序执行 Kruskal
动态 MST 的离线算法通常还需要额外的边收缩、边删除 / 归约、分治等技巧，明显比动态连通复杂。

常见对称扩展

增量最大生成树

与 `IncMST` 完全对称
最小生成树是：

```
新边形成环
删除环上最大边
```

最大生成树改为：

```
新边形成环
删除环上最小边
```

因此只需要让 LCT 路径维护：

```
最小边
```

并把判断：

```
if (a.w < mx)
```

改成：

```
if (a.w > mn)
```

同时应将 MaxOp 换为 MinOp，并把原节点和单位元设为 `{numeric_limits<i64>::max(), 哨兵编号}`；按 pair 的比较顺序选择不会抢占真实边的编号。
不能只翻转上述不等号。

5. 图论

5.1 基础、遍历与建图

5.1.1 建图、遍历与 DAG

```
template<class W = i64>
struct Graph {
    struct Edge {
        int to, id;
        W w;
    };
    int n, edge_cnt{ 0 };
    vector<vector<Edge>> g;
    explicit Graph(int n_) : n(n_), g(n) {}
```

```
    // ===== 基础建图：以下代码是图结构核心，不建议删除 =====
    int add_directed(int u, int v, W w = 1) {
        int id{ edge_cnt ++ };
        g[u].push_back({v, id, w});
        return id;
    }
    int add_undirected(int u, int v, W w = 1) {
        int id{ edge_cnt ++ };
        g[u].push_back({v, id, w});
        g[v].push_back({u, id, w});
        return id;
    }
    // ===== 基础遍历：若只把 Graph 当邻接表使用，可以整段删除 =====
    template<ranges::input_range R>
    vector<int> bfs(R &&source) const {
        vector<int> dis(n, -1), q;
        q.reserve(n);
        for (int s : source) if (dis[s] == -1) {
            dis[s] = 0;
            q.push_back(s);
        }
        for (int h = 0; h < q.size(); ++ h) {
            int u = q[h];
            for (const auto &e : g[u]) if (dis[e.to] == -1) {
                dis[e.to] = dis[u] + 1;
                q.push_back(e.to);
            }
        }
        return dis;
    }
    vector<int> bfs(int s) const { return bfs(array{s}); }
    vector<int> dfs_order(int s) const {
        vector<int> order;
        vector<pair<int, size_t>> stk{{s, 0}};
        vector<char> vis(n);
        vis[s] = true;
        order.push_back(s);
        while (!stk.empty()) {
            auto &[u, next] = stk.back();
            if (next == g[u].size()) { stk.pop_back(); continue; }
            int v{ g[u][next ++].to };
            if (vis[v]) continue;
            vis[v] = true;
            order.push_back(v);
            stk.push_back({v, 0});
        }
        return order;
    }
    // ===== DAG：若不需要拓扑排序或 DAG DP，以下代码可整段删除 =====
    optional<vector<int>> topo_order() const {
        vector<int> deg(n), order;
        order.reserve(n);
        for (int u : views::iota(0, n)) {
            for (const auto &e : g[u]) ++ deg[e.to];
        }
        for (int u : views::iota(0, n)) if (deg[u] == 0) order.push_back(u);
        for (int h = 0; h < order.size(); ++ h) {
            int u{ order[h] };
            for (const auto &e : g[u]) {
                if (-- deg[e.to] == 0) order.push_back(e.to);
            }
        }
        if (order.size() != n) return nullopt;
        return order;
    }
    template<class T = i64, ranges::input_range R>
    optional<vector<T>> dag_path_count(R &&source) const {
        auto order = topo_order();
        if (!order) return nullopt;
        vector<T> dp(n);
        for (int s : source) dp[s] = T{ 1 };
```

```
        for (int u : *order) {
            for (const auto &e : g[u]) dp[e.to] += dp[u];
        }
        return dp;
    }
    template<class T = i64>
    optional<vector<T>> dag_path_count(int s) const {
        return dag_path_count<T>(array{s});
    }
    template<ranges::input_range R>
    optional<vector<W>> dag_longest_path(R &&source) const {
        auto order = topo_order();
        if (!order) return nullopt;
        const W neg = numeric_limits<W>::lowest() / 4;
        vector<W> dis(n, neg);
        for (int s : source) dis[s] = W{};
        for (int u : *order) if (dis[u] != neg) {
            for (const auto &e : g[u]) {
                dis[e.to] = max(dis[e.to], dis[u] + e.w);
            }
        }
        return dis;
    }
    optional<vector<W>> dag_longest_path(int s) const {
        return dag_longest_path(array{s});
    }
    template<ranges::input_range R>
    optional<vector<W>> dag_shortest_path(R &&source) const {
        auto order = topo_order();
        if (!order) return nullopt;
        const W inf = numeric_limits<W>::max() / 4;
        vector<W> dis(n, inf);
        for (int s : source) dis[s] = W{};
        for (int u : *order) if (dis[u] != inf) {
            for (const auto &e : g[u]) {
                dis[e.to] = min(dis[e.to], dis[u] + e.w);
            }
        }
        return dis;
    }
    optional<vector<W>> dag_shortest_path(int s) const {
        return dag_shortest_path(array{s});
    }
    // ===== 可达性：若不需要传递闭包，以下代码可整段删除 =====
    template<size_t N>
    vector<bitset<N>> transitive_closure() const {
        assert(n <= int(N));
        vector<bitset<N>> reach(n);
        for (int u : views::iota(0, n)) {
            reach[u].set(u);
            for (const auto &e : g[u]) reach[u].set(e.to);
        }
        for (int k : views::iota(0, n)) {
            for (int u : views::iota(0, n)) {
                if (reach[u][k]) reach[u] |= reach[k];
            }
        }
        return reach;
    }
    template<size_t N>
    optional<vector<bitset<N>>> dag_transitive_closure() const {
        assert(n <= int(N));
        auto order = topo_order();
        if (!order) return nullopt;
        vector<bitset<N>> reach(n);
        for (int u : views::iota(0, n)) reach[u].set(u);
        for (int u : *order | views::reverse) {
            for (const auto &e : g[u]) reach[u] |= reach[e.to];
        }
        return reach;
```

```
    }
};
```

成员变量

- `n`：点数，点编号为 `0..n-1`
- `edge_cnt`：逻辑边数量
- `g[u]`：从 `u` 出发的邻接边
- `Edge::to`：终点
- `Edge::id`：逻辑边编号
- `Edge::w`：边权
- 无向边的两条邻接边共享同一个 `id`

接口

- `Graph<W>(n)`：建立 `n` 个点的空图，$O(n)$
- `add_directed(u, v, w)`：加入有向边 `u → v`，返回边号，均摊 $O(1)$
- `add_undirected(u, v, w)`：加入无向边，两方向共用边号，均摊 $O(1)$
- `bfs(s)`：单源 BFS，返回无权最短距离，不可达为 `-1`，$O(n+m)$
- `bfs(source)`：多源 BFS，`source` 可以是 `vector<int>`、`array` 等 range，$O(n+m)$
- `dfs_order(s)`：返回从 `s` 可达部分的非递归 DFS 访问序，$O(n+m)$
- `topological_order()`：返回拓扑序；有环返回 `nullopt`，$O(n+m)$
- `dag_path_count<T>(s)`：统计从 `s` 出发到各点的路径条数；有环返回 `nullopt`，$O(n+m)$
- `dag_path_count<T>(source)`：多源版本，每个源点初值为 `1`
- `dag_longest_path(s)`：DAG 单源最长路；有环返回 `nullopt`，$O(n+m)$
- `dag_longest_path(source)`：DAG 多源最长路
- `dag_shortest_path(s)`：DAG 单源最短路，允许负边；有环返回 `nullopt`，$O(n+m)$
- `dag_shortest_path(source)`：DAG 多源最短路
- `transitive_closure<N>()`：一般有向图 Bitset 传递闭包
- `dag_transitive_closure<N>()`：DAG 专用传递闭包；有环返回 `nullopt`

注意事项

- `bfs()` 完全忽略边权，只适合所有边代价相同的无权最短路
- `topological_order()` 每次重新计算入度，因此可以安全重复调用，也不会修改图
- 拓扑排序和全部 `dag_*` 接口要求图由**有向边**组成；加入一条普通无向边等价于加入两个反向弧，通常会形成长度为 `2` 的有向环
- `dag_path_count()` 将平行边视为不同路径；每个源点本身存在一条长度为 `0` 的路径。若计数需要取模，可令 `T` 为取模类
- DAG 最短路不要求边权非负；最长路同理。不可达点分别保留 `numeric_limits<W>::max() / 4` 与 `numeric_limits<W>::lowest() / 4`
- 距离相加前仍需估算数值范围，不能因为设置了无穷元就忽略溢出
- `transitive_closure<N>()` 中 `N` 是编译期最大点数，要求 `n ≤ N`；实际只分配 `n` 行 Bitset
- 若图确定为 DAG，优先使用 `dag_transitive_closure()`，通常明显快于 Floyd 式传递闭包
- `dfs_order()` 只保证得到一个 DFS 风格访问序，不维护灰点/黑点、进入时间和退出时间，不能直接拿来做有向 DFS 边分类

常见用法

无向图与边号

```
Graph g(n);
int id = g.add_undirected(u, v, w);
for (const auto &[v, id, w] : g.g[u]) {
    // ...
}
```

桥、欧拉路等需要区分“父边”而不是“父点”时，可以直接比较 `id`

单源 / 多源 BFS

```
auto dis1 = g.bfs(s);
vector<int> source = {s1, s2, s3};
auto dis2 = g.bfs(source);
```

拓扑排序

```
auto order = g.topo_order();
if (!order) {
    // 有环
} else {
    for (int u : *order) {
```

```
        // ...
    }
}
```

DAG 最长路

```
auto dis = g.dag_longest_path(s);
if (!dis) {
    // 不是 DAG
} else {
    cout << (*dis)[t] << '\n';
}
```

DAG 路径计数

```
auto cnt = g.dag_path_count<i64>(s);
cout << (*cnt)[t] << '\n';
```

使用取模类：

```
auto cnt = g.dag_path_count<Z>(s);
```

DAG 可达性

```
auto reach = g.dag_transitive_closure<2000>();
if (reach && (*reach)[u][v]) {
    // u 可以到达 v
}
```

网格图通常不显式建边

```
constexpr array<pair<int, int>, 4> dir{{
    {1, 0}, {-1, 0}, {0, 1}, {0, -1}
}};
for (auto [dx, dy] : dir) {
    int nx = x + dx, ny = y + dy;
    if (0 <= nx && nx < n && 0 <= ny && ny < m) {
        // 枚举相邻格
    }
}
```

邻接矩阵只在点数较小且图很稠密时考虑：

```
vector matrix(n, vector<i64>(n, INF));
```

复杂度

设 `m` 为邻接表中保存的有向弧数量；无向边会保存为两条弧

操作	时间复杂度
`add_directed()` / `add_undirected()`	均摊 $O(1)$
`bfs()` / `dfs_order()` / `topological_order()` / DAG 路径计数 / DAG 最短路 / 最长路	$O(n+m)$
`transitive_closure<N>()`	$O(n^2N/w)$
`dag_transitive_closure<N>()`	$O((n+m)N/w)$

当 $N=\Theta(n)$ 时，一般传递闭包可写作：

$$O\left(\frac{n^3}{w}\right)$$

DAG 传递闭包为：

$$O\left(\frac{mn}{w}\right)$$

其中 `w` 为机器字长
基础图空间复杂度：

$$O(n+m)$$

传递闭包额外空间复杂度：

$$O\left(\frac{nN}{w}\right)$$

5.1.2 线段树优化建边

```
template<class W = i64>
struct SegmentGraph {
    int n;
    vector<vector<pair<int, W>>> g;
    vector<int> out_id, in_id;
    explicit SegmentGraph(int n_) : n(n_), g(n), out_id(max(1, n << 2)), in_id(max(1, n << 2)) {
        g.reserve(max(1, 3 * n));
        if (n) build(1, 0, n);
    }
    // ===== 基础扩点与建边：额外建模时也可以直接调用 =====
    int new_node() {
        g.emplace_back();
        return int(g.size()) - 1;
    }
    void add_edge(int u, int v, W w = {}) { g[u].push_back({v, w}); }
private:
    // ===== 双线段树：所有区间建边接口依赖，不建议删除 =====
    void build(int p, int l, int r) {
        if (r - l == 1) {
            out_id[p] = in_id[p] = l;
            return;
        }
        out_id[p] = new_node();
        in_id[p] = new_node();
        int mid{ l + r >> 1 };
        build(2 * p, l, mid);
        build(2 * p + 1, mid, r);
        add_edge(out_id[p], out_id[2 * p]);
        add_edge(out_id[p], out_id[2 * p + 1]);
        add_edge(in_id[2 * p], in_id[p]);
        add_edge(in_id[2 * p + 1], in_id[p]);
    }
    void p_to_rng(int u, int ql, int qr, W w, int p, int l, int r) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return add_edge(u, out_id[p], w);
        int mid{ l + r >> 1 };
        p_to_rng(u, ql, qr, w, 2 * p, l, mid);
        p_to_rng(u, ql, qr, w, 2 * p + 1, mid, r);
    }

    void rng_to_p(int ql, int qr, int v, W w, int p, int l, int r) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return add_edge(in_id[p], v, w);
        int mid{ l + r >> 1 };
        rng_to_p(ql, qr, v, w, 2 * p, l, mid);
        rng_to_p(ql, qr, v, w, 2 * p + 1, mid, r);
    }
public:
    // ===== 区间建边接口 =====
    void p_to_rng(int u, int l, int r, W w) { if (l < r) p_to_rng(u, l, r, w, 1, 0, n); }
    void rng_to_p(int l, int r, int v, W w) { if (l < r) rng_to_p(l, r, v, w, 1, 0, n); }
    int rng_to_rng(int l1, int r1, int l2, int r2, W w) {
        if (l1 >= r1 || l2 >= r2) return -1;
        int p{ new_node() };
        rng_to_p(l1, r1, p, w);
        p_to_rng(p, l2, r2, W{});
        return p;
    }
};
```

成员变量

- `n`：原图点数
- 原始点编号固定为 `0..n-1`
- `g`：扩点后的完整邻接表；内部辅助点编号从 `n` 开始
- `out_id[p]`：出线段树节点，对应“一个点连向整个区间”
- `in_id[p]`：入线段树节点，对应“整个区间连向一个点”

接口

- `SegmentGraph<W>(n)`：建立双线段树图，$O(n)$
- `new_node()`：新增一个普通辅助点，均摊 $O(1)$
- `add_edge(u, v, w)`：普通单点到单点有向边，均摊 $O(1)$
- `p_to_rng(u, l, r, w)`：对所有 `v in [l,r)` 建立逻辑边 `u → v`，权值均为 `w`
- `range_to_point(l, r, v, w)`：对所有 `u in [l,r)` 建立逻辑边 `u → v`，权值均为 `w`
- `range_to_range(l1, r1, l2, r2, w)`：对两个区间之间建立完整有向逻辑边，返回本次建立的中转点；空区间返回 `-1`

注意事项

- 所有区间统一为左闭右开 `[l,r)`
- 原始点始终保持 `0..n-1`，线段树辅助点不会改变原点编号
- 基础双线段树在 `n > 0` 时恰好使用 `3n-2` 个点，而不是固定申请 `9n` 个点
- `g.size()` 才是跑最短路时的实际总点数；不要只跑前 `n` 个点
- 出树方向为“父节点 → 子节点”，因此一个辅助节点可以沿零边走向区间内所有原点
- 入树方向为“子节点 → 父节点”，因此区间内任意原点都可以沿零边汇入对应区间节点
- `rng_to_rng()` 只新建一个中转点：源区间以权 `w` 汇入中转点，中转点再以 `0` 权连向目标区间，因此任意合法点对间的总新增代价仍为 `w`
- 若后续跑 Dijkstra，则题目新增边权必须非负；`SegmentGraph` 本身并不限制边权
- 不要枚举区间分解出来的源节点和目标节点两两连边，否则一次区间到区间建边会退化为 $O(\log^2 n)$ 条边

常见用法

点到点

```
SegmentGraph sg(n);
sg.add_edge(u, v, w);
```

点到区间

表示：

$$u \to v, \quad l \le v < r$$

且所有边权均为 `w`：

```
sg.p_to_rng(u, l, r, w);
```

区间到点

表示：

$$u \to v, \quad l \le u < r$$

且所有边权均为 `w`：

```
sg.rng_to_p(l, r, v, w);
```

区间到区间

表示：

$$u \to v, \quad l_1 \le u < r_1, \quad l_2 \le v < r_2$$

且所有逻辑边权均为 `w`：

```
sg.rng_to_rng(l1, r1, l2, r2, w);
```

不需要手动把两个区间各自拆成 $O(\log n)$ 个节点再做笛卡尔积

接入最短路

全部建边完成后，对整张扩展图运行最短路：

```
int N = sg.g.size();
// 对 sg.g[0 .. N) 跑 Dijkstra
```

源点和题目中的普通目标点仍直接使用原编号 `0..n-1`

复杂度

初始化时，双线段树包含：

$$3n-2$$

个节点，以及：

$$4n-4$$

条零权辅助边

操作	新增节点	新增边	时间复杂度
`add_edge()`	0	1	$O(1)$
`p_to_rng()` / `rng_to_p()`	0	$O(\log n)$	$O(\log n)$
`range_to_range()`	1	$O(\log n)$	$O(\log n)$

5.1.3 DAG 可达性、传递闭包与传递约简

解决问题： 在 DAG 上批量判断可达性、构造传递闭包，并删除所有可由其他路径替代的冗余边得到唯一的传递约简

```
struct DAGReachability {
    struct Edge { int u, v; };
    int n, words{};
    vector<Edge> edges;
    vector<vector<int>> out, in;
    vector<int> topo, pos;
    vector<vector<u64>> reach;
    bool built{}, dag{}, closure_ready{};
    explicit DAGReachability(int n_) : n(n_), out(n), in(n) {}
    int add_edge(int u, int v) {
        assert(!built);
        int id = edges.size();
        edges.push_back({u, v});
        out[u].push_back(id);
        in[v].push_back(id);
        return id;
    }
    // ===== 拓扑序：所有后续功能依赖 =====
    bool build() {
        if (built) return dag;
        vector<int> indeg(n);
        for (auto [u, v] : edges) ++ indeg[v];
        queue<int> q;
        for (int u : views::iota(0, n)) if (!indeg[u]) q.push(u);
        topo.reserve(n);
        while (!q.empty()) {
            int u{ q.front() };
            q.pop();
            topo.push_back(u);
            for (int id : out[u]) {
                int v{ edges[id].v };
                if (-- indeg[v] == 0) q.push(v);
            }
        }
        dag = topo.size() == n;
        if (dag) {
            pos.resize(n);
            for (int i : views::iota(0, n)) pos[topo[i]] = i;
        }
        built = true;
        return dag;
    }
```

```
    // ===== 完整传递闭包：中等规模使用 =====
    void build_closure() {
        assert(build());
        if (closure_ready) return;
        words = (n + 63) >> 6;
        reach.assign(n, vector<u64>(words));
        for (int u : topo | views::reverse) {
            reach[u][u >> 6] |= 1ULL << (u & 63);
            for (int id : out[u]) {
                int v{ edges[id].v };
                for (int b : views::iota(0, words)) reach[u][b] |= reach[v][b];
            }
        }
        closure_ready = true;
    }
    bool reachable(int u, int v) const {
        assert(closure_ready);
        return reach[u][v >> 6] >> (v & 63) & 1ULL;
    }
    int reachable_count(int u) const {
        assert(closure_ready);
        int ans{ 0 };
        for (u64 x : reach[u]) ans += popcount(x);
        return ans;
    }
    // ===== 大图离线询问：每 64 个询问并行传播 =====
    vector<char> answer_queries(const vector<pair<int, int>> &queries) const {
        assert(built && dag);
        int q = queries.size();
        vector<char> ans(q);
        if (closure_ready) {
            for (int i : views::iota(0, q)) {
                auto [s, t] = queries[i];
                ans[i] = reachable(s, t);
            }
            return ans;
        }
        vector<u64> mask(n);
        for (int l = 0; l < q; l += 64) {
            ranges::fill(mask, 0);
            int r{ min(l + 64, q) };
            for (int i : views::iota(l, r)) {
                auto [s, t] = queries[i];
                mask[s] |= 1ULL << (i - l);
            }
            for (int u : topo) {
                for (int id : out[u]) mask[edges[id].v] |= mask[u];
            }
            for (int i : views::iota(l, r)) {
                auto [s, t] = queries[i];
                ans[i] = mask[t] >> (i - l) & 1ULL;
            }
        }
        return ans;
    }
    // ===== 传递约简：返回保留的原边编号 =====
    vector<int> transitive_reduction() {
        build_closure();
        vector<int> kept;
        kept.reserve(edges.size());
        vector<u64> covered(words);
        for (int u : views::iota(0, n)) {
            vector<int> ids = out[u];
            ranges::sort(ids, {}, [&](int id) { return pos[edges[id].v]; });
            ranges::fill(covered, 0);
            for (int id : ids) {
                int v{ edges[id].v };
                if (covered[v >> 6] >> (v & 63) & 1ULL) continue;
                kept.push_back(id);
                for (int b : views::iota(0, words)) covered[b] |= reach[v][b];
            }
```

```
        }
        ranges::sort(kept);
        return kept;
    }
};
```

基本结构

DAG 中存在拓扑序

$$u \to v \Longrightarrow pos[u] < pos[v]$$

因此可达性 DP 可以按照逆拓扑序进行
定义 `reach[u]` 为从 `u` 可达的全部节点组成的 Bitset，并约定节点可以零步到达自身

$$u \in reach[u]$$

对于每条边 $u \to v$

$$reach[u] \mid = reach[v]$$

因为逆拓扑处理时 `reach[v]` 已经完整求出

```
for (int u : topo | views::reverse) {
    reach[u][u >> 6] |= 1ULL << (u & 63);
    for (int id : out[u]) {
        int v{ edges[id].v };
        for (int b : views::iota(0, words)) reach[u][b] |= reach[v][b];
    }
}
```

最终 `reach` 就是图的传递闭包

接口说明

`add_edge(u,v)`

加入有向边

```
int id = dag.add_edge(u, v);
```

返回稳定的原边编号
允许重边，但图最终必须是 DAG
自环一定会使 `build()` 判定失败

`build()`

计算拓扑序并判断是否为 DAG

```
if (!dag.build()) {
    // 原图存在环
}
```

成功后

```
dag.topo
```

是拓扑序

```
dag.pos[u]
```

是节点 `u` 的拓扑位置
调用 `build()` 后不能再加边

完整传递闭包

```
dag.build_closure();
```

得到

```
dag.reach[u]
```

表示所有从 `u` 可达的节点
约定可达性包含长度为 0 的路径，因此

```
dag.reachable(u, u) == true
```

若题目只认为至少经过一条边才算可达，在 DAG 中只需额外处理 `u == v`

单次可达查询

```
bool ok = dag.reachable(u, v);
```

复杂度

$$O(1)$$

可达节点数量

```
int cnt = dag.reachable_count(u);
```

包含 `u` 自身
若题目不统计自身

```
int cnt = dag.reachable_count(u) - 1;
```

为什么 Bitset 有效

普通集合合并若逐点处理，一条边可能需要 $O(n)$
把 n 个可达状态压入 $\lceil \frac{n}{64} \rceil$ 个 `u64` 后，每次集合合并只需 $O\left(\frac{n}{64}\right)$ 次按位 OR
因此特别适合

- 中等规模 DAG
- 大量任意两点可达询问
- 可达点计数
- 传递闭包
- 偏序关系
- 传递约简

大图离线可达性询问

完整闭包需要

$$O\left(\frac{n^2}{64}\right)$$

个 `u64`
当 n 很大时可能无法接受
如果只需要回答 q 个离线询问

```
vector<pair<int, int>> queries;
auto ans = dag.answer_queries(queries);
```

则不需要建立完整闭包

原理

每次同时处理最多 64 个询问
第 `i` 个询问

$$s_i \to t_i$$

对应 `u64` 中的一位
初始化

```
mask[s_i] |= 1ULL << i;
```

随后按拓扑序传播

```
mask[v] |= mask[u];
```

若第 `i` 位最终出现在 `mask[t_i]` 中，就说明

$$s_i \rightsquigarrow t_i$$

成立
因此 64 个 DFS 式可达性传播被压成了一次 DAG DP

使用

```
DAGReachability dag(n);
for (auto [u, v] : edges) dag.add_edge(u, v);
if (!dag.build()) {
    // 不是 DAG
}
vector<pair<int, int>> queries{
    {0, 5},
    {3, 8},
    {7, 2}
};
auto ans = dag.answer_queries(queries);
for (int i : views::iota(0, int(queries.size()))) {
    if (ans[i]) {
        // queries[i].first 可以到达 queries[i].second
    }
}
```

如果已经调用过 `build_closure()`，`answer_queries()` 会自动直接使用闭包，以 $O(q)$ 回答

传递闭包

图 G 的传递闭包 G^+ 保留所有可达关系
若

$$u \rightsquigarrow v$$

则闭包中存在边

$$u \to v$$

本板不显式建立这些 $O(n^2)$ 条边，而是用

```
reach[u]
```

保存闭包关系
若确实需要枚举所有可达点

```
dag.build_closure();
for (int u : views::iota(0, n)) {
    for (int b : views::iota(0, dag.words)) {
        for (u64 s = dag.reach[u][b]; s; s &= s - 1) {
            int v{ 64 * b + countr_zero(s) };
            if (v < n) {
                // u 可以到达 v
            }
        }
    }
}
```

传递约简

传递约简要删除所有可由其他路径替代的边，同时保持可达关系完全不变
例如

```
0 → 1 → 2
 \------> 2
```

边

```
0 → 2
```

可以通过

```
0 → 1 → 2
```

替代，因此属于冗余边
约简后只保留

```
0 → 1 → 2
```

DAG 上的性质

对于 DAG，传递约简是唯一的
若 DAG 表示一个偏序关系，其传递约简就是对应的 Hasse 图

`transitive_reduction()`

```cpp
auto kept = dag.transitive_reduction();
```

返回需要保留的原边编号
例如

```cpp
for (int id : kept) {
    auto [u, v] = dag.edges[id];
    // u → v 属于传递约简
}
```

需要直接建立约简后的邻接表

```cpp
vector<vector<int>> reduced(n);
for (int id : dag.transitive_reduction()) {
    auto [u, v] = dag.edges[id];
    reduced[u].push_back(v);
}
```

传递约简原理

固定节点 `u`
将所有出边

$$u \to v$$

按照 `v` 的拓扑位置从小到大处理
维护 `covered`
表示已经可以通过此前保留的直接后继到达的全部节点
若当前 `v` 已经满足

$$v \in covered$$

说明存在此前某个后继 `w`

$$u \to w \rightsquigarrow v$$

于是边

$$u \to v$$

可以删除
否则必须保留该边，并执行

$$covered \mid = reach[v]$$

代码对应

```
for (int id : ids) {
    int v = edges[id].v;
    if (covered[v >> 6] >> (v & 63) & 1ULL) continue;
    kept.push_back(id);
    for (int b : views::iota(0, words)) covered[b] |= reach[v][b];
}
```

为什么必须按拓扑序处理后继

若存在

$$u \to a, \qquad u \to b, \qquad a \rightsquigarrow b$$

必然有

$$pos[a] < pos[b]$$

必须先处理 `a`
这样处理 `b` 时才能发现

$$b \in reach[a]$$

并删掉冗余边 `u → b`
如果顺序反过来，会错误保留这条边

重边

闭包和离线询问都天然允许重边
传递约简中若存在多条完全相同的边

```
u → v
u → v
```

只会保留其中第一条
后续同端点边会发现 `v` 已经处于 `covered`
因此无需提前去重
若不同重边携带不同权值或编号语义，需要自行决定希望保留哪一条
例如希望优先保留最小边权，应在加入图前先处理重边，或修改同终点边的处理顺序

一般有向图怎么办

本板只处理 DAG
如果一般有向图存在 SCC，可以先使用前面的 `SCC` 板子缩点

```
原有向图
    ↓ SCC
缩点 DAG
    ↓ 本板
可达性 / 闭包 / 传递约简
```

对于原图节点 `u,v`
若

```
scc.bel[u] == scc.bel[v]
```

则二者互相可达
否则只需判断缩点 DAG 中

```
bel[u] → bel[v]
```

是否可达
注意：缩点 DAG 的传递约简只约简 SCC 之间的关系
它不会给出原有环图内部的最小等价边集

常见用法

中等规模全可达查询

```
DAGReachability dag(n);
for (auto [u, v] : edges) dag.add_edge(u, v);
if (!dag.build()) {
    // 有环
}
dag.build_closure();
if (dag.reachable(u, v)) {
    // u 可以到达 v
}
```

每个点能到达多少点

```
dag.build_closure();
for (int u : views::iota(0, n)) {
    int cnt = dag.reachable_count(u);
}
```

大图批量询问

```
DAGReachability dag(n);
for (auto [u, v] : edges) dag.add_edge(u, v);
dag.build();
vector<pair<int, int>> query(q);
for (auto &[u, v] : query) cin >> u >> v;
auto ans = dag.answer_queries(query);
```

此时不要调用 `build_closure()`，否则失去低内存优势

求传递约简

```
DAGReachability dag(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    cin >> u >> v;
    dag.add_edge(u, v);
}
if (!dag.build()) {
    // 不是 DAG
}
auto kept = dag.transitive_reduction();
for (int id : kept) {
    auto [u, v] = dag.edges[id];
    cout << u << ' ' << v << '\n';
}
```

快速回忆

```
DAG + 全可达关系
    逆拓扑
    reach[u] |= reach[v]

中等 n
    n 个 Bitset
    单次查询 O(1)

大 n + q 个离线询问
    每 64 个询问占一个 u64
    沿拓扑序传播 mask

传递约简
    先求闭包
```

```
    固定 u
    出边终点按 topo 从小到大
    v 已被 covered 覆盖 → 删除 u → v
    否则保留并 covered |= reach[v]

一般有向图
    先 SCC 缩点成 DAG
```

注意事项

- `build()` 后不能继续加入边
- `build()` 返回 `false` 表示存在有向环，后续 DAG 接口不能使用
- `reachable(u,u)` 定义为 `true`
- 完整闭包适合中等规模，空间通常比时间更先成为瓶颈
- `answer_queries()` 在没有闭包时使用 64 路并行传播，只需 $O(n)$ 额外空间
- 传递约简依赖完整闭包，因此仍需要 $O(n^2/64)$ Bitset 空间
- DAG 的传递约简唯一
- 重边不会影响可达性，约简只会保留其中一条
- 如果边携带权值、容量等额外语义，传递约简只保证可达关系不变，不保证这些额外量保持不变
- 一般有向图先 SCC 缩点后可以处理分量之间的可达关系，但不能把得到的结果理解为原图完整传递约简
- 若只需要普通拓扑 DP，不需要可达矩阵，不应使用完整闭包

复杂度

记

$$B = \left\lceil \frac{n}{64} \right\rceil$$

拓扑排序

$$O(n+m)$$

完整传递闭包

$$O(n + mB)$$

通常写作

$$O\left(\frac{nm}{64} + \frac{n^2}{64}\right)$$

空间复杂度

$$O(nB+m) = O\left(\frac{n^2}{64} + m\right)$$

闭包建立后的单次可达查询

$$O(1)$$

`reachable_count()`

$$O(B)$$

对于 q 个离线询问，不建立完整闭包时

$$O\left(\left\lceil \frac{q}{64} \right\rceil (n+m)\right)$$

额外空间

$$O(n)$$

传递约简在已经具有闭包后，还需要对每个节点的出边按拓扑位置排序，并进行 Bitset 合并

$$O(m \log m + mB)$$

因此包含闭包构造后的总复杂度为

$$O(n + mB + m \log m)$$

总空间仍为

$$O(nB + m)$$

5.2 最短路、环与约束

5.2.1 最短路

算法选择

条件	推荐算法	时间复杂度
DAG，边权任意	拓扑序 DP	$O(n+m)$
所有边权均为 `1`	BFS	$O(n+m)$
边权仅为 `0/1`	0-1 BFS	$O(n+m)$
非负整数边权，最大边权 `C` 很小	Dial	$O(m+nC)$
一般非负边权、稀疏图	堆优化 Dijkstra	$O((n+m)\log n)$
一般非负边权、稠密图	朴素 Dijkstra	$O(n^2)$
存在负边，需要稳定复杂度	Bellman--Ford	$O(nm)$
存在负边，数据或图结构较友好	SPFA	最坏 $O(nm)$
至多经过 `k` 条边	限边 Bellman--Ford	$O(km)$
全源最短路，`n` 较小	Floyd--Warshall	$O(n^3)$
无向简单图最小环，`n` 较小	Floyd 最小环	$O(n^3)$

公共类型

- `INF<T>`：类型 `T` 对应的无穷大值
- `WGraph<T>`：带权邻接表，`g[u]` 中 `{v, w}` 表示边 `u → v`
- `ShortestPathResult<T>::dis`：最短距离
- `ShortestPathResult<T>::par`：一棵最短路树中的父节点
- `restore_path(par, t)`：从 `par` 恢复某个源点到 `t` 的路径，$O(k)$

公共类型与路径恢复

```
template<class T>
inline constexpr T INF = numeric_limits<T>::max() / 4;
template<class T = i64>
using WGraph = vector<vector<pair<int, T>>>;
template<class T>
struct ShortestPathResult {
    vector<T> dis;
    vector<int> par;
};
vector<int> restore_path(const vector<int> &par, int t) {
    vector<int> path;
    for (; t != -1; t = par[t]) path.push_back(t);
    ranges::reverse(path);
    return path;
}
```

BFS

```
template<ranges::input_range R>
ShortestPathResult<int> bfs(const vector<vector<int>> &g, R &&source) {
    int n = g.size();
    vector<int> dis(n, -1), par(n, -1), q;
```

```
    q.reserve(n);
    for (int s : source) if (dis[s] == -1) {
        dis[s] = 0;
        q.push_back(s);
    }
    for (int h = 0; h < q.size(); ++ h) {
        int u = q[h];
        for (int v : g[u]) if (dis[v] == -1) {
            dis[v] = dis[u] + 1;
            par[v] = u;
            q.push_back(v);
        }
    }
    return { move(dis), move(par) };
}
ShortestPathResult<int> bfs(const vector<vector<int>> &g, int s) {
    return bfs(g, array{s});
}
```

- `bfs(g, s)`：单源无权最短路，$O(n+m)$
- `bfs(g, source)`：多源无权最短路，$O(n+m)$
- 不可达点满足 `dis[u] == -1`

堆优化 Dijkstra

```
template<class T, ranges::input_range R>
ShortestPathResult<T> dijkstra(const WGraph<T> &g, R &&source) {
    int n = g.size();
    vector<T> dis(n, INF<T>);
    vector<int> par(n, -1);
    priority_queue<pair<T, int>, vector<pair<T, int>>, greater<>> q;
    for (int s : source) if (dis[s] == INF<T>) {
        dis[s] = T{};
        q.push({ T{}, s });
    }
    while (!q.empty()) {
        auto [d, u] = q.top();
        q.pop();
        if (d != dis[u]) continue;
        for (auto [v, w] : g[u]) if (d + w < dis[v]) {
            dis[v] = d + w;
            par[v] = u;
            q.push({ dis[v], v });
        }
    }
    return { move(dis), move(par) };
}
template<class T>
ShortestPathResult<T> dijkstra(const WGraph<T> &g, int s) {
    return dijkstra(g, array{s});
}
```

朴素 Dijkstra

```
template<class T, ranges::input_range R>
ShortestPathResult<T> dijkstra_dense(const vector<vector<T>> &a, R &&source) {
    int n = a.size();
    vector<T> dis(n, INF<T>);
    vector<int> par(n, -1);
    vector<char> vis(n);
    for (int s : source) dis[s] = T{};
    for (int it = 0; it < n; ++ it) {
        int u{ -1 };
        for (int v = 0; v < n; ++ v) {
            if (!vis[v] && (u == -1 || dis[v] < dis[u])) u = v;
        }
        if (u == -1 || dis[u] == INF<T>) break;
        vis[u] = true;
        for (int v = 0; v < n; ++ v) {
            if (a[u][v] != INF<T> && dis[u] + a[u][v] < dis[v]) {
```

```
                dis[v] = dis[u] + a[u][v];
                par[v] = u;
            }
        }
    }
    return { move(dis), move(par) };
}
template<class T>
ShortestPathResult<T> dijkstra_dense(const vector<vector<T>> &a, int s) {
    return dijkstra_dense(a, array{s});
}
```

- `dijkstra(g, s)`：单源非负权最短路
- `dijkstra(g, source)`：多源非负权最短路
- 不可达点满足 `dis[u] == INF<T>`
- `dijkstra_dense(a, ...)`：邻接矩阵版本

0-1 BFS

```
template<ranges::input_range R>
ShortestPathResult<int> zero_one_bfs(const WGraph<int> &g, R &&source) {
    int n = g.size();
    vector<int> dis(n, INF<int>), par(n, -1);
    deque<pair<int, int>> q;
    for (int s : source) if (dis[s] == INF<int>) {
        dis[s] = 0;
        q.push_back({ 0, s });
    }
    while (!q.empty()) {
        auto [d, u] = q.front();
        q.pop_front();
        if (d != dis[u]) continue;
        for (auto [v, w] : g[u]) {
            int nd{ d + w };
            if (nd >= dis[v]) continue;
            dis[v] = nd;
            par[v] = u;
            if (w == 0) q.push_front({ nd, v });
            else q.push_back({ nd, v });
        }
    }
    return { move(dis), move(par) };
}
ShortestPathResult<int> zero_one_bfs(const WGraph<int> &g, int s) {
    return zero_one_bfs(g, array{s});
}
```

- `zero_one_bfs(g, s)`：单源 0-1 最短路
- `zero_one_bfs(g, source)`：多源版本
- `g` 的边权类型固定为 `int`

最典型的是：**某些操作免费，某些操作代价恰好为 1，并且要求总代价最小**
也就是说，看到“最少多少次某种特殊行为”，很容易转成 `0/1` 边权
例如：

- 正常移动代价 `0`，破坏一堵墙代价 `1`，求最少破墙数
  一个非常典型的抽象是：

```
普通操作 → 权值 0
特殊操作 → 权值 1
答案 = 最少特殊操作次数
```

如果所有边都是 `1`，直接 BFS；只要出现 `0` 和 `1` 混合，优先想到 `0-1 BFS`

Dial 最短路

针对边权值域的性能优化

```
template<ranges::input_range R>
ShortestPathResult<i64> dial(const WGraph<int> &g, R &&source, int max_w) {
```

```
    int n = g.size(), B = max_w + 1;
    vector<i64> dis(n, INF<i64>);
    vector<int> par(n, -1);
    vector<vector<pair<i64, int>>> bucket(B);
    i64 cur{ 0 };
    int pending{ 0 };
    for (int s : source) if (dis[s] == INF<i64>) {
        dis[s] = 0;
        bucket[0].push_back({ 0, s });
        ++ pending;
    }
    while (pending) {
        while (bucket[cur % B].empty()) ++ cur;
        auto [d, u] = bucket[cur % B].back();
        bucket[cur % B].pop_back();
        -- pending;
        if (d != dis[u]) continue;
        for (auto [v, w] : g[u]) {
            i64 nd{ d + w };
            if (nd >= dis[v]) continue;
            dis[v] = nd;
            par[v] = u;
            bucket[nd % B].push_back({ nd, v });
            ++ pending;
        }
    }
    return { move(dis), move(par) };
}
ShortestPathResult<i64> dial(const WGraph<int> &g, int s, int max_w) {
    return dial(g, array{s}, max_w);
}
```

- `dial(g, s, max_w)`：单源小整数非负权最短路
- `dial(g, source, max_w)`：多源版本
- `max_w` 是最大**单边权值**

Dial 可以认为是 Dijkstra 在“小整数权值”下的专门优化
适合：

$$0 \le w \le C$$

且 `C` 很小
例如：

- 网格每种地形代价只有 `0..9`
- 道路收费只有 `1..10`
- 状态转移代价只可能是几个很小的整数
- 字符转换费用限定在很小范围
- 图非常大，`log n` 的堆常数比较明显，而 `C` 只有几十
  比如：

```
草地：1
沙地：2
沼泽：5
山地：8
```

这时虽然普通 Dijkstra 完全能做，但 Dial 可以把优先队列替换成桶
它尤其适合这种条件：

$$C \ll \log n \text{ 对应的实际常数}$$

不过比赛里 Dial 没有 Dijkstra 那么泛用。我的建议是：

- `C = 1`：直接 `0-1 BFS`
- `C` 很小：可以考虑 Dial
- `C` 不确定或达到几百、几千甚至更大：直接 Dijkstra 更稳

Bellman–Ford

```
template<class T = i64>
struct BFEdge {
```

```
    int u, v;
    T w;
};
template<class T>
struct BellmanFordResult {
    vector<T> dis;
    vector<int> par;
    bool neg_cycle{};
};
template<class T, ranges::input_range R>
BellmanFordResult<T> bellman_ford(int n, const vector<BFEdge<T>> &edges, R &&source) {
    vector<T> dis(n, INF<T>);
    vector<int> par(n, -1);
    for (int s : source) dis[s] = T{};
    for (int it = 0; it < n; ++ it) {
        bool changed{ false };
        for (auto [u, v, w] : edges) {
            if (dis[u] == INF<T> || dis[u] + w >= dis[v]) continue;
            dis[v] = dis[u] + w;
            par[v] = u;
            changed = true;
        }
        if (!changed) return { move(dis), move(par), false };
        if (it == n - 1) return { move(dis), move(par), true };
    }
    return { move(dis), move(par), false };
}
template<class T>
BellmanFordResult<T> bellman_ford(int n, const vector<BFEdge<T>> &edges, int s) {
    return bellman_ford(n, edges, array{s});
}
```

- `BFEdge<T>{u, v, w}`：边表中的一条有向边
- `bellman_ford(n, edges, source)`：最短路并判断从源集合可达的负环
- `neg_cycle`：是否存在从源集合可达的负环
- `bellman_ford_k_edges(..., k)`：求至多经过 `k` 条边的最短距离

Bellman–Ford 最核心的用途有两个：
允许负边，以及稳定地判断负环
但是复杂度较高：$O(nm)$

典型含义包括：

- 收益可以抵消成本
- 汇率/价格变化
- 奖励边表现为负费用
- 某些状态转换会降低总代价
  只要不存在从源点可达的负环，就存在有限最短路

判断负环

例如：

- 判断是否存在无限套利
- 判断一组差分约束是否矛盾
- 判断图中是否存在可以无限降低代价的循环
- Wormholes / 虫洞类题
- 货币兑换套利
  如果要判断整张图任意位置有没有负环，可以把所有点都当源点，相当于建立超级源：

```
auto source = views::iota(0, n);
auto res = bellman_ford(n, edges, source);
```

限定至多经过 `k` 条边

```
template<class T, ranges::input_range R>
vector<T> bellman_ford_k_edges(int n, const vector<BFEdge<T>> &edges, R &&source, int k) {
    vector<T> dis(n, INF<T>);
    for (int s : source) dis[s] = T{};
    while (k --) {
```

```
        auto pre = dis;
        for (auto [u, v, w] : edges) if (pre[u] != INF<T>) {
            dis[v] = min(dis[v], pre[u] + w);
        }
    }
    return dis;
}
template<class T>
vector<T> bellman_ford_k_edges(int n, const vector<BFEdge<T>> &edges, int s, int k) {
    return bellman_ford_k_edges(n, edges, array{s}, k);
}
```

> 路径长度（边数）受到限制的最短路

最典型场景就是“最多经过多少次转移”
例如：

- 最多乘坐 `k` 段航班
- 最多中转 `k` 次
- 在 `k` 天 / `k` 回合 / `k` 步以内到达目标。
- 有负边，但只允许走有限步，因此即使存在负环，也不能无限绕
  例如：

```
从城市 s 到城市 t
最多乘坐 5 班飞机
求最低费用
```

这时普通最短路是不够的，因为普通最短路只优化：

$$\text{总代价}$$

而题目还有：

$$\text{边数} \le k$$

的约束
Bellman–Ford 每一轮天然对应：

$$\text{最多再使用一条边}$$

所以执行 `k` 轮即可

与“状态加一维”DP 的关系

它本质上就是：

$$dp[t][v] = \text{至多使用 } t \text{ 条边到达 } v \text{ 的最短距离}$$

Bellman–Ford 只是把第一维滚动掉了
所以看到：

```
最多 k 次
恰好 k 次
不超过 k 条边
```

应该想到这一类 DP/限边最短路

SPFA

差分约束 + 某些实际比较友好的负权图

```
template<class T>
struct SPFAResult {
    vector<T> dis;
    vector<int> par;
    bool neg_cycle{};
};
template<class T, ranges::input_range R>
SPFAResult<T> spfa(const WGraph<T> &g, R &&source) {
    int n = g.size();
    vector<T> dis(n, INF<T>);
    vector<int> par(n, -1), len(n);
    vector<char> inq(n);
    deque<int> q;
```

```
    for (int s : source) if (dis[s] == INF<T>) {
        dis[s] = T{};
        inq[s] = true;
        q.push_back(s);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop_front();
        inq[u] = false;
        for (auto [v, w] : g[u]) if (dis[u] + w < dis[v]) {
            dis[v] = dis[u] + w;
            par[v] = u;
            len[v] = len[u] + 1;
            if (len[v] >= n) return { move(dis), move(par), true };
            if (inq[v]) continue;
            inq[v] = true;
            if (!q.empty() && dis[v] < dis[q.front()]) q.push_front(v);
            else q.push_back(v);
        }
    }
    return { move(dis), move(par), false };
}
template<class T>
SPFAResult<T> spfa(const WGraph<T> &g, int s) {
    return spfa(g, array{s});
}
```

- `spfa(g, source)`：允许负边，并判断从源集合可达的负环
- `len[u]` 维护当前改进路径所含边数
- `neg_cycle == true` 时最短距离不再具有最终意义

差分约束

这是 SPFA 最经典的用途之一
例如约束：

$$x_v - x_u \le w$$

转换成：

$$x_v \le x_u + w$$

对应一条边：

$$u \to v, \quad w$$

然后求一组满足约束的势能
常见形式：

$$x_i - x_j \le c$$

建边：

```
j → i，权值 c
```

因此 SPFA 常见于：

- 一堆变量大小关系
- 区间前缀和约束
- 排程约束
- 最早/最晚时间
- 连续整数选择问题
- 线性不等式约束
- 判断一组约束是否矛盾
  如果出现负环，则通常意味着约束系统不可行

为什么差分约束常用 SPFA

因为这类图往往：

- 比较稀疏
- 边结构特殊
- 实际运行表现通常不错

- 需要顺便判断负环
  于是代码非常自然
  不过不能忘记：

$$\text{SPFA 最坏仍然是 } O(nm)$$

Floyd–Warshall

```
template<class T>
vector<vector<T>> floyd(vector<vector<T>> dis) {
    int n = dis.size();
    for (int i = 0; i < n; ++ i) dis[i][i] = min(dis[i][i], T{});
    for (int k = 0; k < n; ++ k) {
        for (int i = 0; i < n; ++ i) if (dis[i][k] != INF<T>) {
            for (int j = 0; j < n; ++ j) if (dis[k][j] != INF<T>) {
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
            }
        }
    }
    return dis;
}
```

Floyd 最小环

```
template<class T>
pair<vector<vector<T>>, T> floyd_min_cycle(const vector<vector<T>> &a) {
    int n = a.size();
    vector dis = a;
    for (int i = 0; i < n; ++ i) dis[i][i] = T{};
    T cycle = INF<T>;
    for (int k = 0; k < n; ++ k) {
        // 此时 dis[i][j] 只允许编号 < k 的点作为中间点
        for (int i = 0; i < k; ++ i) {
            for (int j = i + 1; j < k; ++ j) {
                if (dis[i][j] == INF<T> ||
                    a[i][k] == INF<T> ||
                    a[k][j] == INF<T>) continue;
                cycle = min(cycle, dis[i][j] + a[i][k] + a[k][j]);
            }
        }
        for (int i = 0; i < n; ++ i) if (dis[i][k] != INF<T>) {
            for (int j = 0; j < n; ++ j) if (dis[k][j] != INF<T>) {
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
            }
        }
    }
    return { move(dis), cycle };
}
```

- `floyd(dis)`：全源最短路
- 不存在边的位置初始化为 `INF<T>`
- `floyd_min_cycle(a)`：返回 `{全源最短路, 最小简单环长度}`

注意事项

- 所有点编号均为 `0base`
- 多源版本可以直接传 `vector<int>`、`array<int, N>`、`views::iota(...)` 等 range
- 多源最短路等价于建立一个到所有源点有零权边的超级源，但无需真的建立该点
- `restore_path()` **不会判断目标是否可达**；应先检查对应 `dis[t]`
- 若存在可达负环，不要对 Bellman–Ford / SPFA 返回的 `par` 直接恢复普通最短路
- `INF<T>` 只是哨兵。所有 `dis[u] + w` 必须保证真实数值不会溢出 `T`；极端负权或负环场景需要考虑扩大到 `i128`
- Dijkstra 要求所有边权非负；零权边可以正常处理
- 0-1 BFS 要求每条边权严格属于 `{0, 1}`
- Dial 要求所有边权是整数且满足 `0 ≤ w ≤ max_w`；`max_w` 很大时应改用 Dijkstra
- SPFA 的 SLF 仅优化常数，没有改善最坏复杂度；一般非负权图不要使用 SPFA
- Bellman–Ford 判断的只是**源集合可达的负环**。判断整张图是否存在负环时，把所有点作为源即可
- `bellman_ford_k_edges()` 每轮必须从上一轮的 `pre` 松弛；若直接原地连续传播，就不再满足“至多 `k` 条边”的限制
- Floyd 可以处理负边；若最终存在 `dis[i][i] < 0`，则存在负环
- Floyd 最小环版本只针对无向图，且 `a[i][j] == a[j][i]`

- 调用 `floyd_min_cycle()` 前，`a[i][i]` 应保持 `INF<T>`；函数内部会单独把 `dis[i][i]` 设为 `0`
- Floyd 最小环只统计至少包含 `3` 个不同点的简单环。自环、两条平行边构成的二元环需要建图时单独处理

常见用法

建立带权图

```
WGraph g(n);
g[u].push_back({ v, w });
g[v].push_back({ u, w }); // 无向图
```

单源 Dijkstra

```
auto [dis, par] = dijkstra(g, s);
if (dis[t] != INF<i64>) {
    auto path = restore_path(par, t);
}
```

多源最短路

```
vector<int> source{ a, b, c };
auto [dis, par] = dijkstra(g, source);
```

也可以直接使用 range：

```
auto res = bellman_ford(n, edges, views::iota(0, n));
bool has_negative_cycle = res.neg_cycle;
```

这等价于所有点同时作为源点，因此可以判断整张图是否存在负环

0-1 BFS

```
WGraph<int> g(n);
g[u].push_back({ v, 0 });
g[x].push_back({ y, 1 });
auto [dis, par] = zero_one_bfs(g, s);
```

Dial

```
int max_w = 10;
auto [dis, par] = dial(g, s, max_w);
```

至多经过 `k` 条边

```
vector<BFEdge<>> edges;
edges.push_back({ u, v, w });
auto dis = bellman_ford_k_edges(n, edges, s, k);
```

Floyd

```
vector dis(n, vector<i64>(n, INF<i64>));
for (int i = 0; i < n; ++ i) dis[i][i] = 0;
for (auto [u, v, w] : edges) {
    dis[u][v] = min(dis[u][v], w);
}
dis = floyd(move(dis));
```

Floyd 判断负环

```
bool neg_cycle = false;
for (int i = 0; i < n; ++ i) {
    neg_cycle |= dis[i][i] < 0;
}
```

Floyd 最小环

```
vector a(n, vector<i64>(n, INF<i64>));
for (auto [u, v, w] : edges) {
    a[u][v] = min(a[u][v], w);
    a[v][u] = min(a[v][u], w);
}
auto [dis, cycle] = floyd_min_cycle(a);
if (cycle == INF<i64>) {
    // 不存在至少含三个不同点的简单环
}
```

复杂度

设点数为 `n`，边数为 `m`，Dial 最大单边权为 `C`

算法	时间复杂度	额外空间
BFS	$O(n+m)$	$O(n)$
堆优化 Dijkstra	$O((n+m)\log n)$	$O(n+m)$
朴素 Dijkstra	$O(n^2)$	$O(n)$
0-1 BFS	$O(n+m)$	$O(n+m)$
Dial	$O(m+nC)$	$O(n+m+C)$
Bellman–Ford	$O(nm)$	$O(n)$
限边 Bellman–Ford	$O(km)$	$O(n)$
SPFA	最坏 $O(nm)$	$O(n)$
Floyd–Warshall	$O(n^3)$	$O(n^2)$
Floyd 最小环	$O(n^3)$	$O(n^2)$

恢复一条包含 `k` 个点的路径：

$$O(k)$$

5.2.2 同余最短路

```
struct ModShortestPath {
    static constexpr i64 INF = numeric_limits<i64>::max();
    struct Step {
        int add;
        i64 cost;
    };
    i64 g{ 1 }, base{ 0 };
    int mod{ 0 };
    vector<Step> step;
    vector<i64> dist;
    i64 max_dist{ 0 };
    explicit ModShortestPath(const vector<i64> &weights) {
        assert(!weights.empty());
        assert(ranges::all_of(weights, [](i64 w) { return w > 0; }));
        base = ranges::min(weights);
        g = 0;
        for (i64 w : weights) g = gcd(g, w);
        i64 m{ base / g };
        assert(m <= INT_MAX);
        mod = int(m);
        // ===== 压缩转移：同余增量相同时，只保留代价最小的权值 =====
        {
            vector<i64> best(mod, INF);
            for (i64 w : weights) {
                int add = int((w / g) % mod);
                if (add && w < best[add]) best[add] = w;
            }
            for (int add : views::iota(1, mod)) {
                if (best[add] != INF) step.push_back({ add, best[add] });
            }
        }
        // ===== 余数图 Dijkstra：以下代码是核心，不建议删除 =====
```

```
        dist.assign(mod, INF);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
        dist[0] = 0;
        q.push({ 0, 0 });
        while (!q.empty()) {
            auto [d, r] = q.top();
            q.pop();
            if (d != dist[r]) continue;
            for (auto [add, w] : step) {
                int nr = int((i64(r) + add) % mod);
                __int128_t nd = (__int128_t)d + w;
                if (nd < dist[nr]) {
                    dist[nr] = i64(nd);
                    q.push({ dist[nr], nr });
                }
            }
        }
        max_dist = *ranges::max_element(dist);
    }
    // ===== 基础查询 =====
    i64 min_value(i64 residue) const {
        residue %= base;
        if (residue < 0) residue += base;
        if (residue % g) return INF;
        return dist[residue / g];
    }
    bool can_make(i64 x) const {
        if (x < 0 || x % g) return false;
        i64 d = dist[(x / g) % mod];
        return d != INF && d <= x;
    }
    u64 count_leq(i64 x) const {
        if (x < 0) return 0;
        u64 ans = 0;
        for (i64 d : dist) if (d != INF && d <= x) {
            ans += u64((x - d) / base) + 1;
        }
        return ans;
    }
    u64 count_range(i64 l, i64 r) const {
        if (l > r || r < 0) return 0;
        l = max<i64>(l, 0);
        return count_leq(r) - (l ? count_leq(l - 1) : 0);
    }
    // ===== 数值半群扩展：若不需要最大不可达数，可以删除 =====
    optional<i64> largest_unreachable() const {
        if (g != 1 || max_dist == INF) return nullopt;
        return max_dist - base;
    }
};
```

成员变量

- `base`：原始权值中的最小值；所有同余类最终都以 `base` 为周期继续增长
- `g`：所有权值的最大公约数
- `mod = base / g`：实际余数图点数；自动约去 `g` 后通常显著小于原始最小权值
- `step`：压缩后的有效转移
- `dist[r]`：满足

$$\frac{x}{g} \equiv r \pmod{mod}$$

的可达非负整数 `x` 中的最小值，仍使用原始数值尺度

- `max_dist`：所有余数类最小可达值的最大值
例如权值为 `{6, 10}`：

$$g = 2, \quad base = 6, \quad mod = \frac{6}{2} = 3$$

内部只建立 `3` 个状态，而查询仍然直接使用原来的 `6、10、20...`，无需手动缩放

接口

- `ModShortestPath(weights)`：构造并运行余数图 Dijkstra；每种正整数权值可以使用任意非负次数
- `min_value(r)`：返回模 `base` 余数为 `r` 的最小可达非负整数；该余数不可能出现时返回 `INF`
- `can_make(x)`：判断非负整数 `x` 是否可以表示
- `count_leq(x)`：返回 `u64`（包含 0，最大可能为 2^63）；统计闭区间 `[0,x]` 中可表示的整数个数
- `count_range(l,r)`：返回 `u64`；统计闭区间 `[l,r]` 中可表示的整数个数
- `largest_unreachable()`：当 `g == 1` 时返回最大不可表示非负整数；`g != 1` 时返回 `nullopt`

建模

设所有权值为

$$a_1, a_2, \ldots, a_k$$

并令

$$g = \gcd(a_1, a_2, \ldots, a_k), \qquad base = \min a_i, \qquad mod = \frac{base}{g}$$

所有可达整数一定是 `g` 的倍数，因此先在状态意义上整体除去 `g`
建立 `mod` 个状态：

$$0, 1, \ldots, mod - 1$$

对于权值 `w`，定义：

$$\Delta = \frac{w}{g} \bmod mod$$

则从任意状态 `r` 存在转移：

$$r \to (r + \Delta) \bmod mod$$

边权仍然使用原始的 `w`
因此 `dist[r]` 就是该同余类中能够组成的最小原始整数
因为 `base` 本身可以无限次使用，一旦 `dist[r]` 可达：

$$dist[r], \quad dist[r] + base, \quad dist[r] + 2base, \quad \ldots$$

全部可达
所以：

$$x \text{ 可达} \iff x \equiv 0 \pmod{g} \quad \text{且} \quad dist[(x/g) \bmod mod] \le x$$

对于某个余数类，若 `dist[r] ≤ x`，则 `[0,x]` 中属于这一类的可达数共有：

$$\left\lfloor \frac{x - dist[r]}{base} \right\rfloor + 1$$

将所有余数类累加即可得到 `count_leq(x)`

为什么可以压缩 `step`

若两个权值 `x < y` 满足：

$$\frac{x}{g} \equiv \frac{y}{g} \pmod{mod}$$

那么它们从任何状态出发都会到达相同的下一个状态
而 `x` 的代价严格更小，因此使用 `y` 的转移不可能出现在最短路中，可以直接删除
此外：

$$w \equiv 0 \pmod{base}$$

对应余数图中的自环，也不可能改善最短距离，因此全部删除
最终有效转移数满足：

$$|step| \le mod - 1$$

注意事项

- 这是完全背包可达性：每个权值可以使用无限次。0/1 背包或有次数上限时不能使用
- 所有权值必须严格为正。`0` 对可达集合没有贡献，负权会破坏“最小值后不断加 `base`”的结构
- 自动约去 `g` 只压缩内部状态，所有接口仍使用原始整数，无需手动对询问除以 `g`
- 实际空间取决于

  $$mod = \frac{\min a_i}{\gcd(a_1, \ldots, a_k)}$$

  而不是单纯取决于最小权值
- 因此 `{10^9, 2\cdot10^9}` 实际只有一个状态；自动约 gcd 比直接以 `10^9` 为模重要得多
- 当约 gcd 后的 `mod` 仍达到几百万甚至更大时，余数图可能无法接受，应重新寻找题目的其他结构

- `dist` 使用 `i64`；代码在松弛时通过 `__int128_t` 避免加法本身溢出，但有限最小可达值必须严格小于 `INF=LLONG_MAX`（该值保留作不可达哨兵）
- `count_leq()` 包含 `0`，因为空选任何权值即可得到 `0`
- `count_range()` 已经处理负左端点和 `l - 1` 下溢问题，区间统计优先直接调用它
- `g > 1` 时有无穷多个整数不可达，例如所有非 `g` 倍数，因此不存在通常意义上的"最大不可达整数"
- `g == 1` 时所有充分大的整数都可达，并且最大不可达整数为

$$\max_r dist[r] - base$$

  即 `largest_unreachable()`
- 若 `base == 1`，所有非负整数均可达，此时 `largest_unreachable()` 返回 `-1`
- `min_value(r)` 中的 `r` 是**原整数模 `base` 的余数**，不是内部压缩后的状态编号
- 无需显式建立余数图邻接表；所有边都由 `step` 即时枚举

常见用法

判断是否可表示

```
vector<i64> weights(k);
for (i64 &w : weights) cin >> w;
ModShortestPath solver(weights);
i64 x;
cin >> x;
cout << (solver.can_make(x) ? "YES\n" : "NO\n");
```

区间内有多少个可表示整数

```
i64 l, r;
cin >> l >> r;
cout << solver.count_range(l, r) << '\n';
```

例如权值：

```
ModShortestPath solver({3, 5});
```

可达集合开始为：

```
0, 3, 5, 6, 8, 9, 10, ...
```

所以：

```
solver.can_make(7);        // false
solver.can_make(8);        // true
solver.count_range(0, 8);  // 5
```

gcd 自动压缩

```
ModShortestPath solver({6, 10});
cout << solver.g << '\n';    // 2
cout << solver.base << '\n'; // 6
cout << solver.mod << '\n';  // 3
solver.can_make(10); // true
solver.can_make(20); // true
solver.can_make(11); // false
```

不需要再手动写：

```
for (i64 &w : weights) w /= g;
x /= g;
```

查询某个原始余数类的最小可达值

```
ModShortestPath solver({6, 10});
solver.min_value(0); // 0
solver.min_value(2); // 20
solver.min_value(4); // 10
solver.min_value(1); // INF
```

这里余数均针对原始的：

$$6 \bmod 6$$

最大不可达整数

```
ModShortestPath solver({3, 5});
auto x = solver.largest_unreachable();
if (x) cout << *x << '\n'; // 7
```

若：

```
ModShortestPath solver({4, 6});
```

则 `g == 2`，所有奇数永远不可达：

```
auto x = solver.largest_unreachable();
assert(!x);
```

复杂度

设：

$$k = \text{输入权值数量}$$

约去最大公约数后的状态数为：

$$s = \frac{\min a_i}{\gcd(a_1, \dots, a_k)}$$

压缩后的有效转移数为 `t = step.size()`，并且：

$$t \le \min(k-1, s-1)$$

构造时：

- 求 `gcd` 与建立有效转移：$O(k+s)$
- 余数图 Dijkstra：

  $$O(st \log s)$$

  因此总时间复杂度为：

  $$O(k + s + st \log s)$$

  通常简写为：

  $$O(st \log s)$$

  空间复杂度：

  $$O(s + t)$$

  查询复杂度：

接口	时间复杂度
`min_value()` / `can_make()` / `largest_unreachable()`	$O(1)$
`count_leq()` / `count_range()`	$O(s)$

其中 `largest_unreachable()` 的最大 `dist` 已在构造结束时预处理

5.2.3 次短路及 K 短路

依赖「最短路基础」中的：

```
template<class T>
inline constexpr T INF = numeric_limits<T>::max() / 4;

template<class T = i64>
using WGraph = vector<vector<pair<int, T>>>;
```

严格次短路

```
pair<vector<i64>, vector<i64>> second_shortest(const WGraph<i64> &g, int s) {
    int n = g.size();
    vector<i64> d1(n, INF<i64>), d2(n, INF<i64>);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
    d1[s] = 0;
    q.push({ 0, s });
    while (!q.empty()) {
        auto [d, u] = q.top();
        q.pop();
        if (d > d2[u]) continue;
        for (auto [v, w] : g[u]) {
            if (d > INF<i64> - w) continue;
            i64 nd{ d + w };
            if (nd < d1[v]) {
                swap(nd, d1[v]);
                q.push({ d1[v], v });
            }
            if (d1[v] < nd && nd < d2[v]) {
                d2[v] = nd;
                q.push({ d2[v], v });
            }
        }
    }
    return { move(d1), move(d2) };
}
```

A* 前 k 短 Walk

```
vector<i64> k_shortest_walks(const WGraph<i64> &g, int s, int t, int k) {
    if (k <= 0) return {};
    int n = g.size();
    // ===== 反图 Dijkstra: 得到 h[u] = u -> t 的最短距离 =====
    WGraph<i64> rg(n);
    for (int u : views::iota(0, n)) {
        for (auto [v, w] : g[u]) rg[v].push_back({ u, w });
    }
    vector<i64> h(n, INF<i64>);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> dq;
    h[t] = 0;
    dq.push({ 0, t });
    while (!dq.empty()) {
        auto [d, u] = dq.top();
        dq.pop();
        if (d != h[u]) continue;
        for (auto [v, w] : rg[u]) {
            if (d > INF<i64> -w || d + w >= h[v]) continue;
            h[v] = d + w;
            dq.push({ h[v], v });
        }
    }
    if (h[s] == INF<i64>) return {};
    // ===== A*: 每个点最多有效弹出 limit 次 =====
    using State = tuple<i64, i64, int>; // f = d + h[u], d, u
    priority_queue<State, vector<State>, greater<>> q;
    vector<int> pop(n);
    vector<i64> ans;
    ans.reserve(k);
    // s == t 时第一次弹出的是长度 0 的空 walk，不计入答案。
    int limit = k + (s == t);
    q.push({ h[s], 0, s });
    while (!q.empty() && ans.size() < k) {
        auto cur = q.top();
        q.pop();
        i64 d{ get<1>(cur) };
        int u{ get<2>(cur) };
        if (++ pop[u] > limit) continue;
        if (u == t && !(s == t && pop[u] == 1)) ans.push_back(d);
        for (auto [v, w] : g[u]) {
            if (h[v] == INF<i64> || pop[v] >= limit) continue;
            if (d > INF<i64> -w) continue;
```

```
            i64 nd = d + w;
            if (nd > INF<i64> -h[v]) continue;
            q.push({ nd + h[v], nd, v });
        }
    }
    return ans;
}
i64 kth_shortest_walk(const WGraph<i64> &g, int s, int t, int k) {
    if (k ⩽ 0) return -1;
    auto ans = k_shortest_walks(g, s, t, k);
    return ans.size() == k ? ans.back() : -1;
}
```

接口说明

`second_shortest(g, s)`

返回：

```
auto [d1, d2] = second_shortest(g, s);
```

其中：

- `d1[v]`：`s → v` 的最短 walk 长度
- `d2[v]`：严格满足

$$d_2[v] > d_1[v]$$

  的最小 walk 长度
- 不存在时为 `INF<i64>`
  因此两条长度相同但经过边不同的最短路，不会使 `d2[v] = d1[v]`

`k_shortest_walks(g, s, t, k)`

返回前 `k` 条非空 `s → t` walk 的长度，按非降序排列
这里的 walk：

- 可以重复经过点
- 可以重复经过边
- 不同边序列分别计数
- 长度相同的不同 walk 会分别占据一个名次
  若不足 `k` 条，返回实际存在的全部结果

`kth_shortest_walk(g, s, t, k)`

返回第 `k` 短非空 walk 长度，`k` 从 `1` 开始
不存在时返回 `-1`

注意事项

- 两个算法都要求所有边权非负
- 零权边允许存在
- `second_shortest()` 求的是严格次短长度，不是“第二条不同路径”
- `k_shortest_walks()` 求的是 walk，不是简单路径
- 如果图中存在零权环，可能存在无限多条长度相同的 walk；这些 walk 仍会分别占据 K 短排名
- 若题目要求“不允许重复点”的第 `k` 短简单路径，应使用 Yen、Eppstein 等其他模型
- `s == t` 时默认排除长度为 `0` 的空 walk，因此返回的是非空闭合 walk
- 反图在函数内部自动建立，调用者只维护 `g`，避免 `g / rg` 不一致
- 若同一终点 `t` 上要进行很多次 K 短查询，可以把反图与 `h[]` 单独预处理并封装，避免重复 Dijkstra
- 所有有限路径长度应小于 `INF<i64>`

常见用法

严格次短路

```
WGraph<i64> g(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    g[u].push_back({ v, w });
}
```

```
auto [d1, d2] = second_shortest(g, s);
if (d2[t] == INF<i64>) {
    // 不存在严格次短 walk
} else {
    cout << d2[t] << '\n';
}
```

前 `k` 短 walk

```
auto ans = k_shortest_walks(g, s, t, k);
for (i64 d : ans) cout << d << '\n';
```

只要第 `k` 短

```
i64 ans = kth_shortest_walk(g, s, t, k);
if (ans == -1) {
    // 不足 k 条
}
```

复杂度

严格次短路：

$$O((n+m)\log n)$$

额外空间：

$$O(n+m)$$

A* 首先进行一次反图 Dijkstra：

$$O((n+m)\log n)$$

之后每个点最多被有效扩展 $O(k)$ 次，因此常用上界为：

$$O(km\log(km))$$

最坏候选状态空间：

$$O(km)$$

5.2.4 Karp 最小平均环

```
optional<ld>
minimum_mean_cycle(int n, const vector<tuple<int, int, i64>> &edges) {
    if (n == 0) return nullopt;
    using i128 = __int128_t;
    const i128 INF = i128(1) << 120;
    // dp[k][v]:
    // 从任意起点出发，恰好走 k 条边到达 v 的最小权值。
    //
    // 不保存全部 dp。
    // 第一遍滚动得到 dp[n]，第二遍重新计算 dp[0..n-1]
    // 并直接代入 Karp 公式，因此额外空间 O(n)。
    vector<i128> pre(n), cur(n), dn;
    // ===== 第一遍：求 dp[n] =====
    for (int len = 1; len <= n; ++ len) {
        ranges::fill(cur, INF);
        for (auto [u, v, w] : edges) {
            if (pre[u] == INF) continue;
            cur[v] = min(cur[v], pre[u] + w);
        }
        swap(pre, cur);
    }
    dn = pre;
    // ===== 第二遍：重新计算 dp[k] 并维护 Karp 公式 =====
    const ld LINF = numeric_limits<ld>::infinity();
    vector<ld> best(n, -LINF);
    ranges::fill(pre, i128{});
    for (int len = 0; len < n; ++ len) {
        for (int v : views::iota(0, n)) {
```

```
            if (dn[v] == INF || pre[v] == INF) continue;
            best[v] = max(best[v], ld(dn[v] - pre[v]) / (n - len));
        }
        if (len + 1 == n) break;
        ranges::fill(cur, INF);
        for (auto [u, v, w] : edges) {
            if (pre[u] == INF) continue;
            cur[v] = min(cur[v], pre[u] + w);
        }
        swap(pre, cur);
    }
    ld ans{ LINF };
    for (int v : views::iota(0, n)) {
        if (dn[v] != INF) ans = min(ans, best[v]);
    }
    if (isinf(ans)) return nullopt;
    return ans;
}
```

接口说明

`minimum_mean_cycle(n, edges)` 求整张有向图中所有有向环的最小平均边权
一条环 C 的平均边权定义为：

$$\mu(C) = \frac{\sum_{e \in C} w_e}{|C|}$$

返回：

```
optional<ld>
```

- 有环：返回最小平均边权
- 无任何有向环：返回 `nullopt`
  边格式：

```
{u, v, w}
```

表示：

$$u \to v$$

边权为 `w`
边权允许为负

原理

定义：

$$dp_k(v)$$

为从任意起点出发，恰好经过 `k` 条边到达 `v` 的最小权值
相当于存在一个超级源，以权值 `0` 连向所有原图节点，因此：

$$dp_0(v) = 0$$

转移：

$$dp_k(v) = \min_{(u,v,w) \in E} (dp_{k-1}(u) + w)$$

Karp 定理给出最小平均环：

$$\boxed{\min_{v} \max_{0 \le k < n} \frac{dp_n(v) - dp_k(v)}{n - k}}$$

其中只考虑 `dp_n(v)` 有限的节点

为什么 `dp[n]` 有限意味着存在环

一个拥有 `n` 个节点的图中，长度恰好为 `n` 的 walk 一定访问至少 `n+1` 个节点位置
因此必然有某个节点重复出现，从而包含有向环
无环图中：

$$dp_n(v) = +\infty$$

对所有 `v` 均成立

为什么只需要 O(n) DP 空间

公式只需要：

- 最终的 dp[n][v]
- 当前正在枚举的 dp[k][v]
  因此：

1. 第一遍滚动 DP 得到 dp[n]
2. 第二遍从 dp[0] 重新滚动
3. 每得到一层 dp[k]，立即更新对应比值
   无需保存：

$$dp[0], dp[1], \ldots, dp[n]$$

注意事项

- 这是有向环的最小平均边权
- 图可以不连通，不需要提前建立超级源；dp[0][v] = 0 已经等价于所有点均可作为起点
- 自环正常参与计算，其平均边权就是自身边权
- 重边正常处理
- 边权允许为负
- 最终答案使用 long double，输出时应设置足够精度
- 该函数只返回最小平均值，不恢复具体环
- 若题目要求最大平均环，可以将所有边权取反后求最小平均环，再把答案取反；注意 LLONG_MIN 不能直接取负
- 若只需要判断：

$$\mu_{\min} < x$$

  也可以考虑把每条边权改为 w-x 后判断负环，配合二分，但这是另一种实现路线

常见用法

```
vector<tuple<int, int, i64>> edges;
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    edges.push_back({ u, v, w });
}
auto ans = minimum_mean_cycle(n, edges);
if (!ans) {
    // 图中不存在有向环
} else {
    cout << fixed << setprecision(12) << *ans << '\n';
}
```

最大平均环

若所有权值都能安全取负：

```
for (auto &[u, v, w] : edges) w = -w;
auto ans = minimum_mean_cycle(n, edges);
if (ans) {
    ld maximum_mean = -*ans;
}
```

复杂度

两遍 DP：

$$O(nm)$$

计算 Karp 公式：

$$O(n^2)$$

因此本实现严格写作：

$$O(nm + n^2)$$

对于边数满足 $m = \Omega(n)$ 的图，可简写为：

$$O(nm)$$

DP 额外空间：

$$O(n)$$

加上输入边表后的总空间：

$$O(n+m)$$

相比保存完整二维 DP 的版本：

$$O(n^2)$$

空间下降为：

$$O(n)$$

5.2.5 差分约束

```
struct DifferenceConstraints {
    struct Edge {
        int u, v;
        i128 w;
    };
    int n;
    vector<Edge> edges;
    explicit DifferenceConstraints(int n_) : n(n_) {}
    // ===== 基础约束接口 =====
    // x[v] - x[u] ≤ w
    void add_leq(int u, int v, i128 w) { edges.push_back({ u, v, w }); }
    // x[v] - x[u] ≥ w
    void add_geq(int u, int v, i64 w) { add_leq(v, u, -i128(w)); }
    // x[v] - x[u] == w
    void add_eq(int u, int v, i64 w) {
        add_leq(u, v, w);
        add_geq(u, v, w);
    }
    // lo ≤ x[v] - x[u] ≤ hi
    void add_range(int u, int v, i64 lo, i64 hi) {
        add_geq(u, v, lo);
        add_leq(u, v, hi);
    }
    // ===== Bellman-Ford: 判断可行性并返回一组解 =====
    optional<vector<i64>> feasible() const {
        vector<i128> x(n);
        if (n == 0) return vector<i64>{};
        // 全部初始化为 0，等价于建立超级源：
        // source → v，边权为 0。
        for (int it = 0; it < n; ++ it) {
            bool changed{ false };
            for (auto [u, v, w] : edges) {
                if (x[v] ≤ x[u] + w) continue;
                x[v] = x[u] + w;
                changed = true;
            }
            if (!changed) {
                vector<i64> ans(n);
                for (int u = 0; u < n; ++ u) {
                    assert(numeric_limits<i64>::min() ≤ x[u] && x[u] ≤ numeric_limits<i64>::max());
                    ans[u] = i64(x[u]);
                }
                return ans;
            }
        }
        // 第 n 轮仍能松弛 ⟺ 存在负环 ⟺ 约束无解
        return nullopt;
    }
};
```

接口说明

差分约束处理形如：

$$x_v - x_u \le w$$

的不等式系统
标准变形：

$$x_v \le x_u + w$$

对应最短路约束边：

$$u \to v, \qquad w$$

即：

```
dc.add_leq(u, v, w);
```

常用接口

小于等于

$$x_v - x_u \le w$$

```
dc.add_leq(u, v, w);
```

大于等于

$$x_v - x_u \ge w$$

等价于：

$$x_u - x_v \le -w$$

```
dc.add_geq(u, v, w);
```

等于

$$x_v - x_u = w$$

```
dc.add_eq(u, v, w);
```

即同时加入：

$$x_v - x_u \le w$$

和：

$$x_v - x_u \ge w$$

区间限制

$$lo \le x_v - x_u \le hi$$

```
dc.add_range(u, v, lo, hi);
```

`feasible()`

```
auto ans = dc.feasible();
```

返回：

- `nullopt`：约束系统无解
- `vector<i64>`：一组满足所有约束的可行解
  因为差分约束只涉及变量之间的差：

$$(x_v + C) - (x_u + C) = x_v - x_u$$

所以所有变量同时加同一个常数后仍然是解，返回解通常不唯一

常见约束翻译

题目约束	调用
$x_v - x_u \le w$	`add_leq(u, v, w)`
$x_v - x_u \ge w$	`add_geq(u, v, w)`
$x_v - x_u = w$	`add_eq(u, v, w)`
$lo \le x_v - x_u \le hi$	`add_range(u, v, lo, hi)`

若变量限定为整数：

$$x_v - x_u < w$$

等价于：

$$x_v - x_u \le w - 1$$

而：

$$x_v - x_u > w$$

等价于：

$$x_v - x_u \ge w + 1$$

这一转换只适用于整数变量

绝对上下界

差分约束本身只描述变量差
若需要表示：

$$x_v \le c$$

增加一个基准变量 `z`，写成：

$$x_v - x_z \le c$$

即：

```
dc.add_leq(z, v, c);
```

类似地：

$$x_v \ge c$$

写为：

```
dc.add_geq(z, v, c);
```

求出一组解后：

```
i64 shift = x[z];
for (i64 &v : x) v -= shift;
```

即可令：

$$x_z = 0$$

而不改变任何差分约束

建模原理

每条约束：

$$x_v \le x_u + w$$

都与最短路中的松弛条件：

$$dis[v] \le dis[u] + w$$

完全一致
因此若图中不存在负环，Bellman--Ford 最终得到的势能就是一组可行解
反之，若存在负环：

$$v_0 \to v_1 \to \cdots \to v_k = v_0$$

且环权和为负，把所有约束相加会得到：

$$x_{v_0} < x_{v_0}$$

矛盾，因此约束无解

为什么不显式建立超级源

通常会建立超级源 `S`：

$$S \to i, \quad 0$$

连接每个变量
本实现直接：

```
vector<i64> x(n);
```

令所有变量初始为 `0`
这与从超级源执行 Bellman--Ford 完全等价，可以少建一个点和 `n` 条边

注意事项

- 当前版本解决的是**可行性**，不是一般线性规划
- 每条约束必须能转成两个变量之差的上下界
- 诸如

  $$x + y \le c$$

  或：

  $$xy \le c$$

  不能直接转成普通差分约束
- `feasible()` 使用 Bellman--Ford，复杂度稳定，不依赖 SPFA 的数据表现
- 全零初始化已经等价于超级源，因此能够检查整张约束图中的任何负环
- 返回解可能为负数，这是正常现象
- 如果题目只要求变量之间的相对大小，可以直接使用返回值
- 如果某个变量必须固定为 `0`，使用基准变量建模并在求解后整体平移
- `add_geq()` 内部会计算 `-w`；不要传入无法安全取负的 `LLONG_MIN`
- 内部用 `i128` 松弛并对下界先扩展类型再取负，避免负环重复松弛和 `LLONG_MIN` 取负溢出；返回的一组势能仍要求位于 `i64` 范围，超出时触发断言
- 实数严格不等式不能简单通过 `±1` 转换
- 差分约束和“同余最短路”没有直接关系；不要因为题目中出现“约束”或“同余”就混用两个模型

常见用法

基本可行性

```
DifferenceConstraints dc(n);
dc.add_leq(u, v, upper);
dc.add_geq(u, v, lower);
auto ans = dc.feasible();
if (!ans) {
    cout << "No\n";
} else {
    for (i64 x : *ans) cout << x << ' ';
}
```

同时限制上下界

题目要求：

$$2 \le x_v - x_u \le 5$$

直接写：

```
dc.add_range(u, v, 2, 5);
```

等式约束

题目要求：

$$x_v = x_u + 7$$

即：

$$x_v - x_u = 7$$

写成：

```
dc.add_eq(u, v, 7);
```

判断矛盾

例如：

$$x - y \le -1$$

且：

$$y - x \le -1$$

可以写成：

```
DifferenceConstraints dc(2);
dc.add_leq(1, 0, -1);
dc.add_leq(0, 1, -1);
auto ans = dc.feasible();
assert(!ans);
```

两式相加得到：

$$0 \le -2$$

因此形成负环，系统无解

时间排程

例如任务 `u` 结束后至少经过 `d` 时间才能开始任务 `v`：

$$start_v - start_u \ge d$$

写作：

```
dc.add_geq(u, v, d);
```

典型应用包括：

- 时间排程
- 先后关系
- 区间前缀和约束
- 最早 / 最晚时间限制
- 相对高度、相对位置
- 一组变量之间的上下界一致性判断

复杂度

设变量数为 `n`，约束数为 `m`
加入一条约束：

$$O(1)$$

`feasible()` 使用 Bellman--Ford：

$$O(nm)$$

额外空间：

$$O(n)$$

保存约束后的总空间：

$$O(n + m)$$

5.2.6 Johnson 全源最短路

解决问题： 在允许负边但不存在负环的稀疏有向图上求所有点对最短路
依赖「最短路」中的

```
template<class T>
inline constexpr T INF = numeric_limits<T>::max() / 4;
template<class T = i64>
using WGraph = vector<vector<pair<int, T>>>;
```

```
optional<vector<vector<i64>>> johnson(const WGraph<i64> &g) {
    using i128 = __int128_t;
    const i128 IINF = i128(1) << 120;
    int n = g.size();
    // ===== 1. Bellman-Ford 求势能 h =====
    // 等价于增加超级源 S，并加入 S → v 权值 0
    vector<i128> h(n);
    for (int it = 0; it < n; ++ it) {
        bool changed{ false };
        for (int u : views::iota(0, n)) {
            for (auto [v, w] : g[u]) {
                i128 nd{ h[u] + w };
                if (nd >= h[v]) continue;
                h[v] = nd;
                changed = true;
                // 第 n 轮仍能松弛 ⇒ 存在负环
                if (it == n - 1) return nullopt;
            }
        }
        if (!changed) break;
    }
    // ===== 2. 每个源点跑重标后的 Dijkstra =====
    vector ans(n, vector<i64>(n, INF<i64>));
    using State = pair<i128, int>;
    for (int s : views::iota(0, n)) {
        vector<i128> dis(n, IINF);
        priority_queue<State, vector<State>, greater<>> q;
        dis[s] = 0;
        q.push({0, s});
        while (!q.empty()) {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u]) {
                i128 nw{ i128(w) + h[u] - h[v] };
                assert(nw >= 0);
                i128 nd{ d + nw };
                if (nd >= dis[v]) continue;
                dis[v] = nd;
                q.push({nd, v});
            }
        }
        // ===== 3. 还原原图距离 =====
        for (int v : views::iota(0, n)) {
            if (dis[v] == IINF) continue;
            i128 d{ dis[v] - h[s] + h[v] };
            assert(numeric_limits<i64>::min() <= d && d <= numeric_limits<i64>::max());
            ans[s][v] = i64(d);
        }
    }
    return ans;
}
```

原理

超级源与势能

增加超级源 `S`
对每个原图节点加入

$$S \to v, \qquad w = 0$$

然后从 `S` 跑 Bellman-Ford
记

$$h[v] = dist(S, v)$$

因为所有节点都有一条权值为 0 的入边，所以初始直接令

```
vector<i128> h(n);
```

也就是

$$h[v] = 0$$

再对原图全部边做 Bellman-Ford，与显式建立超级源完全等价
这样还能检测图中任意位置的负环，而不只是某个原源点可达的负环

重标边权

对原边

$$u \to v$$

权值为 $w(u, v)$
定义新权值

$$w'(u, v) = w(u, v) + h[u] - h[v]$$

Bellman-Ford 完成后满足最短路三角不等式

$$h[v] \le h[u] + w(u, v)$$

移项得到

$$w(u, v) + h[u] - h[v] \ge 0$$

所以

$$w'(u, v) \ge 0$$

此时整张图已经没有负边，可以从每个源点运行 Dijkstra

为什么重标后最短路不变

考虑一条路径

$$P = v_0 \to v_1 \to \cdots \to v_k$$

重标后的路径长度为

$$\sum_{i=0}^{k-1} (w(v_i, v_{i+1}) + h[v_i] - h[v_{i+1}])$$

中间的势能全部抵消

$$\begin{aligned} w'(P) &= \sum_{i=0}^{k-1} (w(v_i, v_{i+1}) + h[v_i] - h[v_{i+1}]) \\ &= \sum_{i=0}^{k-1} w(v_i, v_{i+1}) + h[v_0] - h[v_k] \\ &= w(P) + h[v_0] - h[v_k] \end{aligned}$$

对于固定起点 s 和终点 t

$$h[s] - h[t]$$

是固定值
因此所有 $s \to t$ 路径只是统一增加同一个常数，最短路径不会改变
若重标后的距离为 $d'(s, t)$，原距离为

$$d(s,t) = d'(s,t) - h[s] + h[t]$$

对应代码

```
i128 d = dis[v] - h[s] + h[v];
```

接口

```
auto dis = johnson(g);
```

存在负环

```
if (!dis) {
    // 图中存在负环
}
```

正常情况

```
i64 d = (*dis)[u][v];
```

若

```
d == INF<i64>
```

表示 `u` 无法到达 `v`
否则就是原图中的最短距离

常见用法

```
WGraph<i64> g(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    -- u, -- v;
    g[u].push_back({v, w});
}
auto dis = johnson(g);
if (!dis) {
    cout << "Negative Cycle\n";
} else {
    for (int s : views::iota(0, n)) {
        for (int t : views::iota(0, n)) {
            if ((*dis)[s][t] == INF<i64>) {
                // s 无法到达 t
            } else {
                cout << (*dis)[s][t] << '\n';
            }
        }
    }
}
```

只需要逐行答案时

完整 APSP 矩阵需要

$$O(n^2)$$

空间
如果题目只需要每个源点算完后立即统计答案，不需要保存完整矩阵，可以改成逐源处理
核心只需把

```
vector ans(n, vector<i64>(n, INF<i64>));
```

删除，然后在每次 Dijkstra 结束后处理当前 `s`

```
for (int s : views::iota(0, n)) {
    // Dijkstra ...
    for (int v : views::iota(0, n)) {
        if (dis[v] == IINF) {
            // s → v 不可达
            continue;
        }
        i128 d = dis[v] - h[s] + h[v];
        // 立即统计 d
    }
}
```

这样除图和 Dijkstra 工作数组外，额外空间只需要

$$O(n)$$

与 Floyd 的选型

条件	推荐
稠密图	Floyd
n 较小	Floyd
稀疏图且无负边	每个源点 Dijkstra
稀疏图、有负边、无负环	Johnson
需要判断任意负环	Johnson 的 Bellman-Ford 阶段也可以完成

Floyd 时间复杂度固定为

$$O(n^3)$$

Johnson 的优势主要来自

$$m \ll n^2$$

时 Dijkstra 只遍历实际存在的边

为什么不用 SPFA 求势能

Johnson 第一阶段只需要一次 Bellman-Ford
理论复杂度稳定为

$$O(nm)$$

SPFA 虽然随机数据上可能更快，但可以退化到同级甚至出现很差的常数表现
直接使用 Bellman-Ford 更稳

数值处理

内部的

- 势能 `h`
- 重标边权
- Dijkstra 距离
- 距离恢复
  全部使用

```
__int128_t
```

避免出现

```
w + h[u] - h[v]
```

在 `i64` 中间过程溢出
最终接口仍返回

```
vector<vector<i64>>
```

因此要求真实最短路答案能够放入 `i64`
若题目答案本身可能超过 `i64`，应直接把返回矩阵也改成 `i128`

负环

如果第 n 轮 Bellman-Ford 仍然能够松弛

```
if (it == n - 1) return nullopt;
```

说明存在负环
因为超级源等价地可以到达全部节点，所以这里检测的是**整张图任意位置的负环**
有负环时某些最短距离可能不存在有限下界，因此 Johnson 直接返回失败

自环与重边

重边可以直接保留
Dijkstra 与 Bellman-Ford 会自动选择更优边
自环也允许
若存在负自环

$$u \to u, \qquad w < 0$$

它本身就是负环，Bellman-Ford 会返回 `nullopt`
非负自环不会影响结果

快速回忆

```
稀疏 APSP + 允许负边 + 无负环
    Johnson

1. 超级源 → 所有点权值 0
   等价于 h 全部初始化 0

2. Bellman-Ford 求 h
   第 n 轮还能松弛 ⇒ 负环

3. 重标
   w' = w + h[u] - h[v]
   保证 w' ⩾ 0

4. 每个源点 Dijkstra

5. 恢复
   d = d' - h[s] + h[v]
```

注意事项

- 支持有向图，若是无向图需要自行加入两个方向
- 允许负边，不允许负环
- 支持重边和自环
- Bellman-Ford 的全零初始化等价于增加权值为 0 的超级源
- 不应在原图存在负边时直接对每个源点跑 Dijkstra
- Dijkstra 必须使用重标后的非负边权
- 最终恢复距离时不能忘记 `-h[s]+h[v]`
- 内部使用 `i128` 防止重标过程溢出
- 若只需要统计每个源点的答案，不建议保存整个 $O(n^2)$ 距离矩阵
- 稠密图上通常直接使用 Floyd 更简单

复杂度

Bellman-Ford

$$O(nm)$$

每个源点一次堆优化 Dijkstra

$$O((n+m)\log n)$$

共运行 n 次

$$O(n(n+m)\log n)$$

因此严格总时间复杂度为

$$O(nm+n(n+m)\log n)$$

在通常的稀疏连通图中 $m=\Omega(n)$，常简写为

$$O(nm\log n)$$

若保存全部点对距离，空间复杂度为

$$O(n^2+n+m)$$

通常简写为

$$O(n^2)$$

若逐源处理答案，不保存距离矩阵，则空间复杂度为

$$O(n+m)$$

5.3 生成树与连接子图

5.3.1 最小生成树与严格次小生成树

Kruskal、指定连通块数与严格次小生成树

```
struct MST {
    struct Edge {
        int u, v, id;
        i64 w;
    };
    struct Result {
        i64 cost{};
        vector<int> used;
        int components{};
    };
    int n;
    vector<Edge> edges;
    explicit MST(int n_) : n(n_) {}
    int add_edge(int u, int v, i64 w) {
        int id = edges.size();
        edges.push_back({ u, v, id, w });
        return id;
    }
    // ===== Kruskal: 基础 MST / 最小生成森林，以下代码是核心 =====
    Result kruskal(int components = 1) const {
        assert(1 <= components && components <= n);
        vector<int> order(edges.size());
        ranges::iota(order, 0);
        ranges::sort(order, {}, [&](int id) {
            return pair{ edges[id].w, id };
        });
        DSU dsu(n);
        Result res;
        res.used.reserve(n - components);
        for (int id : order) {
            if (dsu.count() == components) break;
            const auto &[u, v, _, w] = edges[id];
```

```
            if (!dsu.unite(u, v)) continue;
            res.cost += w;
            res.used.push_back(id);
        }
        res.components = dsu.count();
        return res;
    }
private:
    // ===== 严格次小 MST：若不需要，可以连同 strict_second() 删除 =====
    struct Top2 {
        i64 first{}, second{};
        bool has_first{}, has_second{};
        void add(i64 x) {
            if (!has_first) {
                first = x;
                has_first = true;
            } else if (x == first) {
                return;
            } else if (x > first) {
                second = first;
                has_second = true;
                first = x;
            } else if (!has_second || x > second) {
                second = x;
                has_second = true;
            }
        }
    };
    static Top2 merge_top2(Top2 a, const Top2 &b) {
        if (b.has_first) a.add(b.first);
        if (b.has_second) a.add(b.second);
        return a;
    }
public:
    optional<i64> strict_second() const {
        auto mst = kruskal();
        if (mst.components != 1 || n <= 1) return nullopt;
        vector<char> in_mst(edges.size());
        vector<vector<pair<int, i64>>> g(n);
        for (int id : mst.used) {
            in_mst[id] = true;
            const auto &[u, v, _, w] = edges[id];
            g[u].push_back({ v, w });
            g[v].push_back({ u, w });
        }
        // 倍增维护路径上的最大值与严格次大值。
        int lg = bit_width((unsigned)n);
        vector dep(n, -1);
        vector up(lg, vector<int>(n));
        vector mx(lg, vector<Top2>(n));
        vector<int> order{ 0 };
        dep[0] = 0;
        for (int i = 0; i < order.size(); ++ i) {
            int u = order[i];
            for (auto [v, w] : g[u]) if (dep[v] == -1) {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v].add(w);
                order.push_back(v);
            }
        }
        for (int k = 1; k < lg; ++ k) {
            for (int u : views::iota(0, n)) {
                int p = up[k - 1][u];
                up[k][u] = up[k - 1][p];
                mx[k][u] = merge_top2(mx[k - 1][u], mx[k - 1][p]);
            }
        }
        auto path_top2 = [&](int u, int v) {
            Top2 res;
            if (dep[u] < dep[v]) swap(u, v);
```

```
            int d = dep[u] - dep[v];
            for (int k = 0; k < lg; ++ k) if (d >> k & 1) {
                res = merge_top2(res, mx[k][u]);
                u = up[k][u];
            }
            if (u == v) return res;
            for (int k = lg - 1; k >= 0; -- k) {
                if (up[k][u] == up[k][v]) continue;
                res = merge_top2(res, mx[k][u]);
                res = merge_top2(res, mx[k][v]);
                u = up[k][u];
                v = up[k][v];
            }
            res = merge_top2(res, mx[0][u]);
            res = merge_top2(res, mx[0][v]);
            return res;
        };
        optional<i64> ans;
        for (const auto &e : edges) {
            if (in_mst[e.id] || e.u == e.v) continue;
            auto top = path_top2(e.u, e.v);
            if (!top.has_first) continue;
            if (e.w > top.first) {
                i128 wide = i128(mst.cost) + e.w - top.first;
                assert(numeric_limits<i64>::min() <= wide && wide <= numeric_limits<i64>::max());
                i64 cur = i64(wide);
                if (!ans || cur < *ans) ans = cur;
            } else if (e.w == top.first && top.has_second) {
                i128 wide = i128(mst.cost) + e.w - top.second;
                assert(numeric_limits<i64>::min() <= wide && wide <= numeric_limits<i64>::max());
                i64 cur = i64(wide);
                if (!ans || cur < *ans) ans = cur;
            }
        }
        return ans;
    }
};
```

稠密 Prim

```
optional<i64> prim_dense(const vector<vector<i64>> &a) {
    int n = a.size();
    if (!n) return i64{};
    constexpr i64 INF = numeric_limits<i64>::max() / 4;
    vector<i64> best(n, INF);
    vector<char> used(n);
    best[0] = 0;
    i64 ans = 0;
    for (int it = 0; it < n; ++ it) {
        int u{ -1 };
        for (int v : views::iota(0, n)) {
            if (!used[v] && (u == -1 || best[v] < best[u])) u = v;
        }
        if (u == -1 || best[u] == INF) return nullopt;
        used[u] = true;
        ans += best[u];
        for (int v : views::iota(0, n)) if (!used[v]) {
            best[v] = min(best[v], a[u][v]);
        }
    }
    return ans;
}
```

接口说明

`add_edge(u, v, w)`

加入无向边，并返回稳定的逻辑边编号

```
int id = mst.add_edge(u, v, w);
```

`strict_second()` 内部也依靠这一编号区分 MST 边与非树边

`kruskal(components = 1)`

返回：

```
MST::Result { cost, used, components }
```

其中：

- `cost`：实际得到的最小生成森林总权值
- `used`：所选原边编号
- `components`：最终连通块数量

普通 MST：

```
auto res = mst.kruskal();
if (res.components != 1) {
    // 原图不连通
}
```

要求恰好留下 `k` 个连通块：

```
auto res = mst.kruskal(k);
if (res.components != k) {
    // 原图自身的连通性不足，无法合并到 k 块
}
```

当能够达到 `k` 个连通块时，所得森林在所有恰有 `k` 个连通块的生成森林中权值最小

`strict_second()`

返回严格满足

$$W' > W_{\text{MST}}$$

的最小生成树权值
不存在时返回 `nullopt`
这里求的是**严格次小生成树**，因此如果存在多棵权值完全相同的 MST，它们仍然都属于“最小”，不会被视为严格次小

`prim_dense(a)`

邻接矩阵 Prim

- `a[u][v]`：边权
- 无边：`INF`
- 返回 MST 总权值
- 图不连通：`nullopt`

严格次小生成树原理

先得到一棵 MST
对于每条非树边：

$$e = (u, v, w)$$

加入它后会形成唯一环
如果删除环中最大权边 mx_1，新树权值为：

$$W + w - mx_1$$

根据 MST 性质：

$$w \geq mx_1$$

若：

$$w > mx_1$$

直接得到严格更大的候选
但如果：

$$w = mx_1$$

删除最大边只会得到另一棵同权 MST
因此必须删除路径上**严格小于 mx_1 的最大边**，也就是严格次大值 mx_2：

$$W + w - mx_2$$

这就是倍增中维护两个不同权值最大值的原因

注意事项

- MST / Prim / Kruskal 都允许负边
- 自环会被 DSU 自动忽略
- 重边正常处理
- `strict_second()` 维护的是“最大和严格次大权值”，重复出现的同权最大边只算一个值
- 如果非树边与路径最大边等权，而路径不存在严格次大值，则这条边只能产生另一棵同权 MST，无法贡献严格次小答案
- MST 权值和以及替换后的答案必须能够放入 `i64`
- 稀疏图通常直接 Kruskal；邻接矩阵或接近完全图时使用 `prim_dense()`
- 堆优化 Prim 的复杂度为 $O(m \log n)$，但若已经维护边表，Kruskal 通常足够，不必再专门保存一份板子

常见用法

普通 MST

```
MST mst(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    mst.add_edge(u, v, w);
}
auto res = mst.kruskal();
if (res.components != 1) {
    cout << "Impossible\n";
} else {
    cout << res.cost << '\n';
}
```

指定连通块数

```
int k;
cin >> k;
auto res = mst.kruskal(k);
if (res.components == k) {
    cout << res.cost << '\n';
}
```

恢复 MST 边

```
auto res = mst.kruskal();
for (int id : res.used) {
    auto [u, v, _, w] = mst.edges[id];
}
```

严格次小 MST

```
auto ans = mst.strict_second();
if (!ans) {
    // 原图不连通，或不存在严格次小生成树
} else {
    cout << *ans << '\n';
}
```

复杂度

Kruskal:

$$O(m \log m)$$

指定连通块数不会改变最坏复杂度
严格次小生成树:

- Kruskal:

$$O(m \log m)$$

- 倍增预处理：

$$O(n \log n)$$

- 枚举所有非树边：

$$O(m \log n)$$

总复杂度：

$$O(m \log m + (n+m) \log n)$$

空间复杂度：

$$O(m + n \log n)$$

稠密 Prim：

$$O(n^2)$$

空间由邻接矩阵主导：

$$O(n^2)$$

5.3.2 曼哈顿最小生成树

```
struct ManhattanMST {
    struct Point {
        i64 x, y;
    };
    struct Edge {
        int u, v;
        i64 w;
    };
private:
    using i128 = __int128_t;
    struct WorkPoint {
        i128 x, y;
    };
    static i64 distance(const Point &a, const Point &b) {
        i128 dx{ i128(a.x) - b.x };
        i128 dy{ i128(a.y) - b.y };
        if (dx < 0) dx = -dx;
        if (dy < 0) dy = -dy;
        i128 d{ dx + dy };
        assert(d <= numeric_limits<i64>::max());
        return i64(d);
    }
public:
    // ===== O(n) 候选边生成 =====
    static vector<Edge> candidates(const vector<Point> &points) {
        int n = points.size();
        vector<WorkPoint> p(n);
        for (int i : views::iota(0, n)) {
            p[i] = { points[i].x, points[i].y };
        }
        vector<int> order(n);
        ranges::iota(order, 0);
        vector<Edge> edges;
        edges.reserve(4 * n);
        for (int dir = 0; dir < 4; ++ dir) {
            ranges::sort(order, {}, [&](int i) {
                return pair{ p[i].x + p[i].y, p[i].x };
            });
            map<i128, int> sweep;
            for (int i : order) {
                i128 key{ -p[i].y };
                for (auto it = sweep.lower_bound(key);
                     it != sweep.end();) {
                    int j{ it->second };
                    if (p[i].x - p[j].x < p[i].y - p[j].y) break;
                    edges.push_back({ i, j, distance(points[i], points[j]) });
```

```
                    it = sweep.erase(it);
                }
                sweep[key] = i;
            }
            for (auto &q : p) {
                if (dir & 1) q.x = -q.x;
                else swap(q.x, q.y);
            }
        }
        return edges;
    }
    // ===== 直接求 Manhattan MST 权值 =====
    static i64 mst_weight(const vector<Point> &points) {
        int n = points.size();
        if (n <= 1) return 0;
        auto edges = candidates(points);
        ranges::sort(edges, {}, &Edge::w);
        DSU dsu(n);
        i64 ans = 0;
        for (auto [u, v, w] : edges) {
            if (!dsu.unite(u, v)) continue;
            ans += w;
            if (dsu.count() == 1) break;
        }
        assert(dsu.count() == 1);
        return ans;
    }
};
```

接口说明

`Point`

点编号直接等于其在输入 `vector` 中的下标：

```
vector<ManhattanMST::Point> p(n);
```

不再需要额外维护 `Point::id`，从而避免坐标变换后原编号映射错误

`candidates(points)`

返回一个只含

$$O(n)$$

条边的候选边集合，并保证其中存在一棵完整的曼哈顿 MST
因此无需建立包含

$$\Theta(n^2)$$

条边的完全图

`mst_weight(points)`

直接：

1. 生成候选边
2. 按边权排序
3. Kruskal
   返回曼哈顿 MST 权值

原理

二维曼哈顿距离：

$$d(i,j) = |x_i - x_j| + |y_i - y_j|$$

对于每个点，只需要考虑八个方向区域中的最近候选点
利用对称性，可以通过四次坐标变换覆盖全部八个方向：

- 交换 x, y
- 翻转 x
- 再交换

- 再翻转
  每轮按：

$$x + y$$

  排序，并用有序表维护扫描线上的有效点
  每个点在一轮扫描中只会被插入、删除有限次，因此每轮生成线性数量的候选边

注意事项

- 这是**二维曼哈顿距离** MST
- 点编号自动使用输入下标 `0..n-1`
- 重合点完全允许，候选边权可以为 `0`
- `candidates()` 可能生成重复边；Kruskal 会自然忽略，不需要额外去重
- 候选边总数为 $O(n)$，实际至多常数倍于 `n`
- 内部坐标变换使用 `i128`，避免 `x+y`、取负和坐标差溢出
- 最终曼哈顿距离及 MST 总权值仍要求能够存入 `i64`
- 不要用这份候选边处理欧氏距离；欧氏 MST 应使用完全图 Prim 或 Delaunay 三角剖分

常见用法

直接求 MST

```
vector<ManhattanMST::Point> p(n);
for (auto &[x, y] : p) {
    cin >> x >> y;
}
cout << ManhattanMST::mst_weight(p) << '\n';
```

只生成候选边

如果还要处理 MST 上第 `k` 大边、边分类等其他问题：

```
auto edges = ManhattanMST::candidates(p);
ranges::sort(edges, {}, &ManhattanMST::Edge::w);
DSU dsu(n);
for (auto [u, v, w] : edges) {
    if (!dsu.unite(u, v)) continue;
    // 当前边属于所构造的某棵 Manhattan MST
}
```

复杂度

四次扫描，每次排序和 `map` 操作：

$$O(n \log n)$$

产生候选边：

$$O(n)$$

随后 Kruskal：

$$O(n \log n)$$

因此总时间复杂度：

$$O(n \log n)$$

空间复杂度：

$$O(n)$$

5.3.3 最小直径生成树

与普通 MST 的“最小总边权”目标不同，本算法用于解决：

> 在所有生成树中，使树的直径最小

依赖「最短路基础」中的：

```
template<class T>
inline constexpr T INF = numeric_limits<T>::max() / 4;
```

```
template<class T>
vector<vector<T>> floyd(vector<vector<T>> dis);
```

```
optional<i64> minimum_diameter_spanning_tree(
    int n, const vector<tuple<int, int, i64>> &edges) {
    if (n <= 1) return i64{};
    vector d(n, vector<i64>(n, INF<i64>));
    for (int u : views::iota(0, n)) d[u][u] = 0;
    for (auto [u, v, w] : edges) {
        assert(0 <= u && u < n);
        assert(0 <= v && v < n);
        assert(w >= 0);
        d[u][v] = min(d[u][v], w);
        d[v][u] = min(d[v][u], w);
    }
    d = floyd(move(d));
    if (ranges::any_of(d[0], [](i64 x) { return x == INF<i64>; })) return nullopt;
    // order[u]: 所有点按 d[u][v] 从小到大排序。
    vector order(n, vector<int>(n));
    for (int u : views::iota(0, n)) {
        ranges::iota(order[u], 0);
        ranges::sort(order[u], {}, [&](int v) {
            return d[u][v];
        });
    }
    i64 ans{ numeric_limits<i64>::max() };
    // ===== 绝对中心位于原图顶点 =====
    for (int u : views::iota(0, n)) {
        i64 ecc = d[u][order[u].back()];
        ans = min(ans, 2 * ecc);
    }
    // ===== 绝对中心位于原图边内部 =====
    for (auto [u, v, w] : edges) {
        int p{ n - 1 };
        for (int i = n - 2; i >= 0; --i) {
            int a = order[u][i], b = order[u][p];
            if (d[v][a] <= d[v][b]) continue;
            __int128_t cur = (__int128_t)d[u][a] + d[v][b] + w;
            assert(cur <= numeric_limits<i64>::max());
            ans = min(ans, i64(cur));

            p = i;
        }
    }
    return ans;
}
```

接口说明

```
minimum_diameter_spanning_tree(n, edges)
```

边格式：

```
{u, v, w}
```

表示一条无向边
返回：

- 图连通：所有生成树中最小可能直径
- 图不连通：`nullopt`
  要求：
- 点编号 `0base`
- 边权非负

与普通 MST 的区别

普通最小生成树优化：

$$\sum_{e\in T} w_e$$

而最小直径生成树优化：

$$\max_{u,v\in V} d_T(u,v)$$

两者没有包含关系
例如一棵总边权最小的 MST，其直径完全可能不是所有生成树中最小的
因此不能：

```
先求 MST
再求该 MST 的直径
```

来解决 MDST

原理

设图中的一个位置 x 可以位于：

- 原图顶点
- 原图某条边内部
  定义其离心率：

$$ecc(x) = \max_{v\in V} d_G(x,v)$$

图的绝对中心是使离心率最小的位置：

$$R = \min_{x\in G} ecc(x)$$

最小直径生成树满足：

$$\boxed{D_{\min} = 2R}$$

因此问题转化成寻找图的绝对中心

中心位于顶点

若中心就是点 `u`：

$$2R = 2\max_{v} d(u,v)$$

因此直接枚举每个顶点

中心位于边内部

考虑边：

$$(u,v,w)$$

中心距离 `u` 为 (x)，则一个点 `z` 到中心的最短距离为：

$$\min\left(d(u,z)+x,\ d(v,z)+w-x\right)$$

需要求：

$$\min_{0\le x\le w}\ \max_{z} \min\left(d(u,z)+x,\ d(v,z)+w-x\right)$$

将所有点按：

$$d(u,z)$$

排序后，可以线性扫描可能发生最远点切换的位置
当两个控制点分别为 `a,b` 时，对应的两倍半径为：

$$d(u,a)+d(v,b)+w$$

因此每条边只需要：

$$O(n)$$

完成内部中心枚举

注意事项

- 要求无向图
- 要求非负边权，以保证距离具有通常的网络几何意义
- 支持重边；建 Floyd 矩阵时自动保留最小权值
- 图不连通时不存在生成树，返回 `nullopt`
- 返回的是直径值，不是半径
- 绝对中心可能位于边的内部，因此不能只枚举顶点中心
- 中心位置可能是半整数，但最终生成树直径仍按照原边权单位返回，不需要自行除以 `2`
- 当前版本不恢复具体生成树
- 若需要方案，可以记录取得最优值的顶点或边内中心，再按该中心构造最短路径树
- 所有最短距离及最终直径必须能够存入 `i64`

常见用法

```
vector<tuple<int, int, i64>> edges;
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    edges.push_back({ u, v, w });
}
auto ans = minimum_diameter_spanning_tree(n, edges);
if (!ans) {
    cout << "Disconnected\n";
} else {
    cout << *ans << '\n';
}
```

例如：

```
vector<tuple<int, int, i64>> edges{
    { 0, 1, 4 },
    { 1, 2, 3 },
    { 0, 2, 10 }
};
auto ans = minimum_diameter_spanning_tree(3, edges);
```

答案为：7

复杂度

Floyd：

$$O(n^3)$$

预处理每个点的距离顺序：

$$O(n^2 \log n)$$

枚举每条原边：

$$O(mn)$$

所以总复杂度：

$$O(n^3 + n^2 \log n + mn)$$

对于简单图：

$$m = O(n^2)$$

因此可简写为：

$$O(n^3)$$

空间复杂度：

$$O(n^2)$$

5.3.4 Kruskal 重构树

```
struct KruskalTree {
    struct Edge {
        int u, v;
        i64 w;
    };
    int n, tot{}, lg{};
    vector<i64> value;
    vector<array<int, 2>> child;
    vector<int> parent, depth, leaf_count, root_of, roots;
    vector<vector<int>> up;
    explicit KruskalTree(int n_) : n(n_) {}
    // ===== 构造重构森林 =====
    void build(vector<Edge> edges) {
        ranges::sort(edges, {}, &Edge::w);
        int cap = max(0, 2 * n - 1);
        value.assign(cap, 0);
        child.assign(cap, {-1, -1});
        parent.assign(cap, -1);
        leaf_count.assign(cap, 0);
        for (int u : views::iota(0, n)) leaf_count[u] = 1;
        DSU dsu(n);
        vector<int> top(n);
        ranges::iota(top, 0);
        tot = n;
        for (auto [u, v, w] : edges) {
            int x = dsu.find(u), y = dsu.find(v);
            if (x == y) continue;
            int a{ top[x] }, b{ top[y] }, p{ tot ++ };
            value[p] = w;
            child[p] = {a, b};
            parent[a] = parent[b] = p;
            leaf_count[p] = leaf_count[a] + leaf_count[b];
            dsu.unite(x, y);
            top[dsu.find(x)] = p;
        }
        value.resize(tot);
        child.resize(tot);
        parent.resize(tot);
        leaf_count.resize(tot);
        roots.clear();
        for (int u : views::iota(0, n)) {
            if (dsu.is_root(u)) roots.push_back(top[u]);
        }
        // ===== 倍增：若只需要重构森林本身，可以删除 =====
        depth.assign(tot, 0);
        root_of.assign(tot, -1);
        vector<int> order;
        order.reserve(tot);
        for (int r : roots) {
            root_of[r] = r;
            order.push_back(r);
        }
        for (int i = 0; i < order.size(); ++ i) {
            int u = order[i];
            for (int v : child[u]) if (v != -1) {
                depth[v] = depth[u] + 1;
                root_of[v] = root_of[u];
```

```
                order.push_back(v);
            }
        }
        lg = bit_width((unsigned)max(1, tot));
        up.assign(lg, vector<int>(tot));
        for (int u : views::iota(0, tot)) {
            up[0][u] = parent[u] == -1 ? u : parent[u];
        }
        for (int k = 1; k < lg; ++ k) {
            for (int u : views::iota(0, tot)) {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
        }
    }
    // ===== 常用查询 =====
    int lca(int u, int v) const {
        if (root_of[u] != root_of[v]) return -1;
        if (depth[u] < depth[v]) swap(u, v);
        int d = depth[u] - depth[v];
        for (int k = 0; k < lg; ++ k) {
            if (d >> k & 1) u = up[k][u];
        }
        if (u == v) return u;
        for (int k = lg - 1; k >= 0; -- k) {
            if (up[k][u] == up[k][v]) continue;
            u = up[k][u];
            v = up[k][v];
        }
        return up[0][u];
    }
    optional<i64> connect_threshold(int u, int v) const {
        assert(u != v);
        int p = lca(u, v);
        if (p == -1) return nullopt;
        return value[p];
    }
    bool connected(int u, int v, i64 limit) const {
        if (u == v) return true;
        auto w = connect_threshold(u, v);
        return w && *w <= limit;
    }
    int component_node(int u, i64 limit) const {
        for (int k = lg - 1; k >= 0; -- k) {
            int p{ up[k][u] };
            if (p != u && p >= n && value[p] <= limit) u = p;
        }
        return u;
    }
    int component_size(int u, i64 limit) const {
        return leaf_count[component_node(u, limit)];
    }
};
```

接口说明

构造时将边按权值从小到大加入
原图节点编号：

$$[0, n)$$

重构得到的新节点编号：

$$[n, tot)$$

每次两个连通块第一次合并时建立一个新节点 `p`：

```
value[p] = 当前边权;
child[p] = {两个连通块对应的重构树根};
```

因此从叶子向根：

$$value$$

单调不降

`build(edges)`

建立 Kruskal 重构森林
若原图连通：

```
roots.size() == 1
```

否则会得到多棵树
最终：

$$tot \le 2n - 1$$

`lca(u,v)`

求重构森林 LCA
若 `u,v` 原本不连通，返回：

```
-1
```

对于两个不同的原图节点：

```
int p = tree.lca(u, v);
```

若 `p != -1`，则：

```
tree.value[p]
```

就是两点第一次连通所需的最小边权阈值。

`connect_threshold(u,v)`

对于不同节点，返回：

$$\min\{W : \text{只保留 } w \le W \text{ 的边后 } u, v \text{ 连通}\}$$

原图最终仍不连通则返回 `nullopt`
等价于两点之间的最小瓶颈值：

$$\min_{P} \max_{e \in P} w_e$$

`connected(u,v,limit)`

判断只允许使用：

$$w \le limit$$

的边时，两点是否连通

`component_node(u,limit)`

返回只保留：

$$w \le limit$$

的边以后，原点 `u` 所在连通块对应的最高重构树节点

`component_size(u,limit)`

返回该连通块中原图节点数量

常见用法

两点最小瓶颈

```
auto ans = tree.connect_threshold(u, v);

if (!ans) {
    // 原图中 u、v 不连通
} else {
    cout << *ans << '\n';
}
```

阈值连通

```
if (tree.connected(u, v, x)) {
    // 只使用权值 ≤ x 的边已经可以连通
}
```

某点在阈值下所在连通块大小

```
cout << tree.component_size(u, x) << '\n';
```

这类询问正是 Kruskal 重构树非常典型的用途：合并节点权值单调，因此可以在叶子到根的链上倍增寻找最后一个不超过阈值的祖先

注意事项

- 当前版本按边权从小到大构造，对应条件：

$$w \le x$$

- 相同权值的边可能形成若干连续的同权内部节点，这是正常的，不影响 LCA 权值和阈值查询
- 原图允许负边、重边和自环；自环不会导致新的合并
- `value[u]` 对原图叶子 `u < n` 没有实际意义
- `connect_threshold(u,u)` 的"首次连通阈值"没有统一定义，因此接口要求 `u ≠ v`；`connected(u,u,limit)` 始终为真
- 若只需要构造树而不需要在线查询，可以删除 `depth/root_of/up` 以及后续查询函数
- 若题目按"保留权值至少为 `x` 的边"询问，可以建立最大 Kruskal 重构树：边按权值从大到小处理，并相应反转阈值判断

复杂度

设原图边数为 `m`
构造：

$$O(m \log m)$$

重构森林节点数：

$$O(n)$$

倍增预处理：

$$O(n \log n)$$

空间：

$$O(n \log n)$$

查询：

操作	时间复杂度
`lca()`	$O(\log n)$
`connect_threshold()`	$O(\log n)$
`connected()`	$O(\log n)$
`component_node()`	$O(\log n)$
`component_size()`	$O(\log n)$

5.3.5 Chu–Liu / Edmonds 最小树形图

```cpp
struct DirectedMSTEdge {
    int u, v;
    i64 w;
};
optional<i64> directed_mst(
    int n, int root,
    const vector<DirectedMSTEdge> &input) {
    using i128 = __int128_t;
    struct Edge {
        int u, v;
        i128 w;
    };
    const i128 INF = i128(1) << 120;
    vector<Edge> edges;
    edges.reserve(input.size());
    for (auto [u, v, w] : input) {
        edges.push_back({u, v, w});
    }
    i128 ans{ 0 };
    while (true) {
        // ===== 1. 每个非根节点选择最小入边 =====
        vector<i128> in(n, INF);
        vector<int> pre(n, -1);
        for (auto [u, v, w] : edges) {
            if (u != v && w < in[v]) {
                in[v] = w;
                pre[v] = u;
            }
        }
        in[root] = 0;
        if (ranges::any_of(in, [&](i128 x) { return x == INF; })) return nullopt;
        for (i128 w : in) ans += w;
        // ===== 2. 找最小入边形成的环 =====
        int cnt{ 0 };
        vector<int> id(n, -1), vis(n, -1);
        for (int s : views::iota(0, n)) {
            int v{ s };
            while (v != root && vis[v] != s && id[v] == -1) {
                vis[v] = s;
                v = pre[v];
            }
            if (v != root && id[v] == -1) {
                for (int u = pre[v]; u != v; u = pre[u]) {
                    id[u] = cnt;
                }
                id[v] = cnt ++;
            }
        }
        if (cnt == 0) break;
        // ===== 3. 缩环 =====
        for (int v : views::iota(0, n)) {
            if (id[v] == -1) id[v] = cnt ++;
        }
        vector<Edge> next;
        next.reserve(edges.size());
        for (auto [u, v, w] : edges) {
            int x{ id[u] }, y{ id[v] };
            if (x == y) continue;
            next.push_back({ x, y, w - in[v] });
        }
        root = id[root];
        n = cnt;
        edges = move(next);
    }
    assert(ans >= numeric_limits<i64>::min());
    assert(ans <= numeric_limits<i64>::max());
    return i64(ans);
}
```

接口说明

```
directed_mst(n, root, edges)
```

求以 `root` 为根的最小权出树形图
要求最终选出 `n - 1` 条有向边，使得：

- 根 `root` 入度为 `0`
- 每个其他节点恰有一条入边
- 从 `root` 能到达所有节点
  并使总边权最小
  返回：
- 存在：最小总边权
- 不存在：`nullopt`
  Chu–Liu/Edmonds 正是有向图上最小生成树问题的经典算法

原理

每轮先为每个非根节点选取权值最小的入边
如果这些边不构成环，那么每个非根节点都有唯一父边，并已经得到最优树形图
若出现环 C，环中不能把所有边都保留，否则无法形成树
因此将整环缩成一个新节点
假设外部边：

$$u \to v, \qquad v \in C$$

原权为 w
由于当前已经把 `v` 的最小入边：

$$in[v]$$

计入答案，所以缩点后的边权调整为：

$$w - in[v]$$

若之后最终选择这条外部入边，就相当于用它替换掉环中 `v` 原来的最小入边
不断缩环直到不存在环即可

注意事项

- 这是有根有向生成树，不是无向 MST。
- 当前版本选择的是每个非根点的一条入边，因此最终方向从根向外
- 如果题目要求所有节点最终指向根的入树形图，可以先把所有边反向
- 边权可以为负
- 自环会被忽略
- 重边可以直接保留
- 根的入边永远不会被选取
- 某轮存在非根节点没有任何入边时，说明不存在合法树形图，返回 `nullopt`
- 只要求“每个点有入边”并不足以保证原图可从根到达，但缩环过程最终会把与根隔离的封闭部分暴露成没有外部入边的缩点，因此仍会正确判无解
- 当前版本只返回最小权值，不恢复具体边集
- 内部调整权值使用 `i128`，避免连续缩点时 `w - in[v]` 溢出；最终答案仍要求能放入 `i64`

常见用法

```
vector<DirectedMSTEdge> edges;
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    edges.push_back({u, v, w});
}
auto ans = directed_mst(n, root, edges);
if (!ans) {
    cout << "Impossible\n";
} else {
    cout << *ans << '\n';
}
```

复杂度

本实现每次缩环都线性扫描当前边集，而每轮至少缩掉一个环，因此经典朴素实现上界为：

$$O(nm)$$

空间复杂度：

$$O(n+m)$$

更高级的 Edmonds/Tarjan 实现可以达到更好的复杂度，但竞赛中只有在朴素 $O(nm)$ 明显无法通过时才值得使用

5.3.6 一般图 Steiner Tree

依赖「最短路基础」中的：

```
template<class T>
inline constexpr T INF = numeric_limits<T>::max() / 4;
template<class T = i64>
using WGraph = vector<vector<pair<int, T>>>;
```

```
optional<i64> steiner_tree(const WGraph<i64> &g, vector<int> term) {
    ranges::sort(term);
    term.erase(unique(term.begin(), term.end()), term.end());
    int n = g.size(), k = term.size();
    if (k <= 1) return i64{};
    assert(k < 31);
    int full{ 1 << k };
    vector dp(full, vector<i64>(n, INF<i64>));
    for (int i = 0; i < k; ++ i) {
        dp[1 << i][term[i]] = 0;
    }
    using State = pair<i64, int>;
    for (int mask = 1; mask < full; ++ mask) {
        // ===== 1. 合并两个终端集合 =====
        //
        // 固定最低位属于 sub，只枚举每个无序划分一次。
        int low{ mask & -mask };
        for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {
            if (!(sub & low)) continue;
            int other{ mask ^ sub };
            for (int v : views::iota(0, n)) {
                if (dp[sub][v] == INF<i64> || dp[other][v] == INF<i64>) continue;
                chmin(dp[mask][v], dp[sub][v] + dp[other][v]);
            }
        }
        // ===== 2. 多源 Dijkstra 闭包 =====
        //
        // 已经连接 mask 中终端并最终汇合于 u，
        // 再沿最短路移动汇合点
        priority_queue<State, vector<State>, greater<>> q;
        for (int v : views::iota(0, n)) {
            if (dp[mask][v] != INF<i64>) {
                q.push({dp[mask][v], v});
            }
        }
        while (!q.empty()) {
            auto [d, u] = q.top();
            q.pop();
            if (d != dp[mask][u]) continue;
            for (auto [v, w] : g[u]) {
                if (d > INF<i64> - w || d + w >= dp[mask][v]) continue;
                dp[mask][v] = d + w;
                q.push({dp[mask][v], v});
            }
        }
    }
    i64 ans = *ranges::min_element(dp.back());
    if (ans == INF<i64>) return nullopt;
```

```
    return ans;
}
```

接口说明

给定一个无向非负权图，以及少量终端点：

```
term = {t0, t1, ..., tk-1};
```

求总边权最小的连通子图，使全部终端点连通
允许额外使用非终端点，这些额外节点就是 Steiner 点。Steiner Tree 与普通 MST 的区别正是在于只要求指定终端连通，并允许借助其他点降低总代价

状态

定义：

$$dp[S][v]$$

表示连接终端集合 `S`，并使当前连通子图包含节点 `v` 的最小代价

初始化

对于第 `i` 个终端：

$$dp[2^i][t_i] = 0$$

集合合并

若两个终端集合最终都在 `v` 汇合：

$$dp[S][v] = \min_{A \subset S} (dp[A][v] + dp[S \setminus A][v])$$

代码固定 `mask` 的最低位必须位于 `sub`，因此每种无序划分只枚举一次

最短路闭包

合并结束后，还可以沿图中边把汇合位置从 `u` 移动到其他节点：

$$dp[S][v] \le dp[S][u] + w(u, v)$$

因此对每个 `mask` 执行一次多源 Dijkstra

返回值

```
auto ans = steiner_tree(g, term);
```

- 存在连接所有终端的方案：返回最小代价
- 无法全部连通：返回 `nullopt`
- `term` 为空或只有一个不同终端：返回 `0`
  重复终端会自动去重

注意事项

- 当前板子处理无向图 Steiner Tree
- 所有边权必须非负，因为每个状态使用 Dijkstra 做最短路闭包
- 零权边允许存在
- `k` 是指数复杂度的核心，真正限制不仅来自时间，还有：

  $$2^k n$$

  的 DP 内存
- 因此不能简单理解成“`k ≤ 20` 就一定能跑”；例如 `n` 很大时，`k = 20` 的内存已经完全不可接受
- 实战中通常是 `k` 很小，而 `n,m` 可以相对较大
- Steiner Tree 求的是最小连通子图，最优解在非负边权下可以取成树，因此称为 Steiner Tree
- 如果要求每组终端满足不同连接条件，通常需要在 `mask` 上增加合法性限制，而不是直接套完整终端集合
- 网格图无需单独写一份算法：把可走格子映射成图节点，按四邻接或八邻接建边即可
- 若边权允许负数，则“最小连通子图”本身还会涉及是否应该额外吸收负权结构，不能直接使用

常见用法

```
WGraph<i64> g(n);
auto add_edge = [&](int u, int v, i64 w) {
    g[u].push_back({v, w});
    g[v].push_back({u, w});
};
vector<int> terminal{a, b, c};
auto ans = steiner_tree(g, terminal);
if (!ans) {
    cout << "Impossible\n";
} else {
    cout << *ans << '\n';
}
```

网格 Steiner Tree

只需要把每个可走格子编号：

```
id[x][y]
```

然后对相邻格子建立无向边
终端格子的编号加入：

```
term
```

算法主体无需修改

复杂度

集合合并部分：

$$O(3^k n)$$

每个状态执行一次多源 Dijkstra：

$$O\left(2^k (n+m)\log n\right)$$

因此总时间复杂度：

$$O\left(3^k n + 2^k (n+m)\log n\right)$$

DP 空间：

$$O(2^k n)$$

加图后的总空间：

$$O(2^k n + m)$$

5.4 连通性、缩点与支配

5.4.1 SCC 与缩点 DAG

Tarjan SCC

```
struct SCC {
    int n, cur{}, cnt{};
    vector<vector<int>> g, comp;
    vector<int> stk, dfn, low, bel, siz;
    bool built{};
    explicit SCC(int n_) : n(n_), g(n), dfn(n, -1), low(n), bel(n, -1) {
        stk.reserve(n);
        comp.reserve(n);
        siz.reserve(n);
    }
    void add_edge(int u, int v) {
```

```
        assert(!built);
        g[u].push_back(v);
    }
private:
    // ===== Tarjan 核心 =====
    void dfs(int u) {
        dfn[u] = low[u] = cur ++;
        stk.push_back(u);
        for (int v : g[u]) {
            if (dfn[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (bel[v] == -1) {
                // 只有仍在 Tarjan 栈中的点才能更新 low。
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] != low[u]) return;
        comp.push_back({});
        while (true) {
            int v{ stk.back() };
            stk.pop_back();
            bel[v] = cnt;
            comp.back().push_back(v);
            if (v == u) break;
        }
        siz.push_back(comp.back().size());
        ++ cnt;
    }
public:
    int build() {
        if (built) return cnt;
        for (int u : views::iota(0, n)) {
            if (dfn[u] == -1) dfs(u);
        }
        built = true;
        return cnt;
    }
    bool same(int u, int v) const {
        assert(built);
        return bel[u] == bel[v];
    }
    // ===== 缩点 DAG: 若不需要可以删除 =====
    vector<vector<int>> dag() const {
        assert(built);
        vector<vector<int>> res(cnt);
        vector<int> seen(cnt, -1);
        for (int x : views::iota(0, cnt)) {
            for (int u : comp[x]) {
                for (int v : g[u]) {
                    int y{ bel[v] };
                    if (x == y || seen[y] == x) continue;
                    seen[y] = x;
                    res[x].push_back(y);
                }
            }
        }
        return res;
    }
};
```

非递归 Kosaraju

若 `n` 很大、链状图可能导致递归栈溢出，可以替换为这一版
接口和编号方向与上面的 Tarjan 保持一致

```
struct KosarajuSCC {
    int n, cnt{};
    vector<vector<int>> g, rg, comp;
    vector<int> bel, siz;
    bool built{};
```

```
    explicit KosarajuSCC(int n_) : n(n_), g(n), rg(n), bel(n, -1) {
        comp.reserve(n);
        siz.reserve(n);
    }
    void add_edge(int u, int v) {
        assert(!built);
        g[u].push_back(v);
        rg[v].push_back(u);
    }
    int build() {
        if (built) return cnt;
        vector<char> vis(n);
        vector<int> it(n), order, stk;
        order.reserve(n);
        stk.reserve(n);
        // ===== 第一遍：反图退出序 =====
        for (int s : views::iota(0, n)) {
            if (vis[s]) continue;
            vis[s] = true;
            stk.push_back(s);
            while (!stk.empty()) {
                int u{ stk.back() };
                if (it[u] == int(rg[u].size())) {
                    order.push_back(u);
                    stk.pop_back();
                    continue;
                }
                int v{ rg[u][it[u] ++] };
                if (!vis[v]) {
                    vis[v] = true;
                    stk.push_back(v);
                }
            }
        }
        // ===== 第二遍：原图划分 SCC =====
        for (int s : order | views::reverse) {
            if (bel[s] != -1) continue;
            comp.push_back({});
            bel[s] = cnt;
            stk.push_back(s);
            while (!stk.empty()) {
                int u{ stk.back() };
                stk.pop_back();
                comp.back().push_back(u);
                for (int v : g[u]) if (bel[v] == -1) {
                    bel[v] = cnt;
                    stk.push_back(v);
                }
            }
            siz.push_back(comp.back().size());
            ++ cnt;
        }
        built = true;
        return cnt;
    }
    bool same(int u, int v) const {
        assert(built);
        return bel[u] == bel[v];
    }
    vector<vector<int>> dag() const {
        assert(built);
        vector<vector<int>> res(cnt);
        vector<int> seen(cnt, -1);
        for (int x : views::iota(0, cnt)) {
            for (int u : comp[x]) {
                for (int v : g[u]) {
                    int y{ bel[v] };
                    if (x == y || seen[y] == x) continue;
                    seen[y] = x;
                    res[x].push_back(y);
                }
```

```
            }
        }
        return res;
    }
};
```

接口说明

两种实现统一提供：

- `add_edge(u,v)`：添加有向边 $u \to v$
- `build()`：构造 SCC，返回强连通分量数量
- `bel[u]`：原点 `u` 所属 SCC
- `comp[id]`：SCC `id` 包含的原节点
- `siz[id]`：SCC 大小
- `same(u,v)`：判断两点是否强连通
- `dag()`：返回去重后的缩点 DAG
  必须先：

```
scc.build();
```

才能访问 `bel / comp / siz` 或调用 `same()/dag()`
构造完成后不再允许 `add_edge()`

SCC 编号性质

这两个版本特意保持相同编号方向
对于缩点 DAG 中任意一条边：

$$x \to y$$

都有：

$$x > y$$

因此：

$$0, 1, \ldots, cnt - 1$$

是反拓扑序，从汇 SCC 向源 SCC
而：

$$cnt - 1, cnt - 2, \ldots, 0$$

是拓扑序，从源 SCC 向汇 SCC
所以若：

$$dp[x]$$

依赖所有后继 `y`：

```
for (int x = 0; x < scc.cnt; ++ x) {
    for (int y : dag[x]) {
        // y < x, dp[y] 已经完成
        dp[x] = max(dp[x], dp[y]);
    }
}
```

若从源 SCC 向后传播：

```
for (int x = scc.cnt - 1; x >= 0; -- x) {
    for (int y : dag[x]) {
        // x → y
    }
}
```

常见用法

强连通判断

```
SCC scc(n);
for (auto [u, v] : edges) {
    scc.add_edge(u, v);
}
scc.build();
if (scc.same(u, v)) {
    // u、v 强连通
}
```

缩点 DAG

```
auto dag = scc.dag();
for (int x : views::iota(0, scc.cnt)) {
    for (int y : dag[x]) {
        // x → y，并且 x > y
    }
}
```

源 SCC 与汇 SCC

```
auto dag = scc.dag();
vector<int> indeg(scc.cnt);
int sources = 0, sinks = 0;
for (int x : views::iota(0, scc.cnt)) {
    if (dag[x].empty()) ++ sinks;
    for (int y : dag[x]) ++ indeg[y];
}
for (int x : views::iota(0, scc.cnt)) {
    sources += indeg[x] == 0;
}
```

若 `scc.cnt > 1`，添加最少边使整张图强连通：

$$\max(sources, sinks)$$

若只有一个 SCC，则答案为 `0`

所有点都能到达的节点

缩点 DAG 恰好存在一个汇 SCC 时，其中所有原节点都可以被原图任意节点到达

```
auto dag = scc.dag();
int sink{ -1 };
for (int x : views::iota(0, scc.cnt)) {
    if (!dag[x].empty()) continue;
    if (sink != -1) {
        sink = -2;
        break;
    }
    sink = x;
}
int ans = sink >= 0 ? scc.siz[sink] : 0;
```

注意事项

- Tarjan 中访问到已经离开栈的节点时，不能再更新 `low`；本实现通过

```
bel[v] == -1
```

判断节点是否仍在栈中

- 自环和重边不会影响 SCC 划分
- `comp[id]` 内部节点顺序只与搜索过程有关
- `dag()` 自动删除 SCC 之间的重边，但不排序
- Tarjan 只存一份原图，常数和内存较小，通常优先使用

- 非递归 Kosaraju 同时保存原图和反图，边存储量约为两倍，但不会因深 DFS 链爆递归栈
- Kosaraju 第一遍必须记录**退出序**，所以非递归实现需要 `it[u]` 保存当前邻接表扫描位置；不能在第一次访问节点时直接加入 `order`
- 不要根据"Tarjan / Kosaraju"名字猜测 SCC 编号的拓扑方向，应以具体实现为准

复杂度

设点数为 n，边数为 m
Tarjan：

$$O(n+m)$$

空间：

$$O(n+m)$$

Kosaraju：

$$O(n+m)$$

空间：

$$O(n+m)$$

但需要同时保存原图和反图

5.4.2 2-SAT

依赖前面的 `SCC`
要求该 `SCC` 的缩点编号满足对于不同 SCC 间的边 `x → y`

$$bel[x] > bel[y]$$

也就是编号从汇 SCC 向源 SCC递增

```
struct TwoSAT {
    int n;
    SCC scc;
    explicit TwoSAT(int n_) : n(n_), scc(2 * n) {}
    int lit(int x, bool value) const { return 2 * x + value; }
    int neg(int p) const { return p ^ 1; }
    // ===== 基础二元子句 =====
    // (x == xv) OR (y == yv)
    void add_or(int x, bool xv, int y, bool yv) {
        int a = lit(x, xv), b = lit(y, yv);
        scc.add_edge(neg(a), b);
        scc.add_edge(neg(b), a);
    }
    // (x == xv) → (y == yv)
    void imply(int x, bool xv, int y, bool yv) {
        add_or(x, !xv, y, yv);
    }
    // x == value
    void set_value(int x, bool value) {
        scc.add_edge(lit(x, !value), lit(x, value));
    }
    // x == y
    void equal(int x, int y) {
        add_or(x, false, y, true);
        add_or(x, true, y, false);
    }
    // x ≠ y
    void different(int x, int y) {
        add_or(x, true, y, true);
        add_or(x, false, y, false);
    }
    // 禁止 (x == xv) 与 (y == yv) 同时成立
```

```
    void forbid(int x, bool xv, int y, bool yv) { add_or(x, !xv, y, !yv); }
    optional<vector<char>> solve() {
        scc.build();
        vector<char> ans(n);
        for (int x : views::iota(0, n)) {
            int f{ scc.bel[lit(x, false)] }, t{ scc.bel[lit(x, true)] };
            if (f == t) return nullopt;
            ans[x] = t < f;
        }
        return ans;
    }
};
```

建图原理

每个布尔变量 `x` 拆成两个文字节点

```
lit(x, false) = 2 * x;
lit(x, true)  = 2 * x + 1;
```

相反文字只需

```
p ^ 1
```

二元 OR

约束

$$a \vee b$$

等价于

$$\neg a \to b$$

和

$$\neg b \to a$$

因此

```
add_or(x, xv, y, yv);
```

表示

$$(x = xv) \vee (y = yv)$$

无解条件

若某个变量的真文字与假文字处于同一个 SCC

$$bel(x = false) = bel(x = true)$$

则两个状态能够互相推出，公式无解
反之一定存在可行赋值

接口说明

`add_or(x,xv,y,yv)`

加入

$$(x = xv) \vee (y = yv)$$

常见对应关系

约束	调用
$x \vee y$	`add_or(x, true, y, true)`
$\neg x \vee y$	`add_or(x, false, y, true)`
$x \vee \neg y$	`add_or(x, true, y, false)`
$\neg x \vee \neg y$	`add_or(x, false, y, false)`

`imply(x,xv,y,yv)`

加入

$$(x = xv) \to (y = yv)$$

例如

```
sat.imply(x, true, y, false);
```

表示

$$x \to \neg y$$

`set_value(x,value)`

强制

$$x = value$$

例如

```
sat.set_value(x, true);
```

`equal(x,y)`

加入

$$x = y$$

`different(x,y)`

加入

$$x \neq y$$

也就是两者恰好一个为真

`forbid(x,xv,y,yv)`

禁止两个选择同时发生

$$\neg((x = xv) \wedge (y = yv))$$

等价于

$$(x \neq xv) \vee (y \neq yv)$$

这是“每个物品二选一 + 某两个方案冲突”模型中最常用的接口之一

`solve()`

返回

```
optional<vector<char>>
```

无解为 `nullopt`
否则 `ans[x]` 为变量 `x` 的一组可行赋值

```
ans[x] == 0 // false
ans[x] == 1 // true
```

为什么赋值写成 `t < f`

当前 `SCC` 板子的编号满足缩点边

$$x \to y$$

时

$$x > y$$

即编号越小越靠近汇点
因此对变量 `x`

```
ans[x] = bel[true] < bel[false];
```

即可取得一组可行值
如果以后换成 SCC 编号方向相反的实现，这里的比较也必须反转

常见建模

至少一个为真

$$x \vee y$$

```
sat.add_or(x, true, y, true);
```

不能同时为真

$$\neg(x \wedge y)$$

```
sat.forbid(x, true, y, true);
```

等价于

```
sat.add_or(x, false, y, false);
```

不能同时选择两个指定方案

`x == xv` 与 `y == yv` 不能同时发生

```
sat.forbid(x, xv, y, yv);
```

二者相同

```
sat.equal(x, y);
```

表示

$$x = y$$

二者不同

```
sat.different(x, y);
```

表示

$$x \neq y$$

单向依赖

若选择方案 `(x == xv)` 就必须选择 `(y == yv)`

```
sat.imply(x, xv, y, yv);
```

完整示例

要求

- `x0` 或 `x1` 至少一个为真
- `x0=true` 时必须 `x2=false`
- `x1` 与 `x2` 不同
- 强制 `x2=false`

```
TwoSAT sat(3);
sat.add_or(0, true, 1, true);
sat.imply(0, true, 2, false);
sat.different(1, 2);
sat.set_value(2, false);

auto ans = sat.solve();

if (!ans) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int x : views::iota(0, 3)) cout << int((*ans)[x]) << " \n"[x == 2];
}
```

每个物品二选一

规定

```
choice[i] = false: 方案 0
choice[i] = true : 方案 1
```

若两个指定选择不能同时发生

```
for (auto [a, av, b, bv] : conflicts) sat.forbid(a, av, b, bv);
```

求解

```
auto choice = sat.solve();
if (!choice) {
    // 无合法选择
}
```

注意事项

- `TwoSAT(n)` 表示恰有 `n` 个布尔变量
- 变量编号为 `[0,n)`
- 一般使用者只需要 `add_or/imply/set_value/equal/different/forbid`
- 不再公开单边 `imply_node()`，避免漏加逆否边导致建图错误
- `solve()` 后底层 SCC 已完成构建，不能继续增加约束
- 若需要修改约束，应重新构造 `TwoSAT`
- 2-SAT 只能直接处理每个子句最多两个文字的 CNF
- 三个变量的 `a || b || c` 不能直接拆成若干二元 OR
- 大集合的“至多一个为真”若直接两两 `forbid()` 需要 $O(k^2)$ 个约束，可按题意引入辅助变量做线性编码
- 底层 Tarjan SCC 若使用递归实现，超深 implication graph 仍需注意递归栈

复杂度

设变量数为 n，加入的 implication graph 边数为 m
建图空间

$$O(n+m)$$

SCC 求解

$$O(n+m)$$

构造答案

$$O(n)$$

因此总时间复杂度

$$O(n+m)$$

总空间复杂度

$$O(n+m)$$

5.4.3 桥、割点与边双连通分量

一次 Lowlink 同时求：

- 桥
- 割点
- 边双连通分量 EBCC
  之后可以直接建立桥森林

```
struct UndirectedLowlink {
    struct Edge {
        int u, v;
    };
    struct Forest {
        int n;
        vector<array<int, 3>> edges;
        vector<vector<array<int, 2>>> adj;
        vector<int> siz, cnte;
        explicit Forest(int n_) : n(n_), adj(n), siz(n), cnte(n) {}
    };
    int n, timer{}, cnt{};
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> dfn, low, bel;
    vector<vector<int>> comp;
    vector<char> bridge, cut;
    bool built{};
    explicit UndirectedLowlink(int n_) : n(n_), adj(n), dfn(n), low(n), bel(n, -1), cut(n) {}
    int add_edge(int u, int v) {
        assert(!built);
        int id = edges.size();
        edges.push_back({u, v});
        adj[u].push_back(id);
        adj[v].push_back(id);
        bridge.push_back(false);
        return id;
    }
private:
    int other(int id, int u) const { return edges[id].u ^ edges[id].v ^ u; }
    // ===== Lowlink: 桥与割点核心 =====
    void dfs(int u, int parent_edge) {
        dfn[u] = low[u] = ++ timer;
        int child{ 0 };
        for (int id : adj[u]) {
            if (id == parent_edge) continue;
            int v{ other(id, u) };
            if (!dfn[v]) {
                ++ child;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = true;
                if (parent_edge != -1 &&
                    low[v] >= dfn[u]) {
                    cut[u] = true;
                }
```

```
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (parent_edge == -1 && child > 1) cut[u] = true;
    }
public:
    // ===== EBCC: 删除所有桥后求连通块 =====
    int build() {
        if (built) return cnt;
        for (int u : views::iota(0, n)) {
            if (!dfn[u]) dfs(u, -1);
        }
        vector<int> stk;
        stk.reserve(n);
        for (int s : views::iota(0, n)) {
            if (bel[s] != -1) continue;
            comp.push_back({});
            bel[s] = cnt;
            stk.push_back(s);
            while (!stk.empty()) {
                int u{ stk.back() };
                stk.pop_back();
                comp.back().push_back(u);
                for (int id : adj[u]) {
                    if (bridge[id]) continue;
                    int v{ other(id, u) };
                    if (bel[v] == -1) {
                        bel[v] = cnt;
                        stk.push_back(v);
                    }
                }
            }
            ++ cnt;
        }
        built = true;
        return cnt;
    }
    bool same_ebcc(int u, int v) const {
        assert(built);
        return bel[u] == bel[v];
    }
    // ===== 桥森林：若不需要缩点，可以删除 =====
    Forest forest() const {
        assert(built);
        Forest f(cnt);
        for (int c : views::iota(0, cnt)) {
            f.siz[c] = comp[c].size();
        }
        for (int id = 0; id < edges.size(); ++ id) {
            auto [u, v] = edges[id];
            int a{ bel[u] }, b{ bel[v] };
            if (a == b) {
                ++ f.cnte[a];
            } else {
                f.edges.push_back({a, b,id});
                f.adj[a].push_back({b, id});
                f.adj[b].push_back({a, id});
            }
        }
        return f;
    }
};
```

接口说明

`add_edge(u,v)`

加入无向边并返回稳定边编号：

```
int id = low.add_edge(u, v);
```

之后：

```
low.edges[id]
```

仍对应该原图边

`build()`

一次完成：

- `dfn / low`
- 桥标记
- 割点标记
- EBCC 划分
  并返回：

```
low.cnt
```

即边双连通分量数量

`bridge[id]`

编号为 `id` 的原图边是否是桥
对于 DFS 树边：

$$u \to v$$

判桥条件为：

$$low[v] > dfn[u]$$

`cut[u]`

原图点 `u` 是否为割点
非 DFS 根：

$$low[v] \geq dfn[u]$$

存在某个 DFS 子节点 `v` 即为割点
DFS 根节点则必须拥有至少两个 DFS 树子节点

`bel[u]`

原点所属 EBCC 编号：

$$0 \leq bel[u] < cnt$$

`comp[id]`

第 `id` 个 EBCC 包含的全部原图节点

`same_ebcc(u,v)`

判断两点是否属于同一个边双
等价地：

> 不存在删除某一条边后使 `u,v` 断开的情况

`forest()`

返回删桥缩点后的桥森林
成员：

```
Forest::n
```

EBCC 数量

```
Forest::edges[i] = {a, b, id}
```

表示 EBCC `a,b` 之间存在一条桥，对应原图边 `id`

```
Forest::adj[a]
```

中的：

```
{b, id}
```

表示森林相邻分量以及原图桥编号

```
Forest::siz[a]
```

分量 `a` 包含的原图节点数

```
Forest::cnte[a]
```

分量内部的原图边数

常见用法

求全部桥

```
UndirectedLowlink low(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    cin >> u >> v;
    low.add_edge(u, v);
}
low.build();
for (int id = 0; id < low.edges.size(); ++ id) {
    if (!low.bridge[id]) continue;
    auto [u, v] = low.edges[id];
    // 原图边 id: u - v 是桥
}
```

求全部割点

```
low.build();
for (int u : views::iota(0, n)) {
    if (low.cut[u]) {
        // u 是割点
    }
}
```

判断是否属于同一个边双

```
low.build();
if (low.same_ebcc(u, v)) {
    // u、v 位于同一个 EBCC
}
```

遍历所有 EBCC

```
low.build();
for (int c : views::iota(0, low.cnt)) {
    for (int u : low.comp[c]) {
        // 原图节点 u 属于 EBCC c
    }
}
```

建立桥森林

```
auto forest = low.forest();
for (int u : views::iota(0, forest.n)) {
```

```
    for (auto [v, id] : forest.adj[u]) {
        // EBCC u -- v
        // 对应原图桥 id
    }
}
```

若原图连通，压缩后是一棵树；原图不连通时是一片森林

注意事项

父边而不是父点

无向图 Tarjan 最重要的细节是：

```
if (id == parent_edge) continue;
```

不能写成：

```
if (v == parent) continue;
```

例如存在两条平行边：

```
u ===== v
```

DFS 使用其中一条作为树边时，另一条仍然是一条返祖边
如果按父节点跳过，会把两条边全部跳过，错误地把树边判成桥

重边与自环

当前版本完整支持：

- 非连通图
- 重边
- 自环
- 孤立点
  平行边中的任意一条都不可能单独成为桥
  自环：
- 不会成为桥
- 不影响割点
- 属于所在 EBCC 内部边
- 在 `Forest::cnte` 中计数一次
  孤立点本身构成一个 EBCC

递归深度

Lowlink 使用递归 DFS
理论复杂度虽然只有：

$$O(n+m)$$

但极长链图的递归深度可以达到：

$$O(n)$$

若评测栈空间特别严格，需要改为显式栈版本；普通竞赛环境下递归 Tarjan 通常代码更短、更可靠

复杂度

设点数为 n，边数为 m
`add_edge()`：

$$O(1)$$

Lowlink：

$$O(n+m)$$

删除桥后的 EBCC 染色：

$$O(n+m)$$

所以 `build()` 总计：

$$O(n+m)$$

`same_ebcc()`：

$$O(1)$$

`forest()`：

$$O(n+m)$$

总空间：

$$O(n+m)$$

5.4.4 点双连通分量（VBCC）与圆方森林

```
struct VBCC {
    struct Edge { int u, v; };
    int n, timer{}, cnt{};
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> dfn, low, stk;
    vector<vector<int>> comp, belong;
    vector<char> cut;
    bool built{};
    explicit VBCC(int n_) : n(n_), adj(n), dfn(n), low(n), belong(n), cut(n) { stk.reserve(n); }
    int add_edge(int u, int v) {
        assert(!built && u != v);
        int id = edges.size();
        edges.push_back({u, v});
        adj[u].push_back(id);
        adj[v].push_back(id);
        return id;
    }
private:
    int other(int id, int u) const {
        auto [x, y] = edges[id];
        return x ^ y ^ u;
    }
    // ===== Tarjan 核心 =====
    void dfs(int u, int parent_edge) {
        dfn[u] = low[u] = ++ timer;
        stk.push_back(u);
        for (int id : adj[u]) {
            if (id == parent_edge) continue;
            int v{ other(id, u) };
            if (!dfn[v]) {
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    comp.push_back({u});
                    while (true) {
                        int x{ stk.back() };
                        stk.pop_back();
                        comp.back().push_back(x);
                        if (x == v) break;
                    }
                }
            } else {
                low[u] = min(low[u], dfn[v]);
            }
```

```
        }
    }
public:
    int build() {
        if (built) return cnt;
        for (int s : views::iota(0, n)) {
            if (dfn[s]) continue;
            if (adj[s].empty()) {
                dfn[s] = low[s] = ++ timer;
                comp.push_back({s});
            } else {
                dfs(s, -1);
                stk.clear();
            }
        }
        cnt = comp.size();
        for (int cid : views::iota(0, cnt)) {
            for (int u : comp[cid]) belong[u].push_back(cid);
        }
        for (int u : views::iota(0, n)) cut[u] = belong[u].size() > 1;
        built = true;
        return cnt;
    }
    // ===== 圆方森林：若不需要树上处理，可以删除 =====
    vector<vector<int>> block_cut_forest() const {
        assert(built);
        vector<vector<int>> tr(n + cnt);
        for (int cid : views::iota(0, cnt)) {
            int b{ n + cid };
            for (int u : comp[cid]) {
                tr[u].push_back(b);
                tr[b].push_back(u);
            }
        }
        return tr;
    }
};
```

接口说明

定义

本板采用竞赛中常见的点双定义

- 一个桥及其两个端点构成一个大小为 2 的 VBCC
- 一个孤立点单独构成大小为 1 的 VBCC
- 一个割点可以同时属于多个 VBCC
- 两个不同 VBCC 至多共享一个点，共享点一定是割点
- 每条非自环无向边恰好属于一个 VBCC
  因此不能像 EBCC 一样给每个点维护唯一的 `bel[u]`，而应使用

```
belong[u]
```

记录点 `u` 所属的全部 VBCC

`add_edge(u,v)`

加入一条无向边并返回边编号

```
int id = vbcc.add_edge(u, v);
```

点和边均为 `0base`
允许

- 重边
- 非连通图
  当前版本不接受自环

```
assert(u != v);
```

每条无向边只调用一次 `add_edge()`，内部会自动加入两个方向

`build()`

```
int cnt = vbcc.build();
```

完成

- `dfn / low`
- 全部 VBCC
- 每个点所属 VBCC
- 割点判断
  返回 VBCC 数量
  构建后不能继续 `add_edge()`

`comp[cid]`

第 `cid` 个 VBCC 包含的原图节点

```
for (int u : vbcc.comp[cid]) {}
```

`belong[u]`

节点 `u` 所属的全部 VBCC 编号

```
for (int cid : vbcc.belong[u]) {}
```

普通点通常满足

$$|belong[u]| = 1$$

割点满足

$$|belong[u]| \geq 2$$

`cut[u]`

节点 `u` 是否为割点
当前实现直接利用

$$u \text{ 是割点} \iff |belong[u]| > 1$$

因此 Tarjan 主体中不再需要单独维护根节点儿子数

`block_cut_forest()`

建立圆方森林
编号约定

- `[0,n)` 为原图圆点
- `[n,n+cnt)` 为方点
- 方点 `n + cid` 对应 `comp[cid]`
  若原图节点 `u` 属于 VBCC `cid`，圆方森林中加入

  $$u \leftrightarrow n + cid$$

  调用

```
auto tr = vbcc.block_cut_forest();
```

原图连通时得到一棵圆方树，原图不连通时得到圆方森林

Tarjan 原理

DFS 中维护

- `dfn[u]`：节点 `u` 的 DFS 时间戳
- `low[u]`：`u` 的 DFS 子树通过至多一条返祖边能到达的最小 `dfn`
  对于 DFS 树边

  $$u \to v$$

  若

  $$low[v] \geq dfn[u]$$

则 `v` 的 DFS 子树无法绕过 `u` 到达 `u` 的祖先
此时从点栈中不断弹出直到 `v`，再补上 `u`，恰好得到一个 VBCC

```
comp.push_back({u});
while (true) {
    int x = stk.back();
    stk.pop_back();
    comp.back().push_back(x);
    if (x == v) break;
}
```

注意 `u` 本身不会在这里被弹出，因此同一个割点可以继续参与之后形成的其他 VBCC

为什么使用点栈

原版本使用边栈，每得到一个 VBCC 后需要遍历块中所有边，再通过 `mark` 对端点去重
点栈可以直接得到块内节点

```
树边满足 low[v] ⩾ dfn[u]
→ 弹出栈顶节点直到 v
→ 加入 u
```

因此可以删除

- `estk`
- `mark`
- `make_comp()`
- 块内端点去重
  代码更短，分量总复杂度仍为线性

重边为什么安全

DFS 必须跳过父边编号

```
if (id == parent_edge) continue;
```

不能按父节点跳过

```
if (v == parent) continue; // 错误
```

例如存在两条平行边

```
u ===== v
```

其中一条作为 DFS 树边，另一条仍然是返祖边，会使

$$low[v] \leq dfn[u]$$

若按父节点跳过，会把第二条边也忽略
本实现按边编号处理，因此重边安全

圆方森林性质

圆方森林是二分图

- 圆点表示原图节点
- 方点表示 VBCC
- 圆点与方点相连，当且仅当该原图节点属于该 VBC
  任意原图简单路径经过的一系列 VBCC，可以转化成圆方森林中的唯一路径
  因此很多一般无向图问题可以转成树上问题，例如
- 路径上是否经过某割点
- 两点路径涉及多少个点双
- 修改点权后的路径统计
- 圆方树 LCA
- HLD / 树形 DP
- 路径上的块信息维护

注意事项

- VBCC 与 EBCC 不同，EBCC 中每个点只有唯一分量编号，VBCC 中割点可以属于多个分量
- 当前实现使用点栈，要求输入不含自环

- 若题目只关心不同节点之间的点双结构，可以读入时直接忽略 `u == v`
- 若题目明确要求自环所属块，应先确认题目对自环 VBCC 的定义，再单独处理
- 重边完整支持，父边必须按边编号跳过
- 孤立点会得到一个单点 VBCC
- 单条桥会得到一个包含两个端点的 VBCC
- `comp[cid]` 内节点顺序只与 DFS 和弹栈顺序有关
- `belong[u]` 中编号顺序没有拓扑或大小含义
- `build()` 只能真正执行一次，之后再次调用只返回已有 `cnt`
- 构建完成后不能继续加入边，多测时重新构造对象
- 圆方森林中的方点不是真实原图节点，树上统计原点数量或原点权值时必须区分圆点和方点
- 原图不连通时得到森林，LCA/HLD 应逐棵预处理，或按题意增加虚拟超级根
- 递归深度最坏为 $O(n)$，极深链图可能需要扩大程序栈或改成显式栈版本

常见用法

输出全部 VBCC

```
VBCC vbcc(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    cin >> u >> v;
    -- u, -- v;
    if (u != v) vbcc.add_edge(u, v);
}
vbcc.build();
cout << vbcc.cnt << '\n';
for (const auto &vertices : vbcc.comp) {
    cout << vertices.size();
    for (int u : vertices) cout << ' ' << u + 1;
    cout << '\n';
}
```

求全部割点

```
vbcc.build();
for (int u : views::iota(0, n)) {
    if (vbcc.cut[u]) {
        // u 是割点
    }
}
```

查询一个点属于哪些 VBCC

```
vbcc.build();
for (int cid : vbcc.belong[u]) {
    // u 属于 comp[cid]
}
```

特别地

```
bool is_cut = vbcc.belong[u].size() > 1;
```

与

```
vbcc.cut[u]
```

等价

建立圆方森林

```
vbcc.build();
auto tr = vbcc.block_cut_forest();
for (int u : views::iota(0, int(tr.size()))) {
    if (u < n) {
        // 原图圆点 u
    } else {
        int cid = u - n;
        // 方点，对应 comp[cid]
```

```
    }
}
```

给圆点和方点分别赋权

```
auto tr = vbcc.block_cut_forest();
vector<i64> weight(tr.size());
for (int u : views::iota(0, n)) weight[u] = original_weight[u];
for (int cid : views::iota(0, vbcc.cnt)) {
    int b{ n + cid };
    weight[b] = vbcc.comp[cid].size();
}
```

最小自测

```
孤立点
0
⇒ {0}，无割点

单边
0 -- 1
⇒ {0,1}，无割点

三点链
0 -- 1 -- 2
⇒ {0,1}、{1,2}，1 是割点

三角形
0 -- 1
 \  /
   2
⇒ 一个 {0,1,2}，无割点

两个三角形共点
⇒ 两个 VBCC，共点属于两个块且是割点

两点间两条平行边
⇒ 一个 {0,1}，无割点
```

复杂度

设原图有 n 个点、m 条非自环无向边
Tarjan：

$$O(n+m)$$

建立 `belong`：

$$O\left(\sum_C |C|\right)$$

点双连通分量的节点总出现次数满足

$$\sum_C |C| = O(n+m)$$

因此 `build()` 总时间复杂度为

$$O(n+m)$$

建立圆方森林：

$$O(n+m)$$

总空间复杂度：

$$O(n+m)$$

递归栈最坏：

$$O(n)$$

5.4.5 支配树

使用 Lengauer--Tarjan 算法，只处理从 `root` 可达的部分

```
struct DominatorTree {
    int n, root, timer{};
    vector<vector<int>> g, tree;
    vector<int> dfn, rev, parent, semi, idom_idx, uf, label;
    vector<int> idom, tin, tout;
    bool built{};
    DominatorTree(int n_, int root_) : n(n_), root(root_), g(n), tree(n),
        dfn(n, -1), rev(n), parent(n, -1), idom(n, -1) {}
    void add_edge(int u, int v) {
        assert(!built);
        g[u].push_back(v);
    }
private:
    // ===== DFS 编号 =====
    void dfs(int u) {
        dfn[u] = timer;
        rev[timer ++] = u;
        for (int v : g[u]) if (dfn[v] == -1) {
            dfs(v);
            parent[dfn[v]] = dfn[u];
        }
    }
    // ===== Lengauer--Tarjan 并查集 eval/link =====
    void compress(int v) {
        if (uf[uf[v]] == uf[v]) return;
        compress(uf[v]);
        if (semi[label[uf[v]]] < semi[label[v]]) label[v] = label[uf[v]];
        uf[v] = uf[uf[v]];
    }
    int eval(int v) {
        if (uf[v] == v) return label[v];
        compress(v);
        return label[v];
    }
    void dfs_tree(int u, int &clk) {
        tin[u] = clk ++;
        for (int v : tree[u]) dfs_tree(v, clk);
        tout[u] = clk;
    }
public:
    const vector<int> &build() {
        if (built) return idom;
        assert(0 <= root && root < n);
        dfs(root);
        int r{ timer };
        vector<vector<int>> pred(r), bucket(r);
        for (int u : views::iota(0, n)) if (dfn[u] != -1) {
            for (int v : g[u]) if (dfn[v] != -1) pred[dfn[v]].push_back(dfn[u]);
        }
        semi.resize(r);
        idom_idx.assign(r, -1);
        uf.resize(r);
        label.resize(r);
        ranges::iota(semi, 0);
        ranges::iota(uf, 0);
        ranges::iota(label, 0);
        // ===== 半支配点 =====
        for (int w = r - 1; w >= 1; -- w) {
            for (int v : pred[w]) semi[w] = min(semi[w], semi[eval(v)]);
            bucket[semi[w]].push_back(w);
            uf[w] = parent[w];
            int p{ parent[w] };
            for (int v : bucket[p]) {
                int u{ eval(v) };
                idom_idx[v] = semi[u] < semi[v] ? u : p;
            }
            bucket[p].clear();
        }
        // ===== 立即支配点 =====
```

```
        for (int w = 1; w < r; ++ w) {
            if (idom_idx[w] != semi[w]) idom_idx[w] = idom_idx[idom_idx[w]];
        }
        idom[root] = root;
        for (int w = 1; w < r; ++ w) idom[rev[w]] = rev[idom_idx[w]];
        // ===== 支配树与 O(1) 支配查询 =====
        for (int v : views::iota(0, n)) {
            if (v != root && idom[v] != -1) tree[idom[v]].push_back(v);
        }
        tin.assign(n, -1);
        tout.assign(n, -1);
        int clk = 0;
        dfs_tree(root, clk);
        built = true;
        return idom;
    }
    bool reachable(int u) const {
        assert(built);
        return idom[u] != -1;
    }
    bool dominates(int u, int v) const {
        assert(built);
        if (tin[u] == -1 || tin[v] == -1) return false;
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
};
```

定义

在以 `root` 为起点的有向图中，若从 `root` 到 `v` 的每一条路径都经过 `u`，则称 `u` 支配 `v`
记为

$$u \operatorname{dom} v$$

除 `v` 本身外最靠近 `v` 的支配点称为立即支配点

$$idom[v]$$

全部立即支配关系构成支配树

接口

`build()` 返回 `idom`

```
auto &idom = dom.build();
```

约定

```
idom[root] = root;
idom[v] = -1; // root 无法到达 v
```

`tree[u]` 为支配树中 `u` 的儿子
`reachable(u)` 判断 `root` 是否可以到达 `u`
`dominates(u,v)` 判断 `u` 是否支配 `v`
利用支配树性质

$$u \operatorname{dom} v$$

当且仅当 `u` 是支配树中 `v` 的祖先，因此 Euler 序预处理后查询为 $O(1)$

常见用法

```
DominatorTree dom(n, root);
for (auto [u, v] : edges) dom.add_edge(u, v);
dom.build();
if (dom.dominates(u, v)) {
    // 所有 root → v 路径都必须经过 u
}
for (int v : views::iota(0, n)) {
    if (dom.idom[v] == -1) continue;
```

```
    // v 从 root 可达
}
```

常见应用

- 控制流图
- 判断某点是否是所有路径必经点
- 删除某点后哪些节点必然与根断开
- CFG 优化
- 有向图必经路径问题

注意事项

- 支配关系相对于固定根 `root`
- 不可达节点不属于支配树，`idom[v] == -1`
- `root` 支配所有可达节点，并规定 `idom[root] = root`
- 支持重边和自环
- 构造完成后不能继续加边
- 当前 DFS 与支配树 DFS 都使用递归，极长链可能需要扩大栈或改显式栈

复杂度

Lengauer–Tarjan 配合并查集路径压缩近似线性

$$O((n+m)\alpha(n))$$

空间

$$O(n+m)$$

`dominates()`

$$O(1)$$

5.5 Euler 与基环结构

5.5.1 Euler 路与回路

使用非递归 Hierholzer，同时支持有向图和无向图

```
struct EulerTrail {
    struct Arc {
        int to, id;
    };
    struct Result {
        vector<int> vertices, edges;
        bool is_circuit() const { return edges.empty() || vertices.front() == vertices.back(); }
    };
    int n, m{};
    bool directed;
    vector<vector<Arc>> g;
    vector<int> in, out, deg;
    EulerTrail(int n_, bool directed_) : n(n_), directed(directed_), g(n), in(n), out(n), deg(n) {}
    int add_edge(int u, int v) {
        int id{ m ++ };
        g[u].push_back({v, id});
        if (directed) ++ out[u], ++ in[v];
        else g[v].push_back({u, id}), ++ deg[u], ++ deg[v];
        return id;
    }
    optional<Result> solve() const {
        if (!m) {
            Result res;
            if (n) res.vertices.push_back(0);
            return res;
        }
        // ===== 度数条件与起点 =====
```

```
        int start{ -1 };
        if (directed) {
            int plus{}, minus{};
            for (int u : views::iota(0, n)) {
                int d = out[u] - in[u];
                if (d == 1) start = u, ++ plus;
                else if (d == -1) ++ minus;
                else if (d != 0) return nullopt;
            }
            if (!((plus == 1 && minus == 1) || (!plus && !minus))) return nullopt;
            if (!plus) {
                for (int u : views::iota(0, n)) if (out[u]) {
                    start = u;
                    break;
                }
            }
        } else {
            vector<int> odd;
            for (int u : views::iota(0, n)) if (deg[u] & 1) odd.push_back(u);
            if (odd.size() != 0 && odd.size() != 2) return nullopt;
            if (odd.size() == 2) start = odd[0];
            else {
                for (int u : views::iota(0, n)) if (deg[u]) {
                    start = u;
                    break;
                }
            }
        }
        // ===== 非递归 Hierholzer =====
        vector<int> it(n);
        vector<char> used(m);
        vector<pair<int, int>> stk{{start, -1}};
        Result res;
        while (!stk.empty()) {
            int u{ stk.back().first };
            while (it[u] < g[u].size() && used[g[u][it[u]].id]) ++ it[u];
            if (it[u] == g[u].size()) {
                auto [v, id] = stk.back();
                stk.pop_back();
                res.vertices.push_back(v);
                if (id != -1) res.edges.push_back(id);
            } else {
                auto [v, id] = g[u][it[u] ++];
                used[id] = true;
                stk.push_back({v, id});
            }
        }
        if (res.edges.size() != m) return nullopt;
        ranges::reverse(res.vertices);
        ranges::reverse(res.edges);
        return res;
    }
};
```

接口说明

`EulerTrail(n, directed)`

```
EulerTrail euler(n, false); // 无向图
EulerTrail directed_euler(n, true);  // 有向图，独立实例
```

点编号为 `[0,n)`
支持

- 重边
- 自环
- 非连通图输入
  只有所有非零度边最终位于同一可行 Euler 部分时 `solve()` 才会成功

`add_edge(u,v)`

加入一条边并返回原边编号

```
int id = euler.add_edge(u, v);
```

无向边也只调用一次
两个邻接方向共享同一个 `id`，因此 Hierholzer 不会把一条无向边使用两次

`solve()`

返回

```
optional<EulerTrail::Result>
```

无解时为 `nullopt`
有解时

```
res.vertices
```

是经过的点序列

```
res.edges
```

是依次经过的原边编号
始终满足

$$|vertices| = m + 1$$

$$|edges| = m$$

且第 `i` 条边连接

```
vertices[i] -> vertices[i + 1]
```

有向图表示实际方向，无向图则表示本次 Euler 路使用该边的方向

`is_circuit()`

判断结果是否为 Euler 回路

```
if (res.is_circuit()) {}
```

存在条件

无向图

忽略孤立点后必须连通，并且奇度点数量只能是 0 或2
奇度点数为 `0` 时存在 Euler 回路
奇度点数为 `2` 时存在 Euler 路，两个奇度点分别为起点和终点

有向图

Euler 回路要求所有非零度节点满足

$$in(u) = out(u)$$

Euler 路要求恰有一个起点满足

$$out(u) = in(u) + 1$$

恰有一个终点满足

$$in(v) = out(v) + 1$$

其余点满足

$$in(x) = out(x)$$

此外所有边必须能够被同一次 Hierholzer 遍历使用
本实现不额外跑连通性 DFS，而是在最后检查

```
res.edges.size() == m
```

若有未遍历到的边直接返回 `nullopt`

常见用法

无向 Euler 路

```
EulerTrail euler(n, false);
for (int i = 0; i < m; ++ i) {
    int u, v;
    cin >> u >> v;
    -- u, -- v;
    euler.add_edge(u, v);
}
auto ans = euler.solve();
if (!ans) {
    cout << "No Solution\n";
} else {
    for (int u : ans->vertices) cout << u + 1 << ' ';
}
```

输出边编号

```
auto ans = euler.solve();
if (ans) {
    for (int id : ans->edges) {
        // 第 id 条输入边
    }
}
```

恢复无向边实际经过方向

```
auto ans = euler.solve();
if (ans) {
    for (int i : views::iota(0, int(ans->edges.size()))) {
        int id = ans->edges[i];
        int u = ans->vertices[i];
        int v = ans->vertices[i + 1];
        // 原边 id 本次按 u -> v 使用
    }
}
```

字典序

当前代码按邻接表已有顺序选择下一条边，不保证字典序
若题目要求字典序最小，可以在 `solve()` 前按题意排序每个邻接表
例如优先经过编号更小的终点

```
for (auto &adj : euler.g) {
    ranges::sort(adj, [](auto a, auto b) {
        return pair{a.to, a.id} < pair{b.to, b.id};
    });
}
```

有重边时必须明确题目究竟比较点序列还是边编号序列

注意事项

- 无向边只调用一次 `add_edge()`
- 自环在无向图中贡献度数 `2`
- `solve()` 不修改图，可以重复调用
- 空边图在 `n > 0` 时返回单点序列 `{0}`
- `n == 0` 且无边时返回空点序列，但仍视为成功
- 返回空 `optional` 才表示不存在 Euler 路
- 本板只处理纯有向图或纯无向图

- 混合图 Euler 定向需要额外的流量平衡建模，不应写进本板的覆盖范围
- Hierholzer 使用显式栈，不存在递归深度问题

复杂度

每个邻接项只扫描一次，每条边只使用一次

$$O(n+m)$$

空间复杂度

$$O(n+m)$$

5.5.2 函数图

解决问题： 每个点恰有一条出边时，分解环与反向树，并处理入环点、到环距离、第 k 次跳转、有向距离和两点同步首次相遇

```
struct FunctionalGraph {
    int n, lg;
    vector<int> to;
    vector<vector<int>> rev, cycles, up;
    vector<char> in_cycle;
    vector<int> cycle_id, cycle_pos, entry, depth, comp_size;
    explicit FunctionalGraph(vector<int> to_) : n(to_.size()), to(move(to_)), rev(n),
        in_cycle(n), cycle_id(n, -1), cycle_pos(n, -1), entry(n, -1), depth(n, -1) {
        build();
    }
private:
    int lift(int u, int k) const {
        for (int b = 0; k; ++ b, k >>= 1) if (k & 1) u = up[b][u];
        return u;
    }
    // ===== 入度剥离 + 找环 + 反图 BFS + 倍增 =====
    void build() {
        vector<int> indeg(n);
        for (int u : views::iota(0, n)) {
            assert(0 <= to[u] && to[u] < n);
            rev[to[u]].push_back(u);
            ++ indeg[to[u]];
        }
        queue<int> q;
        for (int u : views::iota(0, n)) if (!indeg[u]) q.push(u);
        ranges::fill(in_cycle, true);
        while (!q.empty()) {
            int u{ q.front() };
            q.pop();
            in_cycle[u] = false;
            if (-- indeg[to[u]] == 0) q.push(to[u]);
        }
        // 剥离后剩余点全部位于环上
        for (int s : views::iota(0, n)) if (in_cycle[s] && cycle_id[s] == -1) {
            int cid{ int(cycles.size()) }, u{ s };
            cycles.push_back({});
            do {
                cycle_id[u] = cid;
                cycle_pos[u] = cycles.back().size();
                entry[u] = u;
                depth[u] = 0;
                cycles.back().push_back(u);
                u = to[u];
            } while (u != s);
            comp_size.push_back(cycles.back().size());
        }
        // 从所有环点沿反图向外扩展
        for (int u : views::iota(0, n)) if (in_cycle[u]) q.push(u);
        while (!q.empty()) {
            int u{ q.front() };
            q.pop();
```

```
            for (int v : rev[u]) {
                if (depth[v] != -1) continue;
                cycle_id[v] = cycle_id[u];
                entry[v] = entry[u];
                depth[v] = depth[u] + 1;
                ++ comp_size[cycle_id[v]];
                q.push(v);
            }
        }
        lg = bit_width((unsigned)max(1, n));
        up.assign(lg, vector<int>(n));
        if (n) up[0] = to;
        for (int k = 1; k < lg; ++ k) {
            for (int u : views::iota(0, n)) up[k][u] = up[k - 1][up[k - 1][u]];
        }
    }
public:
    bool on_cycle(int u) const { return in_cycle[u]; }
    bool same_component(int u, int v) const { return cycle_id[u] == cycle_id[v]; }
    int component_count() const { return cycles.size(); }
    int cycle_size(int u) const { return cycles[cycle_id[u]].size(); }
    int component_size(int u) const { return comp_size[cycle_id[u]]; }
    int orbit_size(int u) const { return depth[u] + cycle_size(u); }
    // ===== 任意大的 k 次跳转 =====
    int jump(int u, u64 k) const {
        if (k <= u64(depth[u])) return lift(u, int(k));
        k -= depth[u];
        u = entry[u];
        const auto &cyc = cycles[cycle_id[u]];
        return cyc[(cycle_pos[u] + k % cyc.size()) % cyc.size()];
    }
    // ===== u 沿有向边到 v 的最少步数，不可达返回 -1 =====
    i64 distance(int u, int v) const {
        if (!same_component(u, v)) return -1;
        if (!on_cycle(v)) {
            if (depth[u] < depth[v]) return -1;
            int d{ depth[u] - depth[v] };
            return lift(u, d) == v ? d : -1;
        }
        int len{ cycle_size(u) };
        int around{ (cycle_pos[v] - cycle_pos[entry[u]] + len) % len };
        return i64(depth[u]) + around;
    }
    // ===== 两点每秒同时沿 to 移动，返回首次相遇的 {时间, 节点} =====
    optional<pair<i64, int>> first_meeting(int u, int v) const {
        if (!same_component(u, v)) return nullopt;
        if (u == v) return pair<i64, int>{0, u};
        // 深度相同时，只可能在同一棵反向树中先汇合
        if (depth[u] == depth[v]) {
            if (entry[u] != entry[v]) return nullopt;
            int a{ u }, b{ v }, walked{ 0 }, d{ depth[u] };
            for (int k = lg - 1; k >= 0; -- k) {
                int step = 1 << k;
                if (walked + step <= d && up[k][a] != up[k][b]) {
                    a = up[k][a];
                    b = up[k][b];
                    walked += step;
                }
            }
            return pair<i64, int>{walked + 1, up[0][a]};
        }
        // 深度不同不可能在树上同步相遇，只可能进入环后相遇
        int len{ cycle_size(u) };
        auto phase = [&](int x) {
            int p = (cycle_pos[entry[x]] - depth[x]) % len;
            return p < 0 ? p + len : p;
        };
        if (phase(u) != phase(v)) return nullopt;
        i64 t = max(depth[u], depth[v]);
        return pair<i64, int>{t, jump(u, t)};
```

```
    }
};
```

结构

函数图满足每个节点恰有一条出边

```
to[u]
```

表示

$$u \to to[u]$$

每个弱连通分量都恰好由

```
一个有向环
+
若干棵指向环的树
```

组成
因此可以通过入度剥离一次性得到全部环

```
反复删除入度为 0 的点
→ 最后没有被删除的点恰好是所有环点
```

之后从所有环点沿反图 BFS，即可确定每个树点所属的环和入环位置

成员变量

`cycle_id[u]`

节点 `u` 所属弱连通分量的环编号
两个点位于同一弱连通分量当且仅当

```
cycle_id[u] == cycle_id[v]
```

`cycles[id]`

第 `id` 个环按有向边顺序保存的全部节点

```
for (int u : fg.cycles[id]) {}
```

若

```
fg.cycles[id] = {a, b, c};
```

则有

```
a → b → c → a
```

`cycle_pos[u]`

仅对环点有意义
表示环点 `u` 在 `cycles[cycle_id[u]]` 中的位置
非环点保持 `-1`

`entry[u]`

从 `u` 不断沿 `to` 前进时首次到达的环点
若 `u` 本身位于环上

```
entry[u] == u
```

`depth[u]`

从 `u` 到环的距离

$$depth[u] = \min\{k \geq 0 : f^k(u) \text{ 位于环上}\}$$

因此

```
fg.on_cycle(u)
```

等价于

```
fg.depth[u] == 0
```

`comp_size[id]`

环 `id` 所在整个弱连通分量的节点数

`rev[u]`

函数图的反图邻接表

```
v ∈ rev[u]
```

表示

```
to[v] = u
```

去掉环与环之间的边以后，`rev` 中每个环点都是一棵树的根

常用接口

判断是否在环上

```
if (fg.on_cycle(u)) {}
```

判断是否属于同一弱连通分量

```
if (fg.same_component(u, v)) {}
```

环长

```
int len = fg.cycle_size(u);
```

连通分量大小

```
int siz = fg.component_size(u);
```

从一个点出发最终访问多少个不同节点

```
int cnt = fg.orbit_size(u);
```

因为先经过 `depth[u]` 个树点，再完整经过所属环

$$orbit_size(u) = depth[u] + cycle_size(u)$$

第 k 次跳转

```
int v = fg.jump(u, k);
```

返回

$$f^k(u)$$

`k` 使用 `u64`，可以远大于 `n`
实现先跳到环

```
u → ... → entry[u]
```

剩余步数再对环长取模，因此不需要建立 64 层倍增
例如

```
u64 k = 10000000000000000000ULL;
int v = fg.jump(u, k);
```

仍然可以直接处理

为什么倍增只需要 $O(\log n)$ 层

倍增只负责从树点跳到环
最多需要

$$depth[u] \le n$$

一旦进入环，剩余步数直接对环长取模
因此只需要

$$O(\log n)$$

层

有向距离

```
i64 d = fg.distance(u, v);
```

返回从 `u` 不断沿 `to` 前进首次到达 `v` 所需的步数
不可达返回 `-1`

`v` 不在环上

此时 `v` 必须位于 `u` 到环的唯一路径上
首先必须满足

$$depth[u] \ge depth[v]$$

令

$$d = depth[u] - depth[v]$$

再检查

```
fg.jump(u, d) == v
```

`v` 位于环上

先从 `u` 到达 `entry[u]`
花费

$$depth[u]$$

再沿环走到 `v`
因此总距离为

$$depth[u] + \text{环上顺时针距离}$$

两点同步首次相遇

```
auto meet = fg.first_meeting(u, v);
```

表示 `u` 与 `v` 每次同时执行

```
u = to[u];
v = to[v];
```

返回首次相遇的

```
{时间, 相遇节点}
```

永远不会相遇则返回 `nullopt`

初始深度相同

若

$$depth[u] = depth[v]$$

只有 `entry[u] == entry[v]` 时才可能相遇
此时两条树链最终汇合，可以倍增寻找第一次汇合位置

初始深度不同

若

$$depth[u] \neq depth[v]$$

两点不可能在树上同步相遇
因为每经过一步，两者剩余到环距离同时减一，深度差始终不变
只能等两点都进入环以后判断是否处于同一个同步相位
对环长 L 定义

$$phase(x) = cycle_pos[entry[x]] - depth[x] \pmod{L}$$

若

$$phase(u) \neq phase(v)$$

则永远不会相遇
否则首次相遇时刻为

$$\max(depth[u], depth[v])$$

常见用法

基础分解

```
vector<int> to(n);
for (int &v : to) {
    cin >> v;
    -- v;
}
FunctionalGraph fg(to);
for (int u : views::iota(0, n)) {
    cout << "cycle = " << fg.cycle_id[u] << '\n';
    cout << "entry = " << fg.entry[u] << '\n';
    cout << "depth = " << fg.depth[u] << '\n';
}
```

遍历所有环

```
for (int id : views::iota(0, fg.component_count())) {
    for (int u : fg.cycles[id]) {
        // u 是该环上的节点
    }
}
```

大步跳转

```
int v = fg.jump(u, k);
```

判断一个点能否沿有向边到达另一个点

```
i64 d = fg.distance(u, v);
if (d == -1) {
    // 不可达
} else {
```

```
    // 恰好 d 步到达
}
```

两个人同步移动首次相遇

```
auto res = fg.first_meeting(u, v);
if (!res) {
    // 永不相遇
} else {
    auto [time, node] = *res;
}
```

快速回忆

```
每点出度 = 1

入度剥离
    未被删除的点 = 环点

环点反图 BFS
    cycle_id = 属于哪个环
    entry    = 从哪个环点入环
    depth    = 到环距离

jump(u,k)
    k ≤ depth[u]: 倍增
    否则: 到 entry 后对环长取模

distance(u,v)
    v 在树上: 检查 v 是否是 u 的祖先
    v 在环上: depth[u] + 环上距离

first_meeting(u,v)
    depth 相同: 同 entry 后找树上首次汇合
    depth 不同: 比较环上同步 phase
```

注意事项

- 函数图要求每个节点恰好一条出边
- 自环是长度为 1 的合法环
- 不同弱连通分量之间永远不可达也永远不会相遇
- 每个弱连通分量恰好有一个有向环
- `cycle_pos[u]` 仅对环点有效，树点应使用 `cycle_pos[entry[u]]`
- `depth[u]` 是到环距离，不是一般意义上的树深度
- `rev` 包含环内部的反向边，若只遍历挂在环上的树，需要跳过已经位于环上的邻点
- 函数图与无向图中的基环树不同，不要混用板子
- 当前构造函数完成全部预处理，修改 `to` 后必须重新构造
- 若只需要找环和入环信息，可以删除 `up/lift/jump/distance/first_meeting`
- 若只需要第 k 次跳转，可以保留入度剥离、环分解与 `jump()`，删除同步相遇相关代码

复杂度

设节点数为 n
入度剥离与环分解

$$O(n)$$

反图 BFS

$$O(n)$$

倍增预处理

$$O(n \log n)$$

因此总预处理

$$O(n \log n)$$

空间复杂度

$$O(n \log n)$$

查询复杂度

接口	复杂度
on_cycle()	$O(1)$
same_component()	$O(1)$
cycle_size()	$O(1)$
component_size()	$O(1)$
orbit_size()	$O(1)$
jump()	$O(\log n)$
distance()	$O(\log n)$
first_meeting()	$O(\log n)$

5.5.3 仙人掌图

解决问题： 判定并分解仙人掌图，提取所有环，构造圆方树，并处理环上距离、两点最短路、路径经过环数等问题

定义

无向图是仙人掌图，当且仅当每条边至多属于一个简单环
等价地，每个点双连通分量只能是

- 一条桥
- 一个简单环
  不同简单环可以共享一个割点，但不能共享边，也不能共享两个及以上不同顶点
  支持**不连通图**，此时每个连通分量都必须是仙人掌，也可以称为仙人掌森林

默认边权为 `i64`，不传权值时为 `1`

```
template<class W = i64>
struct Cactus {
    struct Edge {
        int u, v;
        W w;
    };
    struct Cycle {
        // edges[i] 连接 vertices[i] → vertices[(i + 1) % size()]
        vector<int> vertices, edges;
        vector<W> pref;
        W total{};
        int size() const { return vertices.size(); }
        // 从位置 i 沿保存方向走到 j，以及走反方向的长度
        pair<W, W> arc_lengths(int i, int j) const {
            W forward = i ⩽ j ? pref[j] - pref[i] : total - (pref[i] - pref[j]);
            return {forward, total - forward};
        }
        W shortest_arc(int i, int j) const {
            auto [a, b] = arc_lengths(i, j);
            return min(a, b);
        }
    };
    // tree 中：
    // edge ≠ -1: 原图桥，对应原边编号
    // pos  ≠ -1: 原点与环方点的关联，表示原点在该环中的位置
    struct Link {
        int to, edge{-1}, pos{-1};
    };
    int n, timer{}, lg{}, components{};
    vector<Edge> edges;
    vector<vector<int>> adj;
```

```
    // ===== 仙人掌分解 =====
    vector<int> dfn, low, estk, cycle_of_edge, component_id;
    vector<char> bridge;
    vector<Cycle> cycles;
    // belong[u] = {环编号, u 在该环中的位置}
    // 一个割点可以属于多个环
    vector<vector<pair<int, int>>> belong;
    // ===== 仙人掌圆方森林 =====
    // [0, n)             原图节点
    // [n, n + cycles)    环方点
    vector<vector<Link>> tree;
    // ===== 可裁剪: 路径查询 =====
    vector<vector<int>> up;
    vector<int> dep, tree_comp, parent_pos, cycle_pref, anchor_pos;
    vector<W> dist, parent_weight;
    bool built{}, valid{}, query_ready{};
    explicit Cactus(int n_) : n(n_), adj(n) {}
    int add_edge(int u, int v, W w = W{1}) {
        assert(!built && u != v);
        int id = edges.size();
        edges.push_back({u, v, w});
        adj[u].push_back(id);
        adj[v].push_back(id);
        return id;
    }
private:
    int other(int id, int u) const { return edges[id].u ^ edges[id].v ^ u; }
    vector<int> tmp_deg;
    vector<vector<int>> tmp_adj;
    // ===== 将一个点双判定为桥或简单环 =====
    bool make_block(vector<int> ids) {
        if (ids.size() == 1) {
            bridge[ids[0]] = true;
            return true;
        }
        vector<int> vertices;
        for (int id : ids) {
            auto [u, v, w] = edges[id];
            if (tmp_deg[u] ++ == 0) vertices.push_back(u);
            if (tmp_deg[v] ++ == 0) vertices.push_back(v);
            tmp_adj[u].push_back(id);
            tmp_adj[v].push_back(id);
        }
        // 简单环满足 |E| = |V|, 且块内每个点度数为 2
        bool ok{ ids.size() == vertices.size() };
        for (int u : vertices) ok &= tmp_deg[u] == 2;
        if (!ok) {
            for (int u : vertices) {
                tmp_deg[u] = 0;
                tmp_adj[u].clear();
            }
            return false;
        }
        // ===== 按环上的实际顺序恢复点和边 =====
        Cycle cyc;
        int s{ ranges::min(vertices) }, u{ s }, prev{ -1 };
        for (int step = 0; step < ids.size(); ++ step) {
            cyc.vertices.push_back(u);
            auto &inc = tmp_adj[u];
            int id{ inc[0] != prev ? inc[0] : inc[1] };
            cyc.edges.push_back(id);
            int v{ other(id, u) };
            prev = id;
            u = v;
        }
        ok &= u == s;
        if (!ok) {
            for (int v : vertices) {
                tmp_deg[v] = 0;
                tmp_adj[v].clear();
            }
```

```
            return false;
        }
        int cid = cycles.size();
        cyc.pref.assign(cyc.size() + 1, W{});
        for (int i : views::iota(0, cyc.size())) {
            cyc.pref[i + 1] = cyc.pref[i] + edges[cyc.edges[i]].w;
            cycle_of_edge[cyc.edges[i]] = cid;
            belong[cyc.vertices[i]].push_back({cid, i});
        }
        cyc.total = cyc.pref.back();
        cycles.push_back(move(cyc));
        for (int v : vertices) {
            tmp_deg[v] = 0;
            tmp_adj[v].clear();
        }
        return true;
    }
    // ===== 点双 Lowlink，边栈保存当前块 =====
    void dfs(int u, int parent_edge, int cid) {
        component_id[u] = cid;
        dfn[u] = low[u] = ++ timer;
        for (int id : adj[u]) {
            if (id == parent_edge) continue;
            int v{ other(id, u) };
            if (!dfn[v]) {
                estk.push_back(id);
                dfs(v, id, cid);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    vector<int> ids;
                    while (true) {
                        int e{ estk.back() };
                        estk.pop_back();
                        ids.push_back(e);
                        if (e == id) break;
                    }
                    valid &= make_block(move(ids));
                }
            } else if (dfn[v] < dfn[u]) {
                // 无向边只把返祖边压栈一次
                estk.push_back(id);
                low[u] = min(low[u], dfn[v]);
            }
        }
    }
    // ===== 环缩成方点，桥直接保留 =====
    void build_tree() {
        tree.assign(n + cycles.size(), {});
        for (int id : views::iota(0, int(edges.size()))) {
            if (!bridge[id]) continue;
            auto [u, v, w] = edges[id];
            tree[u].push_back({v, id, -1});
            tree[v].push_back({u, id, -1});
        }
        for (int cid : views::iota(0, int(cycles.size()))) {
            int c{ n + cid };
            for (int i : views::iota(0, cycles[cid].size())) {
                int u{ cycles[cid].vertices[i] };
                tree[c].push_back({u, -1, i});
                tree[u].push_back({c, -1, i});
            }
        }
    }
    // ===== 以下全部是可裁剪的路径查询部分 =====
    void build_lca() {
        int N = tree.size();
        lg = bit_width((unsigned)max(1, N));
        up.assign(lg, vector<int>(N));
        dep.assign(N, -1);
        tree_comp.assign(N, -1);
        parent_pos.assign(N, -1);
```

```
        cycle_pref.assign(N, 0);
        anchor_pos.assign(cycles.size(), -1);
        dist.assign(N, W{});
        parent_weight.assign(N, W{});
        for (int s : views::iota(0, n)) {
            if (dep[s] != -1) continue;
            dep[s] = 0;
            tree_comp[s] = component_id[s];
            up[0][s] = s;
            vector<int> stk{s};
            while (!stk.empty()) {
                int u{ stk.back() };
                stk.pop_back();
                for (auto [v, edge, pos] : tree[u]) {
                    if (dep[v] != -1) continue;
                    dep[v] = dep[u] + 1;
                    tree_comp[v] = tree_comp[u];
                    up[0][v] = u;
                    cycle_pref[v] = cycle_pref[u] + (v >= n);
                    W w{};
                    if (u < n && v < n) {
                        // 桥
                        w = edges[edge].w;
                    } else if (u < n && v >= n) {
                        // 从根方向第一次进入这个环，u 是该环锚点
                        anchor_pos[v - n] = pos;
                    } else if (u >= n && v < n) {
                        // 环方点 → 环上其他点
                        int cid{ u - n };
                        parent_pos[v] = pos;
                        w = cycles[cid].shortest_arc(anchor_pos[cid], pos);
                    }
                    parent_weight[v] = w;
                    dist[v] = dist[u] + w;
                    stk.push_back(v);
                }
            }
        }
        for (int k = 1; k < lg; ++ k) {
            for (int u : views::iota(0, N)) {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
        }
    }
    int climb(int u, int k) const {
        for (int b = 0; k; ++ b, k >>= 1) {
            if (k & 1) u = up[b][u];
        }
        return u;
    }
public:
    // ===== 主构造：只分解仙人掌并建立圆方森林，线性复杂度 =====
    bool build() {
        if (built) return valid;
        built = true;
        valid = true;
        int m = edges.size();
        dfn.assign(n, 0);
        low.assign(n, 0);
        bridge.assign(m, false);
        cycle_of_edge.assign(m, -1);
        component_id.assign(n, -1);
        belong.assign(n, {});
        tmp_deg.assign(n, 0);
        tmp_adj.assign(n, {});
        for (int u : views::iota(0, n)) {
            if (dfn[u]) continue;
            int cid{ components ++ };
            dfs(u, -1, cid);
            estk.clear();
        }
```

```
        if (!valid) return false;
        build_tree();
        return true;
    }
    // ===== 基础查询 =====
    bool is_bridge(int id) const {
        assert(built && valid);
        return bridge[id];
    }
    bool edge_on_cycle(int id) const {
        assert(built && valid);
        return cycle_of_edge[id] != -1;
    }
    bool connected(int u, int v) const {
        assert(built && valid);
        return component_id[u] == component_id[v];
    }
    int component_count() const {
        assert(built && valid);
        return components;
    }
    int cycle_count() const {
        assert(built && valid);
        return cycles.size();
    }
    // ===== 可裁剪：LCA / 最短路 / 路径经过环数 =====
    void prepare_queries() {
        assert(built && valid);
        if (query_ready) return;
        build_lca();
        query_ready = true;
    }
    int lca(int u, int v) const {
        assert(query_ready && tree_comp[u] == tree_comp[v]);
        if (dep[u] < dep[v]) swap(u, v);
        u = climb(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int k = lg - 1; k >= 0; --k) {
            if (up[k][u] == up[k][v]) continue;
            u = up[k][u];
            v = up[k][v];
        }
        return up[0][u];
    }
    // 非负边权下的两点最短距离
    optional<W> distance(int u, int v) const {
        assert(query_ready && 0 <= u && u < n && 0 <= v && v < n);
        if (!connected(u, v)) return nullopt;
        int p{ lca(u, v) };
        W ans{ dist[u] + dist[v] - W{2} * dist[p] };
        // LCA 是原图节点时，普通树上距离公式已经正确
        if (p < n) return ans;
        // LCA 是环方点时，需要把“分别走向环锚点”
        // 替换成两个实际入环点之间的最短环弧
        int a{ climb(u, dep[u] - dep[p] - 1) };
        int b{ climb(v, dep[v] - dep[p] - 1) };
        int cid{ p - n };
        ans -= parent_weight[a] + parent_weight[b];
        ans += cycles[cid].shortest_arc(parent_pos[a], parent_pos[b]);
        return ans;
    }
    // u → v 的结构路径经过多少个环
    int cycles_on_path(int u, int v) const {
        assert(query_ready && connected(u, v));
        int p{ lca(u, v) };
        return cycle_pref[u] + cycle_pref[v] - 2 * cycle_pref[p] + (p >= n);
    }
};
```

判定原理

Tarjan 点双分解时使用边栈
当 DFS 树边 `u → v` 满足

$$low[v] \geq dfn[u]$$

时，从边栈中弹出直到当前树边，即得到一个完整点双连通分量
仙人掌的关键性质是每个点双只能有两种形式

桥

只有一条边

```
u ----- v
```

简单环

若一个点双包含多条边，则必须满足

$$|E| = |V|$$

并且块内每个点度数都等于 2
因此代码直接检查

```
ids.size() == vertices.size()
```

以及

```
tmp_deg[u] == 2
```

只要某个点双不是桥也不是环，整张图就不是仙人掌
例如 theta 图

```
      a
     / \
s --     -- t
     \ /
      b
```

存在两点之间三条内部不交路径，其中某些边会属于多个简单环，因此会被拒绝

基础使用

```
Cactus<> cactus(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    cin >> u >> v;
    -- u, -- v;
    cactus.add_edge(u, v);
}
if (!cactus.build()) {
    // 不是仙人掌
}
```

带权

```
Cactus<i64> cactus(n);
for (int i = 0; i < m; ++ i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    cactus.add_edge(u, v, w);
}
```

环信息

所有环保存在

```
cactus.cycles
```

遍历全部环

```
for (int cid : views::iota(0, cactus.cycle_count())) {
    const auto &cyc = cactus.cycles[cid];
    for (int u : cyc.vertices) {
        // 环上的点
    }
    for (int id : cyc.edges) {
        // 环上的原边编号
    }
}
```

`vertices` 与 `edges` 一一对应

```
vertices[0] --edges[0]⟶ vertices[1]
vertices[1] --edges[1]⟶ vertices[2]
...
vertices[k-1] --edges[k-1]⟶ vertices[0]
```

这里的方向只是为了保存顺序
原图仍然是无向图

一个点属于哪些环

```
for (auto [cid, pos] : cactus.belong[u]) {
    // u 位于第 cid 个环
    // 并且 u == cycles[cid].vertices[pos]
}
```

仙人掌中一个割点可以属于多个环，因此不能只保存一个

```
cycle_id[u]
```

这也是为什么我们使用

```
belong[u]
```

保存多个环归属

判断边类型

是否为桥

```
if (cactus.is_bridge(id)) {}
```

是否属于某个环

```
if (cactus.edge_on_cycle(id)) {
    int cid = cactus.cycle_of_edge[id];
}
```

在合法仙人掌中，每条边恰好属于以下一种类型

```
桥
或
某一个简单环
```

环上距离

对于

```
const auto &cyc = cactus.cycles[cid];
```

若已知两个点在环中的位置 `i,j`

```
auto [clockwise, counterclockwise] = cyc.arc_lengths(i, j);
```

得到沿两个方向的弧长
两条弧满足

$$d_1 + d_2 = cycle_length$$

其中

```
cyc.total
```

就是环总长度
非负边权下的最短环上距离

```
W d = cyc.shortest_arc(i, j);
```

即

$$\min(d_1, d_2)$$

仙人掌圆方树

本板将每个环缩成一个新的方点
节点编号为

```
[0, n)                  原图节点
[n, n + cycle_count())  环方点
```

桥直接保留
例如

```
   1
  / \
 0   2 ----- 3
  \ /
```

其中 `0-1-2-0` 为环
会转换成

```
     1
     |
0 --- C --- 2 --- 3
```

`C` 是这个环对应的方点

为什么桥不再建立方点

标准 block-cut tree 可以给每个点双都建立方点
但桥点双只有两个端点

```
u - square - v
```

完全可以直接压缩为

```
u - v
```

因此这里仅为环建立方点，结构更小
这仍然是一棵森林
对于连通仙人掌则是一棵树

遍历圆方树

```
for (int u : views::iota(0, int(cactus.tree.size()))) {
    for (auto e : cactus.tree[u]) {
        int v = e.to;
```

```
    }
}
```

判断节点类型

```
if (u < n) {
    // 原图节点
} else {
    int cid = u - n;
    // 第 cid 个环的方点
}
```

很多仙人掌 DP 到这里就已经变成普通树 DP

连通分量

```
int cc = cactus.component_count();
```

判断两个原图节点是否连通

```
if (cactus.connected(u, v)) {}
```

若题目保证普通意义上的仙人掌图必须连通，只需要检查

```
cactus.build() && cactus.component_count() == 1
```

路径查询预处理

若只需要

- 判定仙人掌
- 找环
- 建圆方树
- 做自己的树 DP
  到这里就可以停止
  只有需要下面的 LCA、最短路、路径环数时才调用

```
cactus.prepare_queries();
```

这部分可以整体裁掉

为什么仙人掌最短路不能直接给圆方树边赋固定权值

对于一个环

```
    a
   /  \
  x    y
   \  /
    b
```

如果简单地将环缩成方点 `C`

```
a - C - b
```

那么 `a → C → b` 的固定两条边无法同时表达环上任意两点之间

$$\min(\text{顺时针弧长}, \text{逆时针弧长})$$

因此普通加权树距离公式并不能直接使用
我们采用一个根方向上的**环锚点**解决这个问题

根方向锚点

把圆方树任意定根
每个环方点都有唯一一个朝向根的原图节点
记为该环的 `anchor`

对于环中的其他节点 `v`
将方点到 `v` 的临时树权定义为

$$dist_{cycle}(anchor, v)$$

这样从根到任意节点的最短距离都是正确的
只有一种情况需要特殊修正

> 两点的 LCA 本身是环方点

此时两条路径从环中的不同位置进入，不能都绕回 `anchor`

两点最短路

要求边权非负
预处理

```
cactus.prepare_queries();
```

查询

```
auto d = cactus.distance(u, v);
if (!d) {
    // 不连通
} else {
    // *d 为最短距离
}
```

查询复杂度

$$O(\log n)$$

LCA 是原图节点

此时普通树距离公式成立

$$dist(u, v) = dist[u] + dist[v] - 2dist[lca]$$

LCA 是环方点

设两条路径从该环上的 `a,b` 两点离开
普通树公式实际上计算了

$$dist(anchor, a) + dist(anchor, b)$$

但真正需要的是

$$dist_{cycle}(a, b)$$

因此修正为

$$\begin{aligned} ans = {} & dist[u] + dist[v] - 2dist[lca] \\ & - dist(anchor, a) - dist(anchor, b) \\ & + dist_{cycle}(a, b) \end{aligned}$$

对应代码

```
ans -= parent_weight[a] + parent_weight[b];
ans += cycles[cid].shortest_arc(parent_pos[a], parent_pos[b]);
```

路径经过多少个环

```
int k = cactus.cycles_on_path(u, v);
```

表示从 `u` 到 `v` 的圆方树路径上有多少个环方点
查询复杂度

$$O(\log n)$$

这个信息非常实用

两点间简单路径数量

在仙人掌中，每经过一个环，都有两种方式从环的一侧走到另一侧
因此若圆方树路径经过 k 个环，则两点间不同简单路径数量为

$$2^k$$

例如模 `mod` 统计

```
int k = cactus.cycles_on_path(u, v);
i64 ans = qpow(2, k, mod);
```

前提是 `u,v` 连通

环贡献 DP

圆方树上做 DP 时

原图节点

通常像普通树节点一样合并儿子

环方点

不能把所有邻点独立处理
因为它们在原图中实际位于同一个环上
此时通常需要

- 按 `cycles[cid].vertices` 的环顺序 DP
- 做前缀 / 后缀
- 环形 DP
- 顺时针与逆时针分别转移
  因此看到

```
u ⩾ n
```

时应想到

```
int cid = u - n;
const auto &cyc = cactus.cycles[cid];
```

而不是把它当普通树节点无脑合并

与 VBCC 的关系

仙人掌判定实际上可以概括成一句话

> 对所有 VBCC，检查它是单边还是简单环

但已有的通用 VBCC 板子主要保存的是点集合
仙人掌常常还需要

- 块内边编号
- 环的实际顺序
- 环上前缀距离
- 原边属于哪个环
  因此我们内部重新做了一遍边栈 Lowlink
  并不是因为算法不同，而是为了保存更适合仙人掌题目的信息

重边

本板使用父边编号而不是父节点，因此可以正确处理平行边
两条平行边

```
u ⩸ v
```

会被视为长度为 `2` 的环
三条及以上平行边则不再是仙人掌，因为每条边会参与多个简单环
如果题目明确保证简单图，则无需关心这一情况

自环

当前 `add_edge()` 直接要求

```
u ≠ v
```

也就是不支持自环
虽然某些定义会把自环视为长度为 `1` 的环，但竞赛中仙人掌通常讨论无自环图
若题目真的允许自环，应按题目定义单独处理

常见用法

判定并枚举环

```
Cactus<> cactus(n);
for (auto [u, v] : edges) cactus.add_edge(u, v);
if (!cactus.build()) {
    cout << "Not a cactus\n";
} else {
    for (int cid : views::iota(0, cactus.cycle_count())) {
        for (int u : cactus.cycles[cid].vertices) {
            // 第 cid 个环上的节点
        }
    }
}
```

判断边是桥还是环边

```
for (int id : views::iota(0, m)) {
    if (cactus.is_bridge(id)) {
        // 桥
    } else {
        int cid = cactus.cycle_of_edge[id];
        // 第 cid 个环
    }
}
```

最短路

```
if (!cactus.build()) return;
cactus.prepare_queries();
auto ans = cactus.distance(u, v);
if (!ans) {
    // 不连通
} else {
    cout << *ans << '\n';
}
```

路径上的环数

```
cactus.prepare_queries();
int k = cactus.cycles_on_path(u, v);
```

若需要简单路径条数

```
i64 ways = qpow(2, k, mod);
```

快速回忆

```
仙人掌
    每条边至多属于一个简单环

Tarjan 边栈
    每个点双：
        1 条边 → 桥
```

```
        多条边 →
            |E| = |V|
            且每点块内度 = 2
            → 简单环

环
    vertices[i]
    edges[i]
    pref[i]
    total

圆方树
    原点 [0,n)
    环方点 [n,n+C)
    桥直接保留
    环替换成方点

只分解
    build()
    O(n+m)

需要路径查询
    prepare_queries()

两点最短路
    LCA 是原点 → 普通树距离
    LCA 是环方点 → 替换一次环内距离

路径经过 k 个环
    不同简单路径数 = 2^k
```

注意事项

- 本板处理无向图
- 支持不连通仙人掌森林
- 支持重边
- 不支持自环
- 一个割点可以同时属于多个环
- `cycle_of_edge[id] == -1` 表示该边不是环边
- 合法仙人掌中的非环边一定是桥
- 环保存方向是任意的，只用于固定顺序
- `Cycle::shortest_arc()` 和 `distance()` 的标准最短路语义要求边权非负
- 若只需要结构分解，不要调用 `prepare_queries()`
- `tree` 是仅给环建立方点的压缩圆方树，不是“每个 VBCC 都建立方点”的完整 block-cut tree
- 圆方树上的环方点做 DP 时通常需要结合原环顺序，不能总当成普通树节点
- Lowlink 使用递归 DFS，极长链需要注意系统栈空间
- 构造完成后不能继续 `add_edge()`

复杂度

设原图有 n 个点、m 条边

仙人掌判定与环分解

每条边只进入和离开边栈一次

$$O(n+m)$$

所有环的点和边总规模也是

$$O(n+m)$$

因此 `build()` 总时间复杂度

$$O(n+m)$$

空间复杂度

$$O(n+m)$$

圆方树规模

若环数量为 c
圆方树节点数为

$$N=n+c$$

并且

$$N=O(n+m)$$

路径查询预处理

倍增 LCA

$$O(N\log N)$$

时间和空间均为

$$O(N\log N)$$

单次查询

`connected()`

$$O(1)$$

`is_bridge()`

$$O(1)$$

环上两个已知位置的距离

$$O(1)$$

`lca()`

$$O(\log N)$$

`distance()`

$$O(\log N)$$

`cycles_on_path()`

$$O(\log N)$$

5.6 特殊图与图计数

5.6.1 弦图与完美消除序列

MCS 使用桶维护，生成候选序与验证均为线性复杂度

```
struct ChordalGraph {
    int n;
    vector<vector<int>> g;
    explicit ChordalGraph(int n_) : n(n_), g(n) {}
    void add_edge(int u, int v) {
        assert(u != v);
        g[u].push_back(v);
```

```
        g[v].push_back(u);
    }
    pair<bool, vector<int>> perfect_elimination_order() const {
        if (!n) return {true, {}};
        // ===== MCS =====
        vector<int> label(n), order(n), pos(n);
        vector<char> used(n);
        vector<vector<int>> bucket(n + 1);
        bucket[0].resize(n);
        ranges::iota(bucket[0], 0);
        int mx{ 0 };
        for (int i = n - 1; i >= 0; --i) {
            while (true) {
                while (!bucket[mx].empty()) {
                    int u{ bucket[mx].back() };
                    if (!used[u] && label[u] == mx) break;
                    bucket[mx].pop_back();
                }
                if (!bucket[mx].empty()) break;
                --mx;
            }
            int u{ bucket[mx].back() };
            bucket[mx].pop_back();
            used[u] = true;
            order[i] = u;
            for (int v : g[u]) if (!used[v]) {
                ++label[v];
                bucket[label[v]].push_back(v);
                mx = max(mx, label[v]);
            }
        }
        for (int i : views::iota(0, n)) pos[order[i]] = i;
        // ===== 线性验证 PEO =====
        vector<int> parent(n, -1), mark(n, -1);
        vector<vector<int>> child(n);
        for (int u : views::iota(0, n)) {
            for (int v : g[u]) {
                if (pos[v] > pos[u] && (parent[u] == -1 || pos[v] < pos[parent[u]])) parent[u] = v;
            }
            if (parent[u] != -1) child[parent[u]].push_back(u);
        }
        for (int p : views::iota(0, n)) if (!child[p].empty()) {
            mark[p] = p;
            for (int v : g[p]) mark[v] = p;
            for (int u : child[p]) {
                for (int v : g[u]) {
                    if (pos[v] > pos[u] && mark[v] != p) return {false, move(order)};
                }
            }
        }
        return {true, move(order)};
    }
    int maximum_clique_size(const vector<int> &peo) const {
        vector<int> pos(n);
        for (int i : views::iota(0, n)) pos[peo[i]] = i;
        int ans{ 0 };
        for (int u : views::iota(0, n)) {
            int cur{ 1 };
            for (int v : g[u]) cur += pos[v] > pos[u];
            ans = max(ans, cur);
        }
        return ans;
    }
    pair<int, vector<int>> optimal_coloring(const vector<int> &peo) const {
        vector<int> color(n, -1), mark(n, -1);
        int colors{ 0 };
        for (int i = n - 1; i >= 0; --i) {
            int u{ peo[i] };
            for (int v : g[u]) if (color[v] != -1) mark[color[v]] = u;
            int c{ 0 };
            while (c < colors && mark[c] == u) ++c;
```

```
            if (c == colors) ++ colors;
            color[u] = c;
        }
        return {colors, move(color)};
    }
};
```

定义与接口

完美消除序列 PEO 满足

> 对序列中的每个点 `u`，所有排在 `u` 后面的邻点构成团

无向简单图是弦图，当且仅当存在 PEO

```
auto [ok, peo] = chordal.perfect_elimination_order();
```

`ok == true` 时 `peo` 是一组合法 PEO
最大团

```
int omega = chordal.maximum_clique_size(peo);
```

最优染色

```
auto [chromatic, color] = chordal.optimal_coloring(peo);
```

弦图是完美图，因此

$$\chi(G) = \omega(G)$$

注意事项

- 输入必须是简单无向图
- 不允许自环
- 不应重复加入同一条边，否则 MCS 的 `label` 会被重复统计
- 输入可能有重边时应先去重
- MCS 得到的候选序只有验证成功后才能作为 PEO
- `maximum_clique_size()` 与 `optimal_coloring()` 要求传入合法 PEO

复杂度

MCS

$$O(n+m)$$

PEO 验证

$$O(n+m)$$

最大团与染色

$$O(n+m)$$

空间

$$O(n+m)$$

稳定婚姻

```
struct StableMarriageResult {
    vector<int> a_to_b, b_to_a;
};
StableMarriageResult stable_marriage(const vector<vector<int>> &pref_a, const vector<vector<int>> &pref_b) {
    int n = pref_a.size();
    assert(pref_b.size() == n);
```

```
    vector rank_b(n, vector<int>(n));
    for (int b : views::iota(0, n)) {
        for (int i : views::iota(0, n)) rank_b[b][pref_b[b][i]] = i;
    }
    vector<int> a_to_b(n, -1), b_to_a(n, -1), nxt(n), free(n);
    ranges::iota(free, 0);
    while (!free.empty()) {
        int a{ free.back() }, b{ pref_a[a][nxt[a] ++] };
        int old{ b_to_a[b] };
        if (old == -1) {
            a_to_b[a] = b;
            b_to_a[b] = a;
            free.pop_back();
        } else if (rank_b[b][a] < rank_b[b][old]) {
            a_to_b[old] = -1;
            a_to_b[a] = b;
            b_to_a[b] = a;
            free.back() = old;
        }
    }
    return {move(a_to_b), move(b_to_a)};
}
```

接口与性质

```
auto [a_to_b, b_to_a] = stable_marriage(pref_a, pref_b);
```

偏好表要求每行都是 `[0,n)` 的完整排列
A 方主动求婚，因此得到

- A 方最优稳定匹配
- B 方最差稳定匹配
  这里的最优和最差均限定在全部稳定匹配中
  若希望 B 方最优，交换双方后运行即可

复杂度

每个 A 最多向每个 B 求婚一次

$$O(n^2)$$

额外空间

$$O(n^2)$$

主要由 `rank_b` 占用

5.6.2 图度序列判定

Erdős–Gallai

```
bool graphical_degree_sequence(vector<int> d) {
    int n = d.size();
    if (ranges::any_of(d, [&](int x) { return x < 0 || x ≥ n; })) return false;
    vector<int> cnt(n);
    for (int x : d) ++ cnt[x];
    for (int x = n - 1, i = 0; x ≥ 0; -- x) while (cnt[x] --) d[i ++] = x;
    vector<i64> pre(n + 1);
    for (int i : views::iota(0, n)) pre[i + 1] = pre[i] + d[i];
    if (pre[n] & 1) return false;
    int p{ n };
    for (int k : views::iota(1, n + 1)) {
        p = max(p, k);
        while (p > k && d[p - 1] ≤ k) -- p;
        i64 rhs = 1LL * k * (k - 1) + 1LL * (p - k) * k + pre[n] - pre[p];
        if (pre[k] > rhs) return false;
    }
```

```
    return true;
}
```

判断非负整数序列能否作为某个简单无向图的度数序列
降序排列后，对所有 $1 \le k \le n$ 检查

$$\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

并要求

$$\sum_{i} d_i \equiv 0 \pmod 2$$

复杂度

$$O(n)$$

Landau 竞赛图得分序列

```
bool tournament_score_sequence(vector<int> s) {
    int n = s.size();
    ranges::sort(s);
    i64 sum { 0 };
    for (int k : views::iota(1, n + 1)) {
        if (s[k - 1] < 0 || s[k - 1] ⩾ n) return false;
        sum += s[k - 1];
        if (k < n && sum < 1LL * k * (k - 1) / 2) return false;
    }
    return sum == 1LL * n * (n - 1) / 2;
}
```

升序得分序列满足

$$\sum_{i=1}^{k} s_i \ge \binom{k}{2} \qquad (1 \le k < n)$$

且

$$\sum_{i=1}^{n} s_i = \binom{n}{2}$$

复杂度

$$O(n \log n)$$

Erdős–Gallai 性能：合法度数位于 `[0,n)`，用计数排序和单调分界指针，判定总时间 O(n)、空间 O(n)；不改变无解和空序列语义。

图度序列判定

解决问题： 判断整数序列能否作为简单无向图、二分图、竞赛图或树的度序列，并在需要时实际构造一张简单无向图

Erdős–Gallai：简单无向图度序列

```
bool graphical_degree_sequence(vector<int> d) {
    int n = d.size();
    if (ranges::any_of(d, [&](int x) { return x < 0 || x ⩾ n; })) return false;
    vector<int> cnt(n);
    for (int x : d) ++ cnt[x];
    for (int x = n - 1, i = 0; x ⩾ 0; -- x) while (cnt[x] --) d[i ++] = x;
    vector<i64> pre(n + 1);
    for (int i : views::iota(0, n)) pre[i + 1] = pre[i] + d[i];
    if (pre[n] & 1) return false;
    int p{ n };
    for (int k : views::iota(1, n + 1)) {
```

```
        p = max(p, k);
        while (p > k && d[p - 1] <= k) -- p;
        i64 rhs = 1LL * k * (k - 1) + 1LL * (p - k) * k + pre[n] - pre[p];
        if (pre[k] > rhs) return false;
    }
    return true;
}
```

判定条件

将序列降序排列

$$d_1 \geq d_2 \geq \cdots \geq d_n$$

它是某张简单无向图的度序列，当且仅当

$$\sum_{i=1}^{n} d_i$$

为偶数，并且对所有 $1 \leq k \leq n$

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

前 k 个点最多互相贡献

$$k(k-1)$$

度
剩余每个点最多向前 k 个点贡献

$$\min(d_i, k)$$

条边

为什么代码需要 `p`

对固定 `k`，找到第一个满足

$$d_p \leq k$$

的位置
于是

$$\sum_{i=k+1}^{n} \min(d_i, k) = (p-k)k + \sum_{i=p+1}^{n} d_i$$

利用前缀和即可快速计算

Havel–Hakimi：判定并构造简单图

若不仅要判断，还需要实际恢复一张满足度序列的简单无向图，可以直接使用 Havel–Hakimi

```
optional<vector<pair<int, int>>> construct_graphical(vector<int> d) {
    int n = d.size();
    if (ranges::any_of(d, [&](int x) { return x < 0 || x >= n; })) return nullopt;
    priority_queue<pair<int, int>> q;
    for (int u : views::iota(0, n)) if (d[u]) q.push({d[u], u});
    vector<pair<int, int>> edges;
    while (!q.empty()) {
        auto [du, u] = q.top();
        q.pop();
        if (du > q.size()) return nullopt;
        vector<pair<int, int>> changed;
        changed.reserve(du);
        while (du --) {
            auto [dv, v] = q.top();
```

```
            q.pop();
            edges.push_back({u, v});
            if (--dv < 0) return nullopt;
            if (dv) changed.push_back({dv, v});
        }
        for (auto x : changed) q.push(x);
    }
    return edges;
}
```

原理

每次取当前度数最大的点 `u`
若

$$deg(u) = d$$

就把它连接到当前剩余度数最大的 d 个节点，并将这些节点剩余度数减一
Havel–Hakimi 定理保证

> 原序列可图，当且仅当执行一次这种操作后得到的新序列可图

不断执行直到所有剩余度数变成 0

使用

```
auto edges = construct_graphical(d);
if (!edges) {
    cout << "Impossible\n";
} else {
    for (auto [u, v] : *edges) {
        // 构造出的简单无向边 u - v
    }
}
```

返回的边数一定为

$$m = \frac{1}{2}\sum_i d_i$$

若只需要判定，不需要构造，优先使用 Erdős–Gallai

Gale–Ryser：二分图度序列

给定二分图左右两侧的度序列

$$a_1, \ldots, a_n$$

和

$$b_1, \ldots, b_m$$

判断是否存在简单二分图满足这些度数

```
bool bipartite_degree_sequence(vector<int> a, vector<int> b) {
    int n = a.size(), m = b.size();
    if (ranges::any_of(a, [&](int x) { return x < 0 || x > m; })) return false;
    if (ranges::any_of(b, [&](int x) { return x < 0 || x > n; })) return false;
    ranges::sort(a, greater{});
    ranges::sort(b, greater{});
    vector<i64> pre_a(n + 1), pre_b(m + 1);
    for (int i : views::iota(0, n)) pre_a[i + 1] = pre_a[i] + a[i];
    for (int i : views::iota(0, m)) pre_b[i + 1] = pre_b[i] + b[i];
    if (pre_a[n] != pre_b[m]) return false;
    for (int k : views::iota(1, n + 1)) {
        int p = lower_bound(b.begin(), b.end(), k, greater<int>()) - b.begin();
        i64 rhs = 1LL * p * k + pre_b[m] - pre_b[p];
        if (pre_a[k] > rhs) return false;
    }
```

```
    return true;
}
```

判定条件

将左侧度数降序

$$a_1 \geq a_2 \geq \cdots \geq a_n$$

首先总度数必须相等

$$\sum_{i=1}^{n} a_i = \sum_{j=1}^{m} b_j$$

并且对所有 $1 \leq k \leq n$

$$\sum_{i=1}^{k} a_i \leq \sum_{j=1}^{m} \min(b_j, k)$$

这就是 Gale–Ryser 判定
代码同样利用降序和前缀和把右侧求和快速计算

Landau：竞赛图得分序列

竞赛图是在完全图每条边上任选一个方向得到的有向图
节点得分定义为出度

```
bool tournament_score_sequence(vector<int> s) {
    int n = s.size();
    if (ranges::any_of(s, [&](int x) { return x < 0 || x >= n; })) return false;
    ranges::sort(s);
    i64 sum = 0;
    for (int k : views::iota(1, n + 1)) {
        sum += s[k - 1];
        if (k < n && sum < 1LL * k * (k - 1) / 2) return false;
    }
    return sum == 1LL * n * (n - 1) / 2;
}
```

判定条件

将得分升序排列

$$s_1 \leq s_2 \leq \cdots \leq s_n$$

对于所有

$$1 \leq k < n$$

必须满足

$$\sum_{i=1}^{k} s_i \geq \binom{k}{2}$$

并且总得分恰好为

$$\sum_{i=1}^{n} s_i = \binom{n}{2}$$

这就是 Landau 定理

为什么是不等号 `≥`

任选前 k 个节点
它们内部恰好存在

$$\binom{k}{2}$$

场比赛
每场一定贡献一个胜场给这 k 个节点中的某个点
因此它们的总出度至少为

$$\binom{k}{2}$$

树的度序列

这个结论很短，不需要单独写算法

`n = 1`

唯一的树只有一个点，因此必须

$$d_1 = 0$$

`n ⩾ 2`

一个正整数序列是某棵树的度序列，当且仅当

$$d_i \geq 1$$

且

$$\sum_{i=1}^{n} d_i = 2(n-1)$$

可以直接写成

```
bool tree_degree_sequence(const vector<int> &d) {
    int n = d.size();
    if (!n) return false;
    if (n == 1) return d[0] == 0;
    return ranges::all_of(d, [](int x) { return x >= 1; }) &&
           accumulate(d.begin(), d.end(), 0LL) == 2LL * (n - 1);
}
```

其充分性可以直接由 Prüfer 序列得到
节点 `i` 在 Prüfer 序列中出现

$$d_i - 1$$

次
总出现次数为

$$\sum_{i}(d_i - 1) = n - 2$$

恰好是一条合法 Prüfer 序列的长度

选型

问题	方法
简单无向图，只判断	Erdős–Gallai
简单无向图，需要恢复边	Havel–Hakimi
简单二分图左右度数	Gale–Ryser
竞赛图出度序列	Landau
树的度序列	总度数 + 正性

常见用法

判断简单图度序列

```
vector<int> d(n);
for (int &x : d) cin >> x;
if (graphical_degree_sequence(d)) {
    // 存在简单无向图
}
```

如果当前环境不支持直接 `cin >> vector`，正常循环读入即可

```
for (int &x : d) cin >> x;
```

构造一张简单图

```
auto res = construct_graphical(d);
if (!res) {
    cout << "Impossible\n";
} else {
    cout << res->size() << '\n';
    for (auto [u, v] : *res) cout << u + 1 << ' ' << v + 1 << '\n';
}
```

判断二分图度序列

```
if (bipartite_degree_sequence(left_degree, right_degree)) {
    // 存在对应简单二分图
}
```

判断竞赛图得分序列

```
if (tournament_score_sequence(score)) {
    // 存在对应竞赛图
}
```

简单图、重边图不要混淆

Erdős–Gallai、Havel–Hakimi、Gale–Ryser 都讨论简单图
也就是

- 不允许自环
- 两点之间最多一条边
  若允许重边，判定条件会发生明显变化
  例如无自环但允许平行边的无向多重图，只需要

$$\sum_i d_i$$

  为偶数，并且最大度数满足

$$\max_i d_i \le \sum_{j \ne i} d_j$$

  若连自环也允许，并规定自环贡献度数 2，则只需要总度数为偶数
  因此做题时第一件事应确认题目中的“图”是否默认简单图

Erdős–Gallai 与 Havel–Hakimi 的选择

只判定时推荐 Erdős–Gallai
排序后检查不等式即可，复杂度稳定
Havel–Hakimi 的优势是能自然构造一张图
但如果度数很大，实际需要显式生成

$$m = \frac{1}{2}\sum_i d_i$$

条边，因此复杂度至少与输出边数有关

注意事项

- Erdős–Gallai 输入允许包含 0
- 简单无向图中必须有 $0 \le d_i < n$
- 总度数必须为偶数
- Havel–Hakimi 返回任意一张满足条件的图，不保证字典序或其他性质

- 二分图中左侧度数不能超过右侧点数，右侧同理
- Gale--Ryser 左右两侧总度数必须相同
- Landau 使用的是竞赛图的出度序列
- 如果题目给的是竞赛图入度序列，仍可直接使用 Landau，因为将所有边反向后仍是竞赛图
- 有向简单图同时给定入度与出度的通用判定属于更一般的有向度序列问题，不建议塞入这份基础板子
- 若要求实际构造二分图，可以使用 Havel--Hakimi 的二分版本或最大流，不必修改 Gale--Ryser 判定函数

快速回忆

```
简单无向图
    Erdős--Gallai
    sum d 偶数
    前 k:
    sum d[1..k] ⩽ k(k-1) + sum min(d[i], k)

需要构造简单图
    Havel--Hakimi
    最大度点连接当前度数最大的若干点

简单二分图
    Gale--Ryser
    sum left = sum right
    前 k 左度 ⩽ sum min(right[j], k)

竞赛图
    Landau
    升序
    前 k 和 ⩾ C(k,2)
    总和 = C(n,2)

树
    n > 1 时全部度 ⩾ 1
    sum d = 2(n-1)
```

复杂度

Erdős--Gallai 使用计数排序，并以单调指针维护分界点
总时间

$$O(n)$$

空间

$$O(n)$$

Havel--Hakimi 若最终构造出 m 条边

$$O((n+m)\log n)$$

最坏简单图中

$$m = O(n^2)$$

因此最坏

$$O(n^2 \log n)$$

输出本身就可能需要

$$O(n^2)$$

空间
Gale--Ryser

$$O((n+m)\log(n+m))$$

其中这里的 m 表示右侧点数，不是边数
若左右规模都记为 $O(N)$，可简写为

$$O(N \log N)$$

Landau

$$O(n \log n)$$

树度序列

$$O(n)$$

Erdős–Gallai 性能：合法度数位于 `[0,n)`，用计数排序和单调分界指针，判定总时间 O(n)、空间 O(n)；不改变无解和空序列语义。

5.6.3 三元环、四元环与团计数

三元环与四元环

二者共用一次按度数排序后的定向

```
struct SmallCycleCounter {
    int n;
    vector<pair<int, int>> edges;
    vector<vector<int>> g, out;
    vector<int> deg, rk;
    bool built{};
    explicit SmallCycleCounter(int n_) : n(n_), g(n), out(n), deg(n), rk(n) {}
    void add_edge(int u, int v) {
        assert(!built && u != v);
        edges.push_back({u, v});
        g[u].push_back(v);
        g[v].push_back(u);
        ++ deg[u], ++ deg[v];
    }
    void build() {
        if (built) return;
        vector<int> order(n);
        ranges::iota(order, 0);
        ranges::sort(order, {}, [&](int u) { return pair{deg[u], u}; });
        for (int i : views::iota(0, n)) rk[order[i]] = i;
        for (auto [u, v] : edges) {
            if (rk[u] > rk[v]) swap(u, v);
            out[u].push_back(v);
        }
        built = true;
    }
    i64 count_triangles() {
        build();
        vector<int> mark(n, -1);
        i64 ans = 0;
        for (int u : views::iota(0, n)) {
            for (int v : out[u]) mark[v] = u;
            for (int v : out[u]) {
                for (int w : out[v]) ans += mark[w] == u;
            }
        }
        return ans;
    }
    i64 count_four_cycles() {
        build();
        vector<int> cnt(n);
        i64 ans = 0;
        for (int u : views::iota(0, n)) {
            for (int v : g[u]) {
                for (int w : out[v]) if (rk[w] > rk[u]) ans += cnt[w] ++;
            }
```

```
            for (int v : g[u]) {
                for (int w : out[v]) if (rk[w] > rk[u]) -- cnt[w];
            }
        }
        return ans;
    }
};
```

定向规则

定义点序为

```
pair{deg[u], u}
```

较小的点指向较大的点

```
低度点 → 高度点
```

度数相同则按编号定向
该定向满足每个点出度为 $O(\sqrt{m})$

三元环

```
i64 ans = counter.count_triangles();
```

对每个 `u`

```
标记 u 的全部出邻居
枚举 u → v → w
检查 w 是否被 u 标记
```

每个三元环恰好统计一次

复杂度

$$O(m\sqrt{m})$$

四元环

```
i64 ans = counter.count_four_cycles();
```

对于固定 `u` 枚举

```
u - v → w
```

并要求

```
rk[w] > rk[u]
```

`cnt[w]` 表示此前已有多少条合法的 `u → ... → w` 长度为 2 的路径
每加入一条新路径

```
ans += cnt[w] ++;
```

就与此前所有路径分别形成一个四元环

复杂度

$$O(m\sqrt{m})$$

注意事项

- 输入必须是简单无向图
- 不允许自环
- 不允许重复边
- 有对角线的四点子图可能包含多个不同四元环，本函数按环的边集合分别计数
- `build()` 后不能再加边
- 三元环与四元环计数可以分别或重复调用

`n ≤ 64` 最大团与极大团

Bron–Kerbosch + pivot

```
struct Clique64 {
    struct Result {
        int maximum{};
        i64 maximum_count{}, maximal_count{};
        u64 maximum_mask{};
    };
    int n;
    vector<u64> adj;
    Result res;
    explicit Clique64(int n_) : n(n_), adj(n) {
        assert(0 <= n && n <= 64);
    }
    void add_edge(int u, int v) {
        assert(u != v);
        adj[u] |= 1ULL << v;
        adj[v] |= 1ULL << u;
    }
private:
    void dfs(u64 r, u64 p, u64 x) {
        if (!p && !x) {
            int siz{ popcount(r) };
            ++ res.maximal_count;
            if (siz > res.maximum) {
                res.maximum = siz;
                res.maximum_count = 1;
                res.maximum_mask = r;
            } else if (siz == res.maximum) ++ res.maximum_count;
            return;
        }
        int pivot{ -1 }, score{ -1 };
        for (u64 s = p | x; s; s &= s - 1) {
            int u{ countr_zero(s) };
            int cur{ popcount(p & adj[u]) };
            if (cur > score) score = cur, pivot = u;
        }
        u64 cand = pivot == -1 ? p : p & ~adj[pivot];
        while (cand) {
            int v{ countr_zero(cand) };
            u64 bit = 1ULL << v;
            cand ^= bit;
            dfs(r | bit, p & adj[v], x & adj[v]);
            p ^= bit;
            x |= bit;
        }
    }
public:
    Result solve() {
        res = {};
        u64 full = n == 64 ? ~0ULL : (1ULL << n) - 1;
        dfs(0, full, 0);
        return res;
    }
};
```

返回值

```
auto res = clique.solve();
```

`res.maximum`：最大团大小
`res.maximum_count`：最大团数量
`res.maximal_count`：全部极大团数量
`res.maximum_mask`：任意一个最大团
恢复节点

```
for (int u : views::iota(0, clique.n)) {
    if (res.maximum_mask >> u & 1ULL) {
```

```
        // u 位于一个最大团
    }
}
```

最大团与极大团

最大团要求点数最大
极大团只要求无法继续加入新节点
每个最大团都是极大团，但极大团不一定最大
原版本的 `mcnt` 实际统计的是极大团数量，本版将

- 最大团大小
- 最大团数量
- 极大团数量
  分别返回

为什么不能按 `best` 剪枝

如果只求最大团，可以使用

```
当前团大小 + 候选点数量 ⩽ best
```

等分支限界
但这里要求准确统计所有极大团
若按照最大团大小剪枝，会遗漏较小极大团，使 `maximal_count` 错误
因此原来的恒假条件

```
if (popcount(r) + popcount(p) ⩽ best && mcnt < 0) return;
```

应直接删除
若只求最大团，应另用专门的最大团分支限界板子

注意事项

- 要求 `n ⩽ 64`
- 输入按简单无向图处理
- 自环没有意义
- `solve()` 会枚举全部极大团，最坏复杂度必然指数级
- pivot 能大幅降低实际搜索量，但不改变指数级最坏界
- 最大团一定是极大团，因此可以在枚举极大团时顺便统计最大团
- `maximum_mask` 只保存任意一个最大团

复杂度

极大团数量经典上界为

$$O(3^{n/3})$$

Bron–Kerbosch + pivot 的运行时间与输出的极大团数量同为指数级量级

邻接位集空间

$$O(n)$$

递归深度

$$O(n)$$

6. 匹配、网络流与图优化模型

6.1 二分图最大匹配（Hopcroft–Karp）

```
struct HopcroftKarp {
    int nl, nr, shortest{ -1 }, matching{ 0 };
    vector<vector<int>> g;
    vector<int> ml, mr, dis, ptr;
    HopcroftKarp(int nl_, int nr_) : nl(nl_), nr(nr_), g(nl), ml(nl, -1), mr(nr, -1), dis(nl), ptr(nl) {}
    void add_edge(int l, int r) { g[l].push_back(r); }
    // ===== Hopcroft--Karp 核心 =====
```

```
    // 每轮只保留最短增广路所在的分层
    bool bfs() {
        queue<int> q;
        ranges::fill(dis, -1);
        for (int u = 0; u < nl; ++ u) if (ml[u] == -1) dis[u] = 0, q.push(u);
        shortest = -1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (shortest != -1 && dis[u] + 1 > shortest) continue;
            for (int v : g[u]) {
                int w = mr[v];
                if (w == -1) {
                    if (shortest == -1) shortest = dis[u] + 1;
                } else if (dis[w] == -1 && (shortest == -1 || dis[u] + 1 < shortest)) {
                    dis[w] = dis[u] + 1;
                    q.push(w);
                }
            }
        }
        return shortest != -1;
    }
    bool dfs(int u) {
        for (int &i = ptr[u]; i < g[u].size(); ++ i) {
            int v = g[u][i], w = mr[v];
            if (w == -1) {
                if (dis[u] + 1 != shortest) continue;
            } else if (dis[w] != dis[u] + 1 || !dfs(w)) {
                continue;
            }
            ml[u] = v;
            mr[v] = u;
            return true;
        }
        dis[u] = -1;
        return false;
    }
    // 可以在已有匹配上继续增广，因此支持增量加边 / 加点后再次调用
    int max_matching() {
        while (bfs()) {
            ranges::fill(ptr, 0);
            for (int u = 0; u < nl; ++ u) if (ml[u] == -1 && dfs(u)) ++ matching;
        }
        return matching;
    }
    // ===== 匹配方案 =====
    vector<pair<int, int>> matching_edges() const {
        vector<pair<int, int>> ans;
        ans.reserve(matching);
        for (int u = 0; u < nl; ++ u) if (ml[u] != -1) ans.emplace_back(u, ml[u]);
        return ans;
    }
    // ===== 动态建图 =====
    int add_left() {
        g.push_back({});
        ml.push_back(-1);
        dis.push_back(0);
        ptr.push_back(0);
        return nl ++;
    }
    int add_right() {
        mr.push_back(-1);
        return nr ++;
    }
    // ===== 删边 / 重算 =====
    // 删除一条 l-r 边；存在重边时只删除一条
    // 若删除的是最后一条承载当前匹配的 l-r 边，会自动解除该匹配
    bool erase_edge(int l, int r) {
        auto it = find(g[l].begin(), g[l].end(), r);
        if (it == g[l].end()) return false;
        g[l].erase(it);
```

```
        if (ml[l] == r && find(g[l].begin(), g[l].end(), r) == g[l].end()) {
            ml[l] = mr[r] = -1;
            -- matching;
        }
        return true;
    }
    // 保留图，只清空当前匹配
    void clear_matching() {
        ranges::fill(ml, -1);
        ranges::fill(mr, -1);
        matching = 0;
    }
    // ===== König 定理扩展：点覆盖 / 独立集 =====
    // 从所有未匹配左点出发，沿“非匹配边 L→R、匹配边 R→L”搜索
    pair<vector<char>, vector<char>> alternating_reachable() const {
        vector<char> zl(nl), zr(nr);
        queue<int> q;
        for (int u = 0; u < nl; ++ u) if (ml[u] == -1) zl[u] = true, q.push(u);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                if (ml[u] == v || zr[v]) continue;
                zr[v] = true;
                if (mr[v] != -1 && !zl[mr[v]]) {
                    zl[mr[v]] = true;
                    q.push(mr[v]);
                }
            }
        }
        return {zl, zr};
    }
    // 最小点覆盖 = (L \ ZL) ∪ ZR
    pair<vector<int>, vector<int>> min_vertex_cover() const {
        auto [zl, zr] = alternating_reachable();
        vector<int> left, right;
        for (int u = 0; u < nl; ++ u) if (!zl[u]) left.push_back(u);
        for (int v = 0; v < nr; ++ v) if (zr[v]) right.push_back(v);
        return {left, right};
    }
    // 最大独立集 = ZL ∪ (R \ ZR)
    pair<vector<int>, vector<int>> max_independent_set() const {
        auto [zl, zr] = alternating_reachable();
        vector<int> left, right;
        for (int u = 0; u < nl; ++ u) if (zl[u]) left.push_back(u);
        for (int v = 0; v < nr; ++ v) if (!zr[v]) right.push_back(v);
        return {left, right};
    }
    // ===== 最小边覆盖 =====
    // 要求当前匹配已经最大，且整个二分图不存在孤立点
    vector<pair<int, int>> min_edge_cover() const {
        vector<pair<int, int>> ans;
        vector<int> any(nr, -1);
        for (int u = 0; u < nl; ++ u) {
            for (int v : g[u]) if (any[v] == -1) any[v] = u;
        }
        for (int u = 0; u < nl; ++ u) if (ml[u] != -1) ans.emplace_back(u, ml[u]);
        for (int u = 0; u < nl; ++ u) if (ml[u] == -1) ans.emplace_back(u, g[u][0]);
        for (int v = 0; v < nr; ++ v) if (mr[v] == -1) ans.emplace_back(any[v], v);
        return ans;
    }
};
```

成员变量

- `g[u]`：左部点 `u` 相邻的所有右部点
- `ml[u]`：左部点 `u` 当前匹配的右部点，未匹配为 `-1`
- `mr[v]`：右部点 `v` 当前匹配的左部点，未匹配为 `-1`
- `matching`：当前匹配边数
- `nl`、`nr`：当前左右部点数

接口

- `HopcroftKarp(nl,nr)`：建立左右部大小分别为 `nl`、`nr` 的二分图，$O(n_l+n_r)$
- `add_edge(l,r)`：加入左部 `l` 到右部 `r` 的边，均摊 $O(1)$
- `max_matching()`：从当前匹配继续增广到最大匹配，返回当前总匹配数，$O(m\sqrt{n_l+n_r})$
- `matching_edges()`：返回当前匹配中的所有 `{左点,右点}`，$O(n_l)$
- `add_left()`：新增一个左部点并返回编号，均摊 $O(1)$
- `add_right()`：新增一个右部点并返回编号，均摊 $O(1)$
- `erase_edge(l,r)`：删除一条 `l-r` 边；若最后一条该重边正被匹配则同时解除匹配，$O(\deg(l))$
- `clear_matching()`：保留图并清空当前匹配，$O(n_l+n_r)$
- `alternating_reachable()`：在最大匹配对应交错图中返回 `{ZL,ZR}` 的可达标记，$O(n+m)$
- `min_vertex_cover()`：恢复一个最小点覆盖，返回 `{左部点集合,右部点集合}`，$O(n+m)$
- `max_independent_set()`：恢复一个最大独立集，返回 `{左部点集合,右部点集合}`，$O(n+m)$
- `min_edge_cover()`：恢复一个最小边覆盖，返回边集合，$O(n+m)$

注意事项

- 左右两侧使用独立的 `0base` 编号：左部为 `0..nl-1`，右部为 `0..nr-1`
- `ml` / `mr` 本身就是一种匹配方案；`matching_edges()` 只是将它整理成边集合
- 最大匹配通常不唯一，`ml` / `mr` 保存的是算法找到的其中一种方案
- 可以在已有最大匹配后继续 `add_edge()`、`add_left()`、`add_right()`，再调用 `max_matching()` 增量增广，不需要清空原匹配
- `erase_edge()` 后建议重新调用 `max_matching()`；删除匹配边后原匹配可能已经不再最大
- `erase_edge()` 使用 `vector::erase`，只适合少量动态修改；若存在大量删边操作，不建议使用本板子模拟全动态匹配
- 改边可以使用 `erase_edge(old_l,old_r)`、`add_edge(new_l,new_r)` 后重新调用 `max_matching()`
- `min_vertex_cover()`、`max_independent_set()`、`alternating_reachable()` 必须在当前图已经得到最大匹配后调用
- 最小点覆盖大小等于最大匹配大小
- 最大独立集大小为 `nl + nr - matching`
- `min_edge_cover()` 必须在最大匹配后调用，并要求所有点都不是孤立点；其大小为 `nl + nr - matching`
- 左部存在完全匹配当且仅当 `matching == nl`
- 右部存在完全匹配当且仅当 `matching == nr`
- 整张二分图存在完美匹配需要 `nl == nr && matching == nl`
- 重边不影响最大匹配正确性，但会增加常数
- 点具有容量的多重匹配更适合转最大流；带权匹配使用 KM 或费用流
- 该模板只处理二分图；一般图最大匹配使用带花树

常见用法

最大匹配并恢复一种方案

```
HopcroftKarp hk(nl, nr);
for (auto [l, r] : edges) hk.add_edge(l, r);
cout << hk.max_matching() << '\n';
for (auto [l, r] : hk.matching_edges()) {
    cout << l << ' ' << r << '\n';
}
```

也可以直接使用 `ml`：

```
hk.max_matching();
for (int l = 0; l < nl; ++ l) {
    if (hk.ml[l] != -1) {
        int r = hk.ml[l];
        // l 与 r 匹配
    }
}
```

判断并恢复左部完全匹配

```
if (hk.max_matching() == hk.nl) {
    for (int l = 0; l < hk.nl; ++ l) {
        int r = hk.ml[l];
        // 每个左点 l 都分配给唯一右点 r
    }
}
```

增量加入边或点

已经求过最大匹配后，不需要重新建图：

```
hk.max_matching();
hk.add_edge(l, r);
int u = hk.add_left();
int v = hk.add_right();
hk.add_edge(u, v);
int matching = hk.max_matching();
```

删除或修改边

```
hk.max_matching();
hk.erase_edge(u, v);
hk.max_matching();
```

修改一条边：

```
hk.erase_edge(u1, v1);
hk.add_edge(u2, v2);
hk.max_matching();
```

最小点覆盖

```
hk.max_matching();
auto [left, right] = hk.min_vertex_cover();
for (int u : left) {
    // 选择左部点 u
}
for (int v : right) {
    // 选择右部点 v
}
```

覆盖中的点数恒为：

```
int cover_size = left.size() + right.size(); // == hk.matching
```

最大独立集

```
hk.max_matching();
auto [left, right] = hk.max_independent_set();
```

其中任意两个被选点之间都不存在二分图边，且：

```
int independent_size = left.size() + right.size();
// == hk.nl + hk.nr - hk.matching
```

最小边覆盖

要求图中没有孤立点：

```
hk.max_matching();
auto cover = hk.min_edge_cover();
for (auto [u, v] : cover) {
    // 选择边 u-v
}
```

边数为：

```
int edge_cover_size = hk.nl + hk.nr - hk.matching;
```

DAG 最小路径覆盖并恢复方案

对于 DAG，将每个原点 `u` 拆成左部 `u` 和右部 `u`；原图存在边 `u → v` 时加入二分图边 `u_L → v_R`

```
HopcroftKarp hk(n, n);
for (auto [u, v] : edges) hk.add_edge(u, v);
int matching = hk.max_matching();
int path_count = n - matching;
```

恢复每条路径：

```
vector<int> pre(n, -1), nxt(n, -1);
for (auto [u, v] : hk.matching_edges()) {
    nxt[u] = v;
    pre[v] = u;
}
vector<vector<int>> paths;
for (int s = 0; s < n; ++ s) if (pre[s] == -1) {
    vector<int> path;
    for (int u = s; u != -1; u = nxt[u]) {
        path.push_back(u);
    }
    paths.push_back(move(path));
}
```

此时 `paths.size() == n - matching`

网格骨牌 / 配对方案

将网格按黑白染色划分为左右部，相邻可配对格子连边：

```
int matching = hk.max_matching();
if (matching * 2 == cells) {
    for (auto [black, white] : hk.matching_edges()) {
        // black 与 white 构成一块骨牌
    }
}
```

复杂度

设 $n = n_l + n_r$，边数为 m

操作	时间复杂度
`add_edge()`	均摊 $O(1)$
`add_left()` / `add_right()`	均摊 $O(1)$
`erase_edge()`	$O(\deg(l))$
`clear_matching()`	$O(n)$
`max_matching()`	$O(m\sqrt{n})$
`matching_edges()`	$O(n_l)$
`alternating_reachable()`	$O(n+m)$
`min_vertex_cover()`	$O(n+m)$
`max_independent_set()`	$O(n+m)$
`min_edge_cover()`	$O(n+m)$

空间复杂度：$O(n+m)$
多次增量修改并反复调用 `max_matching()` 时，总复杂度按各次调用累加，不应直接视为一次 $O(m\sqrt{n})$

6.1.1 Hungarian-KM 二分图最优匹配

```
template<bool MAX = false, class T = i64>
pair<T, vector<int>> hungarian(const vector<vector<T>> &a) {
    int n = a.size();
    if (!n) return {T{}, {}};
    int m = a[0].size();
    assert(n <= m);
    for (const auto &row : a) assert(int(row.size()) == m);
    const T INF = numeric_limits<T>::max() / 4;
    vector<T> u(n + 1), v(m + 1), minv(m + 1);
    vector<int> p(m + 1), way(m + 1);
    vector<char> used(m + 1);
    // 逐个加入左部点，维护势能与到未使用右部点的最短增广路
    for (int i = 1; i <= n; ++ i) {
        p[0] = i;
```

```
        int j0{ 0 };
        ranges::fill(minv, INF);
        ranges::fill(used, false);
        do {
            used[j0] = true;
            int i0{ p[j0] }, j1{ 0 };
            T delta{ INF };
            for (int j = 1; j <= m; ++ j) if (!used[j]) {
                T cur;
                if constexpr (MAX) cur = u[i0] + v[j] - a[i0 - 1][j - 1];
                else cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            for (int j = 0; j <= m; ++ j) {
                if (used[j]) {
                    if constexpr (MAX) u[p[j]] -= delta, v[j] += delta;
                    else u[p[j]] += delta, v[j] -= delta;
                } else {
                    minv[j] -= delta;
                }
            }
            j0 = j1;
        } while (p[j0] != 0);
        // 沿 way[] 翻转增广路
        do {
            int j1{ way[j0] };
            p[j0] = p[j1];
            j0 = j1;
        } while (j0 != 0);
    }
    vector<int> match(n, -1);
    for (int j = 1; j <= m; ++ j) if (p[j]) match[p[j] - 1] = j - 1;
    T value{};
    for (int i = 0; i < n; ++ i) value += a[i][match[i]];
    return {value, match};
}
template<class T = i64>
pair<T, vector<int>> hungarian_min(const vector<vector<T>> &a) { return hungarian<false>(a); }
template<class T = i64>
pair<T, vector<int>> hungarian_max(const vector<vector<T>> &a) { return hungarian<true>(a); }
```

接口

- `hungarian_min(a)`：求最小权指派，返回 `{最小总权值, match}`，时间复杂度 $O(n^2m)$
- `hungarian_max(a)`：求最大权指派，返回 `{最大总权值, match}`，时间复杂度 $O(n^2m)$
- `hungarian<MAX>(a)`：统一核心；`MAX = false` 为最小化，`MAX = true` 为最大化
- `match[i]`：左部点 `i` 匹配到的右部点编号，范围为 $[0, m)$
- 输入为 $n \times m$ 权值矩阵，要求 $n \le m$
- 算法要求每个左部点恰好匹配一个不同的右部点；右部点可以不全部使用

`assert` 条件

- `n <= m`：右部点数量必须不少于左部点，否则无法让所有左部点匹配不同的右部点
- `row.size() == m`：所有行长度必须相同，输入必须是合法矩阵

注意事项

- $n = m$ 时求二分图带权完美匹配；$n < m$ 时所有左部点均匹配，右部有 $m - n$ 个点未匹配
- 若原问题两侧大小不同且只要求较小一侧全部匹配，应将较小的一侧作为矩阵的行
- `hungarian_max()` 直接维护最大化对应的势能，等价于对权值取负后运行最小化版本，但不需要复制或修改输入矩阵
- 权值可以为负数；算法并不要求边权非负
- 默认矩阵中的每个位置都是合法边；它首先解决的是完全二分图上的指派问题
- 若存在少量禁边，可以赋一个足够差的权值，并在求解后检查最终匹配是否经过禁边
- 表示禁边时不要使用接近 `numeric_limits<T>::max()` 或 `lowest()` 的值，否则 `u + v - a`、`a - u - v` 等运算可能溢出
- 若合法边很稀疏或需要自然处理不存在的边，通常更适合使用最小费用流
- 若允许某些左部点不匹配真实右部点，应加入足够多的虚拟右部点
- 松弛量的有效上界须严格小于内部哨兵 `numeric_limits<T>::max() / 4`；大权值应选更宽类型，不能只检查最终答案
- `T` 需要容纳原始权值、势能、松弛量以及最终权值和；范围较大时使用 `i128`
- 不建议使用无符号整数类型，尤其最大化版本的势能和松弛过程中可能出现负值

- Hungarian / KM 解决的是带权一对一二分图指派；无权最大匹配优先使用 Hopcroft–Karp
- 若一个点允许匹配多个对象、边有容量，或还存在流量约束，应使用网络流 / 费用流
- 一般图带权匹配不属于二分图指派，需要使用带权 Blossom

常见用法

最小权指派

```
auto [cost, match] = hungarian_min(a);
for (int i = 0; i < n; ++ i) {
    int j{ match[i] };
    // 左部 i 分配给右部 j
}
```

最大权指派

```
auto [weight, match] = hungarian_max(a);
```

矩形指派

有 n 个人、m 个任务，且 $n \le m$，要求每个人领取一个不同任务：

```
vector<vector<i64>> cost(n, vector<i64>(m));
for (int i = 0; i < n; ++ i) {
    for (int j = 0; j < m; ++ j) {
        cost[i][j] = calc_cost(i, j);
    }
}
auto [ans, match] = hungarian_min(cost);
```

不需要补成方阵，最终自然有 $m - n$ 个任务未被选择

恢复右部对应的左部点

`match` 保存左到右映射；若还需要右到左：

```
vector<int> rev(m, -1);
for (int i = 0; i < n; ++ i) {
    rev[match[i]] = i;
}
```

其中 `rev[j] == -1` 表示右部点 `j` 未匹配

允许左部点不匹配真实对象

给每个左部点准备足够多的虚拟右部点
例如最大化收益，允许放弃某个人：

```
vector<vector<i64>> a(n, vector<i64>(m + n));
for (int i = 0; i < n; ++ i) {
    for (int j = 0; j < m; ++ j) {
        a[i][j] = profit[i][j];
    }
    // 后 n 列为虚拟任务，收益为 0
}
auto [ans, match] = hungarian_max(a);
for (int i = 0; i < n; ++ i) {
    if (match[i] < m) {
        // i 选择真实任务 match[i]
    } else {
        // i 放弃匹配真实任务
    }
}
```

之所以通常准备 n 个虚拟右部点，是因为右部点仍然每个至多只能使用一次；只添加一个公共虚点不能让多个左部点同时选择“不匹配”
最小化问题同理，可把虚拟边设为“不分配”的固定代价

存在少量禁边

例如最小化问题：

```
constexpr i64 BAD = 1E15;
vector<vector<i64>> cost(n, vector<i64>(m, BAD));
for (auto [u, v, w] : edges) cost[u][v] = w;
auto [ans, match] = hungarian_min(cost);
bool ok{ true };
for (int i = 0; i < n; ++ i) {
    if (cost[i][match[i]] == BAD) ok = false;
}
```

需要保证 `BAD` 足够差，同时所有中间运算和答案仍不会溢出

最大化两组对象的总相似度

```
vector<vector<i64>> score(n, vector<i64>(m));
for (int i = 0; i < n; ++ i) {
    for (int j = 0; j < m; ++ j) {
        score[i][j] = calc_score(i, j);
    }
}

auto [ans, match] = hungarian_max(score);
```

复杂度

设左部大小为 n，右部大小为 m，且 $n \le m$

操作	时间复杂度
`hungarian_min()`	$O(n^2m)$
`hungarian_max()`	$O(n^2m)$
方阵 $n = m$	$O(n^3)$

算法额外空间复杂度为 $O(n+m)$；输入矩阵本身占用 $O(nm)$ 空间

6.1.2 一般图最大匹配（Edmonds Blossom）

```
struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, blossom;
    explicit Blossom(int n_) : n(n_), g(n), match(n, -1), p(n), base(n), q(n), used(n), blossom(n) {}
    void add_edge(int u, int v) { if (u != v) g[u].push_back(v), g[v].push_back(u); }
    // 求交错森林中两个点所在花的公共基点
    int lca(int a, int b) {
        vector<char> seen(n);
        while (true) {
            a = base[a];
            seen[a] = true;
            if (match[a] == -1) break;
            a = p[match[a]];
        }
        while (true) {
            b = base[b];
            if (seen[b]) return b;
            b = p[match[b]];
        }
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            blossom[base[v]] = blossom[base[match[v]]] = true;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }
    // 从未匹配根出发 BFS；遇到交错奇环时缩花
```

```
    int find_path(int root) {
        ranges::fill(used, false);
        ranges::fill(p, -1);
        iota(base.begin(), base.end(), 0);
        int ql{ 0 }, qr{ 0 };
        q[qr ++] = root;
        used[root] = true;
        while (ql < qr) {
            int v{ q[ql ++] };
            for (int u : g[v]) {
                if (base[v] == base[u] || match[v] == u) continue;
                if (u == root || (match[u] != -1 && p[match[u]] != -1)) {
                    int b{ lca(v, u) };
                    ranges::fill(blossom, false);
                    mark_path(v, b, u);
                    mark_path(u, b, v);
                    for (int i = 0; i < n; ++ i) if (blossom[base[i]]) {
                        base[i] = b;
                        if (!used[i]) used[i] = true, q[qr ++] = i;
                    }
                } else if (p[u] == -1) {
                    p[u] = v;
                    if (match[u] == -1) return u;
                    u = match[u];
                    used[u] = true;
                    q[qr ++] = u;
                }
            }
        }
        return -1;
    }
    int max_matching() {
        for (int root = 0; root < n; ++ root) if (match[root] == -1) {
            int v{ find_path(root) };
            if (v == -1) continue;
            while (v != -1) {
                int pv{ p[v] }, nv{ pv == -1 ? -1 : match[pv] };
                match[v] = pv;
                if (pv != -1) match[pv] = v;
                v = nv;
            }
        }
        int ans{};
        for (int v : match) ans += v != -1;
        return ans / 2;
    }
};
```

成员变量

- `g[u]`：点 `u` 的邻接表
- `match[u]`：点 `u` 的匹配点，未匹配为 `-1`
- `base[u]`：当前缩花后 `u` 所属花的基点
- `p[u]`：当前交错森林中的前驱

接口

- `Blossom(n)`：建立 `n` 个点的无向图，$O(n)$
- `add_edge(u, v)`：加入无向边，自环自动忽略，均摊 $O(1)$
- `max_matching()`：从当前匹配继续增广至最大基数匹配，返回当前匹配边数，$O(n^3)$
- 求解后 `match[u] = v` 当且仅当 `match[v] = u`，未匹配点为 `-1`

注意事项

- 点编号为 `0base`，调用接口需保证 `0 ≤ u, v < n`
- 解决**一般无向图最大基数匹配**，即最大化匹配边数，不处理边权
- 核心区别于二分图匹配：交错森林中遇到奇环时将其缩成一个花，再继续寻找增广路；常见竞赛实现复杂度为 $O(n^3)$
- 平行边不影响正确性但会增加常数，自环直接忽略
- 可以先调用 `max_matching()`，继续 `add_edge()` 后再次调用；原匹配会作为合法初始匹配继续增广
- 重复调用 `max_matching()` 返回的是**当前最大匹配总边数**，不是本次新增的匹配数
- 不要直接修改 `match`，除非能保证它始终表示一个合法匹配

- 二分图优先使用 Hopcroft–Karp，代码更简单且复杂度更优
- 一般图最大权匹配需要带权 Blossom，是另一套明显更复杂的算法；参考模板体系也将无权 Blossom 与一般图最大权匹配分开整理
- 随机贪心或 `shuffle` 匹配只能用于启发式加速，不能代替 Blossom 保证答案正确

常见用法

```
Blossom bm(n);
for (auto [u, v] : edges) bm.add_edge(u, v);

int matching{ bm.max_matching() };
for (int u = 0; u < n; ++ u) if (bm.match[u] != -1 && u < bm.match[u]) {
    int v{ bm.match[u] };
    // 匹配边 (u, v)
}
```

判断是否存在完美匹配

```
bool perfect{ n % 2 == 0 && bm.max_matching() * 2 == n };
```

动态加入边后继续增广

```
int ans{ bm.max_matching() };

bm.add_edge(u, v);
ans = bm.max_matching();
```

无需清空已有 `match`

复杂度

时间复杂度：$O(n^3)$
空间复杂度：$O(n+m)$

6.1.3 Weighted Blossom 一般图最大权匹配

```
template<class T = i64>
struct WeightedBlossom {
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed);
    struct Edge { int u{}, v{}; T w{}; };
    int n, m{}, id{};
    vector<tuple<int, int, T>> edges;
    vector<vector<Edge>> e;
    vector<vector<int>> b, flower;
    vector<T> lab;
    vector<int> lk, sl, st, f, s, ed, q;
    T slack(const Edge &x) const { return lab[x.u] + lab[x.v] - x.w * 2; }
    void update_slack(int u, int v) { if (!sl[v] || slack(e[u][v]) < slack(e[sl[v]][v])) sl[v] = u; }
    void set_slack(int v) {
        sl[v] = 0;
        for (int u = 1; u <= n; ++ u) if (e[u][v].w > 0 && st[u] != v && s[st[u]] == 0) update_slack(u, v);
    }
    void push(int u) {
        if (u <= n) q.push_back(u);
        else for (int v : flower[u]) push(v);
    }
    void set_st(int u, int v) {
        st[u] = v;
        if (u > n) for (int x : flower[u]) set_st(x, v);
    }
    int get_pos(int u, int v) {
        int p{ int(find(flower[u].begin(), flower[u].end(), v) - flower[u].begin()) };
        if (p & 1) {
            reverse(flower[u].begin() + 1, flower[u].end());
            return int(flower[u].size()) - p;
        }
        return p;
    }
    // ===== 匹配与缩花：核心功能，不建议删除 =====
```

```
    void set_match(int u, int v) {
        lk[u] = e[u][v].v;
        if (u <= n) return;
        Edge w{ e[u][v] };
        int x{ b[u][w.u] }, y{ get_pos(u, x) };
        for (int i = 0; i < y; ++ i) set_match(flower[u][i], flower[u][i ^ 1]);
        set_match(x, v);
        rotate(flower[u].begin(), flower[u].begin() + y, flower[u].end());
    }
    void augment(int u, int v) {
        int w{ st[lk[u]] };
        set_match(u, v);
        if (!w) return;
        set_match(w, st[f[w]]);
        augment(st[f[w]], w);
    }
    int lca(int u, int v) {
        for (++ id; u || v; swap(u, v)) {
            if (!u) continue;
            if (ed[u] == id) return u;
            ed[u] = id;
            u = st[lk[u]];
            if (u) u = st[f[u]];
        }
        return 0;
    }
    void add_blossom(int u, int a, int v) {
        int x{ n + 1 }, i, j;
        while (x <= m && st[x]) ++ x;
        if (x > m) ++ m;
        lab[x] = T{};
        s[x] = st[x] = 0;
        lk[x] = lk[a];
        flower[x].clear();
        flower[x].push_back(a);
        for (i = u; i != a; i = st[f[j]]) {
            flower[x].push_back(i);
            flower[x].push_back(j = st[lk[i]]);
            push(j);
        }
        reverse(flower[x].begin() + 1, flower[x].end());
        for (i = v; i != a; i = st[f[j]]) {
            flower[x].push_back(i);
            flower[x].push_back(j = st[lk[i]]);
            push(j);
        }
        set_st(x, x);
        for (i = 1; i <= m; ++ i) e[x][i].w = e[i][x].w = T{};
        fill(b[x].begin() + 1, b[x].begin() + n + 1, 0);
        for (int u0 : flower[x]) {
            for (int v0 = 1; v0 <= m; ++ v0) if (!e[x][v0].w || slack(e[u0][v0]) < slack(e[x][v0])) {
                e[x][v0] = e[u0][v0];
                e[v0][x] = e[v0][u0];
            }
            for (int v0 = 1; v0 <= n; ++ v0) if (b[u0][v0]) b[x][v0] = u0;
        }
        set_slack(x);
    }
    void expand_blossom(int u) {
        for (int x : flower[u]) set_st(x, x);
        int a{ b[u][e[u][f[u]].u] }, r{ get_pos(u, a) };
        for (int i = 0; i < r; i += 2) {
            int x{ flower[u][i] }, y{ flower[u][i + 1] };
            f[x] = e[y][x].u;
            s[x] = 1;
            s[y] = sl[x] = 0;
            set_slack(y);
            push(y);
        }
        s[a] = 1;
        f[a] = f[u];
```

```
        for (int i = r + 1; i < int(flower[u].size()); ++ i) {
            s[flower[u][i]] = -1;
            set_slack(flower[u][i]);
        }
        st[u] = 0;
    }
    bool on_edge(const Edge &x) {
        int u{ st[x.u] }, v{ st[x.v] };
        if (s[v] == -1) {
            f[v] = x.u;
            s[v] = 1;
            int a{ st[lk[v]] };
            sl[v] = sl[a] = s[a] = 0;
            push(a);
        } else if (s[v] == 0) {
            int a{ lca(u, v) };
            if (!a) {
                augment(u, v);
                augment(v, u);
                return true;
            }
            add_blossom(u, a, v);
        }
        return false;
    }
    // ===== 原始对偶增广：维护标号与 slack =====
    bool bfs() {
        fill(s.begin() + 1, s.begin() + m + 1, -1);
        fill(sl.begin() + 1, sl.begin() + m + 1, 0);
        q.clear();
        size_t h{};
        for (int i = 1; i <= m; ++ i) if (st[i] == i && !lk[i]) {
            f[i] = s[i] = 0;
            push(i);
        }
        if (q.empty()) return false;
        while (true) {
            while (h < q.size()) {
                int u{ q[h ++] };
                if (s[st[u]] == 1) continue;
                for (int v = 1; v <= n; ++ v) if (e[u][v].w > 0 && st[u] != st[v]) {
                    if (slack(e[u][v]) != 0) update_slack(u, st[v]);
                    else if (on_edge(e[u][v])) return true;
                }
            }
            T x{ numeric_limits<T>::max() / 4 };
            for (int i = n + 1; i <= m; ++ i) if (st[i] == i && s[i] == 1) x = min(x, lab[i] >> 1);
            for (int i = 1; i <= m; ++ i) if (st[i] == i && sl[i] && s[i] != 1) {
                x = min(x, slack(e[sl[i]][i]) >> (s[i] + 1));
            }
            for (int i = 1; i <= n; ++ i) if (s[st[i]] != -1) {
                lab[i] += T(s[st[i]] * 2 - 1) * x;
                if (lab[i] <= 0) return false;
            }
            for (int i = n + 1; i <= m; ++ i) if (st[i] == i && s[st[i]] != -1) {
                lab[i] += T(2 - s[st[i]] * 4) * x;
            }
            q.clear();
            h = 0;
            for (int i = 1; i <= m; ++ i) if (st[i] == i && sl[i] && st[sl[i]] != i && slack(e[sl[i]][i]) == 0) {
                if (on_edge(e[sl[i]][i])) return true;
            }
            for (int i = n + 1; i <= m; ++ i) if (st[i] == i && s[i] == 1 && lab[i] == 0) expand_blossom(i);
        }
    }

public:
    explicit WeightedBlossom(int n_) : n(n_) {}
    void add_edge(int u, int v, T w) { if (u != v) edges.emplace_back(u + 1, v + 1, w); }
    pair<T, vector<int>> max_weight_matching() {
        int cap{ 2 * n + 2 };
```

```
        e.assign(cap, vector<Edge>(cap));
        b.assign(cap, vector<int>(n + 1));
        flower.assign(cap, {});
        lab.assign(cap, T{});
        lk.assign(cap, 0);
        sl.assign(cap, 0);
        st.assign(cap, 0);
        f.assign(cap, 0);
        s.assign(cap, 0);
        ed.assign(cap, 0);
        q.clear();
        m = n;
        id = 0;
        for (int i = 1; i <= n; ++ i) st[i] = i;
        for (int i = 1; i <= n; ++ i) for (int j = 1; j <= n; ++ j) e[i][j] = { i, j, T{} };
        T wm{};
        for (auto [u, v, w] : edges) if (w > e[u][v].w) {
            e[u][v].w = e[v][u].w = w;
            wm = max(wm, w);
        }
        for (int i = 1; i <= n; ++ i) b[i][i] = i;
        fill(lab.begin() + 1, lab.begin() + n + 1, wm);
        while (bfs()) {}
        T value{};
        vector<int> match(n, -1);
        for (int i = 1; i <= n; ++ i) if (lk[i]) {
            if (i < lk[i]) value += e[i][lk[i]].w;
            match[i - 1] = lk[i] - 1;
        }
        return { value, match };
    }
};
```

接口

- `WeightedBlossom<T>(n)`：建立 `n` 点一般无向图
- `add_edge(u, v, w)`：加入权值为 `w` 的无向边，自环忽略
- `max_weight_matching()`：返回 `{最大权值和, match}`，其中 `match[u]` 为 `u` 的匹配点，未匹配为 `-1`
- 平行边只保留同一对端点中的最大权值
- 外部点编号为 `0base`

注意事项

- 求的是**允许部分点不匹配时的最大权值匹配**，不是最大基数匹配，也不强制完美匹配
- 内部只处理正权边，权值为 `0` 或负数的边不会被选；在允许空匹配时这不影响最大权值，但不会为了相同权值额外增大匹配边数
- 若需要“先最大化匹配边数，再最大化权值”，可给每条合法边增加一个足够大的统一常数
- 若已知 `|w| ≤ W`，可取 `C > nW`，将边权改为 `w + C`，此时先最大化匹配边数，同基数下再最大化原权值
- 若要求最大权完美匹配，可使用上述变换并检查所有点是否均被匹配；若存在未匹配点则不存在完美匹配
- 最小权完美匹配可将权值变为 `C - cost`，其中 `C` 需同时保证基数优先且中间值不溢出
- `T` 必须是有符号整数类型，且需要容纳 `2w`、标号、slack 和答案和；权值较大时使用 `i128`
- 重复调用 `max_weight_matching()` 会从保存的边重新求解，因此可以继续 `add_edge()` 后再次调用
- 一般二分图最大权指派优先使用 Hungarian / KM，带权 Blossom 只用于一般图
- 该实现使用稠密矩阵维护缩花间的最优边，空间为 $O(n^2)$

常见用法

最大权匹配

```
WeightedBlossom<i64> bm(n);
for (auto [u, v, w] : edges) bm.add_edge(u, v, w);
auto [ans, match] = bm.max_weight_matching();
for (int u = 0; u < n; ++ u) if (match[u] != -1 && u < match[u]) {
    int v{ match[u] };
    // 匹配边 (u, v)
}
```

最大权完美匹配

若 `|w| ≤ W`，先加入足够大的基数奖励

```
i128 C = i128(n) * W + 1;
WeightedBlossom<i128> bm(n);
for (auto [u, v, w] : edges) bm.add_edge(u, v, i128(w) + C);
auto [value, match] = bm.max_weight_matching();
bool ok{ true };
for (int u = 0; u < n; ++ u) {
    if (match[u] == -1) ok = false;
}
if (ok) value -= C * (n / 2);
```

最大基数优先，再最大化权值

同样使用 `w + C`

```
i128 C = i128(n) * W + 1;
WeightedBlossom<i128> bm(n);
for (auto [u, v, w] : edges) bm.add_edge(u, v, i128(w) + C);
auto [value, match] = bm.max_weight_matching();
int cnt{};
for (int u = 0; u < n; ++ u) cnt += match[u] != -1;
cnt /= 2;
i128 original_value = value - C * cnt;
```

复杂度

时间复杂度：$O(n^3)$
空间复杂度：$O(n^2)$

6.2 Dinic 最大流

```
template<class Cap = i64>
struct Dinic {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
    struct Edge {
        int from, to;
        Cap cap, init;
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> level, it;
    explicit Dinic(int n_, int m_hint = 0) : n(n_), g(n), level(n), it(n) {
        e.reserve(2 * m_hint);
    }
    // ===== 基础建边与 Dinic =====
    int add_edge(int u, int v, Cap cap, Cap rcap = 0) {
        int id = e.size() / 2;
        g[u].push_back(e.size());
        e.push_back({u, v, cap, cap});
        g[v].push_back(e.size());
        e.push_back({v, u, rcap, rcap});
        return id;
    }
    bool bfs(int s, int t, Cap scale = 1) {
        ranges::fill(level, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : g[u]) {
                const Edge &edge = e[id];
                if (edge.cap < scale || level[edge.to] != -1) continue;
                level[edge.to] = level[u] + 1;
                q.push(edge.to);
            }
        }
```

```
        return level[t] != -1;
    }
    Cap dfs(int u, int t, Cap f, Cap scale = 1) {
        if (u == t) return f;
        Cap sent = 0;
        for (int &i = it[u]; i < g[u].size() && sent < f; ++ i) {
            int id = g[u][i];
            Edge &edge = e[id];
            if (edge.cap < scale || level[edge.to] != level[u] + 1) continue;
            Cap got = dfs(edge.to, t, min(f - sent, edge.cap), scale);
            if (got == 0) continue;
            edge.cap -= got;
            e[id ^ 1].cap += got;
            sent += got;
        }
        if (sent < f) level[u] = -1;
        return sent;
    }
    Cap max_flow(int s, int t, Cap limit = numeric_limits<Cap>::max()) {
        if (s == t || limit == 0) return 0;
        Cap flow = 0;
        while (flow < limit && bfs(s, t)) {
            ranges::fill(it, 0);
            flow += dfs(s, t, limit - flow);
        }
        return flow;
    }
    // ===== 容量缩放：bfs / dfs 的 scale 保留默认值即可 =====
    Cap max_flow_scaling(int s, int t, Cap limit = numeric_limits<Cap>::max()) {
        if (s == t || limit == 0) return 0;
        Cap mx = 0;
        for (const Edge &edge : e) mx = max(mx, edge.cap);
        Cap scale = 1;
        while (scale <= mx / 2) scale *= 2;
        Cap flow = 0;
        for (; scale > 0 && flow < limit; scale /= 2) {
            while (flow < limit && bfs(s, t, scale)) {
                ranges::fill(it, 0);
                flow += dfs(s, t, limit - flow, scale);
            }
        }
        return flow;
    }
    // ===== 动态容量：用于逐步开放边或修改容量 =====
    void increase_cap(int id, Cap x) {
        assert(x >= 0);
        e[2 * id].cap += x;
        e[2 * id].init += x;
    }
    // 保持当前净流量不变；若新容量无法容纳当前流量则修改失败
    bool set_cap(int id, Cap cap) {
        assert(cap >= 0);
        Edge &edge = e[2 * id];
        Cap f = edge.init - edge.cap;
        if (cap < f) return false;
        edge.cap = cap - f;
        edge.init = cap;
        return true;
    }
    void reset_flow() { for (Edge &edge : e) edge.cap = edge.init; }
    // ===== 方案恢复 =====
    tuple<int, int, Cap, Cap> get_edge(int id) const {
        const Edge &edge = e[2 * id];
        return {edge.from, edge.to, edge.init, edge.init - edge.cap};
    }
    Cap edge_flow(int id) const { return e[2 * id].init - e[2 * id].cap; }
    vector<char> min_cut(int s) const {
        vector<char> vis(n);
        vector<int> stk{s};
        vis[s] = true;
        while (!stk.empty()) {
```

```
            int u = stk.back();
            stk.pop_back();
            for (int id : g[u]) {
                const Edge &edge = e[id];
                if (edge.cap > 0 && !vis[edge.to]) {
                    vis[edge.to] = true;
                    stk.push_back(edge.to);
                }
            }
        }
        return vis;
    }
    // ===== 上下界流辅助 =====
    void disable_edge(int id) { e[2 * id].cap = e[2 * id + 1].cap = 0; }
};
```

成员变量

- `e[2*id]` / `e[2*id+1]`：第 `id` 条逻辑边的正向 / 反向残量边
- `Edge::cap`：当前残量容量
- `Edge::init`：该方向当前设定的初始容量
- `level[u]`：当前分层图中点 `u` 的层数
- `it[u]`：当前弧位置

接口

- `Dinic(n,m_hint)`：建立 `n` 点残量网络；可选预留约 `m_hint` 条逻辑边，$O(n)$
- `add_edge(u,v,cap)`：加入容量为 `cap` 的有向边，返回逻辑边编号，均摊 $O(1)$
- `add_edge(u,v,cap,rcap)`：正反方向初始容量分别为 `cap`、`rcap`
- `max_flow(s,t)`：在**当前残量网络**上继续增广，返回本次新增最大流
- `max_flow(s,t,limit)`：本次最多再发送 `limit` 单位流
- `max_flow_scaling(s,t,limit)`：容量缩放版最大流
- `increase_cap(id,x)`：将第 `id` 条边的正向容量增加 `x`，保留当前流，$O(1)$
- `set_cap(id,cap)`：保持当前流不变，将正向容量修改为 `cap`；若无法容纳当前净流则返回 `false`，$O(1)$
- `reset_flow()`：清空所有已有流量，恢复为当前设定容量，$O(m)$
- `get_edge(id)`：返回 `{from,to,正向容量,当前净正向流量}`，$O(1)$
- `edge_flow(id)`：返回当前净正向流量，$O(1)$
- `min_cut(s)`：返回当前残量网络从 `s` 可达的点集合，$O(n+m)$
- `disable_edge(id)`：直接清空该逻辑边正反两条残量边，仅用于特殊建模，$O(1)$

注意事项

- 点编号为 `0base`，容量使用**有符号整数类型**
- `max_flow()` 修改残量网络，并且返回的是**本次新增流量**，不是历史总流量
- 因此 `ans += flow.max_flow(s, t);` 可以在同一个残量网络上反复继续增广
- 添加新边或增加边容量不会破坏已有流的可行性，因此可以直接继续调用 `max_flow()`
- `set_cap()` 可以降低容量，但要求新容量仍能容纳该边当前净流量
- 若需要把容量降低到当前流量以下，不能只局部修改该边；最简单可靠的方法是 `reset_flow()` 后修改容量并重新求流
- 如果原网络已经达到最大流，而只**降低**若干边容量，并且 `set_cap()` 均成功，则旧流仍然可行，因此仍然是新网络的最大流
- `add_edge(u,v,c,c)` 表示两个方向共享残量关系的无向容量边；两条互相独立的有向边应分别调用两次 `add_edge`
- `get_edge()` / `edge_flow()` 返回净正向流量；若 `rcap > 0`，流量可能为负，范围为 `[-rcap,cap]`
- `min_cut()` 只有在没有因为 `limit` 提前停止、确实已经得到最大流后，才对应最小割
- `disable_edge()` 不是一般意义上的动态删边；若该边已经承载流量，直接清空正反残量会破坏普通流量语义
- `disable_edge()` 主要用于上下界流等建模中移除辅助边；普通删边优先 `reset_flow()` 后 `set_cap(id,0)`
- `reset_flow()` 会恢复所有边，包括之前被 `disable_edge()` 清空的边
- 多源多汇使用超级源汇；超级边容量取可证明的流量上界，不要无脑使用可能溢出的 `INF`
- 总流量、容量之和以及中间结果都必须能放进 `Cap`

> 普通题默认使用 `max_flow()`。容量跨度特别大时可以尝试 `max_flow_scaling()`；若 Dinic 的分层表现依然很差，再考虑 HLPP

常见用法

最大流与最小割

```
Dinic<i64> flow(n, m);
for (auto [u, v, cap] : edges) {
    flow.add_edge(u, v, cap);
}
```

```
i64 ans = flow.max_flow(s, t);
auto side = flow.min_cut(s);
```

保存输入边流量

```
Dinic<i64> flow(n, m);
vector<int> id(m);
for (int i = 0; i < m; ++ i) {
    id[i] = flow.add_edge(u[i], v[i], cap[i]);
}
flow.max_flow(s, t);
for (int i = 0; i < m; ++ i) {
    auto [u, v, cap, used] = flow.get_edge(id[i]);
}
```

查询边的流量与恢复方案

`add_edge()` 返回逻辑边编号，可在求流前保存需要恢复的边：

```
int id = flow.add_edge(u, v, cap);
i64 ans = flow.max_flow(s, t);
i64 f = flow.edge_flow(id); // 边 u → v 的当前净流量
```

限制本次只发送一定流量

`limit` 表示这一次调用最多增加多少流：

```
i64 sent = flow.max_flow(s, t, k);
```

如果需要分阶段发送：

```
i64 ans = 0;
ans += flow.max_flow(s, t, 10);
// 根据当前流量做一些处理
ans += flow.max_flow(s, t, 20);
// 再继续发送至多 20
```

每次都基于上一次留下的残量网络继续

一单位一单位增广

某些需要观察每个流量阶段的题目可以：

```
i64 ans = 0;
while (true) {
    i64 add = flow.max_flow(s, t, 1);
    if (add == 0) break;
    ans += add;
    // 此时最大流恰好又增加了 1
}
```

这种写法会反复执行 Dinic，若不需要中间状态，不应代替一次完整的 `max_flow()`

逐步加入 / 松弛边

已经求出当前图最大流后，可以继续加入新的边：

```
i64 ans = flow.max_flow(s, t);
for (auto [u, v, cap] : new_edges) {
    flow.add_edge(u, v, cap);
}
ans += flow.max_flow(s, t);
```

原有流仍然是一组合法流，因此只需要在新残量网络上继续增广
例如边按照某个阈值逐步开放：

```
Dinic<i64> flow(n);
i64 cur = 0;
for (int i = 0; i < groups.size(); ++ i) {
```

```
    for (auto [u, v, cap] : groups[i]) {
        flow.add_edge(u, v, cap);
    }
    cur += flow.max_flow(s, t);
    if (cur ≥ need) {
        // 第 i 组边开放后第一次达到 need
        break;
    }
}
```

常见于：

- 按边权从小到大逐步允许使用边
- 二分图中逐步允许新的匹配关系
- 随时间开放道路 / 通道
- 按等级逐步松弛建模限制

预先建边，再逐步开放容量

如果边的结构一开始就已知，可以先用容量 `0` 建边：

```
int id = flow.add_edge(u, v, 0);
i64 ans = flow.max_flow(s, t);
flow.increase_cap(id, 5);
ans += flow.max_flow(s, t);
flow.increase_cap(id, 10);
ans += flow.max_flow(s, t);
```

容量依次为：

```
0 → 5 → 15
```

每次增加容量后都可以直接利用旧残量网络继续增广
这通常比每次从头重新建图更合适

同时逐步增加多条边容量

```
i64 ans = flow.max_flow(s, t);
for (auto [id, delta] : changes) {
    flow.increase_cap(id, delta);
}
ans += flow.max_flow(s, t);
```

如果修改全部属于：

```
加边
增加容量
```

则已有流永远保持可行，可以一直复用残量网络

直接修改某条边容量

假设某条边当前：

```
容量 = 10
流量 = 6
```

那么修改为 `8` 是安全的：

```
bool ok = flow.set_cap(id, 8);
```

此时：

```
容量 = 8
流量 = 6
剩余容量 = 2
```

也可以修改为更大：

```
flow.set_cap(id, 20);
i64 add = flow.max_flow(s, t);
```

因为出现了新的剩余容量，所以可以继续增广

安全降低容量

```
i64 used = flow.edge_flow(id);
if (new_cap ⩾ used) {
    flow.set_cap(id, new_cap);
}
```

若网络此前已经是最大流，并且只是降低容量：

```
flow.set_cap(id, new_cap);
```

只要修改成功，当前最大流值不会变化，也不需要重新求最大流
原因是当前流仍然可行，而容量减小不可能产生比原最大流更大的答案

容量降低到当前流量以下

例如：

```
当前容量 = 10
当前流量 = 8
新容量 = 5
```

则：

```
flow.set_cap(id, 5);
```

会返回 `false`
因为原来的 `8` 单位流已经无法通过容量为 `5` 的边
若不需要维护动态流，最稳妥的做法是重新开始：

```
flow.reset_flow();
bool ok = flow.set_cap(id, 5);
assert(ok);
i64 ans = flow.max_flow(s, t);
```

若有很多容量需要同时修改：

```
flow.reset_flow();
for (auto [id, cap] : changes) {
    bool ok = flow.set_cap(id, cap);
    assert(ok);
}
i64 ans = flow.max_flow(s, t);
```

重置后所有边流量均为 `0`，因此任意非负容量都可以安全设置

多轮参数修改

如果参数变化只有增加：

```
i64 ans = flow.max_flow(s, t);
for (int round = 0; round < q; ++ round) {
    for (auto [id, delta] : increase[round]) {
        flow.increase_cap(id, delta);
    }
    ans += flow.max_flow(s, t);
    // ans 为这一轮修改后的新最大流
}
```

这是利用残量网络最重要的动态用法之一

每轮从零重新求

若每轮容量可能任意增加或降低，并且降低可能越过当前流量：

```
for (int round = 0; round < q; ++ round) {
    flow.reset_flow();
    for (auto [id, cap] : new_cap[round]) {
        flow.set_cap(id, cap);
    }
    i64 ans = flow.max_flow(s, t);
}
```

暂时关闭一条普通边

若希望真正按普通网络语义把一条边容量改成 `0`：

```
flow.reset_flow();
flow.set_cap(id, 0);
i64 ans = flow.max_flow(s, t);
```

不要直接：

```
flow.disable_edge(id);
```

`disable_edge()` 是给上下界流等特殊残量图建模使用的

容量缩放

```
Dinic<i64> flow(n, m);
for (auto [u, v, cap] : edges) {
    flow.add_edge(u, v, cap);
}
i64 ans = flow.max_flow_scaling(s, t);
```

普通题优先使用 `max_flow()`；容量跨度非常大且普通 Dinic 表现不好时再尝试

二分图容量匹配 / b-Matching

左部点 `u` 最多匹配 `capL[u]` 次，右部点 `v` 最多匹配 `capR[v]` 次，每条允许匹配边 `(u, v)` 至多使用一次
建立网络：

- `S → L_u`：容量 `capL[u]`
- `L_u → R_v`：容量 `1`
- `R_v → T`：容量 `capR[v]`
  最大流即最大匹配数

```
int S = nl + nr;
int T = S + 1;
Dinic<i64> flow(T + 1, nl + int(es.size()) + nr);
for (int u = 0; u < nl; ++ u) {
    flow.add_edge(S, u, capL[u]);
}
vector<tuple<int, int, int>> edges;
for (auto [u, v] : es) {
    int id = flow.add_edge(u, nl + v, 1);
    edges.emplace_back(u, v, id);
}
for (int v = 0; v < nr; ++ v) {
    flow.add_edge(nl + v, T, capR[v]);
}
i64 ans = flow.max_flow(S, T);
```

这里不需要拆左右部点，因为它们的容量已经分别由：

```
S → 左部点
右部点 → T
```

限制

恢复容量匹配方案

保存所有 `L → R` 边的编号，最大流结束后检查其流量：

```
vector<vector<int>> ml(nl), mr(nr);
for (auto [u, v, id] : edges) {
    if (flow.edge_flow(id) == 1) {
        ml[u].push_back(v);
        mr[v].push_back(u);
    }
}
```

其中：

- `ml[u]`：左部点 `u` 最终匹配的所有右部点
- `mr[v]`：右部点 `v` 最终匹配的所有左部点
  若左右两侧容量均为 `1`，就退化为普通二分图最大匹配

> Tip
>
> 若匹配边 `(u, v)` 还带费用或收益，容量模型不变，
> 将 `L_u → R_v` 改成容量 `1`、对应费用的边，并使用最小费用流

一般图的点容量限制

若一般流网络中点 `u` 最多允许 `cap[u]` 单位流经过，将一个点拆成：

```
u_in → u_out
```

并令这条边容量为 `cap[u]`
原来的边：

```
u → v
```

改为：

```
u_out → v_in
```

例如原边容量均为 `1`：

```
auto in = [&](int u) { return 2 * u; };
auto out = [&](int u) { return 2 * u + 1; };
Dinic<i64> flow(2 * n);
for (int u = 0; u < n; ++ u) {
    flow.add_edge(in(u), out(u), cap[u]);
}
for (auto [u, v] : es) {
    flow.add_edge(out(u), in(v), 1);
}
```

若源点、汇点同样受点容量限制：

```
i64 ans = flow.max_flow(in(s), out(t));
```

若源点或汇点不受点容量限制，可以将对应拆点边容量设为足够大的 `INF`，或根据建图直接从 `out(s)` / 流入 `in(t)`

只恢复有流量的原图边

若原问题的方案就是“哪些原图边最终被使用”，建图时保存这些边：

```
struct Info {
    int u, v, id;
};
vector<Info> edges;
for (auto [u, v, cap] : es) {
    int id = flow.add_edge(u, v, cap);
    edges.push_back({u, v, id});
}
flow.max_flow(s, t);
for (auto [u, v, id] : edges) {
    i64 f = flow.edge_flow(id);
```

```
    if (f > 0) {
        // 原图边 u → v 使用了 f 单位流
    }
}
```

不应直接根据残量 `e[2 * id].cap == 0` 判断边是否被使用：
容量大于 `1` 时，一条边即使还有剩余容量，也可能已经承载了流量。
统一使用 `edge_flow(id)` 更稳妥

复杂度

设点数为 n、逻辑边数为 m

操作	时间复杂度
`add_edge()`	均摊 $O(1)$
`increase_cap()` / `set_cap()`	$O(1)$
`reset_flow()`	$O(m)$
`get_edge()` / `edge_flow()`	$O(1)$
`min_cut()`	$O(n+m)$
`max_flow()`	一般图 $O(n^2m)$
单位网络 `max_flow()`	$O(m\sqrt{n})$

空间复杂度：$O(n+m)$
`max_flow_scaling()` 至多经过 $O(\log U)$ 个容量尺度，其中 U 为当前最大残量容量；可保守视为在 Dinic 上额外增加一个 $O(\log U)$ 的尺度因子，主要作为实践优化使用

6.2.1 HLPP（最高标号预流推进）

解决问题： 求有向图最大流，使用 Highest Label Push-Relabel，并加入 Gap 与 Global Relabel 启发式，适合作为 Dinic 在较难网络上的性能互补版本

```
template<class Cap = i64>
struct HLPP {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
    struct Edge {
        int to, rev;
        Cap cap;
    };
    int n;
    vector<vector<Edge>> g;
    explicit HLPP(int n_) : n(n_), g(n) {}
    void add_edge(int u, int v, Cap cap) {
        if (u == v) return;
        Edge a{ v, int(g[v].size()), cap }, b{ u, int(g[u].size()), 0 };
        g[u].push_back(a);
        g[v].push_back(b);
    }
    Cap max_flow(int s, int t) {
        if (s == t) return 0;
        vector<Cap> excess(n);
        vector<int> h(n, n), cur(n), cnt(2 * n + 2);
        vector<char> active(n);
        vector<vector<int>> bucket(2 * n + 2);
        int highest{};
        // 从汇点沿反向残量边 BFS，重建精确距离标号与活跃桶
        auto global_relabel = [&]() {
            ranges::fill(h, n);
            ranges::fill(cur, 0);
            ranges::fill(cnt, 0);
            ranges::fill(active, false);
            for (auto &v : bucket) v.clear();
            queue<int> q;
            h[t] = 0;
            q.push(t);
            while (!q.empty()) {
                int u{ q.front() };
                q.pop();
```

```
            for (const Edge &e : g[u]) {
                const Edge &r{ g[e.to][e.rev] };
                if (r.cap > 0 && h[e.to] == n) h[e.to] = h[u] + 1, q.push(e.to);
            }
        }
        h[s] = n;
        for (int u = 0; u < n; ++ u) ++ cnt[h[u]];
        highest = 0;
        for (int u = 0; u < n; ++ u) if (u != s && u != t && excess[u] > 0 && h[u] < n) {
            active[u] = true;
            bucket[h[u]].push_back(u);
            highest = max(highest, h[u]);
        }
    };
    auto activate = [&](int u) {
        if (u == s || u == t || excess[u] == 0 || h[u] >= n || active[u]) return;
        active[u] = true;
        bucket[h[u]].push_back(u);
        highest = max(highest, h[u]);
    };
    // 源点饱和推出预流，源点 excess 无需维护
    h[s] = n;
    for (Edge &e : g[s]) if (e.cap > 0) {
        Cap f{ e.cap };
        e.cap = 0;
        g[e.to][e.rev].cap += f;
        excess[e.to] += f;
    }
    global_relabel();
    int work{};
    while (highest >= 0) {
        while (highest >= 0 && bucket[highest].empty()) -- highest;
        if (highest < 0) break;
        int u{ bucket[highest].back() };
        bucket[highest].pop_back();
        active[u] = false;
        if (excess[u] == 0 || h[u] != highest || h[u] >= n) continue;
        while (excess[u] > 0) {
            if (cur[u] == int(g[u].size())) {
                int old{ h[u] }, nh{ 2 * n };
                for (const Edge &e : g[u]) if (e.cap > 0) nh = min(nh, h[e.to] + 1);
                -- cnt[old];
                h[u] = min(nh, 2 * n);
                ++ cnt[h[u]];
                cur[u] = 0;
                // Gap: 高度 old 断层后，old 以上且仍低于 n 的点均无法到达汇点
                if (old < n && cnt[old] == 0) {
                    for (int v = 0; v < n; ++ v) if (v != s && v != t && old < h[v] && h[v] < n) {
                        -- cnt[h[v]];
                        h[v] = n;
                        ++ cnt[n];
                        cur[v] = 0;
                    }
                }
                if (h[u] >= n) break;
                highest = max(highest, h[u]);
                continue;
            }
            Edge &e{ g[u][cur[u]] };
            if (e.cap > 0 && h[u] == h[e.to] + 1) {
                Cap f{ min(excess[u], e.cap) };
                e.cap -= f;
                g[e.to][e.rev].cap += f;
                excess[u] -= f;
                excess[e.to] += f;
                activate(e.to);
            } else ++ cur[u];
        }
        activate(u);
        if (++ work % max(1, 4 * n) == 0) global_relabel();
    }
```

```
        return excess[t];
    }
};
```

接口说明

接口

- `HLPP<Cap>(n)`：建立 `n` 点有向残量网络，$O(n)$
- `add_edge(u, v, cap)`：加入容量为 `cap` 的有向边，自环自动忽略，均摊 $O(1)$
- `max_flow(s, t)`：使用最高标号预流推进求最大流值

`add_edge()` 条件

- `0 ≤ u, v < n`
- `cap ≥ 0`
- `Cap` 必须为有符号整数类型

算法说明

HLPP 不维护从始至终合法的普通流，而是允许中间点暂时拥有正的 `excess`，不断从**当前高度最大的活跃点**向外推流

- `h[u]`：高度标号，只允许沿 `h[u] = h[v] + 1` 的残量边推流
- `excess[u]`：流入减流出的超额流量，`excess[u] > 0` 的普通点为活跃点
- `relabel`：当前点无法继续推流时，将高度提高到可推边所需的最小高度
- Highest Label：优先处理高度最大的活跃点，减少无效往返
- Gap：若某个高度层为空，则更高且仍可达汇点范围内的点实际上都无法到达汇点
- Global Relabel：从汇点沿反向残量边 BFS，重新计算到汇点的精确残量距离

注意事项

- 本版定位是 **Dinic 的性能互补版本**，普通题默认仍优先使用 Dinic
- `max_flow()` 会原地修改残量网络，默认同一对象只求解一次
- 本实现允许最终仍有超额流滞留在无法到达汇点的点上，因此残量状态是**最大预流**而不保证是普通可行流
- 因此不要使用本板恢复逐边实际流量，也不要直接从最终残量图恢复最小割
- 需要边流量、最小割、限流增广、修改容量、上下界流等接口时使用 Dinic
- 源点初始化时不维护 `excess[s]`，避免源点所有出边容量之和超过 `Cap` 时产生无意义的负溢出
- 仍需保证单条容量、各点 `excess` 和最终最大流不会超过 `Cap`
- 无向容量边需要显式加入两个方向，自动建立的反向边仅为残量边
- 自环不会影响 $s-t$ 最大流值，因此直接忽略，也避免自环时 `rev` 的特殊处理
- 平行边可以直接加入，不影响正确性
- Global Relabel 的触发周期 `4 * n` 是启发式参数，可以按实际数据调整

常见用法

```
HLPP<i64> flow(n);
for (auto [u, v, cap] : edges) flow.add_edge(u, v, cap);
i64 ans{ flow.max_flow(s, t) };
```

> Tip
>
> 稀疏普通网络、需要恢复方案时优先 Dinic；只求最大流值，且 Dinic 在稠密图或特殊构造上较慢时再尝试 HLPP

复杂度

通用最坏时间复杂度：$O(n^2m)$
空间复杂度：$O(n+m)$

6.2.2 最大流扩展：最小割、流分解与 Scaling

依赖： 6.2 Dinic 最大流，以下内容均作为 Dinic 的扩展使用，不重复建图与普通最大流代码

最小割恢复

在已经求出最大流后，从源点 `s` 在残量网络中继续沿正残量边搜索，可达点构成最小割的源点侧 S，其余点构成汇点侧 T，这也是常用 Dinic 实现恢复最小割的方法
将以下函数加入 `Dinic`：

```
// ===== 以下函数可直接删除：最小割恢复 =====
vector<char> min_cut_side(int s) const {
    vector<char> vis(n);
    queue<int> q;
    vis[s] = true;
    q.push(s);
    while (!q.empty()) {
        int u{ q.front() };
        q.pop();
        for (int id : g[u]) if (e[id].cap > 0 && !vis[e[id].to]) {
            vis[e[id].to] = true;
            q.push(e[id].to);
        }
    }
    return vis;
}
// 返回跨越 S → T 的原始有容量半边编号，即 e[] 中的下标
vector<int> min_cut_edges(int s) const {
    vector<char> side{ min_cut_side(s) };
    vector<int> ans;
    for (int id = 0; id < int(e.size()); ++ id) {
        const Edge &x{ e[id] };
        if (x.init > 0 && side[x.from] && !side[x.to]) ans.push_back(id);
    }
    return ans;
}
```

接口

- `min_cut_side(s)`：返回源点侧标记，`side[u] = true` 表示 `u` 属于最小割源点侧，$O(n+m)$
- `min_cut_edges(s)`：返回最小割中所有原始有容量半边在 `e[]` 中的编号，$O(n+m)$
- 对普通 `add_edge(u,v,cap)`，正向边编号为 `2 * id`；若使用了非零 `rcap`，反向半边也可能属于割

常见用法

```
flow.max_flow(s, t);
auto side = flow.min_cut_side(s);
for (int u = 0; u < n; ++ u) {
    if (side[u]) {
        // u 位于源点侧 S
    } else {
        // u 位于汇点侧 T
    }
}
```

恢复割边：

```
auto cut = flow.min_cut_edges(s);
for (int eid : cut) {
    auto [u, v] = pair{ flow.e[eid].from, flow.e[eid].to };
    // 原图割边 u → v
}
```

> ⚠ Warning
>
> 只有在 `max_flow(s,t)` 已经增广到最大流后，残量可达集才对应最小割
> 若使用 `limit` 提前停止，不能据此恢复最小割

流分解

一个合法 $s-t$ 流可以分解为若干条带流量的 $s-t$ 路径，再加若干个带流量的环
多数方案输出题只需要路径部分，环流不影响 $s-t$ 流值，因此默认不返回环；需要完整分解时令 `keep_cycles = true`

```
template<class Cap>
struct FlowPart {
    Cap flow{};
    vector<int> vertices; // 依次经过的点，环首尾相同
    vector<int> edges;    // 对应 Dinic::e[] 中的半边编号
```

```
    bool cycle{};
};
template<class Cap>
vector<FlowPart<Cap>> flow_decomposition(const Dinic<Cap> &mf, int s, int t, bool keep_cycles = false) {
    struct Arc {
        int to, eid;
        Cap flow;
    };
    int n{ mf.n };
    vector<vector<int>> g(n);
    vector<Arc> a;
    auto add = [&](int u, int v, int eid, Cap f) {
        g[u].push_back(a.size());
        a.push_back({ v, eid, f });
    };
    // 每对残量边恢复一个净流方向
    for (int id = 0; 2 * id < int(mf.e.size()); ++ id) {
        const auto &x{ mf.e[2 * id] };
        Cap f{ x.init - x.cap };
        if (f > 0) add(x.from, x.to, 2 * id, f);
        else if (f < 0) add(x.to, x.from, 2 * id + 1, -f);
    }
    vector<FlowPart<Cap>> ans;
    // 只提取净 s→t 流量；经过源汇的环流不能计入路径流值。
    Cap remaining{};
    if (s != t) for (int id = 0; 2 * id < int(mf.e.size()); ++ id) {
        const auto &x{ mf.e[2 * id] };
        Cap f{ x.init - x.cap };
        if (x.from == s) remaining += f;
        if (x.to == s) remaining -= f;
    }
    assert(remaining >= 0);
    while (remaining > 0) {
        vector<int> pre(n, -1), pe(n, -1);
        queue<int> q;
        pre[s] = s;
        q.push(s);
        while (!q.empty() && pre[t] == -1) {
            int u{ q.front() };
            q.pop();
            for (int id : g[u]) if (a[id].flow > 0 && pre[a[id].to] == -1) {
                pre[a[id].to] = u;
                pe[a[id].to] = id;
                q.push(a[id].to);
            }
        }
        assert(pre[t] != -1); // 合法非负净流必有足够的正流路径
        vector<int> path;
        for (int v = t; v != s; v = pre[v]) path.push_back(pe[v]);
        ranges::reverse(path);
        Cap f{ remaining };
        for (int id : path) f = min(f, a[id].flow);
        remaining -= f;
        FlowPart<Cap> part;
        part.flow = f;
        part.vertices.push_back(s);
        for (int id : path) {
            a[id].flow -= f;
            part.edges.push_back(a[id].eid);
            part.vertices.push_back(a[id].to);
        }
        ans.push_back(move(part));
    }
    if (!keep_cycles) return ans;
    // 剩余正流一定是 circulation，继续提取环
    while (true) {
        int start{ -1 };
        for (int u = 0; u < n && start == -1; ++ u) {
            for (int id : g[u]) if (a[id].flow > 0) {
                start = u;
                break;
```

```
            }
        }
        if (start == -1) break;
        vector<int> pos(n, -1), vertices, path;
        int u{ start };
        while (pos[u] == -1) {
            pos[u] = vertices.size();
            vertices.push_back(u);
            int id{ -1 };
            for (int x : g[u]) if (a[x].flow > 0) {
                id = x;
                break;
            }
            assert(id != -1);
            path.push_back(id);
            u = a[id].to;
        }
        int k{ pos[u] };
        Cap f{ numeric_limits<Cap>::max() };
        for (int i = k; i < path.size(); ++ i) f = min(f, a[path[i]].flow);
        FlowPart<Cap> part;
        part.flow = f;
        part.cycle = true;
        for (int i = k; i < vertices.size(); ++ i) part.vertices.push_back(vertices[i]);
        part.vertices.push_back(u);
        for (int i = k; i < path.size(); ++ i) {
            a[path[i]].flow -= f;
            part.edges.push_back(a[path[i]].eid);
        }
        ans.push_back(move(part));
    }
    return ans;
}
```

接口

- `flow_decomposition(flow,s,t)`：分解为若干条 $s \to t$ 路径，忽略剩余环流
- `flow_decomposition(flow,s,t,true)`：完整分解为路径与环
- `part.flow`：该路径或环携带的流量
- `part.vertices`：点序列
- `part.edges`：对应 `Dinic::e[]` 的半边编号，可处理平行边
- `part.cycle`：是否为环流分量
- `eid / 2`：对应 `add_edge()` 返回的逻辑边编号
- `eid & 1`：是否使用该逻辑边的反向半边

常见用法

```
flow.max_flow(s, t);
auto parts = flow_decomposition(flow, s, t);
for (auto &[f, vertices, edges, cycle] : parts) {
    // 一条流量为 f 的 s → t 路径
}
```

完整分解：

```
auto parts = flow_decomposition(flow, s, t, true);
for (auto &part : parts) {
    if (part.cycle) {
        // 环流
    } else {
        // s → t 路径流
    }
}
```

注意事项

- 应在一个合法流状态上调用，通常就是 `max_flow()` 之后
- 要求净 s-t 流值非负；s == t 时按循环流处理，仅 keep_cycles=true 时恢复环
- 函数复制当前流量，不修改原 Dinic

- 使用 `init - cap` 恢复每对残量边上的净流，因此也兼容 `rcap != 0`
- 默认去掉环流后得到的路径流总和仍等于原 $s-t$ 流值
- 若只需要每条原边的流量，无需做流分解，直接读取 `e[2 * id].init - e[2 * id].cap`
  最坏情况下至多提取 $O(m)$ 个分量，当前实现时间复杂度 $O(m(n+m))$，额外空间 $O(n+m)$

Dinic Capacity Scaling

现在 Dinic 的 `bfs(s,t,scale)` 已经预留了残量容量阈值，因此不需要再维护一套 Scaling 模板，只新增下面的接口即可
Scaling 从不超过最大残量容量的最高二进制位开始，只让 BFS 使用残量容量至少为 `scale` 的边，处理完后不断减半；KACTL 的 Dinic 也采用这种容量 Scaling，并给出 $O(nm\log U)$ 的复杂度，其中 U 为最大容量

```
// ===== 以下函数可直接删除：Capacity Scaling =====
Cap max_flow_scaling(int s, int t, Cap limit = numeric_limits<Cap>::max()) {
    Cap mx{};
    for (const Edge &x : e) mx = max(mx, x.cap);
    if (s == t || mx == 0 || limit == 0) return 0;
    Cap scale{ 1 };
    while (scale <= mx / 2) scale <<= 1;
    Cap ans{};
    for (; scale && ans < limit; scale >>= 1) {
        while (ans < limit && bfs(s, t, scale)) {
            ranges::fill(it, 0);
            while (ans < limit) {
                Cap f{ dfs(s, t, limit - ans, scale) };
                if (!f) break;
                ans += f;
            }
        }
    }
    return ans;
}
```

这里要求原 Dinic 的 `bfs()` 使用：

```
if (e[id].cap >= scale && level[e[id].to] == -1) {
    // ...
}
```

普通 `max_flow()` 调用 `bfs(s,t)`，默认 `scale = 1`，因此不会增加普通版本的额外代码路径

接口

- `max_flow_scaling(s,t)`：使用 Capacity Scaling 增广到最大流
- `max_flow_scaling(s,t,limit)`：最多再增广 `limit`
- 返回值与普通 `max_flow()` 一样，是本次调用新增的流量
- 可以在已有合法残量状态上继续调用

注意事项

- 只适用于整数容量
- U 为当前残量网络中的最大容量
- 容量范围很小时 Scaling 未必比普通 Dinic 快，甚至可能因为多轮阈值扫描增加常数
- 容量跨度很大、普通 Dinic 表现较差时值得尝试
- 若已经准备 HLPP，Scaling 属于可选性能工具，不建议默认替换普通 Dinic
- Scaling 改变的只是寻找增广路的顺序，最终残量网络仍是普通合法最大流，因此最小割、边流量、流分解等接口都可以正常使用

复杂度

功能	时间复杂度	额外空间
`min_cut_side()`	$O(n+m)$	$O(n)$
`min_cut_edges()`	$O(n+m)$	$O(n)$
`flow_decomposition()`	$O(m(n+m))$	$O(n+m)$
`max_flow_scaling()`	$O(nm\log U)$	沿用 Dinic

> Tip

这三个扩展的定位可以很明确：最小割负责恢复割方案，流分解负责恢复路径方案，Scaling 只负责换一种增广顺序提速

6.3 最小费用流

```
template<class Cap = i64, class Cost = i64>
struct MinCostFlow {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
    static_assert(numeric_limits<Cost>::is_integer && numeric_limits<Cost>::is_signed);
    struct Edge {
        int from, to;
        Cap cap, init;
        Cost cost;
    };
    int n, sc{ 8 };
    bool upd{};
    vector<Edge> e;
    vector<vector<int>> g;
    explicit MinCostFlow(int n_) : n(n_), g(n) {}
    // ===== 最大流 + 整数费用缩放 =====
    int add_edge(int u, int v, Cap cap, Cost cost) {
        int id{ int(e.size()) / 2 };
        g[u].push_back(e.size());
        e.push_back({ u, v, cap, cap, cost });
        g[v].push_back(e.size());
        e.push_back({ v, u, 0, 0, -cost });
        return id;
    }
    pair<Cap, Cost> flow(int s, int t, Cap limit = numeric_limits<Cap>::max()) {
        if (s == t || limit == 0) return { 0, 0 };
        using T = conditional_t<(sizeof(Cost) < sizeof(i128)), i128, Cost>;
        vector<int> dep(n), cur(n);
        Cap f{};
        T c{};
        for (int i = 0; i < int(e.size()); i += 2) c -= T(e[i].init - e[i].cap) * e[i].cost;
        auto dfs = [&](auto &&self, int u, Cap lim) -> Cap {
            if (u == t) return lim;
            for (int &i = cur[u]; i < int(g[u].size()); ++ i) {
                int id{ g[u][i] };
                Edge &x{ e[id] };
                if (x.cap == 0 || dep[x.to] != dep[u] + 1) continue;
                Cap z{ self(self, x.to, min(lim, x.cap)) };
                if (z == 0) continue;
                x.cap -= z;
                e[id ^ 1].cap += z;
                return z;
            }
            return 0;
        };
        while (f < limit) {
            ranges::fill(dep, -1);
            queue<int> q;
            dep[s] = 0;
            q.push(s);
            while (!q.empty()) {
                int u{ q.front() };
                q.pop();
                for (int id : g[u]) if (e[id].cap > 0 && dep[e[id].to] == -1) {
                    dep[e[id].to] = dep[u] + 1;
                    q.push(e[id].to);
                }
            }
            if (dep[t] == -1) break;
            ranges::fill(cur, 0);
            while (f < limit) {
                Cap z{ dfs(dfs, s, limit - f) };
                if (z == 0) break;
                f += z;
```

```
            }
        }
        cancel_negative_cycles();
        for (int i = 0; i < int(e.size()); i += 2) c += T(e[i].init - e[i].cap) * e[i].cost;
        assert(c >= numeric_limits<Cost>::min() && c <= numeric_limits<Cost>::max());
        return { f, Cost(c) };
    }
    // ===== 方案恢复：若不需要逐边流量，以下函数可直接删除 =====
    tuple<int, int, Cap, Cap, Cost> get_edge(int id) const {
        const Edge &x{ e[2 * id] };
        return { x.from, x.to, x.init, x.init - x.cap, x.cost };
    }
    // ===== 流量重设与费用优化 =====
    void change_edge(int id, Cap cap, Cap used) {
        Edge &x{ e[2 * id] }, &r{ e[2 * id + 1] };
        x.init = cap;
        x.cap = cap - used;
        r.init = 0;
        r.cap = used;
    }
    // 保持各点净流量，用整数费用缩放消除残量负环
    Cost cancel_negative_cycles() {
        using T = conditional_t<(sizeof(Cost) < sizeof(i128)), i128, Cost>;
        using F = conditional_t<(sizeof(Cap) < sizeof(i128)), i128, Cap>;
        assert(sc >= 2);
        vector<T> p(n);
        vector<F> b(n);
        vector<int> cur(n);
        T eps{}, ans{};
        for (int i = 0; i < int(e.size()); i += 2) {
            ans -= T(e[i].init - e[i].cap) * e[i].cost;
            T c{ T(e[i].cost) * (n + 1) };
            eps = max(eps, c < 0 ? -c : c);
        }
        auto rc = [&](int id) { return T(e[id].cost) * (n + 1) + p[e[id].from] - p[e[id].to]; };
        auto push = [&](int id, Cap z) {
            e[id].cap -= z;
            e[id ^ 1].cap += z;
            if (e[id].from != e[id].to) b[e[id].from] -= z, b[e[id].to] += z;
        };
        while (eps > 0) {
            eps = max(T(1), eps / sc);
            for (int id = 0; id < int(e.size()); ++ id) if (e[id].cap > 0 && rc(id) < 0) push(id, e[id].cap);
            // 可选价格更新：反向多源最短路，每轮执行一次
            if (upd) {
                constexpr T INF{ numeric_limits<T>::max() };
                vector<T> d(n, INF);
                priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> q;
                for (int u = 0; u < n; ++ u) if (b[u] < 0) d[u] = 0, q.push({ 0, u });
                T mx{};
                while (!q.empty()) {
                    auto [du, u] = q.top();
                    q.pop();
                    if (du != d[u]) continue;
                    mx = du;
                    for (int id : g[u]) if (e[id ^ 1].cap > 0) {
                        int v{ e[id].to };
                        T nd{ du + rc(id ^ 1) + eps };
                        if (nd < d[v]) d[v] = nd, q.push({ nd, v });
                    }
                }
                for (int u = 0; u < n; ++ u) p[u] -= min(d[u], mx);
            }
            ranges::fill(cur, 0);
            queue<int> q;
            for (int u = 0; u < n; ++ u) if (b[u] > 0) q.push(u);
            while (!q.empty()) {
                int u{ q.front() };
                q.pop();
                while (b[u] > 0) {
                    if (cur[u] == int(g[u].size())) {
```

```
                    T d{ numeric_limits<T>::max() };
                    for (int id : g[u]) if (e[id].cap > 0 && e[id].to != u) d = min(d, rc(id));
                    assert(d != numeric_limits<T>::max());
                    p[u] -= d + eps;
                    cur[u] = 0;
                }
                int id{ g[u][cur[u]] }, v{ e[id].to };
                if (e[id].cap == 0 || rc(id) >= 0) { ++ cur[u]; continue; }
                Cap z{ Cap(min(b[u], F(e[id].cap))) };
                F old{ b[v] };
                push(id, z);
                if (v != u && old <= 0 && b[v] > 0) q.push(v);
            }
        }
        if (eps == 1) break;
    }
    for (int i = 0; i < int(e.size()); i += 2) ans += T(e[i].init - e[i].cap) * e[i].cost;
    assert(ans >= numeric_limits<Cost>::min() && ans <= numeric_limits<Cost>::max());
    return Cost(ans);
  }
};
```

成员变量

- `e[2 * id] / e[2 * id + 1]`：第 `id` 条逻辑边的正向 / 反向残量边
- `Edge::cap`：当前残量容量
- `Edge::init`：原边容量，反向残量边为 `0`
- `Edge::cost`：单位流量费用

接口

- `MinCostFlow(n)`：建立 `n` 点费用残量网络，$O(n)$
- `add_edge(u,v,cap,cost)`：加入容量 `cap`、单位费用 `cost` 的有向边，返回逻辑边编号，均摊 $O(1)$
- `flow(s,t)`：在当前残量状态上继续增广至最大流，返回 `{本次新增流量，本次新增最小费用}`
- `flow(s,t,limit)`：最多继续增广 `limit` 单位流
- `get_edge(id)`：返回 `{from,to,capacity,flow,cost}`，$O(1)$
- `change_edge(id,cap,used)`：将逻辑边重设为总容量 `cap`、当前流量 `used`，$O(1)$
- `cancel_negative_cycles()`：保持每个点净流量不变并消除残量负环，返回费用改变量，返回值非正

注意事项

- 点编号为 `0base`，要求 `0 <= s,t,u,v < n`
- `add_edge()` 要求 `cap >= 0`，且 `cost` 与 `-cost` 都能由 `Cost` 表示
- `flow()` 要求 `limit >= 0`
- `flow(s,t,limit)` 表示至多发送，要求恰好 `limit` 时必须检查返回流量
- 允许负费用边和任意位置的负费用环；先用 Dinic 确定新增流量，再用整数费用缩放优化整个残量网络
- `flow()` 返回的费用是本次调用前后总费用之差，包含独立负环的优化；不必预先调用 `cancel_negative_cycles()`
- `s == t` 或 `limit == 0` 时原样返回 `{0,0}`，不优化费用；只优化当前流请调用 `cancel_negative_cycles()`
- `flow()` 会修改残量网络，再次调用表示在当前流基础上继续增广，返回值仍只统计本次增量
- 最大费用流可将所有费用取负，最后将答案费用取反
- `change_edge()` 要求 `0 <= used <= cap`，且调用者必须自行保证修改后整张网络仍满足所需流量守恒
- `cancel_negative_cycles()` 要求当前流满足需要保持的点平衡 / 净流量条件，消圈不会改变各点净流量
- `Cap` 容纳容量和返回流量；内部点盈余至少用 `i128`，必须容纳所有入边容量之和
- 内部费用类型 `T` 至少为 `i128`，必须容纳 `(n+1)*cost`、势能、约化费用、逐边费用乘积及累加中间值；不是任意 `i128` 输入都安全
- `Cost` 必须容纳返回费用差及费用优化阶段的费用差；总费用较大或优化前后差值较大时使用 `i128`
- Dinic 的递归深度可能达到 `n`，注意运行环境栈容量

整数缩放的正确性

每轮先饱和负约化费用边，再推送正盈余并调整势能，最终恢复每个点调用前的净流量。所有残量边的约化费用至少为 `-eps`。
费用先乘 `n+1`；当 `eps == 1` 时，长度至多 `n` 的简单环费用不小于 `-n`，但它必须是 `n+1` 的整数倍，因此不可能为负。没有残量负环即为当前净流量下的最小费用。这是整数精确算法，不使用浮点近似。
费用缩放与 epsilon 最优性的依据见 Goldberg、Tardos、Tarjan：Network Flow Algorithms。

常见用法

恰好发送指定流量

```
MinCostFlow<i64, i64> mcf(n);
for (auto [u, v, cap, cost] : edges) mcf.add_edge(u, v, cap, cost);
```

```
auto [flow, cost] = mcf.flow(s, t, need);
if (flow != need) cout << "Impossible\n";
else cout << cost << '\n';
```

恢复边流量

```
int id{ mcf.add_edge(u, v, cap, cost) };
mcf.flow(s, t);
auto [from, to, capacity, used, unit_cost] = mcf.get_edge(id);
```

存在负费用环

```
Cost cost{};
cost += mcf.cancel_negative_cycles();
auto [f, c] = mcf.flow(s, t, need);
cost += c;
```

上面保留了先优化初始环流的写法，也可以直接调用 `flow()`。正常求流后再次优化返回 `0`：

```
cost += mcf.cancel_negative_cycles();
```

复杂度

设网络有 n 个点、m 条逻辑边，C 为最大单位费用绝对值。

操作	时间复杂度
Dinic 求流量	$O(n^2m)$
`cancel_negative_cycles()`	$O(n^2m\log(2+nC))$，整数费用缩放
`flow()`	Dinic 与费用缩放之和
`get_edge()` / `change_edge()`	$O(1)$

空间复杂度：$O(n+m)$。这份实现优先保持代码紧凑，价格更新默认关闭；小流量简单网络上未必比逐次最短路快。

常用方法：可选启发式

`upd` 默认 `false`，设为 `true` 后，每轮饱和负边后执行一次价格更新：从亏流点反向跑多源 Dijkstra，批量调整势能，减少逐点重标号。单轮额外开销 $O((n+m)\log(n+m))$、空间 $O(n+m)$；短路径、小图不一定更快，建议用目标数据比较后决定。
`sc` 默认 `8`，要求至少 `2`，常用 `4/8/16`。较大值减少缩放轮数，但可能增加单轮推送和重标号，不保证更快。两项只影响运行过程，不改变整数最优性或接口语义。

```
MinCostFlow<i64, i128> mcf(3);
mcf.upd = true; // 开启每轮价格更新
mcf.sc = 4;     // 可省略，默认 8
mcf.add_edge(0, 1, 2, -3);
mcf.add_edge(1, 2, 2, 1);
auto [f, c] = mcf.flow(0, 2); // f=2, c=-4
assert(f == 2 && c == -4);
```

只优化循环流时同样先设置 `upd/sc`，再调用 `cancel_negative_cycles()`；上下界包装见 6.4.1。

价格更新使用边长 `rc+eps`。调用位置在饱和负边后，残量 `rc≥0`，可用 Dijkstra；距离满足 `d[u]≤rc(u,v)+eps+d[v]`，令 `p[u]-=d[u]` 后仍有 `rc≥-eps`。不可达点统一减去最大有限距离，保持跨越可达集合边的同一不等式；没有亏流点时不调整。路径距离、中间加法须可表示且有限距离严格小于内部 `T::max()`。不要把这段直接搬到可能有 `rc<-eps` 的位置。
启发式背景可参见 OR-Tools 的费用流价格更新开关与说明；这里使用紧凑的每轮一次版本。

6.3.1 最小费用循环流

依赖： `6.3 最小费用流` 中的 `MinCostFlow`、`get_edge()`、`change_edge()` 与 `cancel_negative_cycles()`

```
// ===== 依赖 MinCostFlow: 最小费用循环流 =====
template<class Cap, class Cost>
Cost circulation_cost(const MinCostFlow<Cap, Cost> &mcf) {
    Cost ans{};
    for (int id = 0; 2 * id < int(mcf.e.size()); ++ id) {
        const auto &x{ mcf.e[2 * id] };
        ans += Cost(x.init - x.cap) * x.cost;
```

```
    }
    return ans;
}
// 当前状态必须是可行循环流，返回优化后的总费用
template<class Cap, class Cost>
Cost min_cost_circulation(MinCostFlow<Cap, Cost> &mcf) {
    Cost ans{ circulation_cost(mcf) };
    return ans + mcf.cancel_negative_cycles();
}
```

原理

循环流要求每个点满足流量守恒，即

$$\sum_{(v,u)} f_{vu} = \sum_{(u,v)} f_{uv}$$

且每条边满足

$$0 \le f_e \le c_e$$

若当前已经是一组可行循环流，则沿残量网络中的任意环增广都不会改变任何点的净流量
若某个残量环总费用为负，沿该环尽可能增广可以继续降低总费用，因此：

> Tip
>
> **可行循环流为最小费用循环流，当且仅当其残量网络不存在负费用环**

`cancel_negative_cycles()` 使用整数费用缩放恢复原点平衡并消除负环，作为循环流优化核心。实现与数值范围见 6.3。

接口

- `circulation_cost(mcf)`：计算当前流的总费用，$O(m)$
- `min_cost_circulation(mcf)`：将**当前可行循环流**优化为最小费用循环流并返回最终总费用
- 求解后通过 `mcf.get_edge(id)` 恢复每条原边流量

调用条件

- 调用 `min_cost_circulation()` 前，当前流必须满足所有点流量守恒
- 边流量必须满足 `0 ≤ flow ≤ capacity`
- 从刚建好的 `MinCostFlow` 直接调用时，初始零流天然是一组可行循环流
- 若使用 `change_edge()` 手动设置初始流，调用者必须保证设置后仍为可行循环流

注意事项

- 普通只有上界 $0 \le f \le cap$ 的循环流中，**零流永远可行**
- 因此若所有边费用均非负，最优解就是零流，总费用为 `0`
- 非平凡的普通最小费用循环流主要来自负费用环
- 若存在下界 $low \le f \le high$，零流不一定可行，应使用 6.4.1 上下界最小费用流
- 若存在点需求 / 供给约束，也应先通过超级源汇等方式构造一组可行流，再进行负环消除
- `cancel_negative_cycles()` 保持每个点的净流量不变，因此也可用于把某个固定净流量状态继续优化到最小费用
- `min_cost_circulation()` 会修改残量网络，再次调用时若已经不存在负环则费用不再变化
- 最大费用循环流可将所有费用取负，最后将答案取反
- `Cost` 必须能容纳 `Cap * Cost` 与总费用，范围较大时使用 `i128`
- 与普通 `flow()` 共用费用缩放核心；本接口保持各点净流量，不增加源汇流量

常见用法

从零流求最小费用循环流

```
MinCostFlow<i64, i64> mcf(n);
for (auto [u, v, cap, cost] : edges) mcf.add_edge(u, v, cap, cost);
i64 ans{ min_cost_circulation(mcf) };
```

此时初始费用为 `0`，所以也可以直接写

```
i64 ans{ mcf.cancel_negative_cycles() };
```

但统一使用 `min_cost_circulation()` 更适合之后复用到已有可行循环流

恢复方案

```
i64 ans{ min_cost_circulation(mcf) };
for (int id = 0; 2 * id < int(mcf.e.size()); ++ id) {
    auto [u, v, cap, flow, cost] = mcf.get_edge(id);
    // 原边 u → v 使用 flow 单位流量
}
```

从已有可行循环流继续优化

```
// 已通过 change_edge() 等方式构造合法循环流
i64 before{ circulation_cost(mcf) };
i64 after{ min_cost_circulation(mcf) };
```

`after ⩽ before`

典型负环

```
MinCostFlow<i64, i64> mcf(3);
mcf.add_edge(0, 1, 3, -5);
mcf.add_edge(1, 2, 3, 1);
mcf.add_edge(2, 0, 3, 1);
i64 ans{ min_cost_circulation(mcf) }; // -9
```

每绕环一单位费用为 $-5+1+1=-3$，容量限制允许绕 3 单位，因此答案为 -9

与普通最小费用流的关系

`flow(s,t,F)` 固定的是点的净流量：

$$b_s=-F,\quad b_t=F$$

循环流则所有点均有

$$b_v=0$$

两者的最优性条件本质相同：**残量网络不存在负费用环**
普通 `flow()` 已优化整个残量网络，包括与 `s` 无关的负环。以下重复优化保留接口用法，正常返回的改变量为 `0`：

```
auto [f, cost] = mcf.flow(s, t, need);
if (f == need) cost += mcf.cancel_negative_cycles();
```

此时得到的是该固定 $s\to t$ 流量下的全局最小费用，而不是循环流问题本身

复杂度

设有 n 个点、m 条逻辑边，C 为最大单位费用绝对值。

操作	时间复杂度
`circulation_cost()`	$O(m)$
`min_cost_circulation()`	$O(n^2m\log(2+nC))$

空间复杂度：$O(n+m)$。内部费用与盈余范围要求同 6.3，包装函数的费用累加还须能由 `Cost` 表示。

6.4 上下界网络流

可行性阶段从超级源实际只会发送 `need` 单位流，因此任何辅助边都不可能需要超过 `need` 的流量，这样既避免 `INF` 选择问题，也减少溢出风险
上下界转化本身就是先强制发送下界，再把各点的“流入减流出”差额交给超级源汇修正
经典模板也分别通过可行流后继续增广求上下界最大 / 最小流

```
// 依赖 6.2 Dinic<Cap>
template<class Cap = i64>
struct LowerBoundFlow {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
```

```
    struct Edge {
        int from, to;
        Cap low, high;
    };
    int n;
    vector<Edge> edges;
    vector<Cap> answer;
    explicit LowerBoundFlow(int n_) : n(n_) {}
    int add_edge(int u, int v, Cap low, Cap high) {
        edges.push_back({ u, v, low, high });
        return int(edges.size()) - 1;
    }
    bool feasible() { return solve(-1, -1, 0, nullopt).has_value(); }
    optional<Cap> feasible(int s, int t) { return solve(s, t, 0, nullopt); }
    optional<Cap> max_flow(int s, int t) { return solve(s, t, 1, nullopt); }
    optional<Cap> min_flow(int s, int t) { return solve(s, t, -1, nullopt); }
    optional<Cap> flow_with_value(int s, int t, Cap value) { return solve(s, t, 0, value); }
    Cap edge_flow(int id) const { return answer[id]; }
private:
    optional<Cap> solve(int s, int t, int type, optional<Cap> fixed) {
        bool has_st{ s != -1 };
        int ss{ n }, tt{ n + 1 };
        Dinic<Cap> flow(n + 2);
        vector<Cap> balance(n);
        vector<int> id(edges.size());
        // 先强制发送下界，只将 [0, high-low] 留给残量网络
        for (int i = 0; i < edges.size(); ++ i) {
            auto [u, v, low, high] = edges[i];
            id[i] = flow.add_edge(u, v, high - low);
            balance[u] -= low;
            balance[v] += low;
        }
        // 固定净 s→t 流量等价于额外固定 value 单位 t→s 流
        if (fixed) {
            balance[t] -= *fixed;
            balance[s] += *fixed;
        }
        Cap need{};
        for (Cap x : balance) if (x > 0) need += x;
        int back_ts{ -1 }, back_st{ -1 };
        if (has_st && !fixed) {
            // 两个方向同时存在，使基准可行流的净值可以为任意符号
            back_ts = flow.add_edge(t, s, need);
            back_st = flow.add_edge(s, t, need);
        }
        vector<int> auxiliary;
        for (int u = 0; u < n; ++ u) {
            if (balance[u] > 0) auxiliary.push_back(flow.add_edge(ss, u, balance[u]));
            else if (balance[u] < 0) auxiliary.push_back(flow.add_edge(u, tt, -balance[u]));
        }
        if (flow.max_flow(ss, tt) != need) {
            answer.clear();
            return nullopt;
        }
        for (int eid : auxiliary) flow.disable_edge(eid);
        Cap value{ fixed ? *fixed : 0 };
        if (has_st && !fixed) {
            value = flow.edge_flow(back_ts) - flow.edge_flow(back_st);
            flow.disable_edge(back_ts);
            flow.disable_edge(back_st);

            if (type > 0) value += flow.max_flow(s, t);
            if (type < 0) value -= flow.max_flow(t, s);
        }
        answer.resize(edges.size());
        for (int i = 0; i < edges.size(); ++ i) {
            answer[i] = edges[i].low + flow.edge_flow(id[i]);
        }
        return value;
```

```
    }
};
```

原理

对原边

$$low_e \le f_e \le high_e$$

先固定发送 `low`，再令

$$f_e = low_e + x_e, \qquad 0 \le x_e \le high_e - low_e$$

代码中的

```
balance[u] -= low;
balance[v] += low;
```

维护的是下界流造成的

$$balance_v = \text{流入} - \text{流出}$$

于是：

- `balance[u] > 0`：当前流入过多，需要残量网络额外流出这么多，因此加入 `SS → u`
- `balance[u] < 0`：当前流出过多，需要残量网络额外流入这么多，因此加入 `u → TT`
- 所有 `SS` 出边均满流，当且仅当存在满足上下界与流量守恒的方案

接口说明

成员变量

- `edges[id]`：用户加入的第 `id` 条上下界边
- `answer[id]`：最近一次成功求解后，第 `id` 条原边实际流量

接口

- `add_edge(u,v,low,high)`：加入流量范围 `[low,high]` 的有向边，返回原边编号，均摊 $O(1)$
- `feasible()`：判断并构造可行循环流
- `feasible(s,t)`：构造任意可行 `s-t` 流并返回其净流量，无解返回 `nullopt`
- `max_flow(s,t)`：返回最大可行净流量，无解返回 `nullopt`
- `min_flow(s,t)`：返回最小可行净流量，无解返回 `nullopt`
- `flow_with_value(s,t,value)`：构造净流量恰为 `value` 的可行流，`value` 可以为负
- `edge_flow(id)`：返回最近一次成功求解后原边流量，$O(1)$
  这里统一定义

  $$\text{value} = \text{s 的流出} - \text{s 的流入}$$

  因此一般有向图中的 `s-t` 流值允许为负；若题目保证 `s` 无入边、`t` 无出边，则自然非负

固定流量为什么这样处理

要求净 `s → t` 流量恰为 `value`，等价于额外加入一条固定流量为 `value` 的

$$t \to s$$

边，因此只需修改平衡量：

```
balance[t] -= value;
balance[s] += value;
```

不需要真的插入固定边，所以 `value < 0` 也可以直接处理

最大流与最小流

获得任意可行 `s-t` 流后先删除所有超级源汇边以及 `s ⟷ t` 辅助边
若当前净流量为 `value`：

```
value += flow.max_flow(s, t); // 最大流
value -= flow.max_flow(t, s); // 最小流
```

原因是此时原边残量网络准确描述了所有保持上下界合法的调整方向

注意事项

- 依赖 6.2 Dinic 最大流 的 `add_edge()`、`max_flow()`、`edge_flow()`、`disable_edge()`
- 点编号为 `0base`
- `Cap` 必须为有符号整数类型
- `add_edge()` 要求 `0 ≤ low ≤ high`
- 有源汇接口要求 `0 ≤ s,t < n` 且 `s ≠ t`
- 每次求解都会从 `edges` 重新建立残量网络，因此同一对象可连续调用不同接口
- 每次成功求解都会覆盖 `answer`；求解失败会清空 `answer`
- `flow_with_value()` 中的 `value` 可以为负
- 可行阶段完成后必须删除超级源汇辅助边，否则之后的最大 / 最小增广会错误利用这些边
- 非固定有源汇问题还必须删除两条 `s ⟷ t` 辅助边
- 两条 `s ⟷ t` 辅助边容量取 `need` 即可，不需要额外设置 `INF`
- `Cap` 仍需容纳所有 `balance` 的和、流量与最终答案
- 平行边、自环均可正常处理，只要底层 Dinic 支持
- `edge_flow()` 只能在最近一次求解成功后读取

常见用法

最大上下界流并恢复方案

```
LowerBoundFlow<i64> flow(n);
vector<int> id(m);
for (int i = 0; i < m; ++ i) id[i] = flow.add_edge(u[i], v[i], low[i], high[i]);
auto ans = flow.max_flow(s, t);
if (!ans) {
    cout << "No Solution\n";
} else {
    cout << *ans << '\n';
    for (int eid : id) cout << flow.edge_flow(eid) << '\n';
}
```

无源汇可行环流

```
LowerBoundFlow<i64> flow(n);
for (auto [u, v, low, high] : edges) flow.add_edge(u, v, low, high);
if (!flow.feasible()) {
    // 无可行环流
}
```

任意可行有源汇流

```
auto value = flow.feasible(s, t);
if (value) {
    // *value 为该方案中 s 的净流出量
}
```

固定净流量

```
auto ans = flow.flow_with_value(s, t, need);
if (!ans) {
    // 不存在净流量恰为 need 的方案
}
```

负流量同样可以：

```
auto ans = flow.flow_with_value(s, t, -need);
```

复杂度

设原图有 n 个点、m 条边，辅助图有 $n+2$ 个点、$m+O(n)$ 条边

接口	最大流调用次数
`feasible()`	1
`feasible(s,t)`	1
`flow_with_value()`	1
`max_flow()`	2
`min_flow()`	2

建图与方案恢复均为 $O(n+m)$
若沿用 Dinic 的一般上界，一次完整求解写作 $O(n^2m)$ 量级，空间复杂度 $O(n+m)$

> Tip
>
> 这一版最值得保留的设计就是**允许有源汇净流量为负**
> 很多传统板子只加 `t → s` 一条辅助边，因此实际上默认答案非负
> 现在我们用双向辅助边把一般情形统一掉了，而且固定流量又直接通过 `balance` 处理

6.4.1 上下界最小费用流

传统上下界可行流本身就是先固定下界，再通过超级源汇修正各点的流量不平衡
这里在得到任意满足目标净流量的可行流后，再利用负环消圈优化费用即可

```cpp
// 依赖 6.4 LowerBoundFlow<Cap> 与 6.3 MinCostFlow<Cap, Cost>
template<class Cap = i64, class Cost = i64>
struct LowerBoundMinCostFlow {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
    static_assert(numeric_limits<Cost>::is_integer && numeric_limits<Cost>::is_signed);
    struct Edge {
        int from, to;
        Cap low, high;
        Cost cost;
    };
    struct Result {
        bool feasible{};
        Cap value{};
        Cost cost{};
        vector<Cap> flow;
    };
    int n, sc{ 8 };
    bool upd{};
    vector<Edge> edges;
    explicit LowerBoundMinCostFlow(int n_) : n(n_) {}
    int add_edge(int u, int v, Cap low, Cap high, Cost cost) {
        edges.push_back({ u, v, low, high, cost });
        return int(edges.size()) - 1;
    }
    Result circulation() const { return solve(-1, -1, 0, nullopt); }
    Result flow_with_value(int s, int t, Cap value) const { return solve(s, t, 0, value); }
    // ===== 以下函数可直接删除：按流量优先，再最小化费用 =====
    Result max_flow(int s, int t) const { return solve(s, t, 1, nullopt); }
    Result min_flow(int s, int t) const { return solve(s, t, -1, nullopt); }

private:
    Result solve(int s, int t, int type, optional<Cap> fixed) const {
        // 第一步：忽略费用，构造满足上下界及目标净流量的任意可行流
        LowerBoundFlow<Cap> feasible(n);
        for (const Edge &x : edges) feasible.add_edge(x.from, x.to, x.low, x.high);
        Cap value{};
        if (s == -1) {
            if (!feasible.feasible()) return {};
        } else {
            optional<Cap> res;
            if (fixed) res = feasible.flow_with_value(s, t, *fixed);
            else if (type > 0) res = feasible.max_flow(s, t);
            else res = feasible.min_flow(s, t);
            if (!res) return {};
```

```
            value = *res;
        }
        // 第二步：装入费用残量网络，保持净流量不变并消除所有负费用环
        MinCostFlow<Cap, Cost> mcf(n);
        mcf.upd = upd;
        mcf.sc = sc;
        vector<int> id(edges.size());
        Cost total{};
        for (int i = 0; i < int(edges.size()); ++ i) {
            const auto &[u, v, low, high, cost] = edges[i];
            Cap f{ feasible.edge_flow(i) };
            id[i] = mcf.add_edge(u, v, high - low, cost);
            mcf.change_edge(id[i], high - low, f - low);
            total += Cost(f) * cost;
        }
        total += mcf.cancel_negative_cycles();
        vector<Cap> answer(edges.size());
        for (int i = 0; i < int(edges.size()); ++ i) {
            auto [u, v, cap, used, cost] = mcf.get_edge(id[i]);
            answer[i] = edges[i].low + used;
        }
        return { true, value, total, move(answer) };
    }
};
```

原理

第一阶段只处理可行性
对于原边

$$low_e \le f_e \le high_e$$

先使用 `LowerBoundFlow` 得到满足上下界以及指定净流量约束的任意可行流 `f`
第二阶段令

$$x_e = f_e - low_e$$

于是

$$0 \le x_e \le high_e - low_e$$

代码用

```
mcf.add_edge(u, v, high - low, cost);
mcf.change_edge(id, high - low, f - low);
```

直接把当前 `x` 装入费用残量网络
虽然 `x` 本身未必满足普通循环流意义下的流量守恒，但它的各点净流量已经固定
任意两个满足同一上下界与同一净流量约束的可行解之差一定是一个 circulation，因此所有进一步调整都可以表示为残量网络中的环流
若残量网络还存在负费用环，就能沿该环增广并降低总费用
不存在负费用环时即达到该净流量约束下的全局最优费用

接口

- `LowerBoundMinCostFlow(n)`：建立 `n` 点空网络，$O(n)$
- `add_edge(u,v,low,high,cost)`：加入流量范围 `[low,high]`、单位费用 `cost` 的有向边，返回原边编号
- `circulation()`：求最小费用可行环流
- `flow_with_value(s,t,value)`：求净 `s-t` 流量恰为 `value` 的最小费用流，`value` 可以为负
- `max_flow(s,t)`：先最大化净 `s-t` 流量，再在所有最大流中最小化费用
- `min_flow(s,t)`：先最小化净 `s-t` 流量，再在所有最小流中最小化费用
- `Result::feasible`：是否存在可行方案
- `Result::value`：最终 `s` 的净流出量，`circulation()` 中为 `0`
- `Result::cost`：最小总费用
- `Result::flow[id]`：第 `id` 条原边最终实际流量

注意事项

- 本板复用 `6.4 LowerBoundFlow` 求可行解，再复用 `6.3 MinCostFlow::cancel_negative_cycles()` 优化费用，不维护第三套独立费用流实现

- 要求 `0 ≤ low ≤ high`
- 有源汇接口要求 `s ≠ t`
- `Cap` 与 `Cost` 均应使用有符号整数类型
- `flow_with_value()` 中 `value` 可以为负，含义仍是 `s` 的净流出量
- `max_flow()` / `min_flow()` 是**字典序目标**，先优化流量值，再在该流量下最小化费用
- 第一步得到什么可行流不影响最终答案，第二步会在具有相同点净流量的所有可行流之间优化
- 保持指定净流量时调用 `cancel_negative_cycles()`；`flow(s,t)` 还会改变流量值，不能替代固定流量的费用优化
- `Result::feasible == false` 时不要读取 `value`、`cost` 或 `flow`
- 单位费用允许为负，只要所有容量上界有限，最优费用仍有有限下界
- `Cost` 必须能容纳 `flow * cost`、负环费用变化与最终总费用，范围大时使用 `i128`
- 普通无上下界 `s-t` 最小费用流直接使用 `6.3 MinCostFlow`
- 费用优化复用 6.3 的整数费用缩放，包装接口不增加另一套费用流实现；内部盈余、缩放费用与势能的数值范围同 6.3

常见用法

固定流量最小费用

```
LowerBoundMinCostFlow<i64, i64> flow(n);
for (int i = 0; i < m; ++ i) {
    flow.add_edge(u[i], v[i], low[i], high[i], cost[i]);
}
auto ans = flow.flow_with_value(s, t, need);
if (!ans.feasible) {
    cout << "No Solution\n";
} else {
    cout << ans.cost << '\n';
    for (i64 f : ans.flow) cout << f << '\n';
}
```

最小费用可行环流

```
auto ans = flow.circulation();
if (ans.feasible) cout << ans.cost << '\n';
```

上下界最小费用最大流

```
auto ans = flow.max_flow(s, t);
if (ans.feasible) {
    cout << ans.value << ' ' << ans.cost << '\n';
}
```

不再需要先额外建立一个 `LowerBoundFlow` 求最大值再调用 `flow_with_value()`

上下界最小费用最小流

```
auto ans = flow.min_flow(s, t);
if (ans.feasible) {
    cout << ans.value << ' ' << ans.cost << '\n';
}
```

复杂度

设原图为 n 点、m 边，最大单位费用绝对值为 C

接口	可行流阶段	费用优化
`circulation()`	一次 Dinic	$O(n^2m\log(2+nC))$
`flow_with_value()`	一次 Dinic	$O(n^2m\log(2+nC))$
`max_flow()`	至多两次 Dinic	$O(n^2m\log(2+nC))$
`min_flow()`	至多两次 Dinic	$O(n^2m\log(2+nC))$

若把 `LowerBoundFlow` 的复杂度记为 T_{flow}，则总复杂度可写为

$$O(T_{\text{flow}}+n^2m\log(2+nC))$$

空间复杂度为 $O(n+m)$

常用方法：开启价格更新

包装直接把 `upd/sc` 传给 6.3，默认关闭价格更新、缩放因子为 `8`，对全部求解接口生效。

```
LowerBoundMinCostFlow<i64, i128> g(2);
g.upd = true;
g.sc = 4; // 可省略
// 两条边构成循环流，下界要求至少流 1，最优流量为 3
g.add_edge(0, 1, 1, 3, -2);
g.add_edge(1, 0, 1, 3, 1);
auto ans = g.circulation();
assert(ans.feasible && ans.cost == -3);
```

价格更新可能减少重标号，也会增加最短路开销；开启前后分别测试，不能据此放宽数值范围。细节见 6.3 常用方法。

6.5 Stoer–Wagner 全局最小割

```
template<class T = i64>
T stoer_wagner(vector<vector<T>> g) {
    int n{ int(g.size()) };
    if (n <= 1) return 0;
    T ans{ numeric_limits<T>::max() };
    vector<int> v(n);
    iota(v.begin(), v.end(), 0);
    while (v.size() > 1) {
        vector<T> w(n);
        vector<char> used(n);
        int prev{ -1 }, last{ -1 };
        // Maximum Adjacency Search: 最后加入的点给出本阶段最小割
        for (int it = 0; it < int(v.size()); ++ it) {
            int sel{ -1 };
            for (int x : v) if (!used[x] && (sel == -1 || w[x] > w[sel])) sel = x;
            used[sel] = true;
            prev = last;
            last = sel;
            if (it + 1 == int(v.size())) {
                ans = min(ans, w[last]);
                if (ans == 0) return 0;
                // 将阶段最后点 last 缩并到倒数第二点 prev
                for (int x : v) if (x != prev && x != last) {
                    g[prev][x] += g[last][x];
                    g[x][prev] = g[prev][x];
                }
                v.erase(find(v.begin(), v.end(), last));
            } else {
                for (int x : v) if (!used[x]) w[x] += g[sel][x];
            }
        }
    }
    return ans;
}
```

原理

每一轮在当前缩点图上进行 Maximum Adjacency Search
从任意点开始，每次选择与当前已选集合连接总权最大的未选点，并维护

```
w[x] = x 到已选点集合的边权和
```

若最后加入的两个点依次为 `prev`、`last`，则这一阶段得到的 `prev-last` 最小割值就是 `w[last]`
记录该值后将 `last` 缩并进 `prev`，继续在更小的图上执行，所有阶段割值中的最小值即为全局最小割

接口说明

接口

- `stoer_wagner(g)`：返回无向非负容量图的全局最小割值，$O(n^3)$
- `g` 按值传递，缩点只修改内部副本，不影响调用者

注意事项

- `g` 必须是 $n \times n$ 对称矩阵，满足 `g[u][v] == g[v][u]`
- 点编号为 `0base`
- 容量必须非负
- `g[u][u]` 应为 `0`
- 平行边需要预先累加到 `g[u][v]`
- 图不连通时答案为 `0`，代码会在首次得到零割后直接返回
- `n ⩽ 1` 时返回 `0`
- 缩点时边权会不断累加，`T` 必须能容纳缩点后的容量和，范围较大时使用 `i128`
- 该算法只适用于无向图
- 有向全局最小割不是 Stoer–Wagner 的适用范围
- 本板只返回最小割值，不恢复割集
- 若需要所有点对最小割信息，使用 6.5.1 Gomory–Hu 最小割树
- 若只需要指定 `s-t` 最小割，直接使用 Dinic，根据最大流最小割定理求解更合适

常见用法

```
vector<vector<i64>> g(n, vector<i64>(n));
for (auto [u, v, cap] : edges) {
    g[u][v] += cap;
    g[v][u] += cap;
}
i64 cut{ stoer_wagner(g) };
```

判断图是否存在零容量割

```
if (stoer_wagner(g) == 0) {
    // 存在两个非空点集，使跨越它们的总容量为 0
}
```

对于普通正权图，这等价于图不连通

复杂度

时间复杂度：$O(n^3)$
空间复杂度：$O(n^2)$

6.5.1 Gomory–Hu 最小割树

```
// 依赖 6.2 Dinic<Cap>: add_edge()、max_flow()、min_cut()
template<class Cap = i64>
vector<tuple<int, int, Cap>> gomory_hu(int n, const vector<tuple<int, int, Cap>> &edges) {
    static_assert(numeric_limits<Cap>::is_integer && numeric_limits<Cap>::is_signed);
    vector<int> parent(n);
    vector<Cap> value(n);
    for (int s = 1; s < n; ++ s) {
        int t{ parent[s] };
        Dinic<Cap> flow(n);
        for (auto [u, v, cap] : edges) if (u != v) flow.add_edge(u, v, cap, cap);
        Cap cut{ flow.max_flow(s, t) };
        vector<char> side{ flow.min_cut(s) };
        // 与 s 同侧、且仍以 t 为父亲的后续点改挂到 s
        for (int i = s + 1; i < n; ++ i) {
            if (parent[i] == t && side[i]) parent[i] = s;
        }
        // t 的父亲也与 s 同侧时，需要旋转 parent 关系
        if (side[parent[t]]) {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        } else value[s] = cut;
    }
    vector<tuple<int, int, Cap>> tree;
    tree.reserve(max(0, n - 1));
    for (int u = 1; u < n; ++ u) tree.push_back({ u, parent[u], value[u] });
```

```
    return tree;
}
```

原理

Gomory–Hu Tree 将一个无向容量图压缩为一棵带权树，使任意两点 `u,v` 满足

$$\lambda(u,v) = \min_{e \in P(u,v)} w_e$$

其中 $\lambda(u,v)$ 是原图的 `u-v` 最小割，`P(u,v)` 是割树上的唯一路径
算法维护 `parent[u]`，依次对 `s=1..n-1` 求 `s-parent[s]` 最小割，并利用最大流结束后的残量可达集合重新调整父亲关系，因此只需要 $n-1$ 次最大流

接口说明

返回值

`gomory_hu(n, edges)` 返回 `n-1` 条

```
{u, parent[u], value[u]}
```

表示割树边 `u-parent[u]`，权值为 `value[u]`
原图任意两点 `s,t` 的最小割值等于割树上 `s-t` 路径中的最小边权

注意事项

- 只适用于**无向非负容量图**
- 点编号为 `0base`
- 每条无向边使用 `flow.add_edge(u,v,cap,cap)` 建模
- 自环不影响任何两点最小割，直接忽略
- 平行边可以直接加入，无需预先合并
- 图可以不连通，此时不同连通块之间的最小割为 `0`，割树中会出现权值 `0` 的边
- 构造出的树不是原图子图，树边只编码割信息
- `flow.min_cut(s)` 必须返回最大流结束后残量网络中从 `s` 可达的点集
- 若 `6.2 Dinic` 中该接口命名为 `min_cut_side(s)`，这里对应改名即可
- 每轮都必须重新建立 Dinic，因为上一轮最大流已经修改了残量容量
- 容量和任意最小割值必须能放入 `Cap`
- `n ⩽ 1` 时返回空树
- 全局最小割在 `n ⩾ 2` 时等于割树所有边权的最小值
- 只需要一次全局最小割值时优先使用 `6.5 Stoer–Wagner`，无需跑 $n-1$ 次最大流

常见用法

建立割树

```
auto tree = gomory_hu(n, edges);
vector<vector<pair<int, i64>>> g(n);
for (auto [u, v, cut] : tree) {
    g[u].push_back({ v, cut });
    g[v].push_back({ u, cut });
}
```

单次查询两点最小割

查询不多时直接树上 DFS / BFS 即可

```
auto query = [&](int s, int t) {
    vector<i64> mn(n, -1);
    queue<int> q;
    mn[s] = numeric_limits<i64>::max();
    q.push(s);
    while (!q.empty()) {
        int u{ q.front() };
        q.pop();
        if (u == t) return mn[u];
        for (auto [v, w] : g[u]) if (mn[v] == -1) {
            mn[v] = min(mn[u], w);
            q.push(v);
        }
    }
```

```
    return i64{};
};
```

大量点对查询

对割树预处理倍增，维护跳跃过程中最小边权

```
// query(u, v) = 割树路径 u-v 的最小边权
//             = 原图 u-v 最小割
```

也可以直接复用已有 HLD，在路径上维护区间最小值

求全局最小割

```
i64 ans{ numeric_limits<i64>::max() };
for (auto [u, v, cut] : tree) ans = min(ans, cut);
```

`n ⩽ 1` 时需单独按约定处理

复杂度

设一次最大流复杂度为 $MF(n,m)$
构造执行恰好 $n-1$ 次最大流，因此

$$O(n \cdot MF(n,m))$$

每轮重新建图还需要 $O(m)$，已被通常的最大流复杂度覆盖
割树自身空间复杂度为 $O(n)$，单轮最大流工作空间为 $O(n+m)$

> Tip
>
> `Stoer--Wagner` 回答的是“整张图最容易在哪里切开”，Gomory--Hu 则一次预处理后回答“任意两点之间最少要切多少容量才能分开”

6.6 网络流建模与方案恢复索引

模板选择

需求	优先使用
普通最大流 / `s-t` 最小割	`6.2 Dinic`
只求最大流值且 Dinic 较慢	`6.2.1 HLPP`
二分图最大匹配	`6.1 Hopcroft--Karp`
二分图最优匹配 / 指派	`6.1.1 Hungarian--KM`
一般图最大匹配	`6.1.2 Blossom`
一般图最大权匹配	`6.1.3 Weighted Blossom`
最小费用 `s-t` 流	`6.3 MinCostFlow`
最小费用循环流	`6.3.1 最小费用循环流`
上下界可行 / 最大 / 最小流	`6.4 LowerBoundFlow`
上下界最小费用流	`6.4.1 LowerBoundMinCostFlow`
无向全局最小割	`6.5 Stoer--Wagner`
无向任意点对最小割	`6.5.1 Gomory--Hu`
路径方案恢复	`6.2 最大流扩展：流分解`

建模原子

题意	建模
某资源最多使用 `c` 次	容量 `c`
某资源至多使用一次	容量 `1`
某边必须至少使用 `low`、至多 `high`	上下界 `[low,high]`
某边必须恰好使用 `x`	上下界 `[x,x]`

题意	建模
使用一单位资源代价为 `cost`	费用边 `(cap,cost)`
选 `u` 必须选 `v`	`u → v` 容量 `INF`
点 `u` 最多经过 `c` 单位流	拆成 `u_in → u_out`，容量 `c`
一条边不能被最小割切断	容量 `INF`
多个源 / 汇	建超级源 `SS` / 超级汇 `TT`
要求流量守恒且有上下界	`LowerBoundFlow`
固定净 `s-t` 流量为 `F`	`flow_with_value(s,t,F)`
要求恰好发送 `F` 单位最小费用流	`MinCostFlow::flow(s,t,F)` 并检查返回流量

> Tip
>
> 网络流建模的核心通常只有三件事：**容量限制资源数量、最小割表达选择代价、费用表达每单位选择代价**

多源多汇

增加超级源 `SS` 与超级汇 `TT`

```
for (auto [s, cap] : sources) flow.add_edge(SS, s, cap);
for (auto [t, cap] : sinks) flow.add_edge(t, TT, cap);
```

若源 / 汇没有自身流量限制，可将容量取足够大的 `INF`

无向容量边

若一条无向边总容量表示净流量满足

$$|f_{uv}| \le c$$

使用

```
flow.add_edge(u, v, c, c);
```

若题意实际表示 `u → v` 与 `v → u` 各自拥有独立容量 `c`，则加入两条独立有向边

```
flow.add_edge(u, v, c);
flow.add_edge(v, u, c);
```

两种模型在很多纯最大流问题中流值相同，但**逐边流量语义不同**，恢复方案时需要区分

点容量与拆点

将原点 `u` 拆为

```
u_in → u_out
```

并令这条边容量等于点容量
原边

```
u → v
```

改为

```
u_out → v_in
```

若原边没有限制，容量设为 `INF`

最小点割

```
flow.add_edge(in(u), out(u), cost[u]);
flow.add_edge(out(u), in(v), INF);
```

通常源点、汇点本身不能删除，因此 `s_in → s_out`、`t_in → t_out` 取 `INF`
最大流后

```
auto side = flow.min_cut(S);
```

若

```
side[in(u)] && !side[out(u)]
```

则点 `u` 对应拆点边被割，即属于恢复出的点割

点不相交路径

内部点拆边容量设为 `1`

```
flow.add_edge(in(u), out(u), 1);
```

源汇通常使用 `INF`
最大流值即最大点不相交路径数，具体路径使用流分解恢复

边不相交路径

每条可使用边容量设为 `1`

```
flow.add_edge(u, v, 1);
```

最大流值即最大边不相交 `s-t` 路径数
方案使用

```
auto paths = flow_decomposition(flow, s, t);
```

若只关心哪些边被使用，也可直接检查 `edge_flow(id)`

最小割与删除问题

若删除原边 `e` 的代价为 `w_e`，直接令容量为 `w_e`

```
flow.add_edge(u, v, w);
```

最大流值即最小删除代价
最大流结束后：

```
auto side = flow.min_cut(s);
```

对有向边 `u → v`

```
side[u] && !side[v]
```

表示该边跨越最小割
对无向边只需判断

```
side[u] ≠ side[v]
```

> ⚠ Warning
>
> `min_cut(s)` 只有在已经求出完整最大流后才代表最小割
> 若 `max_flow(s,t,limit)` 因 `limit` 提前停止，则不能据此恢复最小割

二选一 / 0-1 最小割模型

令 `side[v] = true` 表示选择状态 `0`，`false` 表示状态 `1`
若：

- 选择状态 `0` 的代价为 `c0`
- 选择状态 `1` 的代价为 `c1`
  建边

```
flow.add_edge(S, v, c1);
flow.add_edge(v, T, c0);
```

于是：

- `v` 位于源侧时切掉 `v → T`，支付 `c0`
- `v` 位于汇侧时切掉 `S → v`，支付 `c1`
  若 `u,v` 状态不同需要额外支付 `w`，加入无向容量边

```
flow.add_edge(u, v, w, w);
```

这类模型是大量“二选一 + 冲突代价”问题的基础

最大权闭合图

每个点有权值 `w[u]`，选中 `u` 后要求所有后继依赖也必须选中
若 `u` 依赖 `v`：

```
flow.add_edge(u, v, INF);
```

点权：

```
if (w[u] > 0) flow.add_edge(S, u, w[u]);
else flow.add_edge(u, T, -w[u]);
```

设

```
sum = Σ max(w[u], 0)
```

则

```
最大闭合子图权值 = sum - maxflow
```

最大流后：

```
auto side = flow.min_cut(S);
```

`side[u] == true` 的原图点即被选入最优闭合子图

项目选择

“选择项目获得收益，但必须购买若干资源”直接视为闭合图：

```
S → 项目        收益
项目 → 所需资源  INF
资源 → T        成本
```

答案：

```
总项目收益 - 最小割
```

源侧的项目和资源就是最终选择方案

`INF` 的选择

不要习惯性写

```
numeric_limits<i64>::max()
```

更安全的方法是找出一个有限解的确定上界 `B`

```
Cap INF{ B + 1 };
```

例如最大权闭合图可以取

```
Cap sum{};
for (auto w : weight) if (w > 0) sum += w;
Cap INF{ sum + 1 };
```

因为存在代价至多 `sum` 的有限割，所以最优割绝不会切容量 `sum + 1` 的依赖边
若 `B + 1` 可能溢出，应直接换更宽类型

二分图最大匹配

网络流建模：

```
S → 左部点      1
左部点 → 右部点 1
右部点 → T      1
```

最大流值等于最大匹配数
但普通二分图匹配应优先使用 `Hopcroft--Karp`，只有出现容量、费用、上下界等额外限制时再转网络流
方案恢复：

```
int id{ flow.add_edge(u, v, 1) };
```

求最大流后若

```
flow.edge_flow(id) == 1
```

则 `(u,v)` 被匹配

二分图多重匹配 / b-Matching

若左点 `u` 最多匹配 `a[u]` 次，右点 `v` 最多匹配 `b[v]` 次：

```
S → u   a[u]
u → v   边自身容量
v → T   b[v]
```

原边容量为 `1` 时表示同一对点最多匹配一次
如果每条匹配还有费用或收益，直接把中间边换为费用边并使用 `MinCostFlow`

二分图最小点覆盖

无权二分图有

$$|\text{最小点覆盖}| = |\text{最大匹配}|$$

最大流后设 `side` 为残量图从 `S` 可达点
则一个最小点覆盖为：

```
左部中不可达点 + 右部中可达点
```

即

```
if (!side[u]) cover.push_back(u); // u in L
if ( side[v]) cover.push_back(v); // v in R
```

二分图最小权点覆盖

要求点权非负

```
S → 左点 u       w[u]
右点 v → T       w[v]
左点 u → 右点 v  INF
```

最小割值即最小权点覆盖
恢复方式仍为：

```
左部不可达 + 右部可达
```

二分图最大权独立集

对于非负点权：

$$\text{最大权独立集} = \text{全部点权和} - \text{最小权点覆盖}$$

方案恰好是最小点覆盖的补集：

```
左部可达 + 右部不可达
```

网格独立集

四邻接网格按棋盘奇偶染色后成为二分图，因此“选若干格子且相邻格不能同时选择”的最大权模型通常直接转最大权独立集

DAG 最小路径覆盖

把每个原点 `u` 拆为二分图中的 `u_L,u_R`
若 DAG 存在边

```
u → v
```

则加入

```
u_L → v_R
```

求二分图最大匹配 `M`

$$\text{最小路径覆盖} = n - |M|$$

拆左右点并使用最大匹配的 DAG 路径覆盖转化

恢复路径

若匹配边为

```
u_L → v_R
```

则令

```
nxt[u] = v;
pre[v] = u;
```

所有

```
pre[u] == -1
```

的点都是一条覆盖路径的起点，沿 `nxt` 即可恢复整条路径

> ⚠ Warning
>
> 上传资料中同时写有“一般无向图最小路径覆盖 = `n - 最大匹配数 / 2`”的表述
> 这个公式**不适用于一般无向图的顶点不交最小路径覆盖**，因此板子中只保留标准的 DAG 版本

偏序最小链覆盖

若题目要求的是偏序链覆盖，应对“可比较关系”建二分图，而不是机械地只使用原图直接边
之后仍使用

$$n - \text{最大匹配}$$

二分图指派 / 最小费用匹配

建图：

```
S → 左点       1, cost = 0
左点 → 右点    1, cost = w
右点 → T       1, cost = 0
```

要求恰好匹配 `k` 对：

```
auto [f, cost] = mcf.flow(S, T, k);
if (f != k) {
    // 不存在大小为 k 的匹配
}
```

最大权匹配令中间边费用为 `-w`
恢复：

```
auto [u, v, cap, used, cost] = mcf.get_edge(id);
if (used == 1) {
    // 选择了匹配边 u-v
}
```

资料中的“分配问题 / 二分图最优匹配”也是通过费用流处理
若是标准完整指派问题，应优先使用 Hungarian / KM，代码更短且更快

运输 / 供需模型

供应点 `i` 有 `supply[i]`，需求点 `j` 有 `demand[j]`

```
S → 供应点 i       supply[i], 0
供应点 i → 需求点 j 可运输量, transport_cost
需求点 j → T       demand[j], 0
```

令

```
Cap need = accumulate(demand.begin(), demand.end(), Cap{});
auto [f, cost] = mcf.flow(S, T, need);
```

`f == need` 才表示所有需求均被满足
如果供应和需求还有下界、固定量等约束，转 `LowerBoundMinCostFlow`

固定数量的最小费用不相交路径

边不相交：

```
原边容量 = 1
```

点不相交：

```
u_in → u_out 容量 = 1
```

边权作为费用，要求 `k` 条路径时：

```
auto [f, cost] = mcf.flow(s, t, k);
```

只有 `f == k` 才存在 `k` 条满足条件的路径
资料中也单独收录了“最大权不相交路径”的费用流建模

最大密度子图

经典定义：

$$\rho(S) = \frac{|E(S)|}{|S|}$$

资料中将其归为 0-1 分数规划 + 网络流 模型 【佚名B】算法模板
二分答案 `g`，判断是否存在

$$|E(S)| - g|S| > 0$$

可以转为最大权闭合图：

- 每条原边建立一个“边点”，权值 `+1`
- 每个原顶点建立一个“点点”，权值 `-g`
- 边点向它的两个端点各连一条 `INF` 依赖边
- 若最大闭合子图权值 `> 0`，说明存在密度大于 `g` 的子图
  若 Dinic 只支持整数容量，可将 `g=p/q` 后整体乘 `q`：

```
边点权值 = q
顶点权值 = -p
```

最终源侧的原顶点即对应恢复出的候选子图

上下界模型

原边：

$$low \le f \le high$$

直接使用：

```
LowerBoundFlow<Cap> flow(n);
int id = flow.add_edge(u, v, low, high);
```

常见目标：

```
flow.feasible();                    // 可行环流
flow.feasible(s, t);                // 任意可行 s-t 流
flow.max_flow(s, t);                // 最大净流量
flow.min_flow(s, t);                // 最小净流量
flow.flow_with_value(s, t, value);  // 固定净流量
```

恢复：

```
Cap f = flow.edge_flow(id);
```

上下界转化的本质是先发送所有下界，再用超级源汇修正各点的流量不平衡

上下界 + 费用

直接使用：

```
LowerBoundMinCostFlow<Cap, Cost> flow(n);
```

对应：

```
flow.circulation();
flow.flow_with_value(s, t, value);
flow.max_flow(s, t);
flow.min_flow(s, t);
```

其中 `max_flow()` / `min_flow()` 的目标是：

1. 先最大化 / 最小化净流量
2. 再在该流量值下最小化费用
   方案直接读取：

```
ans.flow[id]
```

方案恢复速查

模型	恢复方法
普通边流量	`flow.edge_flow(id)`
费用流边流量	`mcf.get_edge(id)` 中的 `used`
上下界流量	`LowerBoundFlow::edge_flow(id)`
上下界费用流	`Result::flow[id]`
最小割两侧	`flow.min_cut(s)`
有向最小割边	`side[u] && !side[v]`
无向最小割边	`side[u] != side[v]`
最小点割	`side[in(u)] && !side[out(u)]`
最大权闭合图	原点位于源侧即被选择
二分图匹配	匹配边流量为 `1`
二分图最小点覆盖	左部不可达 + 右部可达
二分图最大独立集	左部可达 + 右部不可达
DAG 路径覆盖	匹配边建立 `nxt/pre`
`s-t` 路径方案	流分解
Gomory–Hu 查询	割树路径最小边权

最常用的两个恢复接口

```
int id{ flow.add_edge(u, v, cap) };
Cap used{ flow.edge_flow(id) };
auto side = flow.min_cut(s);
```

建议建模时就保存逻辑边编号：

```
vector<int> id(m);
for (int i = 0; i < m; ++ i) id[i] = flow.add_edge(u[i], v[i], cap[i]);
```

不要赛后再从邻接表位置猜原边

常见错误

- **依赖边方向写反**：`选 u ⇒ 必须选 v` 应建 `u → v` 的 `INF` 边
- **`INF` 不够大**：必须严格大于任意可能的有限最优割值
- **`INF` 过大导致溢出**：优先用“有限答案上界 + 1”
- **点限制直接放在所有邻边上**：点容量、点割、点不相交通常需要拆点
- **多源汇直接任选一个源汇**：应增加超级源汇
- **费用流要求恰好 `k` 却不检查返回流量**：必须判断 `f == k`
- **最大流尚未跑完就恢复最小割**：只有完整最大流后的残量可达集才是最小割

- **把反向残量边当原图边**：恢复原边时使用 `add_edge()` 返回的逻辑编号
- **把最大流方案直接当若干路径**：流中可能存在分叉或环，需要流分解
- **无向边建模语义混淆**：`add_edge(u,v,c,c)` 表示共享的无向容量，两个独立 `add_edge()` 表示两个方向各有容量
- **能用专门算法却强上网络流**：普通匹配用 HK、标准指派用 Hungarian、单次无向全局最小割用 Stoer–Wagner

> Tip
>
> 赛场看到模型时可以按这个顺序判断：**有没有"每个资源最多多少次" → 容量；有没有"选 A 必须选 B" → 最小割 / 闭合图；有没有"每单位代价" → 费用流；有没有"至少多少" → 上下界；有没有"点只能经过几次" → 拆点；最后再想怎么从残量图恢复方案**

7. 字符串

7.1 KMP

函数版

```cpp
template<class Seg>
vector<int> kmp(const Seg &s) {
    int n = s.size();
    vector<int> f(n + 1);
    for (int i = 1, j = 0; i < n; ++ i) {
        while (j && s[i] != s[j]) j = f[j];
        if (s[i] == s[j]) ++ j;
        f[i + 1] = j;
    }
    return f;
}
template<class Seg>
vector<int> match(const Seg &p, const Seg &s) {
    int n = p.size(), m = s.size();
    vector<int> f = kmp(p), pos;
    if (n == 0) return pos;
    for (int i = 0, j = 0; i < m; ++ i) {
        while (j && s[i] != p[j]) j = f[j];
        if (s[i] == p[j]) ++ j;
        if (j == n) {
            pos.push_back(i - n + 1);
            j = f[j];
        }
    }
    return pos;
}
template<class Seg>
int period(const Seg &s) {
    if (s.empty()) return 0;
    int n = s.size();
    auto f = kmp(s);
    return n - f[n];
}
template<class Seg>
int cycle(const Seg &s) {
    int n = s.size(), p{ period(s) };
    return p && n % p == 0 ? p : n;
}
template<class Seg>
vector<int> borders(const Seg &s) {
    auto f = kmp(s);
    vector<int> res;
    for (int x = f.back(); x; x = f[x]) res.push_back(x);
    ranges::reverse(res);
    return res;
}
```

- `kmp(s)`：求前缀函数，返回 `f`，`f[i]` 表示 `s[0..i)` 的最长相等真前后缀长度，$O(n)$
- `match(p, s)`：求模式串 `p` 在文本串 `s` 中出现的所有位置，返回 `0base` 下标，$O(n+m)$
- `period(s)`：返回字符串最小正周期，等于 `n - f[n]`，$O(n)$
- `cycle(s)`：返回字符串最小循环节长度；若不能由短串重复构成，则返回 `n`，$O(n)$

- `borders(s)`：返回 `s` 所有 border 的长度，升序排列，$O(n)$

固定模式串版

```
template<class Seg>
struct KMP {
    int n{ 0 };
    Seg p;
    vector<int> f{0};
    KMP() = default;
    KMP(const Seg &p_) : p(p_) { build(); }
    void build() {
        n = p.size();
        f.assign(n + 1, 0);
        for (int i = 1, j = 0; i < n; ++ i) {
            while (j && p[i] != p[j]) j = f[j];
            if (p[i] == p[j]) ++ j;
            f[i + 1] = j;
        }
    }
    vector<int> match(const Seg &s) const {
        int m = s.size();
        vector<int> pos;
        if (n == 0) return pos;
        for (int i = 0, j = 0; i < m; ++ i) {
            while (j && s[i] != p[j]) j = f[j];
            if (s[i] == p[j]) ++ j;
            if (j == n) {
                pos.push_back(i - n + 1);
                j = f[j];
            }
        }
        return pos;
    }
    int period() const { return n ? n - f.back() : 0; }
    int cycle() const {
        int q{ period() };
        return q && n % q == 0 ? q : n;
    }
    vector<int> borders() const {
        vector<int> res;
        for (int x = f.back(); x; x = f[x]) res.push_back(x);
        ranges::reverse(res);
        return res;
    }
};
```

- `KMP<T> kmp(p)`：以模式串 `p` 构造 KMP 匹配器，$O(n)$
- `kmp.build()`：重新构造当前 `p` 的前缀函数，$O(n)$
- `kmp.match(s)`：求当前模式串 `p` 在文本串 `s` 中出现的所有位置，$O(m)$
- `kmp.period()`：返回模式串 `p` 的最小正周期，$O(1)$
- `kmp.cycle()`：返回模式串 `p` 的最小循环节长度，$O(1)$
- `kmp.borders()`：返回模式串 `p` 的所有 border 长度，$O(n)$

注意事项

- `f` 的长度是 `n + 1`，不是 `n`；字符串 `s` 的最小正周期为 `n - f[n]`
- `period(s)` 不要求整除 `n`；`cycle(s)` 才要求循环节长度整除 `n`
- 拼接法 `p + '#' + s` 也能查找子串，但分隔符必须保证不在字符集中；本板子用直接扫描，避免该问题
- `T` 只需要支持 `size()` 和 `operator[]`，可用于 `string`、`vector<int>` 等序列
- 若固定一个模式串匹配多个文本串，优先用结构体版；只临时求一次前缀函数或周期，优先用函数版

常见用法

统计本质不同子串数量

```
string s;
cin >> s;
i64 ans{ 0 };
string cur;
```

```
for (char c : s) {
    string t = (cur += c);
    ranges::reverse(t);
    auto f = kmp(t);
    int mx{ *ranges::max_element(f) };
    ans += (int)cur.size() - mx;
}
cout << ans << '\n';
```

Z 函数

```
template<class Seg>
vector<int> z_function(const Seg &s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++ i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++ z[i];
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    if (n) z[0] = n;
    return z;
}
```

- `z_function(s)`：返回 `z` 数组，`z[i]` 表示 `s` 与 `s[i..]` 的最长公共前缀长度，$O(n)$

注意事项

- 规定 `z[0] = s.size()`
- `Seq` 需要支持 `size()` 和 `operator[]`，如 `string`、`vector<int>`
- 若使用 `p + sep + s` 查找模式串，`sep` 必须不在原字符集中出现
- 若不想处理分隔符，优先使用 `ExKMP`

常见用法

```
auto z = z_function(s);
{
    int n = s.size();
    if (z[i] >= len) {} // 前缀在某位置出现，即 s[0..len) == s[i..i+len)
    if (z[n - len] == len) {} // 判断前缀是否为后缀，即 s[0..len) == s[n-len..n)
}
{// 求最小正周期
    int p{ n };
    for (int d = 1; d < n; ++ d) if (n % d == 0 && z[d] == n - d) {
        p = d;
        break;
    }
}
{ // 查找模式串 p 出现位置
    string t = p + '#' + s;
    auto z = z_function(t);
    vector<int> pos;
    int n = p.size();
    for (int i = n + 1; i < t.size(); ++ i) if (z[i] >= n) pos.push_back(i - n - 1);
}
```

ExKMP

```
template<class Seg>
vector<int> exkmp(const Seg &p, const Seg &s) {
    int n = p.size(), m = s.size();
    vector<int> z = z_function(p), ext(m);
    if (n == 0) return ext;
    for (int i = 0, l = 0, r = -1; i < m; ++ i) {
        if (i <= r) ext[i] = min(r - i + 1, z[i - l]);
        while (ext[i] < n && i + ext[i] < m && p[ext[i]] == s[i + ext[i]]) ++ ext[i];
        if (i + ext[i] - 1 > r) l = i, r = i + ext[i] - 1;
    }
```

```
    return ext;
}
```

- `exkmp(p, s)`：返回 `ext` 数组，`ext[i]` 表示 `p` 与 `s[i..]` 的最长公共前缀长度，$O(|p|+|s|)$

注意事项

- 依赖 `z_function()`
- `ext.size() == s.size()`
- `Seq` 需要支持 `size()` 和 `operator[]`，如 `string`、`vector<int>`
- 若 `p` 为空串，当前实现返回全 `0`；算法竞赛中通常不讨论空模式串
- 只需要单串前缀与后缀 LCP 时，使用 `Z 函数` 即可

常见用法

```
auto ext = exkmp(p, s);
// 查找模式串出现位置
vector<int> pos;
for (int i = 0; i < s.size(); ++ i) if (ext[i] == p.size()) pos.push_back(i);
// 求模式串与文本每个后缀的 LCP
int lcp{ ext[i] }; // p 与 s[i..] 的 LCP
// 比较模式串和文本后缀
int l{ ext[i] };
if (l == p.size()) { // p 是 s[i..] 的前缀
} else if (i + l == s.size()) { // s[i..] 是 p 的前缀
} else if (p[l] < s[i + l]) { // p < s[i..]
} else { // p > s[i..]
}
```

7.2 字符串哈希

`u64` 自然溢出哈希

```
struct RollingHash {
    u64 base;
    vector<u64> pw, h;
    static u64 default_base() {
        static const u64 b = [] {
            u64 x{ u64(chrono::steady_clock::now().time_since_epoch().count()) };
            x += 0x9e3779b97f4a7c15ULL;
            x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
            x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
            x ^= x >> 31;
            return x | (1ULL << 32) | 1;
        }();
        return b;
    }
    template<class T>
    static u64 encode(const T &x) {
        if constexpr (is_same_v<decay_t<T>, char>) return u64((unsigned char)x) + 1;
        else return u64(x) + 1;
    }
    template<class Seg>
    explicit RollingHash(const Seg &s, u64 base_ = default_base())
        : base(base_), pw(s.size() + 1, 1), h(s.size() + 1) {
        for (int i = 0; i < s.size(); ++ i) {
            pw[i + 1] = pw[i] * base;
            h[i + 1] = h[i] * base + encode(s[i]);
        }
    }
    int size() const { return int(h.size()) - 1; }
    u64 get(int l, int r) const { return h[r] - h[l] * pw[r - l]; }
    bool equal(int l1, int r1, int l2, int r2) const {
        return r1 - l1 == r2 - l2 && get(l1, r1) == get(l2, r2);
    }
    int lcp(int l1, int r1, int l2, int r2) const {
        int lo{ 0 }, hi{ min(r1 - l1, r2 - l2) };
        while (lo < hi) {
```

```
            int mid{ (lo + hi + 1) / 2 };
            if (get(l1, l1 + mid) == get(l2, l2 + mid)) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }
    u64 concat(u64 a, u64 b, int len_b) const { return a * pw[len_b] + b; }
};
```

双模哈希

```
struct DoubleHash {
    using H = pair<i64, i64>;
    static constexpr i64 mod1{ 1000000007 };
    static constexpr i64 mod2{ 1000000009 };
    i64 base;
    vector<i64> pw1, pw2, h1, h2;
    static i64 default_base() {
        static const i64 b{
            256 + i64(RollingHash::default_base() % (mod1 - 257))
        };
        return b;
    }
    template<class T>
    static u64 encode(const T &x) {
        if constexpr (is_same_v<decay_t<T>, char>) return u64((unsigned char)x) + 1;
        else return u64(x) + 1;
    }
    template<class Seg>
    explicit DoubleHash(const Seg &s, i64 base_ = default_base())
        : base(base_), pw1(s.size() + 1, 1), pw2(s.size() + 1, 1),
          h1(s.size() + 1), h2(s.size() + 1) {
        for (int i = 0; i < int(s.size()); ++ i) {
            i64 x1{ i64(encode(s[i]) % mod1) }, x2{ i64(encode(s[i]) % mod2) };
            pw1[i + 1] = pw1[i] * base % mod1;
            pw2[i + 1] = pw2[i] * base % mod2;
            h1[i + 1] = (h1[i] * base + x1) % mod1;
            h2[i + 1] = (h2[i] * base + x2) % mod2;
        }
    }
    int size() const { return int(h1.size()) - 1; }
    H get(int l, int r) const {
        i64 x{ (h1[r] - h1[l] * pw1[r - l]) % mod1 };
        i64 y{ (h2[r] - h2[l] * pw2[r - l]) % mod2 };
        if (x < 0) x += mod1;
        if (y < 0) y += mod2;
        return { x, y };
    }
    bool equal(int l1, int r1, int l2, int r2) const {
        return r1 - l1 == r2 - l2 && get(l1, r1) == get(l2, r2);
    }
    int lcp(int l1, int r1, int l2, int r2) const {
        int lo{ 0 }, hi{ min(r1 - l1, r2 - l2) };
        while (lo < hi) {
            int mid{ (lo + hi + 1) / 2 };
            if (get(l1, l1 + mid) == get(l2, l2 + mid)) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }
    H concat(H a, H b, int len_b) const {
        return { (a.first * pw1[len_b] + b.first) % mod1,
            (a.second * pw2[len_b] + b.second) % mod2
        };
    }
};
```

接口说明

- `Hash(s)`：预处理序列 `s`，$O(n)$

- `Hash(s,base)`：使用指定底数预处理
- `get(l,r)`：返回 `[l,r)` 的归一化子串哈希，$O(1)$
- `equal(l1,r1,l2,r2)`：判断同一序列中的两个子串是否相等，$O(1)$
- `lcp(l1,r1,l2,r2)`：求两个子串的 LCP 长度，$O(\log n)$
- `concat(a,b,len_b)`：已知两个字符串的哈希，求其拼接哈希，$O(1)$
- `size()`：原序列长度，$O(1)$

`Seg` 只要求支持 `size()`、`operator[]`，可以直接用于：

```
string
vector<int>
vector<i64>
```

等广义字符串

注意事项

- 所有区间均为 `[l,r)
- 默认底数在运行时随机，但同一程序中同类型哈希对象共享同一个默认底数
- 如果显式传入底数，两个不同对象之间比较哈希时必须保证底数相同
- `DoubleHash` 碰撞概率明显低于单个 `u64` 哈希，但理论上仍可能碰撞
- 对最终只有少量候选且正确性要求极高的题，可以哈希筛选后再逐元素确认
- **哈希值不携带长度**；不同长度字符串作为集合键时必须同时保存长度
- 如果题目本身已有 KMP / Z / SA 等确定性算法能够直接完成，不要为了方便无条件换成哈希
- `concat()` 中的 `len_b` 是右侧字符串 `b` 的长度

常见用法

子串相等

```
RollingHash h(s);
if (h.equal(l1, r1, l2, r2)) {
    // s[l1..r1) == s[l2..r2)
}
```

更重视碰撞风险时：

```
DoubleHash h(s);
```

接口不变

两个不同字符串比较子串

默认构造的两个对象共享底数：

```
RollingHash a(s);
RollingHash b(t);
bool same{ r1 - l1 == r2 - l2 && a.get(l1, r1) == b.get(l2, r2) };
```

若显式设置底数，应写：

```
u64 base{ RollingHash::default_base() };
RollingHash a(s, base);
RollingHash b(t, base);
```

求子串 LCP

同一个字符串：

```
int len{ h.lcp(l1, r1, l2, r2) };
```

不同字符串：

```
int lo{ 0 }, hi{ min(r1 - l1, r2 - l2) };
while (lo < hi) {
    int mid{ (lo + hi + 1) / 2 };
    if (hs.get(l1, l1 + mid) == ht.get(l2, l2 + mid)) lo = mid;
    else hi = mid - 1;
}
```

得到两个子串的最长公共前缀长度 `lo`

字典序比较两个子串

先求：

```
int k{ h.lcp(l1, r1, l2, r2) };
int n1{ r1 - l1 }, n2{ r2 - l2 };
```

若：

```
k == min(n1, n2)
```

则较短者字典序更小；否则比较：

```
s[l1 + k] < s[l2 + k]
```

因此哈希可以把一次任意子串字典序比较做到：

$$O(\log n)$$

若需要大量字符串排序，SA + LCP/RMQ 往往更合适

判断回文

建立正串与反串哈希：

```
string rs{ s.rbegin(), s.rend() };
RollingHash h(s);
RollingHash rh(rs);
```

原串 `[l,r)` 在反串中对应：

```
[n-r, n-l)
```

因此：

```
bool pal{ h.get(l, r) == rh.get(n - r, n - l) };
```

即可 $O(1)$ 判断回文

判断周期

长度为：

```
int len{ r - l };
```

若 `p` 满足：

```
h.get(l, r - p) == h.get(l + p, r)
```

则 `p` 是 `s[l..r)` 的一个周期
即：

$$\boxed{p \text{ 是周期} \iff s[l..r-p) = s[l+p..r)}$$

如果进一步要求该子串由长度 `p` 的完整块重复组成，还需要：

```
len % p == 0
```

判断 Border

长度为 `k` 的前缀同时也是后缀：

```
if (h.get(0, k) == h.get(n - k, n)) {
    // k 是一个 border 长度
}
```

对子串 `[l,r)` 同理：

```
h.get(l, l + k) == h.get(r - k, r)
```

拼接哈希

若：

```
auto ha = h.get(l1, r1);
auto hb = h.get(l2, r2);
```

则：

```
auto hab = h.concat(ha, hb, r2 - l2);
```

满足：

$$H(A+B) = H(A) \cdot base^{|B|} + H(B)$$

这个结论常用于：

- 线段树维护字符串哈希
- 分治拼接
- 动态字符串
- 区间翻转哈希

哈希去重

若所有子串长度相同：

```
set<u64> st;
for (...) {
    st.insert(h.get(l, r));
}
```

即可概率去重
不同长度时必须把长度也加入键：

```
set<pair<int, u64>> st;
st.insert({ r - l, h.get(l, r) });
```

双哈希则使用：

```
(length, hash1, hash2)
```

作为键

二级结论

LCP + 哈希 = 子串比较基础工具

预处理 $O(n)$ 后：

- 子串相等：$O(1)$
- LCP：$O(\log n)$
- 字典序比较：$O(\log n)$
- 回文判断：$O(1)$
- 周期判断：$O(1)$
  因此很多“任意子串比较”问题都可以统一转成：

```
Hash + 二分 LCP
```

正反哈希

同时维护：

```
forward hash
reverse hash
```

即可把大量与：

- 回文
- 翻转
- 镜像
  有关的问题转为普通哈希相等

动态字符串

静态 `RollingHash` 本身不能修改字符
若题目有单点修改 / 区间拼接，可利用：

$$H(A+B) = H(A)B^{|B|} + H(B)$$

将：

```
(hash, length)
```

作为线段树 `Info`，即可维护动态字符串哈希

复杂度

设序列长度为 n

操作	时间复杂度
构造	$O(n)$
`get()` / `equal()` / `concat()` / 回文 / 周期判断	$O(1)$
`lcp()`	$O(\log n)$

空间复杂度：$O(n)$

7.3 Trie

```
template<int A = 26, int offset = 'a'>
struct Trie {
    struct Node {
        array<int, A> next{};
        int pass{ 0 }, cnt{ 0 };
    };
    vector<Node> t;
    explicit Trie(int m = 0) {
        t.reserve(m + 1);
        t.emplace_back();
    }
    template<class Seg>
    static const Seg &view(const Seg &s) { return s; }
    template<size_t N>
    static string_view view(const char (&s)[N]) {
        return {s, N - (N > 0 && s[N - 1] == '\0')};
    }
    // ===== 基础函数：以下代码被所有接口依赖，不建议删除 =====
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    int new_node() {
        t.emplace_back();
        return int(t.size()) - 1;
    }
    // ===== 基础接口：插入 / 查找 / 单步转移 =====
    template<class Seg>
    int add(const Seg &s) {
        int p{ 0 };
        ++ t[p].pass;
        for (const auto &c : view(s)) {
            int x{ id(c) };
            if (!t[p].next[x]) t[p].next[x] = new_node();
            p = t[p].next[x];
            ++ t[p].pass;
        }
        ++ t[p].cnt;
        return p;
```

```
    }
    template<class Seg>
    int find(const Seg &s) const {
        int p{ 0 };
        for (const auto &c : view(s)) {
            p = t[p].next[id(c)];
            if (!p) return -1;
        }
        return p;
    }
    template<class T>
    int step(int p, const T &c) const {
        if (p < 0) return -1;
        int q{ t[p].next[id(c)] };
        return q ? q : -1;
    }
    int size() const { return int(t.size()); }
    int total() const { return t[0].pass; }
    // ===== 查询接口：若只建 Trie 后自行 DP / DFS，可按需删除 =====
    template<class Seg>
    int count(const Seg &s) const {
        int p{ find(s) };
        return p == -1 ? 0 : t[p].cnt;
    }
    template<class Seg>
    bool contains(const Seg &s) const { return count(s) > 0; }
    // 当前有多少字符串以 s 为前缀
    template<class Seg>
    int prefix_count(const Seg &s) const {
        int p{ find(s) };
        return p == -1 ? 0 : t[p].pass;
    }
    // 枚举当前字典中所有作为 s 前缀的字符串终止节点
    template<class Seg, class F>
    void match_prefixes(const Seg &s, F f) const {
        int p{ 0 };
        if (t[p].cnt) f(p);
        for (const auto &c : view(s)) {
            p = t[p].next[id(c)];
            if (!p) return;
            if (t[p].cnt) f(p);
        }
    }
    // 当前有多少已存字符串是 s 的前缀
    template<class Seg>
    int count_prefixes(const Seg &s) const {
        int ans{ 0 };
        match_prefixes(s, [&](int p) {
            ans += t[p].cnt;
        });
        return ans;
    }
    // ===== 删除接口：若不需要删除字符串，可直接删除 =====
    // 只修改 pass / cnt，不回收节点或删除边
    template<class Seg>
    bool erase(const Seg &s) {
        int p{ find(s) };
        if (p == -1 || !t[p].cnt) return false;
        -- t[p].cnt;
        p = 0;
        -- t[p].pass;
        for (const auto &c : view(s)) {
            p = t[p].next[id(c)];
            -- t[p].pass;
        }
        return true;
    }
};
```

成员变量

- `t[0]`：根节点，表示空前缀
- `t[u].next[c]`：节点 `u` 沿字符编号 `c` 的儿子，`0` 表示不存在
- `t[u].pass`：当前仍存在且经过节点 `u` 的字符串数量
- `t[u].cnt`：当前仍存在且恰好在节点 `u` 结束的字符串数量

因此：

$$t[0].pass = \text{当前字符串总数}$$

对于非根节点 `u`：

> `t[u].pass` 同时等于 Trie 中以该节点前缀开头的字符串多重集合大小

基础接口

- `Trie(m)`：构造 Trie，`m` 为预计插入的总字符数，仅用于 `reserve`
- `add(s)`：插入一次字符串 `s`，返回终止节点，$O(|s|)$
- `find(s)`：返回 `s` 对应的 Trie 节点，不存在返回 `-1`，$O(|s|)$
- `step(p,c)`：从节点 `p` 沿字符 `c` 走一步，不存在返回 `-1`，$O(1)$
- `size()`：返回历史创建过的 Trie 节点总数，$O(1)$
- `total()`：返回当前字符串多重集合大小，$O(1)$

查询接口

- `count(s)`：返回字符串 `s` 当前出现次数，$O(|s|)$
- `contains(s)`：判断当前是否存在字符串 `s`，$O(|s|)$
- `prefix_count(s)`：返回当前有多少字符串以 `s` 为前缀，$O(|s|)$
- `match_prefixes(s,f)`：枚举当前字典中所有是 `s` 前缀的字符串终止节点，$O(|s|)$
- `count_prefixes(s)`：返回当前有多少已存字符串是 `s` 的前缀，$O(|s|)$
- `erase(s)`：删除一次 `s`，不存在则返回 `false`，$O(|s|)$

注意两种容易混淆的查询：

```
prefix_count(s)   : s 是多少字符串的前缀
count_prefixes(s) : 多少字符串是 s 的前缀
```

注意事项

- 默认处理小写字母；大写字母使用 `Trie<26, 'A'>`
- 已经映射到 `[0,A)` 的整数序列使用 `Trie<A, 0>`
- 必须保证所有字符满足 `0 ≤ id(c) < A`
- `erase()` 不回收节点，也不会删除已经创建的 Trie 边，只维护 `pass / cnt`
- 因此删除后 `find(s)` 仍可能找到历史路径；判断当前字符串是否存在应使用 `count()` / `contains()`
- 同理，`step()` 可能走入当前 `pass == 0` 的历史节点；需要当前有效路径时应额外检查 `t[p].pass`
- `size()` 表示历史节点数，不表示当前有效前缀数
- `pass / cnt` 均按字符串多重集合统计，重复插入同一字符串会重复计数
- Trie 本身就是一棵以 `0` 为根的树；很多前缀、LCP 问题都可转成树上祖先、LCA 或子树问题
- 01 Trie、可持久化 01 Trie、Patricia Trie 等整数位结构应继续独立维护

常见用法

字符串计数与前缀计数

```cpp
int sum{ 0 };
for (auto &s : a) sum += s.size();
Trie<> tr(sum);
for (auto &s : a) {
    tr.add(s);
}
cout << tr.count("abc") << '\n';
cout << tr.prefix_count("ab") << '\n';
```

例如字典为：

```
a
ab
ab
abc
b
```

则：

```
tr.count("ab");          // 2
tr.prefix_count("ab");   // 3: ab, ab, abc
tr.count_prefixes("abc");// 4: a, ab, ab, abc
```

枚举查询串的所有字典前缀

例如分词、前缀匹配：

```
tr.match_prefixes(s, [&](int p) {
    // t[p].cnt > 0
    // 节点 p 对应的字典字符串是 s 的一个前缀
});
```

若需要区分每个前缀的长度，可以手写转移：

```
int p{ 0 };
if (tr.t[p].cnt) {
    // 空串
}
for (int i = 0; i < int(s.size()); ++ i) {
    p = tr.step(p, s[i]);
    if (p == -1) break;
    if (tr.t[p].cnt) {
        // s[0..i] 是字典中的字符串
    }
}
```

从文本某个位置开始沿 Trie 匹配

```
int p{ 0 };
for (int r = l; r < int(s.size()); ++ r) {
    p = tr.step(p, s[r]);
    if (p == -1) break;
    if (tr.t[p].cnt) {
        // s[l..r] 是字典中的字符串
    }
}
```

这种写法常用于：

- 字典分词 DP
- Word Break
- 从每个位置枚举匹配模式
- Trie + 区间 / DP 转移
  若文本和模式串规模都很大，且需要从大量位置反复开始匹配，通常应进一步考虑 AC 自动机

Trie 的树结构

每个非根节点唯一对应一个非空前缀
可以直接从 `next` 构造父亲和深度：

```
vector<int> fa(tr.size(), -1);
vector<int> dep(tr.size());
for (int u = 0; u < tr.size(); ++ u) {
    for (int c = 0; c < A; ++ c) {
        int v{ tr.t[u].next[c] };
        if (!v) continue;
        fa[v] = u;
        dep[v] = dep[u] + 1;
    }
}
```

其中：

```
dep[u]
```

就是节点 `u` 对应前缀的长度

前缀关系 = Trie 祖先关系

设字符串 `a`、`b` 的终止节点分别为 `u`、`v`
则：

$$\boxed{a \text{ 是 } b \text{ 的前缀} \iff u \text{ 是 } v \text{ 的 Trie 祖先}}$$

如果需要大量前缀关系查询，可以对 Trie 求 Euler 序：

```
tin[u], tout[u]
```

于是：

$$u \text{ 是 } v \text{ 的祖先} \iff tin[u] \le tin[v] < tout[u]$$

两个字符串的 LCP = Trie 上 LCA 深度

设字符串 `a`、`b` 的终止节点为 `u`、`v`
则：

$$\boxed{\mathrm{LCP}(a,b) = dep[\mathrm{LCA}(u,v)]}$$

因为两个字符串共有的前缀，恰好对应两条根路径的公共部分
同理，对于多个字符串：

$$\boxed{\mathrm{LCP}(S_1,S_2,\dots,S_k) = dep[\mathrm{LCA}(pos_1,pos_2,\dots,pos_k)]}$$

因此当题目同时涉及：

- 大量字符串
- 前缀关系
- 两两 LCP
- 路径 / 祖先查询
  可以把字符串问题直接转成 Trie 树上的问题

所有字符串两两 LCP 之和

对于当前 Trie 中的字符串多重集合：

$$\boxed{\sum_{1\le i<j\le n}\mathrm{LCP}(S_i,S_j) = \sum_{u\neq 0}\binom{pass[u]}{2}}$$

代码：

```
i64 ans{ 0 };
for (int u = 1; u < tr.size(); ++ u) {
    i64 x{ tr.t[u].pass };
    ans += x * (x - 1) / 2;
}
```

原因是：
一对字符串若 LCP 长度为 `k`，则它们恰好共同经过 Trie 上前 `k` 个非根节点，因此右侧恰好为这对字符串贡献 `k` 次
该公式在执行 `erase()` 后仍然成立，因为 `pass` 维护的是**当前**多重集合

LCP 恰好等于某个前缀的字符串对数

设节点 `u` 对应前缀 P_u
所有同时经过 `u` 的字符串对有：

$$\binom{pass[u]}{2}$$

若两字符串继续进入同一个儿子，则它们的 LCP 会更长
因此：

$$\boxed{exact[u] = \binom{pass[u]}{2} - \sum_{v\in child(u)}\binom{pass[v]}{2}}$$

即 `exact[u]` 为：

> LCP 恰好等于节点 `u` 所对应前缀的字符串对数

```
vector<i64> exact(tr.size());
for (int u = 0; u < tr.size(); ++ u) {
    i64 x{ tr.t[u].pass };
    exact[u] = x * (x - 1) / 2;
    for (int c = 0; c < A; ++ c) {
        int v{ tr.t[u].next[c] };
```

```
        if (!v) continue;
        i64 y{ tr.t[v].pass };
        exact[u] -= y * (y - 1) / 2;
    }
}
```

若再按 `dep[u]` 汇总，即可得到：

> LCP 长度恰好为 `k` 的字符串对数

所有字符串长度之和

每个字符串长度为 `L` 时，会给其路径上的 `L` 个非根节点各贡献一次 `pass`
因此：

$$\boxed{\sum_i |S_i| = \sum_{u \neq 0} pass[u]}$$

代码：

```
i64 sum_len{ 0 };
for (int u = 1; u < tr.size(); ++ u) {
    sum_len += tr.t[u].pass;
}
```

前缀关系字符串对计数

严格前缀

在节点 `u` 结束的字符串有：

```
cnt[u]
```

个
在其子树中但不恰好结束于 `u` 的字符串有：

```
pass[u] - cnt[u]
```

个
所以一个字符串是另一个字符串的严格前缀的无序字符串对数为：

$$\boxed{\sum_u cnt[u]\Big(pass[u] - cnt[u]\Big)}$$

代码：

```
i64 ans{ 0 };
for (int u = 0; u < tr.size(); ++ u) {
    ans += 1LL * tr.t[u].cnt * (tr.t[u].pass - tr.t[u].cnt);
}
```

允许两个字符串相等

若把相同字符串也视作互为前缀，则再加：

$$\boxed{\sum_u \binom{cnt[u]}{2}}$$

不同前缀数量

如果从未删除字符串，每个非根 Trie 节点都对应一个实际存在的不同非空前缀，因此：

```
int distinct_prefixes = tr.size() - 1;
```

但是存在 `erase()` 时，死节点不会回收
此时：

- 历史创建过的不同非空前缀数：

```
tr.size() - 1
```

- 当前字符串集合实际拥有的不同非空前缀数：

```
int ans{ 0 };
for (int u = 1; u < tr.size(); ++ u) {
    ans += tr.t[u].pass > 0;
}
```

不同字符串数量

当前 Trie 中不同字符串的数量为：

```
int distinct{ 0 };
for (int u = 0; u < tr.size(); ++ u) {
    distinct += tr.t[u].cnt > 0;
}
```

当前字符串总数则直接为：

```
tr.total();
```

区别在于重复字符串只为 `distinct` 贡献一次，但会按重复次数计入 `total()`

最短唯一前缀

如果每个字符串只插入一次，则字符串 `s` 的路径上第一个满足：

```
tr.t[p].pass == 1
```

的节点，就是 `s` 的最短唯一前缀

```
int p{ 0 };
for (int i = 0; i < s.size(); ++ i) {
    p = tr.step(p, s[i]);
    if (tr.t[p].pass == 1) {
        // s[0..i] 是最短唯一前缀
        break;
    }
}
```

若允许重复字符串，则 `pass` 按多重集合计数：

- 同一字符串出现多次时，不存在能够区分这些副本的唯一前缀
- 若题目要求区分“不同字符串”而非字符串副本，需要额外维护去重后的通过数量

字典序遍历

Trie 中：

1. 当前节点对应字符串本身
2. 再依次遍历字符 `0,1,...,A-1` 的儿子
   就是字符串字典序

```
auto dfs = Y([&](auto self, int u) -> void {
    for (int i = 0; i < tr.t[u].cnt; ++ i) {
        // 当前前缀出现一次
    }
    for (int c = 0; c < A; ++ c) {
        int v{ tr.t[u].next[c] };
        if (!v || !tr.t[v].pass) continue;
        // 当前字符串末尾加入字符 c
        self(v);
        // 回退字符 c
    }
});
dfs(0);
```

必须先处理 `cnt[u]`，再进入儿子，因为：

```
"a" < "aa" < "ab"
```

字典序第 k 个字符串

`pass[u]` 恰好是 Trie 子树中的字符串多重集合大小
因此可以逐层跳过整个字符子树
假设 `k` 为 `1-indexed`：

```
int u{ 0 };
while (true) {
    if (k ⩽ tr.t[u].cnt) {
        // 当前前缀就是答案
        break;
    }
    k -= tr.t[u].cnt;
    for (int c = 0; c < A; ++ c) {
        int v{ tr.t[u].next[c] };
        if (!v || !tr.t[v].pass) continue;
        if (k ⩽ tr.t[v].pass) {
            // 答案位于字符 c 的子树
            // 将 c 加入答案
            u = v;
            break;
        }
        k -= tr.t[v].pass;
    }
}
```

要求：

```
1 ⩽ k && k ⩽ tr.total()
```

复杂度为：

$$O(LA)$$

其中 `L` 为答案长度
当 `A` 是固定小常数时通常记作 $O(L)$
重复字符串按照 `cnt[u]` 重复出现在字典序序列中

Trie 上树形 DP

Trie 的每个节点就是一个前缀，因此很多字符串集合问题可以直接变成树形 DP：

```
auto dfs = Y([&](auto self, int u) -> void {
    for (int c = 0; c < A; ++ c) {
        int v{ tr.t[u].next[c] };
        if (!v || !tr.t[v].pass) continue;
        self(v);
        // 用 v 的 DP 信息更新 u
    }
});
```

常见建模包括：

- 选择若干字符串，使公共前缀代价最优
- 对同一前缀下的字符串分组
- Trie 子树统计
- 前缀贡献合并
- 字符串集合上的背包 / 匹配
- 按 LCP 层次进行贪心
  核心对应关系是：

```
Trie 节点     ⟷ 一个前缀
Trie 子树     ⟷ 以该前缀开头的所有字符串
祖先关系      ⟷ 前缀关系
LCA 深度      ⟷ LCP
pass[u]       ⟷ 当前子树中的字符串多重集合大小
cnt[u]        ⟷ 恰好等于当前前缀的字符串数量
```

复杂度

设：

- S：历史创建的 Trie 节点数
- A：字符集大小

操作	时间复杂度
`add(s)` / `find(s)` / `count(s)` / `contains(s)` / `prefix_count(s)` / `count_prefixes(s)` / `erase(s)` / `match_prefixes(s,f)`	$O(\|S\|)$
`step(p,c)`	$O(1)$
Trie 全树遍历	$O(SA)$
字典序第 `k` 个	$O(LA)$

空间复杂度：

$$O(SA)$$

当 `A` 为固定小常数时：

- 插入 / 查询一个长度为 n 的字符串：$O(n)$
- 全树遍历：$O(S)$
- 空间：$O(S)$
  严格来说，每次创建新节点都需要初始化一个长度为 `A` 的 `array`，因此若不把 `A` 视为常数，插入的最坏复杂度为：

$$O(|s|A)$$

  字符集很大且转移非常稀疏时，应考虑：
- 先离散化字符集
- 每节点使用稀疏边结构
  而不是直接增大 `A`

7.4 AC 自动机

```
template<int A = 26, int offset = 'a'>
struct AC {
    struct Node {
        int len{ 0 }, link{ 0 };
        array<int, A> next{};
    };
    vector<Node> t;
    vector<int> ord;
    explicit AC(int m = 0) : t(2) {
        t.reserve(m + 2);
        ord.reserve(m + 2);
        t[0].next.fill(1);
        t[0].len = -1;
    }
    // ===== 基础函数：插入与 build() 依赖，不建议删除 =====
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    int new_node() {
        t.emplace_back();
        return int(t.size()) - 1;
    }
    // ===== Trie 构建：所有模式串必须在 build() 前插入 =====
    template<class Seg>
    int add(const Seg &s) {
        int p{ 1 };
        for (const auto &c : s) {
            int x{ id(c) };
            if (!t[p].next[x]) {
                t[p].next[x] = new_node();
                t[t[p].next[x]].len = t[p].len + 1;
            }
            p = t[p].next[x];
        }
        return p;
    }
    // ===== AC 构建：计算 Fail 并补全所有自动机转移 =====
    // ord 为 Fail 父亲在儿子之前的拓扑序，后续统计与 DP 常用
    void build() {
```

```
        queue<int> q;
        q.push(1);
        ord.clear();
        while (!q.empty()) {
            int u{ q.front() };
            q.pop();
            ord.push_back(u);
            for (int c = 0; c < A; ++ c) {
                int &v = t[u].next[c];
                if (!v) {
                    v = t[t[u].link].next[c];
                } else {
                    t[v].link = t[t[u].link].next[c];
                    q.push(v);
                }
            }
        }
    }
    // ===== 状态接口：手写匹配与自动机 DP 使用 =====
    int next(int p, int x) const { return t[p].next[x]; }
    template<class T>
    int next(int p, const T &c) const { return next(p, id(c)); }
    int link(int p) const { return t[p].link; }
    int len(int p) const { return t[p].len; }
    int size() const { return int(t.size()); }
    template<class Seg, class F>
    void scan(const Seg &s, F f) const {
        int p{ 1 };
        for (const auto &c : s) {
            p = next(p, c);
            f(p);
        }
    }
    // ===== Fail 信息传播：统计、权值、禁用状态与状压 DP 共用 =====
    // propagate: Fail 父亲 → 儿子；f(a[u], a[link[u]])
    template<class T, class F>
    void propagate(vector<T> &a, F f) const {
        for (int i = 1; i < int(ord.size()); ++ i) {
            int u{ ord[i] };
            f(a[u], a[t[u].link]);
        }
    }
    // aggregate: 儿子 → Fail 父亲；f(a[link[u]], a[u])
    template<class T, class F>
    void aggregate(vector<T> &a, F f) const {
        for (int i = int(ord.size()) - 1; i > 0; -- i) {
            int u{ ord[i] };
            f(a[t[u].link], a[u]);
        }
    }
    template<class T>
    void propagate_sum(vector<T> &a) const {
        propagate(a, [](T &x, const T &y) { x += y; });
    }
    template<class T>
    void aggregate_sum(vector<T> &a) const {
        aggregate(a, [](T &x, const T &y) { x += y; });
    }
    // ===== 常用统计：统计层不占用 AC 的成员空间，可按需删除 =====
    // score[u] 表示到达状态 u 时当前位置产生的贡献
    template<class Seg, class T>
    T query(const Seg &s, const vector<T> &score) const {
        T ans{};
        scan(s, [&](int p) {
            ans += score[p];
        });
        return ans;
    }
    // 返回每个状态对应 Trie 字符串在文本中的出现次数
    template<class Seg>
    vector<i64> count_states(const Seg &s) const {
```

```
        vector<i64> cnt(size());
        scan(s, [&](int p) { ++ cnt[p]; });
        aggregate_sum(cnt);
        return cnt;
    }
    // ===== Fail 树：若不需要树上问题，可直接删除 =====
    vector<vector<int>> fail_tree() const {
        vector<vector<int>> g(size());
        for (int u = 2; u < size(); ++ u) {
            g[t[u].link].push_back(u);
        }
        return g;
    }
};
```

成员变量

- `0`：哨兵状态，不属于实际 AC 自动机
- `1`：根节点，表示空串
- `t[u].next[c]`：
  - `build()` 前表示 Trie 中字符编号 `c` 的儿子，`0` 表示不存在
  - `build()` 后表示 AC 自动机沿字符编号 `c` 的完整转移
- `t[u].link`：状态 `u` 的 Fail 指针
- `t[u].len`：状态 `u` 对应 Trie 字符串的长度
- `ord`：`build()` 得到的 BFS 序，同时满足 Fail 父亲一定在儿子之前

核心接口

- `AC(m)`：构造空自动机，`m` 为预计所有模式串的总长度，仅用于预留空间
- `add(s)`：插入模式串 `s`，返回其终止状态，$O(|s|)$
- `build()`：构造 Fail 指针并补全自动机转移，$O(SA)$
- `next(p,x)`：从状态 `p` 沿字符编号 `x` 转移，$O(1)$
- `next(p,c)`：从状态 `p` 沿字符 `c` 转移，$O(1)$
- `link(p)`：返回状态 `p` 的 Fail 状态，$O(1)$
- `len(p)`：返回状态对应字符串长度，$O(1)$
- `size()`：返回状态数，包含哨兵 `0` 与根 `1`，$O(1)$
- `scan(s,f)`：扫描文本 `s`，每到达一个状态调用一次 `f(state)`，$O(|s|)$

Fail 信息传播

- `propagate(a,f)`：按照 `ord` 正序执行 `f(a[u],a[link[u]])`，即 Fail 父亲向儿子传播，$O(S)$
- `aggregate(a,f)`：按照 `ord` 逆序执行 `f(a[link[u]],a[u])`，即儿子向 Fail 父亲汇总，$O(S)$
- `propagate_sum(a)`：执行 `a[u] += a[link[u]]`，$O(S)$
- `aggregate_sum(a)`：执行 `a[link[u]] += a[u]`，$O(S)$

`propagate()` / `aggregate()` 不限定运算必须是加法，例如：

```
ac.propagate(mask, [](auto &x, const auto &y) {
    x |= y;
});
```

即可传播模式集合

统计接口

- `query(s,score)`：扫描文本，对每个到达状态 `u` 累加 `score[u]`，$O(|s|)$
- `count_states(s)`：返回每个状态对应 Trie 字符串在文本中的出现次数，$O(|s|+S)$
- `fail_tree()`：构造 Fail 树邻接表，$O(S)$

其中 S 为自动机状态数

注意事项

- 所有 `add()` 必须发生在 `build()` 之前
- `build()` 会把缺失 Trie 边改成完整自动机转移，因此调用后不能继续插入模式串，也不能再用 `next == 0` 判断 Trie 边是否存在
- 当前实现按“一次构建”设计，同一个对象不要重复调用 `build()`
- `add()` 返回的终止状态在 `build()` 前后保持不变，应保存下来用于后续统计
- `ord[0] = 1`，即根节点；逆序上传时应跳过根，因此 `aggregate()` 内部只处理 `ord[1..]`
- 默认处理小写字母；大写字母使用 `AC<26, 'A'>`
- 已经映射到 `[0,A)` 的整数序列使用 `AC<A, 0>`
- 必须保证所有字符满足 `0 ≤ id(c) < A`

- 当前采用稠密转移表，空间为 $O(SA)$；字符集很大且转移稀疏时不适合直接开很大的 `A`
- 默认模式串非空；空模式串的出现次数通常定义为 $|text|+1$，需要单独处理
- 原先的 `val / sum` 已从 `Node` 中移除；模式串终止状态、权值、禁用标记、状态贡献等全部放在外部数组中
- 原来的 `query(text)` 功能仍然存在，但现在写作 `query(text, score)`；这是统计层与自动机核心分离后的通用形式

常见用法

所有模式串出现总次数

保存每个模式串的终止状态：

```
AC<> ac(sum_len);
vector<int> pos;
for (auto &s : pattern) {
    pos.push_back(ac.add(s));
}
ac.build();
```

令每个模式串权值为 `1`：

```
vector<i64> score(ac.size());
for (int p : pos) {
    ++ score[p];
}
ac.propagate_sum(score);
```

此时 `score[u]` 表示：

> 到达状态 `u` 时，当前位置能够结束的所有模式串数量

因此：

```
cout << ac.query(text, score) << '\n';
```

即为所有模式串在文本中的出现次数总和
重复插入相同模式串时，终止状态相同，但 `score` 会被重复增加，因此仍会按照模式串的多重集合统计

统计每个模式串出现次数

```
AC<> ac(sum_len);
vector<int> pos(n);
for (int i = 0; i < n; ++ i) {
    pos[i] = ac.add(pattern[i]);
}
ac.build();
auto cnt = ac.count_states(text);
for (int i = 0; i < n; ++ i) {
    cout << cnt[pos[i]] << '\n';
}
```

例如模式串为 `a`、`aa`，文本为 `aaa`，对应答案分别为 `3`、`2`
`count_states()` 内部等价于：

```
vector<i64> cnt(ac.size());
ac.scan(text, [&](int p) {
    ++ cnt[p];
});
ac.aggregate_sum(cnt);
```

其本质为：

$$occ[u] = \sum_{v \in subtree_{fail}(u)} visit[v]$$

也就是说：

AC 出现次数统计本质上是 Fail 树子树求和

重复模式串对应同一个终止状态，因此自然得到相同的出现次数

扫描文本 / 手写自动机 DP

最简单写法：

```
int p{ 1 };
for (char c : text) {
    p = ac.next(p, c);
    // 当前文本前缀对应的 AC 状态为 p
}
```

也可以直接使用：

```
ac.scan(text, [&](int p) {
    // 当前状态为 p
});
```

扫描长度为 n 的文本总复杂度为 $O(n)$

构造 Fail 树

直接：

```
auto g = ac.fail_tree();
```

或者手写：

```
vector<vector<int>> g(ac.size());
for (int u = 2; u < ac.size(); ++ u) {
    g[ac.link(u)].push_back(u);
}
```

根节点为 `1`
Fail 树常用于：

- Fail 子树统计
- 祖先 / 后代关系
- Euler 序
- 树状数组 / 线段树维护
- LCA 类后缀关系问题

Fail 的核心定理

设状态 `u` 对应 Trie 字符串为 S_u
则：

$$\boxed{link[u] = S_u \text{ 的最长真后缀中，同时也是某个 Trie 前缀的状态}}$$

不断执行：

```
u
link[u]
link[link[u]]
...
1
```

即可枚举 S_u 的所有同时也是 Trie 前缀的后缀

Fail 树祖先与后缀

将：

```
link[u] → u
```

作为树边，则：

$$\boxed{u \text{ 是 } v \text{ 的 Fail 树祖先} \iff S_u \text{ 是 } S_v \text{ 的后缀}}$$

扫描文本后当前自动机状态为 `v` 时：

$$\boxed{\text{模式状态 } u \text{ 在当前位置结尾出现} \iff u \text{ 是 } v \text{ 的 Fail 树祖先}}$$

这个结论是出现次数统计、动态启停模式串和 Fail 树建模的核心

`ord` 的方向

任意非根状态都满足：

$$len[link[u]] < len[u]$$

因此 `build()` 得到的 BFS 序满足：

> Fail 父亲一定出现在儿子之前

正序：

```
for (int i = 1; i < int(ac.ord.size()); ++ i) {
    int u{ ac.ord[i] };
    // ac.link(u) → u
}
```

适合从 Fail 祖先向后继传播：

- 模式串权值
- 禁止状态
- 模式集合状压
- 其他继承型信息

即 `propagate()`

逆序：

```
for (int i = int(ac.ord.size()) - 1; i > 0; -- i) {
    int u{ ac.ord[i] };
    // u → ac.link(u)
}
```

适合沿 Fail 向上汇总：

- 模式串出现次数
- Fail 子树贡献
- 文本访问次数

即 `aggregate()`

顺序	典型用途
`ord` 正序	`sum[u] += sum[link[u]]`、传播 `bad` / `mask`
`ord` 逆序	`cnt[link[u]] += cnt[u]`、出现次数汇总

统计所有模式串的带权匹配总和

设模式串 `i` 的权值为 `w[i]`：

```
AC<> ac(sum_len);
vector<int> pos(n);
for (int i = 0; i < n; ++ i) {
    pos[i] = ac.add(pattern[i]);
}
ac.build();
vector<i64> score(ac.size());
for (int i = 0; i < n; ++ i) {
    score[pos[i]] += w[i];
}
```

沿 Fail 正序继承后缀模式的权值：

```
ac.propagate_sum(score);
```

此时：

```
score[u]
```

表示：

> 到达状态 `u` 时，当前位置能够结束的所有模式串权值之和

因此：

```
i64 ans{ ac.query(text, score) };
```

即为文本中所有模式匹配的总权值
当所有 `w[i] = 1` 时，就退化为所有模式串出现总次数

判断是否已经匹配禁止模式串

保存所有禁止模式串的终止状态：

```
vector<int> terminal;
for (auto &s : pattern) {
    terminal.push_back(ac.add(s));
}
ac.build();
```

标记终点：

```
vector<char> bad(ac.size());
for (int p : terminal) {
    bad[p] = true;
}
```

因为 Fail 祖先代表当前字符串的后缀，因此需要沿 Fail 正序传播：

```
ac.propagate(bad, [](auto &x, const auto &y) {
    x |= y;
});
```

此时：

```
bad[u]
```

表示到达状态 `u` 时，当前字符串后缀中是否已经出现至少一个禁止模式串

AC + 自动机 DP：禁止出现某些模式串

定义：

```
dp[i][u]
```

表示已经构造长度为 `i` 的字符串，且当前处于状态 `u`
转移：

```
for (int u = 1; u < ac.size(); ++ u) if (!bad[u]) {
    for (int c = 0; c < A; ++ c) {
        int v{ ac.next(u, c) };
        if (bad[v]) continue;
        dp[i + 1][v] += dp[i][u];
    }
}
```

即可求：

> 长度为 n，且不包含任意禁止模式串的字符串数量

复杂度为：

$$O(nSA)$$

若 n 极大，则状态转移本质为固定矩阵：

$$\boxed{\text{AC 自动机} + \text{矩阵快速幂}}$$

AC + 权值 DP

先构造每个状态的当前位置匹配权值：

```
vector<i64> score(ac.size());
for (int i = 0; i < n; ++ i) {
    score[pos[i]] += w[i];
```

```
}
ac.propagate_sum(score);
```

从状态 `u` 加字符 `c` 后：

```
int v{ ac.next(u, c) };
```

当前新产生的所有模式匹配贡献正好为：

```
score[v]
```

因此例如最大化长度为 `n` 的字符串的匹配权值：

```
ndp[v] = max(ndp[v], dp[u] + score[v]);
```

可用于：

- 最大 / 最小模式匹配权值
- 固定长度字符串最优构造
- AC + Max-Plus 矩阵快速幂
  若 n 极大：

AC + Max-Plus 矩阵快速幂

AC + 状压：记录匹配过的模式集合

模式串数量较小时，例如 $m \le 20$：

```
vector<int> mask(ac.size());
for (int i = 0; i < m; ++ i) {
    mask[pos[i]] |= 1 << i;
}
```

沿 Fail 正序传播：

```
ac.propagate(mask, [](auto &x, const auto &y) {
    x |= y;
});
```

此时：

```
mask[u]
```

表示到达状态 `u` 时，在当前位置结束的所有模式串集合
之后可以在：

```
(state, mask)
```

上进行 BFS / DP
典型问题：

- 最短包含全部给定模式串的字符串
- 长度为 n 时至少出现指定模式集合
- 匹配模式集合满足某条件的字符串计数
- 出现至少 / 恰好 k 种模式串

AC + 数位 DP

若模式串由数字组成，例如禁止：

```
13
666
2026
```

建立：

```
AC<10, '0'> ac;
```

将禁止状态按前面的方式构造后，把 AC 状态直接加入数位 DP：

```
dp[pos][state][tight][lead]
```

即可处理：

- `[0,N]` 中不包含禁止数字串的整数数量
- 包含指定数字模式的整数数量
- 数字模式出现次数或权值统计
  即：

$$\boxed{\text{AC 状态可以直接作为数位 DP 的一维状态}}$$

注意 `lead` 是否应参与 AC 转移取决于题目对前导零的定义

动态启用 / 禁用模式串：Fail 树 + BIT

假设所有模式串固定，但会动态：

```
activate(i)
deactivate(i)
```

并询问文本中当前启用模式串的匹配贡献
首先构造 Fail 树，并求 Euler 序：

```
tin[u], tout[u]
```

使 Fail 子树对应：

```
[tin[u], tout[u])
```

根据：

$$u \text{ 在当前状态 } p \text{ 匹配} \iff u \text{ 是 } p \text{ 的 Fail 树祖先}$$

激活模式终止状态 `u`、权值 `w`，等价于：

```
range_add(tin[u], tout[u], +w);
```

禁用：

```
range_add(tin[u], tout[u], -w);
```

扫描文本：

```
int p{ 1 };
i64 ans{ 0 };
for (char c : text) {
    p = ac.next(p, c);
    ans += point_query(tin[p]);
}
```

因为给 `u` 的整个 Fail 子树加 `w`，正好意味着：

> 所有以 `u` 为 Fail 祖先的状态都会获得该模式串贡献

因此：

$$\boxed{\text{动态模式匹配} = \text{AC} + \text{Fail 树 Euler 序} + \text{BIT 区间加单点查}}$$

如果 BIT 使用差分形式，则对子树区间 `[l,r)` 加 `w` 可以写成：

```
bit.add(l, w);
bit.add(r, -w);
```

查询状态 `p` 时取 `tin[p]` 的前缀和即可

复杂度

设：

- M：所有模式串总长度
- S：自动机状态数，$S \le M+2$
- A：字符集大小
- L：文本长度

操作	时间复杂度
插入全部模式串	$O(M)$
`build()`	$O(SA)$
单次 `next()`	$O(1)$
`scan(text)` / `query(text,score)`	$O(L)$
`propagate()` / `aggregate()` / `fail_tree()`	$O(S)$
`count_states(text)`	$O(L+S)$
禁止串普通自动机 DP	$O(nSA)$

空间复杂度为：

$$O(SA)$$

当字符集大小 A 为固定常数，例如小写字母的 `26` 时，通常简记为：

- 建立自动机：$O(M)$
- 扫描文本：$O(L)$
- 空间：$O(M)$

7.5 子序列自动机

```
template<int A = 26, int offset = 'a'>
struct SubseqAutomaton {
    int n{ 0 };
    vector<array<int, A>> next;
    SubseqAutomaton() : next(1) { next[0].fill(0); }
    template<class Seg>
    explicit SubseqAutomaton(const Seg &s) { build(s); }
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    template<class Seg>
    void build(const Seg &s) {
        n = s.size();
        next.resize(n + 1);
        next[n].fill(n);
        for (int i = n - 1; i >= 0; --i) {
            next[i] = next[i + 1];
            next[i][id(s[i])] = i;
        }
    }
    // 从原串位置 p 开始，返回字符 c 第一次出现的位置；
    // 不存在时返回 n。
    template<class T>
    int step(int p, const T &c) const { return next[p][id(c)]; }
    // 从原串位置 p 开始匹配 s。
    // 成功返回最后一个匹配位置的后一位，失败返回 -1。
    template<class Seg>
    int match(const Seg &s, int p = 0) const {
        for (const auto &c : s) {
            if (p >= n) return -1;
            int q{ step(p, c) };
            if (q == n) return -1;
            p = q + 1;
        }
        return p;
    }
    template<class Seg>
    bool contains(const Seg &s) const { return match(s) != -1; }
};
```

成员变量

- `n`：原串长度
- `next[p][c]`：原串中下标不小于 `p` 的位置里，字符 `c` 第一次出现的位置；不存在时为 `n`
- 状态 `p` 表示：**下一次允许从原串位置 `p` 开始寻找**

接口

- `SubseqAutomaton(s)`：直接对原串 `s` 建表，$O(nA)$
- `build(s)`：重新对原串 `s` 建表，$O(nA)$
- `step(p,c)`：查询从位置 `p` 开始字符 `c` 的第一次出现位置，$O(1)$
- `match(t,p)`：判断 `t` 能否作为原串后缀区间 `[p,n)` 的子序列；成功返回最后一次匹配位置的后一位，失败返回 `-1`，$O(|t|)$
- `contains(t)`：判断 `t` 是否为整个原串的子序列，$O(|t|)$

注意事项

- 这里最重要的是状态含义：`p` 表示**下一次搜索的起始下标**，不是“当前已经匹配到的位置”
- `step(p,c)` 返回的是字符实际所在下标 `q`；继续匹配时应令 `p = q + 1`
- `n` 同时作为“不存在”的哨兵值
- `p` 合法范围为 `[0,n]`
- 默认小写字母；整数编号序列使用 `SubseqAutomaton<A, 0>`
- 必须保证 `0 ≤ id(c) < A`
- 构建需要复制长度为 `A` 的整张下一位置表，因此更适合固定小字符集
- 如果字符集非常大而查询很少，更适合为每种字符保存出现位置数组后用 `lower_bound`

常见用法

判断是否为子序列

```
SubseqAutomaton<> sa(s);
if (sa.contains(t)) {
    // t 是 s 的子序列
}
```

求最早匹配结束位置

```
int r{ sa.match(t) };
if (r != -1) {
    // 可以在 s[0..r) 中匹配出 t
    // 且 r 最小
}
```

由于每次都贪心选择当前字符最靠前的位置，因此得到的 `r` 是最早可能结束的位置

从指定位置之后继续匹配

```
int r{ sa.match(t, l) };
if (r != -1) {
    // t 是 s[l..n) 的子序列
    // 匹配实际落在区间 [l, r) 内
}
```

手动逐字符转移

```
int p{ 0 };
for (char c : t) {
    int q{ sa.step(p, c) };
    if (q == sa.n) {
        // 无法继续匹配
        break;
    }
    p = q + 1;
}
```

贪心定理

假设：

```
sa.match(t, l)
```

每个字符都取当前能选到的**最靠左位置**
那么：

$$\boxed{\text{它得到的是所有 } t \text{ 的匹配方案中，结束位置最早的方案}}$$

这是因为局部把某字符选得更靠左，永远不会损害后续字符的可选空间

可以 $O(|t|)$ 判断区间子序列

要判断：

> `t` 是否是 `s[l,r)` 的子序列

只需要：

```
int p = sa.match(t, l);
if (p != -1 && p <= r) {
    // 是子序列
}
```

原因：
`match()` 已经给出了**最早结束位置**
所以：

$$\boxed{t \subseteq s[l,r) \iff match(t,l) \le r}$$

正反两个子序列自动机

再对反串建立一个自动机，或者建立：

```
prev[p][c]
```

即可得到：

- 从左向右：最早结束位置
- 从右向左：最晚开始位置

> 某子序列最紧覆盖区间
> 删除一段后还能否匹配
> 前后两部分分别匹配

都可以直接做

统计不同子序列数

设状态 `p` 表示：

> 下一次从 `s[p..n)` 中选字符

定义：

$$f[p] = s[p..n) \text{ 的不同子序列数量，包含空串}$$

对于每个字符 `c`：

```
q = next[p][c];
```

若 `q < n`，所有以 `c` 开头的不同子序列，与：

```
s[q+1..]
```

的任意不同子序列一一对应
因此：

$$\boxed{f[p] = 1 + \sum_{\substack{c \\ next[p][c] < n}} f[next[p][c] + 1]}$$

边界：

$$f[n] = 1$$

最终：

```
f[0] - 1
```

就是不同非空子序列数量
这里不会重复计数，因为不同首字符天然属于不同集合，而同一首字符永远固定选择它的第一次可用出现位置

第 k 个字典序不同子序列

有了上面的 `f[p]` 就很自然
对于状态 `p`，按字符：

```
'a', 'b', ...
```

枚举
若：

```
(q = next[p][c]) < n
```

那么以字符 `c` 开头的不同子序列总数为：

$$f[q+1]$$

于是可以跳过整个字典序块
而选中字符 `c` 后：

```
c
```

本身是这一块里的第一个字符串，然后才是：

```
c + 非空子序列
```

因此可以 $O(AL)$ 求第 `k` 个不同子序列，其中 `L` 是答案长度

最短不存在子序列

定义：

$$g[p] = \text{不是 } s[p..n) \text{ 子序列的最短字符串长度}$$

若存在字符 `c`：

```
next[p][c] == n
```

那么直接：

$$g[p] = 1$$

否则：

$$\boxed{g[p] = 1 + \min_c g[next[p][c]+1]}$$

边界：

$$g[n] = 1$$

记录最优字符还能恢复答案
如果最小长度相同优先选较小字符，就直接得到：

> 字典序最小的最短不存在子序列

大字符集时应该换实现

目前：

$$\text{build} = O(nA), \qquad \text{query} = O(m)$$

非常适合：

```
A = 2
A = 26
A = 52
```

但如果字符值域很大，例如整数序列，`A=10^9` 肯定不能这么建
这时应该用：

```
vector<vector<int>> pos;
```

为每种字符保存出现位置，然后每次：

```
ranges::lower_bound(pos[c], current)
```

复杂度变成大约：

$$O(n)$$

预处理，以及单次查询：

$$O(m \log n)$$

场景	方法
小固定字符集、大量查询	子序列自动机
大字符集、查询不太多	出现位置数组 + `lower_bound`
只有一个查询	双指针即可

复杂度

原串长度为 n，查询串长度为 m，字符集大小为 A

操作	时间复杂度
`build(s)`	$O(nA)$
`step(p,c)`	$O(1)$
`match(t)` / `contains(t)`	$O(m)$

空间复杂度为 $O(nA)$

7.6 Manacher

```
struct Manacher {
    int n;
    vector<int> r;
    template<class Seg>
    explicit Manacher(const Seg &s) : n(s.size()) {
        int m{ 2 * n + 1 };
        r.assign(m, 0);
        auto equal = [&](int x, int y) {
            if ((x & 1) != (y & 1)) return false;
            return !(x & 1) || s[x / 2] == s[y / 2];
        };
        for (int i = 0, p = 0; i < m; ++ i) {
            if (i < p + r[p]) r[i] = min(r[2 * p - i], p + r[p] - i);
            while (i - r[i] >= 0 && i + r[i] < m && equal(i - r[i], i + r[i])) ++ r[i];
            if (i + r[i] > p + r[p]) p = i;
        }
    }
    // ===== 全部只依赖 r，可按需删除 =====
    // 以虚拟位置 c 为中心的最长回文子串长度
    int longest_at(int c) const { return r[c] - 1; }
    // 以虚拟位置 c 为中心的回文子串个数
    int count_at(int c) const { return r[c] / 2; }
    int longest() const { return *ranges::max_element(r) - 1; }
    pair<int, int> longest_segment() const {
        int best{ 0 }, l{ 0 };
        for (int c = 0; c < r.size(); ++ c) {
            int len{ longest_at(c) };
            if (len > best) {
                best = len;
                l = (c - len) / 2;
            }
        }
        return { l, l + best };
    }
    i64 count() const {
```

```
        i64 ans{ 0 };
        for (int c = 0; c < r.size(); ++ c) ans += count_at(c);
        return ans;
    }
    bool is_pal(int l, int rr) const {
        if (l < 0 || l > rr || rr > n) return false;
        return r[l + rr] > rr - l;
    }
    // ===== 全部枚举：输出规模可能达到 O(n^2)，不需要时可删除 =====
    vector<pair<int, int>> enumerate() const {
        vector<pair<int, int>> res;
        for (int c = 0; c < r.size(); ++ c) {
            for (int len = (c & 1 ? 1 : 2); len < r[c]; len += 2) {
                int l{ (c - len) / 2 };
                res.push_back({ l, l + len });
            }
        }
        return res;
    }
};
```

成员变量

- `n`：原串长度
- `r[c]`：虚拟串中以 `c` 为中心的回文半径
- 虚拟串为 `#s[0]#s[1]#...#s[n-1]#`
- 虚拟位置 `2i + 1` 对应原串字符 `s[i]`
- 虚拟偶数位置对应字符之间的空隙

接口

- `Manacher(s)`：预处理所有回文中心，$O(n)$
- `longest_at(c)`：以虚拟中心 `c` 为中心的最长回文子串长度，$O(1)$
- `count_at(c)`：以虚拟中心 `c` 为中心的回文子串个数，$O(1)$
- `longest()`：最长回文子串长度，$O(n)$
- `longest_segment()`：返回一个最长回文子串区间 `[l,r)`，$O(n)$
- `count()`：回文子串出现次数总和，$O(n)$
- `is_pal(l,r)`：判断 `[l,r)` 是否为回文，$O(1)$
- `enumerate()`：枚举所有回文子串区间 `[l,r)`，$O(K)$，其中 K 为回文子串出现次数

注意事项

- 所有原串区间均为 `0base` 的 `[l,r)`
- 实现没有真的向原串插入特殊字符，因此 `Seg` 只要求支持 `size()`、`operator[]` 和元素相等比较
- `r[c]` 是虚拟串半径，不是原串回文长度
- `longest_at(c) = r[c] - 1`
- `count_at(c) = r[c] / 2`
- `count()` 统计的是**回文子串出现次数**，不是本质不同回文串数量；后者应使用 PAM
- `enumerate()` 最坏会产生 $\Theta(n^2)$ 个区间，因此不可能保证线性复杂度
- 空串 `[l,l)` 会被 `is_pal()` 判为回文

常见用法

最长回文子串

```
Manacher man(s);
int len{ man.longest() };
auto [l, r] = man.longest_segment();
cout << s.substr(l, r - l) << '\n';
```

$O(1)$ 判断任意区间是否为回文

对于原串区间 `[l,r)`：

$$c = l + r, \quad len = r - l$$

它是回文当且仅当：

$$\boxed{r[c] > len}$$

因此：

```
if (man.is_pal(l, r)) {
    // s[l..r) 是回文串
}
```

这个接口特别适合：

- 区间 DP
- 回文分割
- 枚举切分点后快速判断两侧是否回文
- 与哈希 / DP 配合

一个中心对应哪些回文串

对于虚拟中心 `c`，最长回文长度为：

```
int mx{ man.r[c] - 1 };
```

该中心下所有回文长度依次为：

- `c` 为奇数：`1, 3, 5, ..., mx`
- `c` 为偶数：`2, 4, 6, ..., mx`
  所以一个中心贡献的回文子串个数为：

$$\boxed{\left\lfloor \frac{r[c]}{2} \right\rfloor}$$

进一步：

$$\boxed{\text{所有回文子串出现次数} = \sum_{c} \left\lfloor \frac{r[c]}{2} \right\rfloor}$$

这就是 `count()`

统计长度恰好为 L 的回文子串

长度为 `L` 的区间 `[l,l+L)` 的虚拟中心为：

$$c = 2l + L$$

因此长度恰好为 `L` 的回文子串数量，就是满足：

$$c \equiv L \pmod 2, \qquad r[c] > L$$

的中心数量
直接扫描：

```
int ans{ 0 };
for (int c = L & 1; c < man.r.size(); c += 2) {
    ans += man.r[c] > L;
}
```

若要回答大量不同长度的查询，可以进一步离线统计半径

最长回文前缀 / 后缀

预处理后直接枚举端点：

```
int pre{ 0 };
for (int r = 1; r ≤ s.size(); ++ r) {
    if (man.is_pal(0, r)) pre = r;
}
int suf{ int(s.size()) };
for (int l = s.size(); l ≥ 0; -- l) {
    if (man.is_pal(l, s.size())) suf = l;
}
```

得到：

- 最长回文前缀：`s[0..pre)`
- 最长回文后缀：`s[suf..n)`

复杂度

操作	时间复杂度
`Manacher(s)` / `longest()` / `longest_segment()` / `count()`	$O(n)$
`longest_at(c)` / `count_at(c)` / `is_pal(l,r)`	$O(1)$
`enumerate()`	$O(K)$

其中 K 为所有回文子串出现次数，最坏为 $\Theta(n^2)$
空间复杂度：$O(n)$

7.7 PAM（回文树）

```
template<int A = 26, int offset = 'a'>
struct PAM {
    struct Node {
        int len{ 0 }, link{ 0 }, num{ 0 };
        i64 occ{ 0 };
        array<int, A> next{};
    };
    vector<Node> t;
    vector<int> s;
    int suff{ 1 };
    explicit PAM(int m = 0) : t(2) {
        t.reserve(m + 2);
        s.reserve(m);
        t[0].len = -1;
    }
    template<class Seg>
    requires ranges::sized_range<Seg>
    explicit PAM(const Seg &a) : PAM(int(a.size())) { build(a); }
    // ===== 基础函数：插入依赖，不建议删除 =====
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    int new_node(int len) {
        t.emplace_back();
        t.back().len = len;
        return int(t.size()) - 1;
    }
    // 找最长回文后缀 p，使其两侧可以再加入当前字符
    int get_link(int p) const {
        int i{ int(s.size()) - 1 };
        while (i - t[p].len - 1 < 0 || s[i - t[p].len - 1] != s[i]) {
            p = t[p].link;
        }
        return p;
    }
    // ===== 在线插入：每次最多新建一个本质不同回文串 =====
    bool add_id(int x) {
        s.push_back(x);
        int p{ get_link(suff) };
        bool created{ false };
        if (!t[p].next[x]) {
            int q{ new_node(t[p].len + 2) };
            if (t[q].len == 1) t[q].link = 1;
            else t[q].link = t[get_link(t[p].link)].next[x];
            t[q].num = t[t[q].link].num + 1;
            t[p].next[x] = q;
            created = true;
        }
        suff = t[p].next[x];
        ++ t[suff].occ;
        return created;
    }
    template<class T>
    bool add(const T &c) { return add_id(id(c)); }
    template<class Seg>
    void build(const Seg &a) { for (const auto &c : a) add(c); }
    // ===== 基础状态接口 =====
    int size() const { return int(t.size()); }
```

```
    int distinct() const { return int(t.size()) - 2; }
    int longest_suffix() const { return suff; }
    int longest_suffix_len() const { return t[suff].len; }
    int suffix_count() const { return t[suff].num; }
    // 枚举当前串的所有非空回文后缀
    template<class F>
    void suffixes(F f) const {
        for (int p = suff; p > 1; p = t[p].link) f(p);
    }
    // ===== 出现次数统计：不需要完整出现次数时可删除 =====
    vector<i64> count() const {
        vector<i64> res(t.size());
        for (int p = 2; p < t.size(); ++ p) res[p] = t[p].occ;
        for (int p = int(t.size()) - 1; p >= 2; -- p) res[t[p].link] += res[p];
        return res;
    }
    // ===== 后缀链接树：不需要树上问题时可删除 =====
    vector<vector<int>> link_tree() const {
        vector<vector<int>> g(t.size());
        for (int p = 2; p < t.size(); ++ p) g[t[p].link].push_back(p);
        return g;
    }
};
```

成员变量

- `t[0]`：奇根，`len = -1`
- `t[1]`：偶根，`len = 0`
- `t[p].len`：节点 `p` 对应回文串长度
- `t[p].link`：节点 `p` 的最长真回文后缀
- `t[p].next[c]`：由当前回文串两侧各加入字符 `c` 后得到的回文节点
- `t[p].num`：节点 `p` 的非空回文后缀数量
- `t[p].occ`：节点 `p` 被作为当前前缀最长回文后缀访问的次数；不是最终完整出现次数
- `suff`：当前整个字符串的最长回文后缀节点
- `s`：已经加入的字符编号序列
  真实回文串节点范围为：

```
[2, size())
```

接口

- `PAM(m)`：构造空 PAM，`m` 仅用于预留长度
- `PAM(s)`：直接构造字符串 `s` 的 PAM，$O(n)$
- `add(c)`：向末尾加入字符，返回是否新建了一个本质不同回文串，均摊 $O(1)$
- `build(s)`：依次插入整个字符串，$O(n)$
- `size()`：包括两个根的总节点数，$O(1)$
- `distinct()`：当前本质不同非空回文子串数量，$O(1)$
- `longest_suffix()`：当前最长回文后缀节点，$O(1)$
- `longest_suffix_len()`：当前最长回文后缀长度，$O(1)$
- `suffix_count()`：当前前缀的非空回文后缀个数，$O(1)$
- `suffixes(f)`：枚举当前所有非空回文后缀节点，$O(k)$
- `count()`：返回每个回文节点的完整出现次数，$O(S)$
- `link_tree()`：构造回文后缀链接树，$O(S)$

注意事项

- 必须保证字符满足 `0 ≤ id(c) < A`
- 默认小写字母；大写字母可用 `PAM<26, 'A'>`
- 已经映射到 `[0,A)` 的整数序列使用 `PAM<A, 0>`
- 每加入一个字符，最多产生一个新的本质不同回文串，因此长度为 n 的字符串最多只有 n 个本质不同非空回文子串
- `add(c) == true` 只表示这一步新建了一个本质不同回文串，并不表示该回文串只出现一次
- `t[p].occ` 不是最终完整出现次数；必须调用 `count()` 后读取结果
- 节点创建时 `link` 一定指向更早创建的节点，因此可以按节点编号逆序汇总出现次数
- 若只需要静态区间回文判断，应优先使用 Manacher；PAM 的优势是在线维护不同回文串及其结构
- `suffixes()` 最坏一次可能枚举 $O(n)$ 个回文后缀；大量回文 DP 若因此退化，应使用 Series PAM

常见用法

本质不同回文子串数量

```
PAM<> pam(s);
cout << pam.distinct() << '\n';
```

核心结论：

$$\boxed{\text{PAM 中每个真实节点恰好对应一个本质不同回文子串}}$$

因此：

$$\boxed{\#\text{distinct palindrome} = size() - 2 \leq n.}$$

在线统计每个前缀新增的本质回文串

```
PAM<> pam;
for (char c : s) {
    bool created{ pam.add(c) };
    if (created) {
        // 当前字符加入后恰好新增了一个本质不同回文串
    }
    cout << pam.distinct() << '\n';
}
```

每次加入一个字符时，所有新出现的回文必须是当前前缀的后缀，而其中只有最长的那个可能此前没有出现过，因此一次最多新建一个节点

当前前缀的最长回文后缀

```
pam.add(c);
int p{ pam.longest_suffix() };
int len{ pam.t[p].len };
```

因此 PAM 可以在线维护每个前缀的最长回文后缀

枚举当前所有回文后缀

不断沿 `link`：

```
suff
link[suff]
link[link[suff]]
...
1
```

恰好枚举当前字符串的所有非空回文后缀

```
pam.suffixes([&](int p) {
    int len{ pam.t[p].len };
    // 当前前缀存在一个长度为 len 的回文后缀
});
```

核心结论：

$$\boxed{link[p] = p \text{ 对应回文串的最长真回文后缀}}$$

因此整个 `link` 链就是所有回文后缀

当前前缀的回文后缀数量

构造节点时：

$$num[p] = num[link[p]] + 1$$

所以：

```
pam.add(c);
cout << pam.suffix_count() << '\n';
```

就是当前前缀的非空回文后缀数量

所有回文子串出现次数总和

每一个回文子串都有唯一的右端点
对于前缀 `s[0..i]`，以 `i` 结尾的回文子串数量正好等于：

```
pam.suffix_count()
```

因此：

```
PAM<> pam;
i64 ans{ 0 };
for (char c : s) {
    pam.add(c);
    ans += pam.suffix_count();
}
```

最终：

$$\boxed{ans = \text{字符串中所有回文子串的出现次数}}$$

这与 Manacher 的：

```
man.count()
```

得到的是同一个量，只是 PAM 可以在线计算

每个本质回文串的出现次数

```
PAM<> pam(s);
auto occ = pam.count();
for (int p = 2; p < pam.size(); ++ p) {
    cout << pam.t[p].len << ' ' << occ[p] << '\n';
}
```

为什么需要沿 `link` 逆序上传？
每个位置只给其**最长回文后缀**的 `occ` 加 `1`，但它的所有回文后缀也都在该位置出现。
因此：

$$\boxed{occ[link[p]] \mathrel{+}= occ[p]}$$

即可把所有出现贡献上传完整

求 $\max |P| \cdot occ(P)$

```
auto occ = pam.count();
i64 ans{ 0 };
for (int p = 2; p < pam.size(); ++ p) {
    ans = max(ans, 1LL * pam.t[p].len * occ[p]);
}
```

这是 PAM 最经典的统计用途之一

按长度统计回文串

统计本质不同回文串：

```
vector<int> distinct_by_len(s.size() + 1);
for (int p = 2; p < pam.size(); ++ p) {
    ++ distinct_by_len[pam.t[p].len];
}
```

统计回文出现次数：

```
auto occ = pam.count();
vector<i64> occ_by_len(s.size() + 1);
for (int p = 2; p < pam.size(); ++ p) {
    occ_by_len[pam.t[p].len] += occ[p];
}
```

后缀链接树

将：

```
link[p] -> p
```

连成树
则：

$$\boxed{u \text{ 是 } v \text{ 的 link 树祖先} \iff P_u \text{ 是 } P_v \text{ 的回文后缀}}$$

由于 P_v 本身是回文，所以它的回文后缀同时也是回文前缀

```
auto g = pam.link_tree();
```

可以进一步进行：

- 子树统计
- Euler 序
- LCA
- 树上 DP
- 动态维护某些回文节点

恢复节点对应的实际回文串

核心结构不额外保存位置，但当 `add()` 新建节点时：

```
PAM<> pam;
vector<int> first_end(s.size() + 2, -1);
for (int i = 0; i < s.size(); ++ i) {
    if (pam.add(s[i])) first_end[pam.longest_suffix()] = i;
}
```

于是节点 `p` 的一个首次出现区间为：

```
int r{ first_end[p] + 1 };
int l{ r - pam.t[p].len };
// s[l..r) 即节点 p 对应的回文串
```

这样无需为了少数恢复题在 PAM 核心里永久增加 `first_pos` 字段

复杂度

设字符串长度为 n，PAM 节点数为 $S \le n+2$

操作	时间复杂度
单次 `add(c)`	均摊 $O(1)$
`build(s)`	$O(n)$
`distinct()` / `longest_suffix()` / `suffix_count()`	$O(1)$
`suffixes()`	$O(k)$
`count()` / `link_tree()`	$O(S)$

固定字符集大小 `A` 时空间复杂度为 $O(n)$；严格写为 $O(SA)$

7.8 Series PAM（回文系列优化）

```
template<int A = 26, int offset = 'a'>
struct SeriesPAM {
    struct Node {
        int len{ 0 }, link{ 0 };
        int diff{ 0 }, slink{ 0 };
        array<int, A> next{};
    };
    vector<Node> t;
    vector<int> s;
    int suff{ 1 };
    explicit SeriesPAM(int m = 0) : t(2), s(1, -1) {
        t.reserve(m + 2);
        s.reserve(m + 1);
        t[0].len = -1;
    }
    template<class Seg>
```

```
    requires ranges::sized_range<Seg>
    explicit SeriesPAM(const Seg &a) : SeriesPAM(int(a.size())) { build(a); }
    // ===== 基础函数：插入依赖，不建议删除 =====
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    int new_node(int len) {
        t.emplace_back();
        t.back().len = len;
        return int(t.size()) - 1;
    }
    int get_link(int p) const {
        int i{ int(s.size()) - 1 };
        while (s[i - t[p].len - 1] != s[i]) p = t[p].link;
        return p;
    }
    // ===== 在线插入：维护 diff / slink 回文系列 =====
    int add_id(int x) {
        s.push_back(x);
        int p{ get_link(suff) };
        if (!t[p].next[x]) {
            int q{ new_node(t[p].len + 2) };
            if (t[q].len == 1) t[q].link = 1;
            else t[q].link = t[get_link(t[p].link)].next[x];
            t[q].diff = t[q].len - t[t[q].link].len;
            if (t[q].diff == t[t[q].link].diff) t[q].slink = t[t[q].link].slink;
            else t[q].slink = t[q].link;
            t[p].next[x] = q;
        }
        return suff = t[p].next[x];
    }
    template<class T>
    int add(const T &c) { return add_id(id(c)); }
    template<class Seg>
    void build(const Seg &a) { for (const auto &c : a) add(c); }
    int size() const { return int(t.size()); }
    int distinct() const { return int(t.size()) - 2; }
    int longest_suffix() const { return suff; }
    // 枚举当前回文后缀的各个 series 代表节点
    template<class F>
    void series(F f) const {
        for (int p = suff; p > 1; p = t[p].slink) f(p);
    }
};
```

成员变量

除普通 PAM 的 `len / link / next / suff` 外：

- `t[p].diff = len[p] - len[link[p]]`
- `t[p].slink`：回文系列链接
- `s`：字符编号序列，`s[0] = -1` 是哨兵，因此 `get_link()` 不需要额外判断左边界
  这里刻意不维护普通 PAM 的：

```
num
occ
```

因为 Series PAM 的职责只是为回文 DP 提供快速后缀分组；需要普通出现次数统计时应使用普通 PAM

接口

- `SeriesPAM(m)`：建立空结构
- `SeriesPAM(s)`：建立整个字符串的回文系列树，$O(n)$
- `add(c)`：在末尾加入字符，返回当前最长回文后缀节点，均摊 $O(1)$
- `build(s)`：依次插入字符串，$O(n)$
- `size()`：节点总数，$O(1)$
- `distinct()`：本质不同回文串数量，$O(1)$
- `longest_suffix()`：当前最长回文后缀节点，$O(1)$
- `series(f)`：沿 `slink` 枚举当前所有回文后缀系列

注意事项

- 只有当 DP 转移需要频繁枚举所有回文后缀、普通 `link` 链可能退化为 $O(n^2)$ 时，才需要 Series PAM
- 普通查询、不同回文串计数、出现次数统计应优先使用普通 PAM
- 必须保证所有字符编号位于 `[0,A)`
- `s[0] = -1` 是内部哨兵，因此合法字符编号不能为 `-1`
- `diff / slink` 是结构性字段，不能像普通统计数组那样随意删除
- 回文系列 DP 通常还需要一个按 PAM 节点编号维护的辅助数组 `g`
- `g` 往往需要跨前缀复用，不要习惯性在每一轮清空
- 使用 series link 后的具体复杂度取决于 DP 转移；典型回文划分算法通常可做到 $O(n \log n)$，部分专门转移还能进一步优化

常见用法

回文系列的含义

定义：

$$diff[p] = len[p] - len[link[p]]$$

如果：

$$diff[p] = diff[link[p]]$$

说明 `p` 与 `link[p]` 属于同一个系列
这一系列中的回文长度构成等差数列，公差就是：

$$diff[p]$$

`slink[p]` 会一次跳过当前整段相同 `diff` 的回文后缀
因此：

```
p
slink[p]
slink[slink[p]]
...
```

不是枚举每个回文后缀，而是枚举每个回文长度等差系列的代表

普通 PAM 为什么可能慢

若直接枚举：

```
for (int p = pam.longest_suffix(); p > 1; p = pam.t[p].link) {
    // 枚举所有回文后缀
}
```

例如字符串：

```
aaaaaaa...
```

第 `i` 个前缀可能有 $\Theta(i)$ 个回文后缀
总复杂度可能达到：

$$\Theta(n^2)$$

Series PAM 就是用 `diff / slink` 把大量结构相似的后缀合并处理

枚举当前所有回文系列

```
int last{ pam.add(c) };
for (int p = last; p > 1; p = pam.t[p].slink) {
    int q{ pam.t[p].slink };
    // p 表示当前一个回文后缀系列
    // q 是下一个系列
}
```

也可以：

```
pam.series([&](int p) {
    // 处理系列 p
});
```

回文划分方案数 DP

设：

```
dp[i]
```

表示前缀 `s[0..i)` 划分成若干回文串的方案数
朴素转移需要枚举所有回文后缀：

$$dp[i] = \sum_{\substack{j<i \\ s[j..i) \text{ 为回文}}} dp[j]$$

使用 Series PAM：

```
SeriesPAM<> pam(s.size());
vector<i64> dp(s.size() + 1);
vector<i64> g(s.size() + 2);
dp[0] = 1;
for (int i = 1; i <= s.size(); ++ i) {
    int last{ pam.add(s[i - 1]) };
    for (int p = last; p > 1; p = pam.t[p].slink) {
        int q{ pam.t[p].slink };
        int j{ i - (pam.t[q].len + pam.t[p].diff) };
        g[p] = dp[j];
        if (pam.t[p].diff == pam.t[pam.t[p].link].diff) {
            g[p] += g[pam.t[p].link];
        }
        dp[i] += g[p];
    }
}
```

若题目要求取模，把 `i64` 换成取模类型或在加法处取模。
其中：

```
g[p]
```

把同一个等差回文系列中的多个朴素 DP 转移压缩到一起。

最少回文划分 DP

设：

```
dp[i]
```

表示 `s[0..i)` 最少划分成多少个回文串

```
constexpr int INF = 1E9;
SeriesPAM<> pam(s.size());
vector<int> dp(s.size() + 1, INF);
vector<int> g(s.size() + 2, INF);
dp[0] = 0;
for (int i = 1; i <= int(s.size()); ++ i) {
    int last{ pam.add(s[i - 1]) };
    for (int p = last; p > 1; p = pam.t[p].slink) {
        int q{ pam.t[p].slink };
        int j{ i - (pam.t[q].len + pam.t[p].diff) };
        g[p] = dp[j];
        if (pam.t[p].diff == pam.t[pam.t[p].link].diff) {
            g[p] = min(g[p], g[pam.t[p].link]);
        }
        dp[i] = min(dp[i], g[p] + 1);
    }
}
```

最终：

```
dp[s.size()]
```

就是整个字符串的最少回文划分数

核心 DP 下标公式

对于当前前缀长度 `i`、系列代表节点 `p`，令：

```
int q{ pam.t[p].slink };
```

则该系列最短成员对应的前驱位置为：

$$\boxed{j = i - \Big(len[q] + diff[p]\Big)}$$

所以系列 DP 中经常出现：

```
int j{ i - (pam.t[q].len + pam.t[p].diff) };
```

这是 Series PAM 最需要快速回忆的公式

复杂度

PAM 本体仍满足：

操作	时间复杂度
单次 `add(c)`	均摊 $O(1)$
`build(s)`	$O(n)$
`distinct()`	$O(1)$

空间复杂度为 $O(nA)$，固定字符集时为 $O(n)$
Series DP 的复杂度取决于具体转移；典型使用中通过 `slink` 避免逐个枚举全部回文后缀，常见复杂度约为 $O(n \log n)$，某些专门递推可以进一步做到线性

7.9 SA（后缀数组）

```
struct SuffixArray {
    struct MinRMQ {
        static constexpr int B{ 64 };
        using u64 = unsigned long long;
        int n{};
        vector<u64> stk;
        vector<vector<int>> st;
        void build(const vector<int> &a) {
            n = a.size();
            stk.assign(n, 0);
            st.clear();
            if (!n) return;
            int m{ (n + B - 1) / B };
            st.assign(__lg(m) + 1, vector<int>(m));
            for (int b = 0; b < m; ++ b) {
                int l{ b * B }, r{ min(n, l + B) };
                st[0][b] = *min_element(a.begin() + l, a.begin() + r);
                u64 s{};
                for (int i = l; i < r; ++ i) {
                    while (s && a[i] < a[l + __lg(s)]) s ^= 1ULL << __lg(s);
                    s |= 1ULL << (i - l);
                    stk[i] = s;
                }
            }
            for (int k = 1; k < st.size(); ++ k) {
                for (int i = 0; i + (1 << k) <= m; ++ i) {
                    st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
                }
            }
        }
        int small(const vector<int> &a, int l, int r) const {
            return a[l + __builtin_ctzll(stk[r - 1] >> (l - l / B * B))];
        }
        int query(const vector<int> &a, int l, int r) const {
            if (l / B == (r - 1) / B) return small(a, l, r);
            int lb{ l / B + 1 }, rb{ (r - 1) / B };
            int ans{ min(small(a, l, lb * B), small(a, rb * B, r)) };
            if (lb < rb) {
                int k{ __lg(rb - lb) };
```

```
            ans = min({ ans, st[k][lb], st[k][rb - (1 << k)] });
        }
        return ans;
    }
};
int n{};
vector<int> sa, rk, lc;
MinRMQ rmq;
// ===== 基础构造：构造 sa / rk / lc，核心代码不建议删除 =====
template<class Seq>
explicit SuffixArray(const Seq &a) {
    n = a.size();
    sa.resize(n);
    rk.resize(n);
    lc.assign(n ? n - 1 : 0, 0);
    if (!n) return;
    ranges::iota(sa, 0);
    ranges::sort(sa, [&](int i, int j) { return a[i] < a[j]; });
    rk[sa[0]] = 0;
    for (int i = 1; i < n; ++ i) rk[sa[i]] = rk[sa[i - 1]] + (a[sa[i - 1]] != a[sa[i]]);
    vector<int> tmp, cnt(n);
    tmp.reserve(n);
    for (int k = 1; rk[sa.back()] != n - 1; k <<= 1) {
        tmp.clear();
        for (int i = n - k; i < n; ++ i) tmp.push_back(i);
        for (int i : sa) if (i >= k) tmp.push_back(i - k);
        ranges::fill(cnt, 0);
        for (int x : rk) ++ cnt[x];
        for (int i = 1; i < n; ++ i) cnt[i] += cnt[i - 1];
        for (int x : views::reverse(tmp)) sa[-- cnt[rk[x]]] = x;
        swap(rk, tmp);
        rk[sa[0]] = 0;
        for (int i = 1; i < n; ++ i) {
            int x{ sa[i - 1] }, y{ sa[i] };
            int x2{ x + k < n ? tmp[x + k] : -1 };
            int y2{ y + k < n ? tmp[y + k] : -1 };
            rk[y] = rk[x] + (tmp[x] != tmp[y] || x2 != y2);
        }
    }
    // Kasai：由相邻后缀在线性时间内构造 LCP 数组
    for (int i = 0, k = 0; i < n; ++ i) {
        if (!rk[i]) {
            k = 0;
            continue;
        }
        if (k) -- k;
        int j{ sa[rk[i] - 1] };
        while (i + k < n && j + k < n && a[i + k] == a[j + k]) ++ k;
        lc[rk[i] - 1] = k;
    }
    // 若删除 lcp() / cmp() / range()，同步删除本行、rmq 和 MinRMQ
    rmq.build(lc);
}
// ===== LCP / 子串比较：不需要时可整段删除 =====
int lcp(int i, int j) const {
    if (i == j) return n - i;
    if (i == n || j == n) return 0;
    int x{ rk[i] }, y{ rk[j] };
    if (x > y) swap(x, y);
    return rmq.query(lc, x, y);
}
// 比较原序列的 a[l1,r1) 与 a[l2,r2)，返回 -1 / 0 / 1
int cmp(int l1, int r1, int l2, int r2) const {
    int n1{ r1 - l1 }, n2{ r2 - l2 };
    int k{ min({ lcp(l1, l2), n1, n2 }) };
    if (k == min(n1, n2)) {
        if (n1 == n2) return 0;
        return n1 < n2 ? -1 : 1;
    }
    return rk[l1 + k] < rk[l2 + k] ? -1 : 1;
}
```

```
    // ===== 模式匹配：复用 LCP RMQ；不需要时可整段删除 =====
    // a 必须与构造 SuffixArray 时使用的原序列相同
    template<class Seq, class Pat>
    pair<int, int> range(const Seq &a, const Pat &t) const {
        int at{ -1 }, z{};
        auto compare = [&](int p) {
            int m = t.size(), i{ at < 0 ? 0 : min(z, lcp(at, p)) };
            while (i < m && p + i < n && a[p + i] == t[i]) ++ i;
            if (i > z) at = p, z = i;
            if (i == m) return 0;
            if (p + i == n) return -1;
            return a[p + i] < t[i] ? -1 : 1;
        };
        int l{ 0 }, r{ n };
        while (l < r) {
            int m{ l + r >> 1 };
            if (compare(sa[m]) < 0) l = m + 1;
            else r = m;
        }
        int L{ l };
        r = n;
        while (l < r) {
            int m{ l + r >> 1 };
            if (compare(sa[m]) <= 0) l = m + 1;
            else r = m;
        }
        return { L, l };
    }
};
```

成员变量

- `n`：原序列长度
- `sa[i]`：字典序第 `i` 小的后缀起点，即后缀 `a[sa[i], n)`
- `rk[i]`：后缀 `a[i, n)` 在 `sa` 中的排名，满足 `sa[rk[i]] = i`
- `lc[i]`：后缀 `a[sa[i], n)` 与 `a[sa[i + 1], n)` 的最长公共前缀长度，长度为 `max(n - 1, 0)`
- `rmq`：`lc` 上的静态最小值结构，供 `lcp()` / `cmp()` / `range()` 使用

接口

- `SuffixArray(a)`：构造 `sa` / `rk` / `lc` 并预处理 LCP 查询，$O(n \log n)$
  - `a` 需要支持 `size()`、`operator[]`
  - 元素需要支持 `<`、`==`、`!=`
  - 可直接用于 `string`、`vector<int>`、`vector<i64>` 等可比较序列
- `lcp(i, j)`：返回后缀 `a[i,n)` 与 `a[j,n)` 的最长公共前缀长度，$O(1)$
  - 要求 `0 <= i,j <= n`
  - `i = n` 或 `j = n` 表示空后缀
- `cmp(l1, r1, l2, r2)`：按字典序比较 `a[l1,r1)` 与 `a[l2,r2)`，返回 `-1 / 0 / 1`，$O(1)$
- `range(a, t)`：返回所有以模式序列 `t` 为前缀的后缀在 `sa` 中构成的区间 `[L,R)`，$O(|t| + \log n)$
  - 出现次数为 `R - L`
  - `sa[L..R)` 即各次出现的起点

注意事项

- 所有下标均为 `0-based`，区间均为左闭右开 `[l,r)`
- `sa` 不包含空后缀，因此长度恒为 `n`
- 本板子的 `lc[i]` 表示 `LCP(sa[i], sa[i + 1])`；有些资料的 `height[i]` 定义为 `LCP(sa[i - 1], sa[i])`，下标相差一位
- `lcp(n, j)` 中的 `n` 只表示查询时的空后缀，不能访问 `rk[n]`
- `range(a, t)` 中传入的 `a` 必须与构造 `SuffixArray` 时的原序列一致；结构体故意不保存原序列副本，以减少空间和复制开销
- 空模式 `t` 的 `range()` 返回 `[0,n)`，即所有非空后缀；若题目把空串在原串中的出现次数定义为 `n + 1`，需要单独处理
- `range()` 得到的出现位置按后缀字典序排列；若题目要求位置从小到大，需要额外排序
- 序列元素的比较关系决定字典序；处理自定义类型时应保证比较关系一致
- 不要在构造后修改 `sa` / `rk` / `lc` / `rmq`
- 若只需要 `sa` / `rk` / `lc`：
  - 删除 `MinRMQ
  - 删除成员 `rmq`
  - 删除构造函数末尾的 `rmq.build(lc)`
  - 删除 `lcp()` / `cmp()` / `range()`
  - 同步删除 `range()`，其跳过已知公共前缀需要 RMQ

常见用法

基础构造

```
string s;
cin >> s;
SuffixArray sa(s);
for (int p : sa.sa) {
    cout << p << ' ';
}
cout << '\n';
```

本质不同子串个数

长度为 `n` 的字符串共有 $\frac{n(n+1)}{2}$ 个子串计重；按后缀顺序加入时，`lc` 恰好统计与前一个后缀重复的前缀：

```
i64 ans = 1LL * sa.n * (sa.n + 1) / 2;
for (int x : sa.lc) ans -= x;
cout << ans << '\n';
```

最长重复子串

最长重复子串长度等于 `lc` 的最大值：

```
int ans = 0;
for (int x : sa.lc) ans = max(ans, x);
cout << ans << '\n';
```

若还需要一个出现位置：

```
int len = 0, pos = -1;
for (int i = 0; i < sa.lc.size(); ++ i) {
    if (sa.lc[i] > len) {
        len = sa.lc[i];
        pos = sa.sa[i];
    }
}
```

查询模式串出现次数

```
string t;
cin >> t;
auto [L, R] = sa.range(s, t);
cout << R - L << '\n';
```

判断是否出现：

```
auto [L, R] = sa.range(s, t);
bool exists = L < R;
```

枚举模式串出现位置

```
auto [L, R] = sa.range(s, t);
vector<int> pos(sa.sa.begin() + L, sa.sa.begin() + R);
ranges::sort(pos);
```

整数序列模式匹配

不需要手动转成字符串，也不要求值域连续：

```
vector<int> a(n), t(m);
for (int &x : a) cin >> x;
for (int &x : t) cin >> x;
SuffixArray sa(a);
auto [L, R] = sa.range(a, t);
cout << R - L << '\n';
```

复杂度

设原序列长度为 `n`，模式长度为 `m`

操作	时间复杂度
构造 `sa / rk`	$O(n \log n)$
构造 `lc`	$O(n)$
构造 LCP RMQ	$O\left(n + \frac{n}{64} \log \frac{n}{64}\right)$
`lcp(i,j)`	$O(1)$
`cmp(...)`	$O(1)$
`range(a,t)`	$O(m + \log n)$
枚举 `range()` 的全部出现位置	$O(R - L)$

核心 `sa / rk / lc` 空间复杂度为 $O(n)$
带 LCP RMQ 后总空间复杂度为

$$O\left(n + \frac{n}{64} \log \frac{n}{64}\right)$$

性能说明：`range()` 保留已比较出的最长模式前缀，用已有 LCP RMQ 跳过相同部分；模式字符最多线性向前扫描，再加两次二分的 O(log n) 个分支。长重复前缀不再在每个中点重新扫描。

7.10 SAM

```
template<int A = 26, int offset = 'a'>
struct SAM {
    struct Node {
        int len{ 0 }, link{ 0 };
        array<int, A> next{};
    };
    vector<Node> t;
    vector<int> occ;
    int last{ 1 };
    explicit SAM(int n = 0) {
        t.reserve(2 * n + 2);
        occ.reserve(2 * n + 2);
        t.assign(2, Node());
        occ.assign(2, 0);
        t[0].len = -1;
        t[0].next.fill(1);
    }
    template<class T>
    static int id(const T &c) { return int(c) - offset; }
    // ===== 核心扩展：单串与广义 SAM 均依赖，不建议删除 =====
    int new_node() {
        t.emplace_back();
        occ.push_back(0);
        return int(t.size()) - 1;
    }
    // c 为 0..A-1 的字符编号
    int extend(int p, int c) {
        if (t[p].next[c]) {
            int q{ t[p].next[c] };
            if (t[q].len == t[p].len + 1) return q;
            int r{ new_node() };
            t[r].len = t[p].len + 1;
            t[r].link = t[q].link;
            t[r].next = t[q].next;
            t[q].link = r;
            while (t[p].next[c] == q) {
                t[p].next[c] = r;
                p = t[p].link;
            }
            return r;
        }
        int cur{ new_node() };
        t[cur].len = t[p].len + 1;
        while (!t[p].next[c]) {
            t[p].next[c] = cur;
```

```
            p = t[p].link;
        }
        t[cur].link = extend(p, c);
        return cur;
    }
    // ===== 顺序建串：若只从 Trie / 外部状态调用 extend(p, c)，可删除 last 与以下接口 =====
    template<class T>
    int extend(const T &c) {
        last = extend(last, id(c));
        ++ occ[last];
        return last;
    }
    void new_string() { last = 1; }
    template<class Seg>
    void build(const Seg &s) { for (const auto &c : s) extend(c); }
    // ===== 基础访问：简单封装，可按需删除 =====
    int next(int p, int c) const { return t[p].next[c]; }
    template<class T>
    int next(int p, const T &c) const { return next(p, id(c)); }
    int link(int p) const { return t[p].link; }
    int len(int p) const { return t[p].len; }
    int size() const { return int(t.size()); }
    // ===== 出现次数：若不统计，可删除 occ / add_occ() / order() / build_occ() / get_occ() =====
    // 同时删除 occ.reserve / occ.assign / occ.push_back / ++ occ[last]
    void add_occ(int p, int v = 1) { occ[p] += v; }
    vector<int> order() const {
        int mx{ 0 };
        for (int u = 1; u < size(); ++ u) mx = max(mx, t[u].len);
        vector<int> cnt(mx + 1), ord(size() - 1);
        for (int u = 1; u < size(); ++ u) ++ cnt[t[u].len];
        for (int i = 1; i ⩽ mx; ++ i) cnt[i] += cnt[i - 1];
        for (int u = size() - 1; u ⩾ 1; -- u) ord[-- cnt[t[u].len]] = u;
        return ord;
    }
    // 原地求 endpos 大小，节省一份 O(S) 数组；调用后 occ 不再是原始终点计数
    void build_occ() {
        auto ord{ order() };
        for (int i = int(ord.size()) - 1; i ⩾ 0; -- i) {
            int u{ ord[i] };
            if (u ≠ 1) occ[t[u].link] += occ[u];
        }
    }
    // 非破坏性版本：不修改 occ，但额外复制 O(S) 空间
    vector<int> get_occ() const {
        vector<int> res{ occ };
        auto ord{ order() };
        for (int i = int(ord.size()) - 1; i ⩾ 0; -- i) {
            int u{ ord[i] };
            if (u ≠ 1) res[t[u].link] += res[u];
        }
        return res;
    }
    // ===== 常用查询：各功能可单独删除；contains() 依赖 find() =====
    i64 count_substrings() const {
        i64 ans{ 0 };
        for (int u = 2; u < size(); ++ u) ans += t[u].len - t[t[u].link].len;
        return ans;
    }
    template<class Seg>
    int find(const Seg &s) const {
        int p{ 1 };
        for (const auto &c : s) {
            p = t[p].next[id(c)];
            if (!p) return 0;
        }
        return p;
    }
    template<class Seg>
    bool contains(const Seg &s) const { return find(s) ≠ 0; }
    template<class Seg>
    int lcs(const Seg &s) const {
```

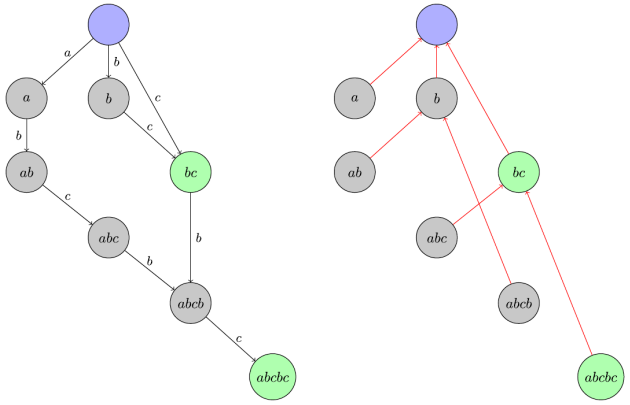
```
        int p{ 1 }, l{ 0 }, ans{ 0 };
        for (const auto &x : s) {
            int c{ id(x) };
            while (!t[p].next[c]) {
                p = t[p].link;
                l = t[p].len;
            }
            p = t[p].next[c];
            ans = max(ans, ++ l);
        }
        return ans;
    }
};
```

成员变量

- `t[u].len`：状态 `u` 中最长子串的长度
- `t[u].link`：状态 `u` 的后缀链接
- `t[u].next[c]`：状态 `u` 经过字符编号 `c` 的转移
- 状态 `u` 表示的所有子串长度为 `(t[t[u].link].len, t[u].len]`
- `occ[u]`：
  - 建串完成后、统计前：直接落在状态 `u` 的前缀终点数量
  - `build_occ()` 后：状态 `u` 的 `endpos` 大小，即该状态对应子串的出现次数
- `last`：当前顺序插入字符串对应状态
- `0`：虚点，`len = -1`，所有字符均转移到根
- `1`：根，表示空串

接口

- `SAM<A, offset>(n)`：构造，并按预计总插入长度 `n` 预留约 `2n + 2` 个状态
- `id(c)`：将元素映射到字符编号 `0..A-1`
- `extend(p, c)`：从状态 `p` 扩展字符编号 `c`，返回所得状态；用于广义 SAM / Trie 建 SAM
- `extend(c)`：向当前字符串末尾加入元素 `c`，更新 `last` 并记录一次前缀终点
- `new_string()`：令 `last = 1`，开始构造新的独立字符串
- `build(s)`：从当前 `last` 开始顺序插入序列 `s`
- `next(p, c)`：查询状态转移
- `link(p)`：返回后缀链接
- `len(p)`：返回状态最大长度
- `size()`：返回状态数，包含虚点 `0` 与根 `1`
- `add_occ(p, v)`：给状态增加 `v` 个直接终点标记，常用于 Trie 建广义 SAM
- `order()`：按 `len` 非递减返回全部真实状态，包含根、不包含虚点
- `build_occ()`：原地沿后缀链接汇总出现次数
- `get_occ()`：非破坏性返回所有状态的出现次数
- `count_substrings()`：返回本质不同非空子串数量
- `find(s)`：若 `s` 出现，返回其对应状态，否则返回 `0`
- `contains(s)`：判断 `s` 是否出现
- `lcs(s)`：返回 `s` 与当前 SAM 所表示字符串集合的最长公共子串长度



以字符串 `abcbc` 为例：

- 左图黑边是字符转移 `next`
  - 从状态 `u` 沿字符 `c` 到 `v`，表示在 `u` 所表示的字符串后追加 `c`
  - 从根出发沿字符依次走，就在 SAM 中匹配一个子串

- 右图红边是后缀链接 `link`
  - `u → link[u]` 指向 `u` 所表示字符串类的最长严格后缀类
  - 始终有 `len[link[u]] < len[u]`
  - 因而 `link` 构成一棵以根为根的树
    图中节点标出的 `ab`、`abc`、`bc` 等只是该状态的**最长代表串**。状态 `u` 实际表示一整个 `endpos` 等价类，其子串长度范围为 `(len[link[u]], len[u]]`
    即

```
len[link[u]] + 1 ... len[u]
```

例如图中的状态 `bc`：

```
len[bc] = 2
link[bc] = root
```

因此它表示长度 `1..2` 的两个子串：

```
c
bc
```

它们在 `abcbc` 中的结束位置完全相同：

```
endpos(c)  = {3, 5}
endpos(bc) = {3, 5}
```

所以被压缩进同一个 SAM 状态
同理，状态 `abcb` 满足：

```
len[abcb] = 4
len[link[abcb]] = len[b] = 1
```

因此它表示长度 `2..4` 的：

```
cb
bcb
abcb
```

这些字符串的 `endpos` 也完全相同

三个最重要的关系

看到 SAM 时优先想起：

$$\boxed{u \text{ 表示长度 } (len[link[u]], len[u]] \text{ 的一组子串}}$$

同一状态内所有子串：

$$\boxed{endpos \text{ 相同} \Longrightarrow occ \text{ 相同}}$$

后缀链接满足：

$$\boxed{endpos(u) \subseteq endpos(link[u])}$$

因此出现次数可以按 `len` 从大到小传播：

```
occ[link[u]] += occ[u];
```

图与代码的对应

```
左图字符边              t[u].next[c]
右图红色后缀边          t[u].link
节点最大字符串长度       t[u].len
同一节点的出现次数       occ[u]
根                      1
虚点                    0（图中通常省略）
```

例如右图中：

```
abcbc → bc → root
abc   → bc → root
abcb  → b  → root
ab    → b  → root
```

这也解释了为什么从 `last = abcbc` 沿 `link` 一直走，会依次得到原串的后缀类：

```
abcbc
bc
空串
```

图中绿色节点正对应这条从 `last` 沿后缀链接到根的链，即原串各后缀所在的状态

做统计题时的速记

一个状态 `u` 表示：

```
长度：
len[link[u]] + 1 ... len[u]

数量：
len[u] - len[link[u]]

出现次数：
occ[u]
```

因此：
本质不同子串数

```
ans += len[u] - len[link[u]];
```

出现至少 `k` 次的最长子串

```
if (occ[u] ≥ k) ans = max(ans, len[u]);
```

P3804：长度 × 出现次数最大值

```
if (occ[u] > 1) {
    ans = max(ans, i64(occ[u]) * len[u]);
}
```

因为同一状态中的子串 `occ` 相同，所以需要最大长度时总是直接取 `len[u]`

> SAM 状态 = endpos 等价类；len 给最长串，link 给长度下界，occ 对整个状态相同

注意事项

- `SAM<A, offset>` 要求元素满足 `0 ≤ int(c) - offset < A`
- 小写字母使用默认 `SAM<>`
- 整数编号序列使用 `SAM<A, 0>`
- `extend(p, c)` 中的 `c` 是已经映射好的 `0..A-1` 编号；字符需要写 `extend id(ch)`
- `build(s)` 不会自动重置 `last`；插入多个互相独立的字符串时，每个字符串前调用 `new_string()`
- 单串 `extend(c)` 会自动执行 `++ occ[last]`
- clone 节点的初始 `occ` 必须为 `0`，不能复制原节点的 `occ`
- `occ` 使用 `int` 即可：出现次数不超过所有插入字符串总长度；最终乘积需要在运算前转为 `i64`
- `build_occ()` 会直接修改 `occ`：
  - 只能在全部字符串构造完成后调用
  - 不要调用两次
  - 调用后不要继续 `extend()` / `build()` / `add_occ()`
  - `build_occ()` 与 `get_occ()` 通常二选一，不要先 `build_occ()` 再调用 `get_occ()`
- 若需要保留原始终点计数或多次重新计算，使用 `get_occ()`
- `0` 是虚点，不参与普通状态答案；`1` 是根
- 固定 `array<int, A>` 适合小字符集；字符集很大时空间为 $O(SA)$，应考虑稀疏转移

常见用法

本质不同子串数量

```
i64 ans{ sam.count_substrings() };
```

查询模式串出现次数

普通规模、需要保留原 `occ`：

```
auto occ{ sam.get_occ() };
int p{ sam.find(t) };
cout << (p ? occ[p] : 0) << '\n';
```

原地统计出现次数

内存敏感时：

```
sam.build_occ();
int p{ sam.find(t) };
cout << (p ? sam.occ[p] : 0) << '\n';
```

P3804：长度乘出现次数最大值

```
SAM<> sam(s.size());
sam.build(s);
sam.build_occ();
i64 ans{ 0 };
for (int u = 2; u < sam.size(); ++ u) {
    if (sam.occ[u] > 1) { // 题目要求次数大于 1
        ans = max(ans, i64(sam.occ[u]) * sam.len(u));
    }
}
```

这里对每个状态只需要取 `len[u]`，因为同一状态中的所有子串拥有相同 `endpos`，出现次数相同，而长度越长乘积越大

最长出现至少 `k` 次的子串

```
sam.build_occ();
int ans{ 0 };
for (int u = 2; u < sam.size(); ++ u) {
    if (sam.occ[u] ≥ k) {
        ans = max(ans, sam.len(u));
    }
}
```

多个独立字符串

```
SAM<> sam(total_len);
for (const string &s : a) {
    sam.new_string();
    sam.build(s);
}
```

这样不会产生跨两个字符串边界的子串

Trie 建广义 SAM

设 Trie 中 `v` 是 `u` 经过字符编号 `c` 到达的儿子：

```
sam_pos[root] = 1;
queue<int> q;
q.push(root);
while (!q.empty()) {
    int u{ q.front() };
    q.pop();
    for (int c = 0; c < A; ++ c) {
        int v{ trie[u][c] };
        if (!v) continue;
        sam_pos[v] = sam.extend(sam_pos[u], c);
        sam.add_occ(sam_pos[v], cnt[v]);
        q.push(v);
```

```
    }
}
```

若只需要子串集合，不统计出现次数，可以删掉 `add_occ()`

复杂度

记：

- `M`：所有插入字符串总长度
- `S`：SAM 状态数，`S = O(M)`
- `L`：最长插入字符串长度
- `m`：查询串长度
- `A`：字符集大小

操作	时间复杂度
`extend(c)` / `extend(p, c)`	均摊 $O(A)$
`build(s)`	$O(\|s\|)$
`new_string()` / `next` / `link` / `len` / `size` / `add_occ()`	$O(1)$
`order()` / `build_occ()` / `get_occ()`	$O(S+L)$
`count_substrings()`	$O(S)$
`find(s)` / `contains(s)` / `lcs(s)`	$O(m)$

固定小字符集下，`A` 视为常数，因此构造总时间为 $O(M)$
空间复杂度：

- SAM 本体：$O(SA)$
- `occ`：$O(S)$
- `build_occ()` 临时空间：$O(S+L)$
- `get_occ()` 临时空间：$O(S+L)$，并额外保留一份 $O(S)$ 的结果数组
  固定小字符集下可视为 $O(S)$

7.11 循环最小表示与 Lyndon 分解

Booth 最小表示

```cpp
template<class Seg>
int minimum_rotation(const Seg &s) {
    int n = s.size();
    if (!n) return 0;
    int i{ 0 }, j{ 1 }, k{ 0 };
    while (i < n && j < n && k < n) {
        const auto &a{ s[(i + k) % n] };
        const auto &b{ s[(j + k) % n] };
        if (a == b) {
            ++ k;
            continue;
        }
        if (a > b) i += k + 1;
        else j += k + 1;
        if (i == j) ++ j;
        k = 0;
    }
    return min(i, j);
}
template<class Seg>
auto minimum_rotation_sequence(const Seg &s) {
    using T = decay_t<decltype(s[0])>;
    int n = s.size(), p{ minimum_rotation(s) };
    vector<T> res;
    res.reserve(n);
    for (int i = 0; i < n; ++ i) {
        res.push_back(s[(p + i) % n]);
    }
    return res;
}
```

循环同构标准化

```
template<class Seg>
auto canonical_cycle(const Seg &s, bool allow_reverse = false) {
    using T = decay_t<decltype(s[0])>;
    vector<T> a(s.size());
    for (int i = 0; i < s.size(); ++ i) a[i] = s[i];
    vector<T> ans{ minimum_rotation_sequence(a) };
    if (allow_reverse) {
        ranges::reverse(a);
        ans = min(ans, minimum_rotation_sequence(a));
    }
    return ans;
}
```

Duval Lyndon 分解

```
template<class Seg>
vector<pair<int, int>> lyndon_factorization(const Seg &s) {
    int n = s.size();
    vector<pair<int, int>> res;
    for (int i = 0; i < n;) {
        int j{ i + 1 }, k{ i };
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else ++ k;
            ++ j;
        }
        int len{ j - k };
        while (i <= k) {
            res.push_back({ i, i + len });
            i += len;
        }
    }
    return res;
}
```

接口说明

- `minimum_rotation(s)`：返回字典序最小循环移位的起始下标，$O(n)$
- `minimum_rotation_sequence(s)`：返回字典序最小循环表示，$O(n)$
- `canonical_cycle(s,false)`：返回只允许旋转时的标准表示，$O(n)$
- `canonical_cycle(s,true)`：允许旋转和整体反向时的标准表示，$O(n)$
- `lyndon_factorization(s)`：返回 Duval / Chen–Fox–Lyndon 分解的 `[l,r)` 区间，$O(n)$

`Seg` 只需要：

- `size()`
- `operator[]`
- 元素支持 `==`、`<` / `<=`

可以直接处理：

```
string
vector<int>
vector<pair<int, int>>
```

等可字典序比较的广义字符串

注意事项

- 所有位置和区间均为 `0base`
- 空序列的最小表示起点定义为 `0`
- `minimum_rotation()` 返回的是原序列中的起点，不会实际修改序列
- 当最小循环表示存在多个相同起点时，该实现返回其中最小的下标
- 循环最小表示和“字典序最小后缀”不是同一个问题
- `canonical_cycle(..., true)` 表示允许整个环反向后再任意旋转，适用于无方向环同构
- 若题目只允许旋转，不能错误地开启 `allow_reverse`
- Lyndon 分解和普通“按最小字符切分”没有关系，应直接使用 Duval
- Lyndon 分解的因子区间彼此首尾相接，覆盖整个字符串
- 这里使用的是通常的字典序；若题目字符顺序特殊，应先映射为正确的比较顺序

常见用法

得到最小循环表示

```
int p{ minimum_rotation(s) };
rotate(s.begin(), s.begin() + p, s.end());
```

或者不修改原串：

```
auto t = minimum_rotation_sequence(s);
```

判断两个字符串是否循环同构

```
bool same{ a.size() == b.size() && canonical_cycle(a) == canonical_cycle(b) };
```

这是**确定性**的 $O(n)$ 判定，不需要哈希

无方向环 / 雪花同构

如果一个环正向、反向都视为同一个对象：

```
auto key = canonical_cycle(a, true);
```

然后：

```
set<vector<int>> st;
st.insert(key);
```

即可对环进行标准化去重
典型场景：

- 雪花
- 多边形边标签环
- 环形排列
- 无方向循环序列

周期串的多个最小表示

如果字符串具有周期，则可能有多个位置产生完全相同的最小循环表示
例如：

```
abab
```

起点 `0` 和 `2` 都得到：

```
abab
```

`minimum_rotation()` 返回最小起点 `0`
因此：

> “最小表示字符串”唯一，但对应的起点不一定唯一

Lyndon 分解

```
auto fac = lyndon_factorization(s);
for (auto [l, r] : fac) {
    // s[l..r) 是一个 Lyndon word
}
```

Chen–Fox–Lyndon 定理给出唯一分解：

$$\boxed{s = w_1 w_2 \cdots w_k}$$

其中每个 w_i 都是 Lyndon word，并且：

$$\boxed{w_1 \geq w_2 \geq \cdots \geq w_k}$$

按字典序非增排列

判断整个字符串是否为 Lyndon word

对于非空字符串：

```
auto fac = lyndon_factorization(s);
bool is_lyndon{ fac.size() == 1 };
```

Lyndon word 的核心性质

非空字符串 w 是 Lyndon word，当且仅当它严格小于自己的任意非平凡循环移位。
因此 Lyndon word 一定是本原串，即不存在：

$$w = x^k, \qquad k > 1$$

反过来，如果一个字符串是本原串，那么它的字典序最小循环表示一定是一个 Lyndon word
所以 Booth 最小表示与 Lyndon 之间有关系，但解决的问题不同：

- Booth：在一个循环同构类里找最小代表
- Duval：把任意线性字符串唯一分解成 Lyndon words

分解区间恢复

```
for (auto [l, r] : lyndon_factorization(s)) {
    string part{ s.substr(l, r - l) };
}
```

如果只需要边界而不是复制子串，保留 `[l,r)` 会比直接返回 `vector<string>` 更省空间

二级结论

循环同构标准形

定义：

$$C(s) = \min_{0 \le i < n} \text{rotate}(s, i)$$

那么：

$$\boxed{a, b \text{ 循环同构} \iff C(a) = C(b)}$$

因此最小表示可以直接作为循环对象的哈希前标准化、排序键或 `map/set` 键

允许翻转时

定义：

$$D(s) = \min(C(s), C(reverse(s)))$$

则：

$$\boxed{D(a) = D(b)}$$

当且仅当两者可以通过：

- 任意旋转
- 可选的一次整体反向
  互相得到

Lyndon 与最小旋转

如果 s 是本原串，则：

$$\boxed{minimum_rotation(s)}$$

得到的字符串一定是 Lyndon word
若 s 不是本原串，则其最小表示可能是某个 Lyndon word 的幂

复杂度

设字符串长度为 n

操作	时间复杂度
`minimum_rotation()`	$O(n)$
`minimum_rotation_sequence()`	$O(n)$
`canonical_cycle()`	$O(n)$
`lyndon_factorization()`	$O(n)$

除输出结果外：

- Booth 额外空间：$O(1)$
- Lyndon 分解保存区间：$O(k)$，其中 `k` 为因子数
- `canonical_cycle()`：$O(n)$

7.12 Bitset LCS

```
struct BitsetLCS {
    int m, words;
    u64 last_mask{ ~0ULL };
    array<vector<u64>, 256> pos;
    explicit BitsetLCS(string_view b)
        : m(int(b.size())), words((m + 63) / 64) {
        for (auto &v : pos) v.assign(words, 0);
        for (int i = 0; i < m; ++ i) {
            pos[(unsigned char)b[i]][i >> 6] |= 1ULL << (i & 63);
        }
        if (m % 64) last_mask = (1ULL << (m % 64)) - 1;
    }
    int query(string_view a) const {
        if (!m) return 0;
        vector<u64> s(words);
        for (unsigned char c : a) {
            u64 carry{ 1 }, borrow{ 0 };
            for (int i = 0; i < words; ++ i) {
                u64 old{ s[i] }, x{ pos[c][i] | old }, y{ (old << 1) | carry };
                carry = old >> 63;
                u64 subtrahend{ y + borrow }, sub{ x - subtrahend };
                bool overflow{ subtrahend < y };
                borrow = overflow || x < subtrahend;
                s[i] = x & ~sub;
            }
            s.back() &= last_mask;
        }
        int ans{ 0 };
        for (u64 x : s) ans += popcount(x);
        return ans;
    }
};
int lcs_bitset(string_view a, string_view b) {
    if (a.size() < b.size()) swap(a, b);
    return BitsetLCS(b).query(a);
}
```

接口说明

`BitsetLCS`

将固定字符串 `b` 预处理成字符位置位集。

- `BitsetLCS(b)`：预处理 `b`
- `query(a)`：返回 `LCS(a,b)` 的长度
  适用于：

  > 一个固定 `b`，需要和多个字符串分别求 LCS

`lcs_bitset(a,b)`

一次性求：

```
LCS(a, b)
```

内部自动让较短字符串作为位集维度，以降低空间和常数
返回值只有 LCS 长度

注意事项

- 这里求的是最长公共子序列，不是最长公共子串

- 当前实现针对字节字符集，因此使用 `unsigned char` 的 `0..255`
- `string_view` 中 UTF-8 中文会按字节处理，不是按 Unicode 字符处理
- 如果字符已经离散化到 `[0,A)`，可以把 `256` 改成 `A`，并把字符索引改成离散值
- 该算法支持重复字符，是一般 LCS，不是“位置映射后求 LIS”的特殊版本
- 当前只求长度，不直接恢复一个 LCS
- 需要恢复方案时优先使用普通 DP、Hirschberg 等专门方法
- 多字版本中：
  - 左移进位
  - 无符号减法借位
  - 最后一个 `u64` 的无效高位清零
    都不能删除
- 当 `b.size() ⩽ 64` 时整个 DP 状态只有一个 `u64`，速度尤其快

核心公式

设：

- `S` 表示当前 LCS DP 的差分位集
- `M[c]` 表示字符 `c` 在 `b` 中出现的位置集合
  每处理 `a` 中一个字符 `c`：

$$X = M[c] \mid S$$

$$Y = (S \ll 1) \mid 1$$

$$\boxed{S = X \,\&\, \neg(X - Y)}$$

最终：

$$\boxed{LCS(a,b) = popcount(S)}$$

板子中复杂的多字借位逻辑，只是在多个 `u64` 上精确实现：

```
X - Y
```

这个大整数减法

常见用法

一次性求 LCS

```
cout << lcs_bitset(a, b) << '\n';
```

无需手动判断哪一个字符串更短

固定一个字符串，多次查询

例如有很多询问：

```
LCS(a1, b)
LCS(a2, b)
LCS(a3, b)
...
```

只预处理一次：

```
BitsetLCS lcs(b);
for (auto &a : query) {
    cout << lcs.query(a) << '\n';
}
```

此时位置位集 `pos` 不需要重复构造

小字符集整数序列

假设元素已经离散化到：

```
[0,A)
```

可以将：

```
array<vector<u64>, 256> pos;
```

改成：

```
array<vector<u64>, A> pos;
```

并将：

```
pos[(unsigned char)b[i]]
```

改成：

```
pos[b[i]]
```

核心算法完全不变

与普通 LCS DP 的选择

经典 DP：

$$O(nm)$$

Bitset LCS：

$$O\left(n\left\lceil\frac{m}{64}\right\rceil\right)$$

个 `u64` 运算
因此当：

```
n,m ≈ 10^4 ~ 10^5
```

且只需要长度时，Bitset 优化往往非常有价值

为什么让较短串作为位集维度

算法复杂度近似：

$$|a|\cdot\left\lceil\frac{|b|}{64}\right\rceil$$

交换二者不改变主要乘积数量级，但令 `b` 较短可以减少：

- `pos` 空间
- 单轮位集长度
- 尾部无效位
  所以一次性接口自动执行：

```
if (a.size() < b.size()) swap(a, b);
```

与“LCS 转 LIS”的区别

如果第二个序列中的元素唯一，可以将：

```
元素 → 第二序列位置
```

映射后求 LIS
但一般存在重复元素时：

```
LCS ≠ 简单位置映射后的 LIS
```

Bitset LCS 没有这个限制，可以直接处理重复字符

复杂度

令：

- $n=|a|$
- $m=|b|$
- $W=\lceil m/64\rceil$

预处理

`BitsetLCS(b)`：

$$O(m+256W)$$

空间：

$$O(256W)$$

单次查询

`query(a)`：

$$O(nW)$$

因此一次性：

$$\boxed{O\left(\max(n,m)\left\lceil\frac{\min(n,m)}{64}\right\rceil\right)}$$

实际额外 DP 空间只有：

$$O(W)$$

固定模式串的位置表为：

$$O(256W)$$

7.13 周期结构、Runs 与 Main–Lorentz

Runs：枚举所有极大周期段

```
struct Run {
    int period, l, r;
    int length() const { return r - l; }
};
template<class Seg>
vector<Run> enumerate_runs(const Seg &src) {
    using T = decay_t<decltype(*begin(src))>;
    vector<T> s(begin(src), end(src));
    int n{ int(s.size()) };
    // by_period[p] 暂存周期 p 对应的极大候选区间
    vector<vector<pair<int, int>>> by_period(n + 1);
    // 处理跨过左右分界线的周期段
    auto crossing = [&](const vector<T> &left, const vector<T> &right) {
        vector<tuple<int, int, int>> res;
        int nl{ int(left.size()) }, nr{ int(right.size()) };
        vector<T> a{ left }, b{ right };
        ranges::reverse(a);
        // 不需要分隔符：后续查询长度会被 nl / nr 截断
        b.insert(b.end(), left.begin(), left.end());
        b.insert(b.end(), right.begin(), right.end());
        auto zl = z_function(a);
        auto zr = z_function(b);
        for (int p = 1; p <= nl; ++ p) {
            int x{ p == nl ? p : min(p + zl[p], nl) };
            int y{ min(zr[nl + nr - p], nr) };
            if (x + y >= 2 * p) res.push_back({ p, x, y });
        }
        return res;
    };
    // ===== Main-Lorentz 风格分治：左右递归 + 处理跨中点周期 =====
    auto dfs = [&](auto self, int l, int r) -> void {
        if (r - l <= 1) return;
        int m{ (l + r) / 2 };
        self(self, l, m);
        self(self, m, r);
        vector<T> left(s.begin() + l, s.begin() + m);
        vector<T> right(s.begin() + m, s.begin() + r);
        for (auto [p, x, y] : crossing(left, right)) {
            by_period[p].push_back({ m - x, m + y });
        }
        ranges::reverse(left);
        ranges::reverse(right);
```

```
        for (auto [p, x, y] : crossing(right, left)) {
            by_period[p].push_back({ m - y, m + x });
        }
    };
    dfs(dfs, 0, n);
    // ===== 去除同周期下被包含的区间，并保留每个 Run 的最小周期 =====
    vector<Run> res;
    set<pair<int, int>> used;
    for (int p = 1; p <= n; ++ p) {
        auto &v = by_period[p];
        sort(v.begin(), v.end(), [](const auto &a, const auto &b) {
            if (a.first != b.first) return a.first < b.first;
            return a.second > b.second;
        });
        int mx{ -1 };
        for (auto [l, r] : v) {
            if (r <= mx) continue;
            mx = r;
            // 周期从小到大扫描，因此同一极大区间第一次出现即为最小周期
            if (used.insert({ l, r }).second) res.push_back({ p, l, r });
        }
    }
    return res;
}
```

Main–Lorentz：压缩表示所有相邻重复 XX

```
struct TandemGroup {
    int period, l, r;
    // 对任意 start ∈ [l, r):
    // s[start..start + 2 * period) = X + X
    int count() const { return r - l; }
};
template<class Seg>
vector<TandemGroup> main_lorentz(const Seg &src) {
    using T = decay_t<decltype(*begin(src))>;
    vector<T> a(begin(src), end(src));
    vector<TandemGroup> res;
    auto get_z = [](const vector<int> &z, int i) {
        return 0 <= i && i < int(z.size()) ? z[i] : 0;
    };
    // 用 optional 的 nullopt 作为绝不会与普通字符相等的分隔符
    auto with_sep = [](const vector<T> &x, const vector<T> &y) {
        vector<optional<T>> res;
        res.reserve(x.size() + y.size() + 1);
        for (const auto &v : x) res.emplace_back(v);
        res.emplace_back(nullopt);
        for (const auto &v : y) res.emplace_back(v);
        return res;
    };
    // 对固定半长 period，把所有合法 l1 压成连续的起点区间
    auto emit = [&](int shift, bool left, int center, int period, int k1, int k2) {
        int lo{ max(1, period - k2) }, hi{ min(period, k1) };
        // 防止与递归子问题重复统计
        if (left && hi == period) -- hi;
        if (lo > hi) return;
        int l, r;
        if (left) {
            l = shift + center - hi;
            r = shift + center - lo + 1;
        } else {
            l = shift + center - period - hi + 1;
            r = shift + center - period - lo + 2;
        }
        res.push_back({ period, l, r });
    };
    // ===== Main-Lorentz 分治 =====
    auto dfs = [&](auto self, const vector<T> &s, int shift) -> void {
        int n = s.size();
        if (n <= 1) return;
```

```
        int nl{ n / 2 }, nr{ n - nl };
        vector<T> left(s.begin(), s.begin() + nl);
        vector<T> right(s.begin() + nl, s.end());
        vector<T> rleft(left.rbegin(), left.rend());
        vector<T> rright(right.rbegin(), right.rend());
        self(self, left, shift);
        self(self, right, shift + nl);
        auto z1 = z_function(rleft);
        auto z2 = z_function(with_sep(right, left));
        auto z3 = z_function(with_sep(rleft, rright));
        auto z4 = z_function(right);
        for (int center = 0; center < n; ++ center) {
            int period, k1, k2;
            if (center < nl) {
                // 中心落在左半部分
                period = nl - center;
                k1 = get_z(z1, nl - center);
                k2 = get_z(z2, nr + 1 + center);
            } else {
                // 中心落在右半部分
                period = center - nl + 1;
                k1 = get_z(z3, n - (center - nl));
                k2 = get_z(z4, center - nl + 1);
            }
            if (k1 + k2 ⩾ period) {
                emit(shift, center < nl, center, period, k1, k2);
            }
        }
    };
    dfs(dfs, a, 0);
    return res;
}
```

基本定义

周期

对于字符串区间：

```
s[l..r)
```

长度：

$$L = r - l$$

若对于所有：

$$l + p \le i < r$$

都有：

$$s[i] = s[i - p]$$

则称 `p` 是该字符串的一个**周期**
等价地：

$$\boxed{p \text{ 是周期} \iff s[l..r-p) = s[l+p..r)}$$

注意：

> 周期不要求整除字符串长度

例如：

```
abababa
```

长度为 `7`，但 `2` 仍然是它的周期

最小周期

所有周期中最小的一个称为：

$$p_{\min}$$

字符串的指数定义为：

$$\boxed{\exp(s) = \frac{|s|}{p_{\min}}}$$

指数不要求为整数
例如：

```
ababa
```

最小周期为 `2`，指数为：

$$\frac{5}{2}$$

Tandem repetition / Square

若：

$$S = XX$$

即两个完全相同的字符串相邻出现，则称其为 tandem repetition / square
若 `|X| = p`，则区间长度为：

$$2p$$

例如：

```
abcabc
```

是半长为 `3` 的 tandem repetition

Run

一个 Run 表示：

```
(period, l, r)
```

满足：

1. `[l,r)` 的**最小周期**为 `period`
2. 至少重复两次：

$$r - l \geq 2 \cdot period$$

3. 不能继续向左保持该周期：

$$l = 0 \quad \text{或} \quad s[l-1] \neq s[l-1+period]$$

4. 不能继续向右保持该周期：

$$r = n \quad \text{或} \quad s[r] \neq s[r-period]$$

也是：

$$\boxed{\text{Run} = \text{极大的指数至少为 2 的周期段}}$$

接口说明

`Run`

```
struct Run {
    int period, l, r;
};
```

表示：

```
s[l..r)
```

是一个 Run，最小周期为 `period`

- `length()`：返回 `r - l`
  指数为：

```
double exponent = 1.0 * run.length() / run.period;
```

比较指数时通常不需要浮点数

`enumerate_runs(s)`

```
auto runs = enumerate_runs(s);
```

返回字符串所有 Runs
每个结果：

```
run.period
run.l
run.r
```

满足：

```
s[run.l..run.r)
```

是一个极大周期段
结果按照 `period` 非降序产生

`TandemGroup`

```
struct TandemGroup {
    int period, l, r;
};
```

表示对于：

```
for (int start = l; start < r; ++ start)
```

都有：

```
s[start .. start + period) == s[start + period .. start + 2 * period)
```

也就是说，一个 `TandemGroup` 压缩了一批：

```
XX
```

重复

`main_lorentz(s)`

返回所有 tandem repetition 的压缩表示：

```
auto groups = main_lorentz(s);
```

不会直接展开所有答案
这是必要的，因为所有 tandem repetition 的数量本身可能达到：

$$\Theta(n^2)$$

注意事项

- 所有区间均为 `0base` 的 `[l,r)`
- `Run::period` 是**最小周期**
- `TandemGroup::period` 是 `XX` 中单个 `X` 的长度，不保证它是 `XX` 的最小周期
- `enumerate_runs()` 只需要元素支持相等比较
- `main_lorentz()` 使用 `optional<T>` 构造安全分隔符，因此同样不要求字符集中预留特殊字符
- 两份算法都是静态算法；字符串修改后需要重新计算
- Run 的数量是线性的，但所有普通周期子串、所有 tandem repetition 的数量都可能是平方级
- 因此不要看到 Runs 数量少，就错误地认为"展开所有周期子串"也是 $O(n)$
- 如果题目只判断一个指定 `p` 是否为某区间周期，直接用哈希 / LCP 等工具通常更简单
- 如果需要枚举整个字符串的周期结构、最大重复段、最大指数等，Runs 才真正有优势

常见用法

枚举所有 Runs

```
auto runs = enumerate_runs(s);
for (auto [p, l, r] : runs) {
    // s[l..r) 的最小周期为 p
    // 且 [l,r) 已无法继续向左右扩展
}
```

求最大指数

Run 的指数为：

$$\frac{r-l}{p}$$

如果只比较大小，不需要浮点数：

```
auto runs = enumerate_runs(s);
Run best{ 1, 0, 0 };
for (auto run : runs) {
    if (1LL * run.length() * best.period > 1LL * best.length() * run.period) {
        best = run;
    }
}
```

若 `runs` 可能为空，应单独处理

判断是否存在指数至少为 `k` 的重复段

```
bool ok{ false };
for (auto [p, l, r] : enumerate_runs(s)) {
    if (r - l ≥ k * p) {
        ok = true;
        break;
    }
}
```

因为任何指数至少为 `2` 的周期子串，都可以沿其周期继续向两边扩展，最终落入某个极大周期结构中

最长 Run

```
auto runs = enumerate_runs(s);
int ans{ 0 };
for (auto [p, l, r] : runs) {
    ans = max(ans, r - l);
}
```

得到最长的极大重复周期段长度

按周期分类

```
vector<vector<pair<int, int>>> by_period(s.size() + 1);
for (auto [p, l, r] : enumerate_runs(s)) {
    by_period[p].push_back({ l, r });
}
```

之后可以快速处理：

- 某周期 `p` 出现在哪些极大周期段
- 对周期长度加权
- 对不同周期做离线统计

Main–Lorentz：统计所有 `XX` 数量

不需要真正展开答案：

```
auto groups = main_lorentz(s);
i64 ans{ 0 };
for (auto g : groups) {
    ans += g.count();
}
```

这样即使真实 `XX` 数量达到平方级，也仍然可以直接求总数，而不用保存每个区间

Main–Lorentz：求最长 `XX`

对于一个 `TandemGroup`，其中所有重复长度都是：

$$2 \cdot period$$

因此：

```
int ans{ 0 };
for (auto g : main_lorentz(s)) {
    ans = max(ans, 2 * g.period);
}
```

Main–Lorentz：找到任意一个 `XX`

```
auto groups = main_lorentz(s);
if (!groups.empty()) {
    auto [p, l, r] = groups[0];
    int start{ l };
    // s[start..start + p) == s[start + p..start + 2p)
}
```

展开所有 tandem repetition

只有题目真的要求输出所有答案时才这样做：

```
for (auto [p, l, r] : main_lorentz(s)) {
    for (int start = l; start < r; ++ start) {
        int L{ start }, R{ start + 2 * p };
        // s[L..R) = XX
    }
}
```

复杂度必须计入输出量：

$$O(n \log n + K)$$

其中 `K` 是实际 tandem repetition 数量，最坏可为：

$$\Theta(n^2)$$

二级结论

周期与 Border

长度为 `n` 的字符串具有周期 `p`，当且仅当它存在长度：

$$n - p$$

的 Border
即：

$$\boxed{p \text{ 是周期} \iff s[0..n-p) = s[p..n)}$$

因此：

```
period p
⟷
border n - p
```

是同一个结构的两种描述

前缀函数与最小周期

设：

```
pi[n - 1]
```

为整个字符串最长真 Border 长度
则：

$$\boxed{p = n - \pi[n-1]}$$

是字符串的最小周期
但若要求字符串恰好由某个块完整重复：

```
X X X ... X
```

还需要：

$$n \bmod p = 0$$

所以要区分：

```
最小周期
```

与：

```
最短循环节长度
```

后者通常还要求整除

Fine–Wilf 定理

若长度为 `L` 的字符串同时具有周期 `p` 和 `q`，且：

$$L \geq p + q - \gcd(p, q)$$

则：

$$\boxed{\gcd(p, q)}$$

也是它的周期
这是周期题中非常重要的结构定理
它说明：

> 一个足够长的字符串如果同时表现出两种周期性，那么两种周期会被强制合并成更小的公共周期

为什么 Runs 能压缩周期结构

普通周期子串可能极多
例如：

```
aaaaaaaa...
```

存在平方级数量的周期子串和 tandem repetition
但是极大周期段 Run 的数量满足：

$$\boxed{\#Runs = O(n)}$$

事实上 Runs Theorem 给出了更强的线性上界
因此很多题可以把：

```
枚举所有周期子串
```

改写成：

```
枚举所有 Runs
+
在每个 Run 内做数学统计
```

这也是 Runs 最核心的价值

Tandem repetition 与 Run

一个：

$$XX$$

必然具有周期：

$$|X|$$

可以沿这个周期向左右继续扩展，直到无法继续，于是得到一个极大周期区域
因此：

> tandem repetition 是局部重复；
> Run 是把这种重复尽可能向两边扩展后的极大结构

区别在于：

- 一个 Run 可以压缩大量 tandem repetitions
- Run 只保留最小周期
- Main–Lorentz 可以直接描述所有具体 `XX`
  所以通常：

需求	优先算法
极大周期段	Runs
最大指数	Runs
存在长重复周期结构	Runs
枚举所有 `XX`	Main–Lorentz
统计所有 `XX`	Main–Lorentz 压缩组
单次判断某区间周期	Hash / LCP

一个 Run 内固定最小周期的基本重复

设 Run：

```
(p,l,r)
```

长度：

$$L = r - l$$

那么最基本的长度 `2p` tandem repetition 的起点可以位于：

$$l, l+1, \dots, r-2p$$

共有：

$$\boxed{L - 2p + 1}$$

个位置
但如果要统计所有可能半长的 tandem repetitions，不能简单只用这个公式；不同半长和局部更小周期会产生更复杂的情况，直接使用 `main_lorentz()` 更稳妥

Main–Lorentz 思想快速回忆

将当前串分成：

```
u | v
```

递归解决：

```
完全位于 u 中的重复
完全位于 v 中的重复
```

只剩下：

```
跨越中点的重复
```

对于固定候选周期 `p`：

- 向左能匹配多少：LCS
- 向右能匹配多少：LCP
  如果：

$$LCS + LCP \geq p$$

就存在长度至少 `2p` 的重复结构
LCP / LCS 可以通过若干 Z 函数一次性在线性时间求完，因此每层分治总复杂度为：

$$O(n)$$

递归深度：

$$O(\log n)$$

最终：

$$\boxed{O(n \log n)}$$

复杂度

设字符串长度为 `n`

Runs

`enumerate_runs(s)`：

$$O(n \log n)$$

时间
返回的 Run 数量：

$$O(n)$$

当前实现为了保持代码短且清晰，会保存分治产生的候选区间，额外空间可按：

$$O(n \log n)$$

估计

Main–Lorentz

构造压缩组：

```
main_lorentz(s)
```

时间：

$$O(n \log n)$$

压缩组数量为：

$$O(n \log n)$$

若只做：

- 计数
- 求最长 repetition
- 判断是否存在
  无需展开全部答案，仍保持 $O(n \log n)$
  若展开所有 tandem repetitions：

  $$O(n \log n + K)$$

  其中：

  $$K = \text{实际输出数量}$$

  且最坏：

  $$K = \Theta(n^2)$$